

차례

Proven C 라이브러리 완전 매뉴얼 (v0.1.1)	4
목차	4
저작권	4
따라 하며 익히기: 아홉 개의 짧은 프로그램	5
이 글이 있는 이유	5
목차	5
1과 — 출력, 그리고 되는 빌드	5
2과 — 자기 길이를 아는 텍스트	6
3과 — 실패할 수 있는 호출은 그렇다고 말한다	7
4과 — 값과 오류는 함께 온다	8
5과 — 기억을 누가 주는가는 인자다	9
6과 — 0장의 프로그램을 한 줄씩	10
7과 — 한꺼번에 풀려나는 기억	11
8과 — 컨테이너는 자기 할당자를 기억한다	12
9과 — 바깥 세상도 실패하고, 같은 방식으로 말한다	13
다음에 읽을 것	15
Chapter 0: 여기서부터 시작	16
목차	16
1. 이 매뉴얼은 누구를 위한 것인가	16
2. 이 라이브러리가 존재하는 이유	16
3. 첫 프로그램	19
4. 빌드와 include	20
5. 모든 페이지에서 만나게 될 다섯 가지 계약	21
6. 의도와 설계 철학	22
7. 빌드와 include 모델	23
8. 전역 계약(Global contracts)	24
9. 소유권과 파괴 매트릭스	25
10. 연산 동작 클래스	28
11. 매뉴얼 챕터	29
12. 플랫폼 지원과 검증	30
13. 부록 B: 용어집	30
14. 부록 C: 공개 헤더 맵	35
15. 부록 D: libc 대응표	36
16. 다음에 읽을 것	37
1장: 기초 — 에러, 타입, 메모리 view	38
목차	38
1. 에러는 값이다	38
2. 타입, 그리고 왜 표기가 다른가	42
3. 메모리 view: 포인터와 길이를 하나로	43
4. 감길 수 없는 크기 산술	45
5. 정렬	46
6. Panic: 에러를 돌려줄 상대가 남지 않았을 때	49
7. 버전 매크로	50
8. 예제와 오용 사례	51
2장: 할당 — 힙, 아레나, 풀, 버퍼	53
목차	53
1. 할당이 매개변수인 이유, 그리고 heap allocator	53
2. Arena: 여러 객체, 하나의 수명	54

3. Pool: 여러 객체, 하나의 크기	57
4. Allocator 트레잇	59
5. 원시 바이트 버퍼	60
6. 예제와 오용 사례	61
3장: 문자열, 포매팅, 스캐닝	70
목차	70
1. U8 문자열과 view	70
2. U16 문자열과 view	74
3. 포매팅	75
4. 스캐닝	78
5. 예제와 오용 사례	81
4장: 컨테이너와 알고리즘	91
목차	91
1. 동적 배열	91
2. 침습적(intrusive) 리스트	93
3. 링 버퍼	96
4. 해시 맵	99
5. 알고리즘	102
6. 용도별 해싱	102
7. 바이트를 텍스트로: hex와 Base64	105
8. 예제와 오용 사례	109
Chapter 8: Formatting and Scanning (v0.1.1)	121
목차	121
1. 설계 모델	121
2. 포매터 데이터 모델	122
3. 포매터 생성자와 선택자	124
4. 형식 문자열 문법	128
5. 형식화 API	130
6. 콘솔 출력 헬퍼	134
7. 스캐너 데이터 모델	134
8. 스캐너 기본 API	135
9. 스캔 인자 모델	136
10. 구조적 스캔 문법	136
11. 스캔 형식화 API	137
12. 예제와 오용 사례	138
13. 프리스탠딩과 빌드 모드 참고	143
Chapter 5: Hosted Services	152
목차	152
1. 파일시스템 API	152
2. 시스템 입출력과 환경변수	157
3. 메모리 매핑	158
4. 시간 API	160
5. 예제와 오용 사례	163
디렉터리를 한 번에 한 엔트리씩 읽기	167
트리 순회	168
스트림: writer와 reader	171
표준 스트림	175
난수, 용도별	177
6장: 실행, 별칭, PAL, Freestanding, 크로스 빌드	197

목차	197
1. 스택리스 코루틴	197
2. Job 시스템	198
3. 스레드, allocator, 포인터 프로버넌스	200
4. 별칭 계층	203
5. PAL 계약	204
6. Freestanding 부분집합	205
7. 크로스 컴파일	205
8. 예제와 오용 사례	206
Proven Freestanding 모드 (v0.1.1)	211
0. 이 모드가 존재하는 이유, 그리고 라이브러리 전체가 이 제약에 의해 빚어진 이유	211
1. 빌드 프로파일	212
2. 사용 가능한 소스 파일	212
3. 모듈 사용 가능성	213
4. 최소 정적 arena 설정	215
5. Panic 핸들러 오버라이드	216
6. Cortex-M 컴파일 명령 예제	217
7. RISC-V ELF 컴파일 명령 예제	217
8. Freestanding 모드에서의 포매팅	218
8a. Float 큰 정수 용량 조정	218
9. 호스티드 API 피하기	219
10. 수명 규칙은 여전히 적용된다	219
11. 검증	220
부록 A: alias_xcv.h Alias 인덱스 (7장)	221
사용 규칙	221
Alias 표	221

Proven C 라이브러리 완전 매뉴얼 (v0.1.1)

C23 시스템 라이브러리 proven의 매뉴얼입니다. 각 장은 `manual-ko/` 아래 파일 하나이며, 영문판([manual/](#))의 미러입니다. 코드와 API 계약을 빌드가 검증하는 정보는 영문판이고, 두 문서가 어긋나면 영문본이 기준입니다.

목차

- 따라 하며 익히기 — 짧은 프로그램 아홉 개
- 0장 — 여기서부터
- 1장 — 기초
- 2장 — 할당
- 3장 — 문자열과 텍스트
- 4장 — 컨테이너와 알고리즘
- 8장 — 형식화와 파싱
- 5장 — Hosted 서비스
- 6장 — 실행과 플랫폼
- 프리스탠딩
- 부록 A — Alias 인덱스

저작권

- 지은이: rubidus — rubidus@gmail.com
- 저장소: github.com/rubidus-api/proven_c_lib
- 판: v0.1.1
- 라이선스: MIT — Copyright (c) 2026 rubidus-api. 라이브러리와 이 매뉴얼은 같은 라이선스를 따르며, 전문은 저장소 루트의 LICENSE에 있습니다.
- 이 매뉴얼의 모든 코드 예제는 빌드가 컴파일하고, 인용된 예제는 실행까지 합니다.

따라 하며 익히기: 아홉 개의 짧은 프로그램

Part I — 여기서 시작합니다. 입문용 C 책 한 권 외에 사전 지식은 필요 없습니다. **6과를 마치면 0장**의 인사말 프로그램을 한 줄씩 읽어 낼 수 있고, 레퍼런스 장들이 낯선 낱말의 벽으로 보이지 않게 됩니다. **9과를 마치면** 아레나, 자라는 컨테이너, 파일을 한 번씩 써 본 것이 됩니다 --- 텍스트를 다룰 줄 알게 된 프로그램이 다음으로 찾는 것들입니다.

이 글이 있는 이유

0장은 첫 쪽부터 도는 프로그램을 보여 줍니다. 그런데 그 서론 줄에는 새 개념이 다섯 개나 한꺼번에 들어 있습니다 — 인자로 건네는 할당자, 반드시 확인해야 하는 result, 자기 길이를 아는 view, 자르는 대신 거부하는 append, 만든 할당자로 짝지어 부르는 destroy. C 책 한 권을 막 뗐다면 다섯은 넷이 많습니다.

그래서 이 실습은 하나씩만 더합니다. 프로그램 아홉 개, 새로 나오는 것은 매번 딱 하나입니다. 앞의 여섯은 0장의 프로그램까지 이끌고, 뒤의 셋은 그 너머의 아레나·컨테이너·파일로 갑니다. 아홉 모두 manual/examples/ 아래의 진짜 파일이고 빌드가 컴파일해 **실제로 돌립니다**. 그러니 여기 실린 것은 정말로 돌아간 코드입니다.

빌드는 어느 C 프로그램과 같습니다. 이 라이브러리는 여러분의 코드와 함께 컴파일하는 소스라서 설치할 것이 없습니다.

```
cc -std=c23 -Iinclude your_program.c src/proven/*.c platform/*.c -o your_program
```

이 줄이 낯설다면 **0장 §4**를 먼저 읽고 오세요.

목차

1. 1과 — 출력, 그리고 되는 빌드
2. 2과 — 자기 길이를 아는 텍스트
3. 3과 — 실패할 수 있는 호출은 그렇다고 말한다
4. 4과 — 값과 오류는 함께 온다
5. 5과 — 기억을 누가 주는가는 인자다
6. 6과 — 0장의 프로그램을 한 줄씩
7. 7과 — 한꺼번에 풀려나는 기억
8. 8과 — 컨테이너는 자기 할당자를 기억한다
9. 9과 — 바깥 세상도 실패하고, 같은 방식으로 말한다
10. 다음에 읽을 것

1과 — 출력, 그리고 되는 빌드

새로 나오는 것 하나: proven_printf, 그리고 그 인자가 검사된다는 사실.

printf("%d", 3.5)는 컴파일됩니다. 하지만 틀렸고, 운이 나쁘면 쓰레기를 찍거나 죽습니다. 서식 문자열은 그냥 *문자열*이라, 뒤에 오는 인자와 이어 주는 것이 아무것도 없기 때문입니다. proven_printf는 자리표로 {}를 쓰고 인자마다 PROVEN_ARG로 감쌉니다. 이 매크로가 인자의 타입을 함께 실어 보내므로, 서식과 인자가 어긋날 수가 없습니다.

```
/*
 * 1과 - 한 줄을 빌드해서 돌려 본다.
 *
 * 아직 이 라이브러리의 생각에 대한 것은 하나도 없다. 이 프로그램이 답하는 물음은
 * "내 빌드 명령이 되는가" 하나뿐이고, 그것만 따로 물어 볼 값어치가 있다. 뒤의
 * 모든 과가 그것을 전제하기 때문이다.
 *
 * printf(fmt, ...) 는 검사되지 않는다. "%d" 자리에 double 을 건넸다고 컴파일러가
 * 알려 주지 못한다. proven_printf 은 검사한다. 인자마다 PROVEN_ARG 로 감싸
 * 자기 타입을 함께 들고 가기 때문이다.
 */
```

```
int main(void) {
```

```

proven_println("hello from proven");

/* {} 가 자리표다. 값은 PROVEN_ARG 를 거치면서 어떤 타입인지를 함께 싣는다.
 * 그래서 서식 문자열과 인자가 어긋날 수가 없다. */
proven_println("one number: {}", PROVEN_ARG(42));
proven_println("two of them: {} and {}", PROVEN_ARG(1), PROVEN_ARG(2));

return EXAMPLE_OK();
}

```

돌리면 세 줄이 찍힙니다. 빌드되고 돌아왔다면 include 경로와 소스 목록이 맞다는 뜻이고, 이다음부터는 빌드가 아니라 라이브러리 이야기입니다.

처음에 잘 걸리는 것. {}는 %s가 아닙니다. 틀릴 글자가 아예 없고, long이나 int냐를 기억할 필요도 없습니다 — PROVEN_ARG가 압니다.

2과 — 자기 길이를 아는 텍스트

새로 나오는 것 하나: 뷰(view) --- 포인터와 크기가 함께 다니는 것.

여러분이 아는 C에서 문자열은 포인터 하나이고, 길이는 「처음 0 바이트가 나오는 자리」입니다. 그것을 만지는 함수마다 얼마나 되는지 알아내려고 바이트를 훑어야 하고, 0이 없으면 끝을 넘어 훑습니다. 이 설계 하자가 이 언어의 보안 권고 상당수의 뿌리입니다.

view는 그 빠진 반쪽을 드러냅니다. 바이트를 소유하지 않고 빌려 쓰며, 아무것도 해제하지 않습니다.

```

/*
 * 2과 - 자기 길이를 아는 텍스트.
 *
 * 여러분이 아는 C 에서 문자열은 포인터 하나이고, 길이는 "처음 0 바이트가 나오는
 * 자리"다. 함수마다 얼마나 되는지 알아내려고 바이트를 훑어야 하고, 0 이 없으면
 * 끝을 넘어 훑는다.
 *
 * 뷰(view)는 C 가 암묵으로 남겨 둔 그 짝이다. 포인터와 크기가 함께 다닌다.
 * 빌려 쓸 뿐이라 바이트를 소유하지 않고 해제하지도 않는다.
 */

int main(void) {
    /* PROVEN_LIT 은 리터럴에서 뷰를 만든다. 크기는 컴파일 때 정해지므로 여기서
     * 훑는 일은 없다 - strlen 과 다른 점이다. */
    proven_u8str_view_t hello = PROVEN_LIT("hello");

    EXAMPLE_REQUIRE(hello.size == 5, "the view already knows its own length");
    EXAMPLE_REQUIRE(hello.ptr != NULL, "and it points at the literal's bytes");

    /* 길이가 포인터와 함께 다니므로, 비교는 크기 검사에 memcmp 하나다.
     * 훑을 일도 없고 끝을 넘어갈 길도 없다. */
    EXAMPLE_REQUIRE(proven_u8str_view_eq(hello, PROVEN_LIT("hello")), "same text");
    EXAMPLE_REQUIRE(!proven_u8str_view_eq(hello, PROVEN_LIT("hell")), "shorter text differs");

    /* 뷰는 무언가의 *일부*를 복사 없이 가리킬 수 있다. 아래는 리터럴의 가운데다 -
     * 여전히 빌린 것이고, 할당은 없다. */
    proven_u8str_view_t ell = { .ptr = hello.ptr + 1, .size = 3 };
    EXAMPLE_REQUIRE(proven_u8str_view_eq(ell, PROVEN_LIT("ell")), "a window onto the same bytes");

    proven_println("whole: {} (size {})", PROVEN_ARG(hello), PROVEN_ARG(hello.size));
    proven_println("part : {} (size {})", PROVEN_ARG(ell), PROVEN_ARG(ell.size));

    return EXAMPLE_OK();
}

```

눈여겨볼 것. `PROVEN_LIT`은 크기를 컴파일 때 정하므로 `strlen`이 아닙니다. 그리고 세 번째 `view`는 리터럴의 * 가운데*를 복사 없이 가리킵니다 --- `view`가 무언가의 일부를 가리킬 수 있다는 것, 이것이 이 라이브러리의 자르기와 파싱이 할당을 하지 않는 이유입니다.

처음에 잘 걸리는 것. `view`는 빌린 것입니다. 가리키던 바이트가 사라지면 --- 스코프를 벗어난 지역 버퍼, 이미 `destroy` 한 문자열 --- 그 `view`는 어느 C 포인터와 똑같이 매달린 포인터가 됩니다. 이 라이브러리는 몰래 수명을 늘려 주지 않습니다.

3과 — 실패할 수 있는 호출은 그렇다고 말한다

새로 나오는 것 하나: `proven_err_t`, 그리고 자르는 대신 거부하기.

C는 실패를 서로 무관한 세 방식으로 알립니다. 특별한 반환값, 널 포인터, 그리고 다음 호출이 덮어 쓰는 전역 `errno`. 셋 다 안 보기 쉽고, 안 봐도 아무도 뭐라 하지 않습니다. 여기서는 실패할 수 있는 호출이 오류를 * 값*으로 돌려주고, 그 함수들에는 `[[nodiscard]]`가 붙어 있어 버리면 습관이 아니라 컴파일러 경고가 됩니다.

```
/*
 * 3과 - 실패할 수 있는 호출은 반환값으로 그렇다고 말한다.
 *
 * 여러분이 아는 C 는 실패를 세 가지 방식으로 알린다. 특별한 반환값(-1), 널
 * 포인터, 그리고 다음 호출이 덮어써 버리는 errno 라는 전역. 셋 다 안 보기 쉽고,
 * 안 봐도 아무도 뭐라 하지 않는다.
 *
 * 여기서는 실패할 수 있는 호출이 proven_err_t 를 돌려준다. 그냥 값이라서 담아
 * 둘 수도, 비교할 수도, 위로 넘길 수도 있다. 그리고 이 함수들에 붙은
 * [[nodiscard]] 덕분에 무시하는 것은 습관이 아니라 컴파일러 경고가 된다.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* 딱 8 바이트짜리 자리. (그 자리가 어디서 오는지는 2장의 주제다. 지금은
     * 우리가 "얼마나"를 말했다는 것만 보면 된다.) */
    proven_result_u8str_t s = proven_u8str_create(alloc, 8);
    EXAMPLE_REQUIRE(proven_is_ok(s.err), "8 bytes should be available");

    /* 이걸 들어간다. */
    proven_err_t err = proven_u8str_append(&s.value, PROVEN_LIT("12345678"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "eight bytes into eight bytes fits exactly");

    /* 이걸 안 들어간다 - 그리고 라이브러리는 *거부*한다. 들어갈 만큼만 붙이지
     * 않는다. 잘린 낱말은 짧아진 낱말이 아니라 다른 낱말이기 때문이다. */
    proven_err_t too_much = proven_u8str_append(&s.value, PROVEN_LIT("9"));
    EXAMPLE_REQUIRE(!proven_is_ok(too_much), "one byte more than capacity must fail");
    EXAMPLE_REQUIRE(too_much == PROVEN_ERR_OUT_OF_BOUNDS, "and it says why: out of bounds");

    /* 거부는 아무것도 바꾸지 않았다. 문자열은 그대로다. 이것이 "실패 원자성"의
     * 뜻이다 - 실패한 호출은 반쯤 된 상태를 남기지 않는다. */
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&s.value), PROVEN_LIT("12345678")),
        "the refused append must not have written anything");

    proven_println("after the refusal, the string is still: {}",
        PROVEN_ARG(proven_u8str_as_view(&s.value)));

    proven_u8str_destroy(alloc, &s.value);
    return EXAMPLE_OK();
}
```

눈여겨볼 것. 들어가지 못한 append는 아무것도 바꾸지 않습니다. 들어갈 만큼만 넣지도 않습니다. 이것을 실패 원자성이라고 하는데, 이유는 잘린 문자열이 짧아진 메시지가 아니라 다른 메시지이기 때문입니다 --- 잘린 경로는 다른 파일을 가리키고, 잘린 명령은 다른 명령입니다.

처음에 잘 걸리는 것. 검사는 proven_is_ok(err)이고 err == PROVEN_OK도 같은 말입니다. 하면 안 되는 것은 그것을 무시하고 값을 읽는 일입니다.

4과 — 값과 오류는 함께 온다

새로 나오는 것 하나: result 구조체 --- .err와 .value가 한 번에 돌아온다.

돌려줄 값이 없는 호출에는 오류 코드만으로 충분합니다. 돌려줄 값이 있으면 라이브러리는 둘을 작은 구조체 하나에 담아 줍니다. 규칙은 한 문장입니다. **err를 보기 전에는 value는 아무 뜻도 없다.**

```
/*
 * 4과 - 돌려줄 값이 있는 호출은, 값과 오류가 함께 온다.
 *
 * 돌려줄 것이 없을 때는 proven_err_t 하나로 충분하다. 돌려줄 것이 *있으면*
 * 필드 둘짜리 작은 구조체를 받는다. `err` 와 `value` 다. 규칙은 한 문장이다.
 * `err` 를 보기 전에는 `value` 는 아무 뜻도 없다.
 *
 * malloc 이 NULL 인지 확인하던 것과 같은 규율인데, 확인해야 한다는 사실이
 * 기억해야 할 관습이 아니라 타입의 일부라는 점이 다르다.
 */

/* 여러분의 함수도 이것을 돌려줄 수 있다. 라이브러리에 마법은 없다. */
static proven_result_size_t safe_div(proven_size_t a, proven_size_t b) {
    proven_result_size_t res = {0};
    if (b == 0) {
        res.err = PROVEN_ERR_INVALID_ARG;    /* value 는 0 그대로 - 아무 뜻도 없다 */
        return res;
    }
    res.err = PROVEN_OK;
    res.value = a / b;
    return res;
}

int main(void) {
    proven_result_size_t ok = safe_div(10, 2);
    EXAMPLE_REQUIRE(proven_is_ok(ok.err), "dividing by 2 is fine");
    EXAMPLE_REQUIRE(ok.value == 5, "and only now may we read the value");

    proven_result_size_t bad = safe_div(10, 0);
    EXAMPLE_REQUIRE(!proven_is_ok(bad.err), "dividing by zero must fail");
    EXAMPLE_REQUIRE(bad.err == PROVEN_ERR_INVALID_ARG, "and say which rule was broken");
    /* bad.value 는 0 이지만 그것은 답이 아니다. 답이 없다는 뜻이다. */

    /* 라이브러리 자신의 호출도 같은 모양이다. proven_u8str_create 는 만든
     * 문자열과 그것을 지키는 오류를 함께 돌려준다. */
    proven_allocator_t alloc = proven_heap_allocator();
    proven_result_u8str_t s = proven_u8str_create(alloc, 16);
    if (!proven_is_ok(s.err)) return 1;    /* 만들어진 것이 없으니 지울 것도 없다 */

    proven_println("10 / 2 = {}", PROVEN_ARG(ok.value));

    proven_u8str_destroy(alloc, &s.value);
    return EXAMPLE_OK();
}
```

눈여겨볼 것. `safe_div`는 라이브러리 함수가 아니라 여러분의 함수입니다. 이 방식은 그냥 구조체를 돌려주는 것일 뿐, 밑에 깔린 장치가 없습니다. 그래서 이 쪽을 읽은 순간부터 여러분 코드에 그대로 쓸 수 있습니다.

처음에 잘 걸리는 것. 실패했을 때 `value` 자리에는 보통 0이 들어 있습니다. 그 0은 답이 아니라 *답이 없다는 것*입니다. 둘을 가르는 것이 `err` 검사입니다.

5과 — 기억을 누가 주는가는 인자다

새로 나오는 것 하나: `proven_allocator_t` --- 가정하지 않고 건네받는다.

`malloc`은 여러분 대신 내려진 결정입니다. 힙 하나, 전략 하나, 그리고 호출 자리에는 그런 티가 나지 않습니다. 이 라이브러리에서는 기억이 필요한 것이면 무엇이든 할당자를 인자로 받고 그것만 씁니다. 따라 나오는 둘이 요점입니다 --- 「이건 누가 할당했지?」에 호출을 읽어 답할 수 있고, 같은 코드에 다른 할당자를 건네도 코드는 그대로입니다.

```
/*
 * 5과 - 기억을 누가 주는가는 전역이 아니라 인자다.
 *
 * malloc 은 여러분 대신 내려진 전역 결정이다. 힙 하나, 전략 하나, 그리고 호출
 * 자리에는 보이지 않는다. 여기서는 기억이 필요한 것이면 무엇이든
 * proven_allocator_t 를 받아 그것만 쓴다. 따라 나오는 둘이 요점이다.
 *
 * - "이건 누가 할당했지?" 에 호출을 보고 언제나 답할 수 있다.
 * - 같은 코드에 다른 할당자를 건네도 코드는 그대로다.
 */

/* 이 함수는 기억이 어디서 오는지 알지도, 신경 쓰지도 않는다. */
static proven_result_u8str_t make_greeting(proven_allocator_t alloc,
                                           proven_u8str_view_t name) {
    proven_result_u8str_t out = proven_u8str_create(alloc, 64);
    if (!proven_is_ok(out.err)) return out;

    proven_err_t err = proven_u8str_append(&out.value, PROVEN_LIT("hello, "));
    if (proven_is_ok(err)) err = proven_u8str_append(&out.value, name);
    if (!proven_is_ok(err)) {
        proven_u8str_destroy(alloc, &out.value); /* 만든 것을 되돌린다 */
        out.err = err;
        out.value = (proven_u8str_t){0};
    }
    return out;
}

int main(void) {
    /* (a) 보통의 힙 - 밑에서 malloc 과 free 가 돈다 */
    proven_allocator_t heap = proven_heap_allocator();

    proven_result_u8str_t a = make_greeting(heap, PROVEN_LIT("world"));
    EXAMPLE_REQUIRE(proven_is_ok(a.err), "the heap should be able to give 64 bytes");
    proven_println("from the heap : {}", PROVEN_ARG(proven_u8str_as_view(&a.value)));
    /* 만든 것과 *같은* 할당자로 지운다. 그 짝이 이 라이브러리 소유 규칙의
     * 전부다. */
    proven_u8str_destroy(heap, &a.value);

    /* (b) 아레나 - 기억 덩이 하나를 차례로 나눠 주고 한꺼번에 되돌린다.
     * 위의 make_greeting 은 조금도 바뀌지 않았다는 데 주목할 것. */
    alignas(PROVEN_MAX_ALIGN) proven_byte_t backing[512];
    proven_arena_t arena = proven_arena_create((proven_mem_mut_t){
        .ptr = backing, .size = sizeof backing });
    proven_allocator_t from_arena = proven_arena_as_allocator(&arena);
```

```

proven_result_u8str_t b = make_greeting(from_arena, PROVEN_LIT("arena"));
EXAMPLE_REQUIRE(proven_is_ok(b.err), "the arena has room for this too");
proven_println("from an arena : {}", PROVEN_ARG(proven_u8str_as_view(&b.value)));

/* 여기엔 지우는 반복문이 없다. 아레나는 reset 으로 전부 되돌린다. 그 이야기는
 * 2장의 몫이고, 지금의 요점은 *부르는 쪽이 골랐다*는 것뿐이다. */
proven_arena_reset(&arena);

return EXAMPLE_OK();
}

```

눈여겨볼 것. make_greeting은 한 번 쓰였고 힙에서도, 아레나에서도 돕니다. 아레나는 여러분이 가진 기억 덩이 하나를 포인터를 밀어 가며 나눠 주고 한꺼번에 되돌리는 방식입니다. 함수는 바뀌지 않았고, 전역으로 설정한 것도 없습니다.

소유 규칙 한 줄: 만든 쪽이 지운다, 그것도 같은 할당자로.

처음에 잘 걸리는 것. 아레나에서는 destroy가 아무것도 되돌리지 않습니다 --- 아레나는 reset 으로 해제합니다. 그래도 부르세요. 그 쪽이 있어야 나중에 할당자를 바꿔도 코드가 옳습니다.

6과 — 0장의 프로그램을 한 줄씩

새로 나오는 것 하나: 없습니다. 그것이 이 과제입니다.

```

/*
 * 6과 - 0장의 프로그램을 한 줄씩 읽는다.
 *
 * 새로 나오는 것은 없다. 매뉴얼이 첫 쪽에 실는 그 인사말 프로그램 그대로이고,
 * 이 과의 요점은 이제 그 모든 부분에 이름을 달 수 있다는 것이다. 건네받는
 * 할당자(5과), 반드시 확인해야 하는 result(4과), 거부가 돌려주는 오류(3과),
 * 자기 길이를 들고 다니는 뷰(2과), 그리고 인자를 검사하는 출력(1과).
 *
 * 이것이 이제 평범하게 읽힌다면 이 실습은 할 일을 다한 것이고, 레퍼런스 장들이
 * 여러분에게 열린 것이다.
 */

int main(void) {
    /* 5과 - 기억이 어디서 오는지는 부르는 쪽이 정한다 */
    proven_allocator_t alloc = proven_heap_allocator();

    /* 4과 - 문자열과 그것을 지키는 오류가 함께 온다 */
    proven_result_u8str_t greeting = proven_u8str_create(alloc, 64);
    if (!proven_is_ok(greeting.err)) return 1;

    /* 2과 - 자기 크기를 아는, 빌려 쓰는 텍스트 */
    proven_u8str_view_t name = PROVEN_LIT("world");

    /* 3과 - 각 append 는 들어가거나 거부한다. 자르는 것은 하나도 없다 */
    proven_err_t err = proven_u8str_append(&greeting.value, PROVEN_LIT("hello, "));
    if (proven_is_ok(err)) err = proven_u8str_append(&greeting.value, name);
    if (proven_is_ok(err)) err = proven_u8str_append(&greeting.value, PROVEN_LIT("!"));

    if (!proven_is_ok(err)) {
        proven_u8str_destroy(alloc, &greeting.value);
        return 1;
    }

    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&greeting.value),
        PROVEN_LIT("hello, world!")),

```

```

        "the three appends should have built the whole greeting");

/* 1과 - 서식 문자열과 인자는 어긋날 수 없다 */
proven_println("{ }", PROVEN_ARG(proven_u8str_as_view(&greeting.value)));

/* 다시 5과 - 만든 할당자로 지운다 */
proven_u8str_destroy(alloc, &greeting.value);
return EXAMPLE_OK();
}

```

저 주석들이 새 정보가 아니라 *아는 것에 붙은 이름표*로 읽힌다면, 이제 레퍼런스 장을 펼칠 준비가 된 것입니다. 다음 세 과는 그 장들로 들어가는 첫걸음 --- 아레나, 컨테이너, 파일 --- 이고, 모양은 같습니다. 과마다 새 생각 하나.

7과 — 한꺼번에 풀려나는 기억

새로 나오는 것 하나: proven_arena_reset --- 아레나(arena)가 나눠 준 것을 호출 하나로 전부 돌려받는다.

5과는 make_greeting에 아레나를 건네고 끝에서 reset 했지만, 그 reset 이 무엇을 하는지는 말하지 않았습니 다. 생각은 이것이 전부입니다. 아레나는 여러분이 가진 기억 덩어리를 포인터를 앞으로 밀며 조각조각 나눠 줍니다. 조각 하나를 돌려받는 방법은 없습니다 --- 아레나에 destroy를 불러도 돌아오는 것이 없습니다. reset 하면 *전부*가 한꺼번에 돌아옵니다.

이것은 대부분의 프로그램이 가진 모양에 맞습니다. 바퀴마다 임시 글을 좀 만들어 쓰고, 쓰고 나면 끝나는 루프입니다. 바퀴 안에서는 마음껏 할당하고 끝에서 reset 하면, 따라다닐 것도 셀 것도 없습니다.

```

/*
 * 7과 - 한꺼번에 풀려나는 기억.
 *
 * 5과는 make_greeting 에 아레나를 건넌 뒤 reset 했지만, 그 reset 이 무엇인지는
 * 말하지 않았다. 이번 과는 그 호출 하나에 대한 것이다.
 *
 * 아레나는 여러분이 가진 기억 덩어리 위에서 포인터를 앞으로 밀어 가며 나눠 준다.
 * 객체 하나하나를 푸는 free 는 없다. 아레나에 destroy 를 불러도 돌아오는 것은
 * 없다. 모든 것이 proven_arena_reset 한 번에 한꺼번에 돌아온다. 제약처럼 들리지만
 * 그것이 아레나를 쓰는 이유다. 일감마다 임시 글을 만드는 루프는 문자열을 몇 개를
 * 만들었던 문장 하나로 전부 버릴 수 있다.
 */

```

```

int main(void) {
    /* 이 배열의 주인은 아레나가 아니라 우리다. 아레나는 조각을 나눠 줄 뿐이다. */
    alignas(PROVEN_MAX_ALIGN) proven_byte_t backing[256];
    proven_arena_t arena = proven_arena_create((proven_mem_mut_t){
        .ptr = backing, .size = sizeof backing });
    proven_allocator_t scratch = proven_arena_as_allocator(&arena);

    static const proven_u8str_view_t names[] = {
        PROVEN_LIT("ada"), PROVEN_LIT("grace"), PROVEN_LIT("barbara"),
    };

    for (proven_size_t i = 0; i < sizeof names / sizeof names[0]; ++i) {
        /* 한 바퀴에 문자열 둘, 둘 다 아레나에서 나온다. */
        proven_result_u8str_t line = proven_u8str_create(scratch, 32);
        proven_result_u8str_t note = proven_u8str_create(scratch, 32);
        EXAMPLE_REQUIRE(proven_is_ok(line.err) && proven_is_ok(note.err),
            "each round fits easily in 256 bytes");
        if (!proven_is_ok(line.err) || !proven_is_ok(note.err)) return 1;

        proven_err_t err = proven_u8str_append(&line.value, PROVEN_LIT("hello, "));
        if (proven_is_ok(err)) err = proven_u8str_append(&line.value, names[i]);
    }
}

```

```

if (proven_is_ok(err)) err = proven_u8str_append(&note.value, PROVEN_LIT("(scratch)"));
EXAMPLE_REQUIRE(proven_is_ok(err), "the appends fit their capacity");

/* 바퀴마다 첫 할당은 `backing` 의 맨 앞에 떨어진다. 앞 바퀴 끝의
 * reset 이 전부 돌려주었기 때문이다. */
EXAMPLE_REQUIRE((const void *)proven_u8str_as_view(&line.value).ptr == (const void *)backing,
    "each round starts again at the beginning of the block");

proven_println("round {}: {} {} ({} bytes in use)",
    PROVEN_ARG(i), PROVEN_ARG(proven_u8str_as_view(&line.value)),
    PROVEN_ARG(proven_u8str_as_view(&note.value)),
    PROVEN_ARG(arena.offset));

/* 문장 하나로 두 문자열이 사라진다. destroy 루프도, 잊을 것도 없다.
 * 여기서부터 `line` 과 `note` 는 회수된 기억을 가리킨다. */
proven_arena_reset(&arena);
EXAMPLE_REQUIRE(arena.offset == 0, "reset reclaims everything the round allocated");
}

/* 아레나는 자라지 않는다. 덩어리보다 많이 달라고 하면 3과에서 본 것과 같은
 * 오류 값이 온다. 죽지도 않고, 몰래 malloc 으로 넘어가지도 않는다. */
proven_result_u8str_t too_big = proven_u8str_create(scratch, 1024);
EXAMPLE_REQUIRE(too_big.err == PROVEN_ERR_NOMEM, "a 256-byte arena refuses 1 KiB");

proven_arena_destroy(&arena);
return EXAMPLE_OK();
}

```

눈여겨볼 것. 바퀴마다 쓰는 바이트 수가 같고, 바퀴마다 첫 문자열이 `backing`의 맨 앞에 떨어집니다 --- 프로 그램이 그것을 확인합니다. `reset` 은 문자열 두 개를 푼 것이 아니라 숫자 하나를 0으로 되돌렸을 뿐이고, 그 래서 한 바퀴에 문자열을 둘 만들든 이천 개 만들든 값이 같습니다. 그리고 아레나가 너무 작으면 답은 `PROVEN_ERR_NOMEM`, 3과와 같은 오류 값이지 죽는 것이 아닙니다.

처음에 잘 걸리는 것. `reset` 뒤에도 `line`과 `note`는 옛 포인터를 그대로 들고 있고, 그 포인터는 이제 다음 바퀴 가 덮어쓸 기억을 가리킵니다. `reset` 은 아레나가 내준 모든 포인터를 한꺼번에 허공을 가리키게 만듭니다. 아 레나 밖으로 복사해 두지 않은 것은 바퀴 너머로 들고 가지 마세요.

8과 — 컨테이너는 자기 할당자를 기억한다

새로 나오는 것 하나: 컨테이너는 할당자(allocator)를 만들 때 한 번 받아서 간직한다.

3과의 문자열은 자리가 정해져 있었고, 할당자를 `create`에도 `destroy`에도 건넸습니다. 자라는 배열은 그렇게 할 수 없습니다. 자리를 넘치는 `push` 는 호출자가 계획하지 않은 순간에 다시 할당해야 하기 때문입니다. 그 래서 배열은 만들 때 받은 할당자를 간직합니다. 한 번 건네고 나면 `push`, `get`, `destroy` 는 배열만 받습니다.

```

/*
 * 8과 - 컨테이너는 자기 할당자를 기억한다.
 *
 * 문자열은 할당자로 만들고 같은 할당자로 없앤다. 두 번 다 여러분이 건넨다.
 * 자라는 배열은 push 가 저장 공간을 넘칠 때마다 나중에 다시 할당해야 하므로,
 * 만들 때 받은 할당자를 간직한다. 새로 나오는 것은 그 하나다. 할당자는 만들 때
 * 한 번 건네고, 그 뒤의 push, get, destroy 는 배열 말고는 아무것도 받지 않는다.
 *
 * PROVEN_ARRAY_* 매크로는 원소 타입을 인자로 받아 값을 그 타입으로 만든다.
 * 그래서 그 타입으로 바뀔 수 없는 값 - int 배열에 push 하는 구조체 - 은
 * 컴파일되지 않는다. 적어 준 타입이 배열을 만들 때의 타입과 같은지는 알지
 * 못한다. 매번 같은 타입을 적는 것은 여러분 몫이다.
 */

```

```

int main(void) {

```

```

proven_allocator_t alloc = proven_heap_allocator();

/* 2 는 처음 용량이지 한도가 아니다. */
proven_result_array_t made = PROVEN_ARRAY_INIT(alloc, int, 2);
EXAMPLE_REQUIRE(proven_is_ok(made.err), "creating an empty array must succeed");
if (!proven_is_ok(made.err)) return 1;
proven_array_t squares = made.value;

/* 둘 들어갈 자리에 열 번 push 한다. 들어가지 않는 push 마다 배열이 기억한
 * 할당자로 저장 공간을 늘린다. 여기에는 할당자가 없다. */
for (int i = 1; i <= 10; ++i) {
    proven_err_t err = PROVEN_ARRAY_PUSH(&squares, int, i * i);
    EXAMPLE_REQUIRE(proven_is_ok(err), "the heap can grow a ten-int array");
    if (!proven_is_ok(err)) {
        PROVEN_ARRAY_DESTROY(&squares);
        return 1;
    }
}
EXAMPLE_REQUIRE(squares.len == 10, "ten pushes, ten elements");

/* get 은 원소를 가리키는 포인터를 돌려주고, 인덱스가 끝을 넘으면 NULL 을
 * 돌려준다. 범위를 벗어난 인덱스는 엉뚱한 곳 읽기가 아니라 검사할 수 있는 답이다. */
const int *third = PROVEN_ARRAY_GET(&squares, int, 2);
EXAMPLE_REQUIRE(third && *third == 9, "element 2 is 3 * 3");
EXAMPLE_REQUIRE(PROVEN_ARRAY_GET(&squares, int, 10) == NULL, "index 10 is past the end");

int sum = 0;
for (proven_size_t i = 0; i < squares.len; ++i) {
    sum += *PROVEN_ARRAY_GET(&squares, int, i);
}
EXAMPLE_REQUIRE(sum == 385, "1 + 4 + 9 + ... + 100");
proven_println("{} squares, sum {}", PROVEN_ARG(squares.len), PROVEN_ARG(sum));

/* destroy 는 배열만 받는다. 간직해 둔 할당자로 풀어 준다. */
PROVEN_ARRAY_DESTROY(&squares);
return EXAMPLE_OK();
}

```

눈여겨볼 것. 배열은 둘 들어갈 자리로 만들어져 push 를 열 번 받았는데, 그 어디에도 할당자가 나오지 않습니다. 끝을 넘는 PROVEN_ARRAY_GET은 NULL입니다 --- 저장 공간 너머에 무엇이 있든 읽어 버리는 것이 아니라, 확인할 수 있는 답입니다. 매크로가 원소 타입(int)을 인자로 받으므로 그 타입으로 바뀔 수 없는 값(구조체 같은)은 컴파일되지 않습니다. 배열을 만들 때와 같은 타입을 적는 것은 여전히 여러분 몫입니다.

처음에 잘 걸리는 것. PROVEN_ARRAY_GET이 돌려주는 포인터는 배열의 저장 공간 *안*을 가리킵니다. 다음 push 가 저장 공간을 더 큰 덩어리로 옮길 수 있고, 그러면 옛 포인터는 허공을 가리킵니다. 꺼내서 쓰고, push 너머로 들고 가지 마세요. 4장에 이 실수가 통째로 나옵니다. 7과의 걸림돌이 같은 이유로 다시 나온 것입니다 --- 그 기억이 언제 옮겨질지는 여러분의 포인터가 아닌 다른 것이 정합니다.

9과 — 바깥 세상도 실패하고, 같은 방식으로 말한다

새로 나오는 것 하나: 여러분 프로그램 탓이 아닌 실패도 같은 오류 값으로 온다.

지금까지의 실패는 모두 프로그램 안에 원인이 있었습니다 --- 자리가 모자라거나 인자가 틀렸거나. 파일은 바깥의 실패를 데려옵니다. 없는 이름, 쓸 권한이 없는 디렉터리, 가득 찬 디스크. 어느 것도 버그가 아니고, 모두 멀쩡한 프로그램에게 일어납니다. 라이브러리는 이것들도 앞에서와 같은 proven_err_t 값으로 알려므로, 새로 배울 두 번째 오류 체계가 없습니다.

쓰는 호출은 통파일 호출입니다. proven_fs_write_file_atomic은 호출 하나로 파일을 쓰고, proven_fs_read_all_u8str은 파일을 소유하는 문자열로 되웁니다. open 도, 읽기 루프도, 잊어버릴 close 도 없습니다.

```
/*
 * 9과 - 바깥 세상도 실패하고, 같은 방식으로 그렇다고 말한다.
 *
 * 지금까지는 프로그램 안의 이유로만 실패할 수 있었다. 자리가 모자라거나 인자가
 * 틀렸거나. 파일은 없을 수도, 읽을 수 없을 수도, 디스크가 가득 찼을 수도 있고,
 * 그 어느 것도 여러번 코드의 버그가 아니다. 라이브러리는 그런 실패도 앞에서와
 * 같은 오류 값으로 알린다. 그러니 이미 배운 검사가 검사의 전부다.
 *
 * 여기 쓰는 것은 통파일 호출이다. 호출 하나로 파일을 쓰고, 하나로 되웁니다.
 * open 도, 읽기 루프도, 잊어버릴 close 도 없다.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* 현재 디렉터리의 이름 하나. 프로그램은 끝나기 전에 이것을 지운다. */
    proven_u8str_view_t path = PROVEN_LIT("proven_tutorial_notes.tmp");
    proven_u8str_view_t text = PROVEN_LIT("buy milk\ncall home\n");

    /* 원자적이다. 읽는 쪽은 옛 파일이나 새 파일을 보지, 반쪽을 보지 않는다.
     * 할당자는 호출이 경로를 다루며 쓸 수 있는 임시 작업 공간이다. */
    proven_err_t err = proven_fs_write_file_atomic(alloc, path, proven_mem_view_from_u8(text));
    EXAMPLE_REQUIRE(proven_is_ok(err), "writing a small file in the current directory should work");
    if (!proven_is_ok(err)) return 1;

    /* 읽기는 소유하는 문자열을 돌려준다. 4과의 모양, 5과의 규칙 그대로다.
     * `alloc` 으로 만들어졌으니 `alloc` 으로 없앤다. */
    proven_result_u8str_t back = proven_fs_read_all_u8str(alloc, path);
    EXAMPLE_REQUIRE(proven_is_ok(back.err), "the file we just wrote can be read");
    if (proven_is_ok(back.err)) {
        EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&back.value), text),
            "the bytes come back exactly as written");
        proven_println("read {} bytes back", PROVEN_ARG(proven_u8str_as_view(&back.value).size));
        proven_u8str_destroy(alloc, &back.value);
    }

    err = proven_fs_remove(alloc, path);
    EXAMPLE_REQUIRE(proven_is_ok(err), "removing the file we created should work");

    /* 이제 파일은 없고, 그것을 읽는 것은 프로그램 바깥에서 온 실패다.
     * 그 실패도 보통 값으로 오고, 어떤 실패인지 말해 준다. */
    proven_result_u8str_t missing = proven_fs_read_all_u8str(alloc, path);
    EXAMPLE_REQUIRE(missing.err == PROVEN_ERR_NOT_FOUND, "a removed file reads as NOT_FOUND");
    if (proven_is_ok(missing.err)) proven_u8str_destroy(alloc, &missing.value);
    proven_println("reading it again: not found, as expected");

    return EXAMPLE_OK();
}
```

눈여겨볼 것. proven_fs_read_all_u8str이 돌려주는 문자열은 소유하는 것입니다. alloc으로 만들고 alloc으로 없앱니다 --- 5과의 규칙 그대로입니다. 지운 파일을 다시 읽으면 PROVEN_ERR_NOT_FOUND가 옵니다. 3과의 거부와 같은 종류의 값이고, 무엇이 잘못됐는지 이름을 댁니다. 쓰기 이름의 *atomic(원자적)*은 읽는 쪽이 옛 파일이나 새 파일을 보지 둘이 섞인 것은 보지 않는다는 뜻입니다. 무엇을 약속하지 않는지(예컨대 정전을 건디는 것)는 5장에 있습니다.

처음에 잘 걸리는 것. 쓰기와 proven_fs_remove에 건네는 할당자는 경로를 다루는 데 쓰는 임시 작업 공간이지 무언가의 주인이 아닙니다. 그 호출들은 가져간 것을 돌아오기 전에 다 돌려줍니다. destroy 할 것이 생기는 것은 문자열을 건네주는 읽기뿐입니다.

다음에 읽을 것

알고 싶은 것	읽을 곳
위 내용의 완전한 판	0장
타입·오류·계약의 자세한 것	1장
아레나·풀·버퍼 제대로	2장
소유하는 문자열, 자르기, 인코딩	3장
배열·맵·리스트·해시	4장
파일·스트림·시간·난수	5장
모르는 낱말	용어집

Chapter 0: 여기서부터 시작

Part I — 여기서 시작합니다. 입문용 C 책 한 권 외에 사전 지식은 필요 없습니다. 이 장을 마치면 proven을 링크한 프로그램을 빌드할 수 있고, 다른 어떤 장이든 읽을 수 있으며, 모르는 용어를 찾아볼 수 있습니다.

문체에 대해. 이 장만 경어체(합니다체)로 쓰였고, 1장부터의 레퍼런스 장들은 평서체(해라체)를 씁니다. 의도한 것입니다 — 0장은 독자에게 직접 말을 거는 입문 장이고, 나머지는 찾아보는 문서이기 때문입니다.

목차

1. 이 매뉴얼은 누구를 위한 것인가
2. 이 라이브러리가 존재하는 이유
3. 첫 프로그램
4. 빌드와 include
5. 모든 페이지에서 만나게 될 다섯 가지 계약
6. 의도와 설계 철학
7. 빌드와 include 모델
8. 전역 계약(Global contracts)
9. 소유권과 파괴 매트릭스
10. 연산 동작 클래스
11. 매뉴얼 챕터
12. 플랫폼 지원과 검증
13. 부록 B: 용어집
14. 부록 C: 공개 헤더 맵
15. 부록 D: libc 대응표
16. 다음에 읽을 것

1. 이 매뉴얼은 누구를 위한 것인가

이 매뉴얼은 여러분이 입문용 C 책을 한 권 끝냈다고 가정합니다. 구체적으로는 변수와 제어 흐름, 함수, 배열, struct, 포인터와 `*/&`, `malloc`과 `free`, `printf`, `strlen`을 쓰는 `char *` 문자열, 그리고 `gcc main.c -o main`으로 프로그램을 컴파일하는 것에 익숙하다고 봅니다. 이것들이 모두 익숙하다면 충분합니다.

반대로 다음은 **가정하지 않습니다**: 규율로서의 ownership(소유), 빌려 쓰는(borrowed) 데이터와 소유(owned) 데이터의 차이, 아레나(arena)나 풀(pool), 인터페이스로 쓰이는 함수 포인터 테이블, C23 어트리뷰트, 컴파일러가 적극적으로 이용해 먹는 대상으로서의 미정의 동작(undefined behaviour), "그냥 되던데" 수준을 넘어선 정렬 (alignment), atomic, 그리고 라이브러리가 최선을 다하는 대신 연산을 *거부*할 수도 있다는 발상. 이 하나 하나는 처음 등장하는 자리에서 설명하며, 전부 § 13의 용어집에도 있습니다.

이 문서는 C 튜토리얼이 아닙니다. 포인터가 무엇인지는 설명하지 않습니다. 대신 이 라이브러리가 왜 `errno`를 설정하는 대신 에러가 담긴 struct를 돌려주는지는 길게 설명합니다. 그것은 여러분이 동의하지 않을 권리가 있는 설계 결정이고, 아무도 설명해 주지 않은 결정에는 반대할 수조차 없기 때문입니다.

2. 이 라이브러리가 존재하는 이유

C는 여러분에게 거의 아무것도 주지 않으면서 여러분을 완전히 믿습니다. 그것이 C의 큰 장점입니다 — 런타임도 없고, 숨은 할당도 없고, 여러분이 쓰지 않은 비용도 없습니다 — 그리고 그래서 C는 여전히 운영체제와 임베디드 장치, 작고 예측 가능해야 하는 모든 것의 언어입니다.

그리고 같은 다섯 가지 버그가 50년째 출하되고 있는 이유이기도 합니다. 이 라이브러리는 그 다섯 가지 버그에 대한 답의 모음입니다. 각 답에는 대가가 따르고, 이 절은 그 대가가 무엇인지 말합니다.

문자열 함수는 무엇이 얼마나 큰지 모릅니다

```
char buf[64];
strcpy(buf, name);           /* wrong: how long is name? strcpy never asks */
strcat(buf, ", welcome!");   /* wrong: and how much room is left now? */
```

strcpy는 목적지 포인터와 원본 포인터를 받습니다. 그 시그니처 어디에도 목적지의 크기가 실려 있지 않으므로, 아무것도 그것을 검사할 수 없습니다. 이 함수는 64바이트 버퍼의 200번째 바이트에도 기꺼이 쓰고, 그렇게 망가뜨리는 것은 컴파일러가 마침 그 뒤에 배치한 무엇이든입니다 — 흔히 반환 주소입니다. 이것은 부주의한 사람들이 가끔 저지르는 실수가 아닙니다. 이 언어의 역사에서 가장 많이 악용된 버그 유형이며, API가 그것을 기본 동작으로 만들어 두었습니다.

strncpy는 전통적인 해답이지만 그 자체가 또 하나의 함정입니다. 항상 NUL로 중단하지는 않기 때문에, "안전한" 버전이 조용히 문자열이 아닌 문자열을 만들어 냅니다.

이 라이브러리는 대신 이렇게 합니다. 문자열이 자기 길이를 지니고 다닙니다. proven_u8str_view_t는 포인터와 크기가 항상 함께 있는 것입니다. append는 목적지의 용량을 알기 때문에 그 용량을 검사하고, 텍스트가 들어가지 않으면 **에러를 반환하고 아무것도 쓰지 않습니다** — 자르지 않습니다. 잘린 경로는 틀린 경로이고, 잘린 명령은 다른 명령이기 때문입니다. [챕터 3](#)을 참조하세요.

대가. 모든 문자열이 한 워드가 아니라 두 워드이고, NUL 종단 형태를 따로 요청하지 않으면 proven 문자열을 printf("%s")에 바로 넘길 수 없습니다.

실패를 확인하게 만드는 장치가 없습니다

```
char *p = malloc(n);
p[0] = 'x';           /* wrong: malloc returns NULL when it fails */
```

malloc은 NULL을 반환해서 실패를 알리는데, 여러분이 끝내 들여다보지 않아도 C는 한마디도 하지 않습니다. fopen도, realloc도, 센티널 값을 반환하는 모든 함수가 마찬가지입니다. 에러는 제공되어 있고, 알아차리는 것은 선택 사항입니다.

errno는 더 나쁩니다. 호출 이후까지 살아남는 전역이기 때문입니다. 다른 라이브러리 호출이 덮어쓸 기회를 갖기 전에, 정확히 알맞은 순간에 확인해야 하고, 성공한 호출도 거기에 쓰레기 값을 넣을 수 있다는 사실을 기억해야 합니다.

이 라이브러리는 대신 이렇게 합니다. 실패할 수 있는 함수는 에러를 값으로 반환하고, 돌려줄 결과까지 있을 때는 둘이 하나의 struct에 담겨 함께 돌아옵니다:

```
proven_result_u8str_t s = proven_u8str_create(alloc, 64);
if (!proven_is_ok(s.err)) return 1;    /* s.value means nothing until you check */
```

하는 일 자체가 실패할 수 있는 함수에는 [[nodiscard]]가 붙어 있어서, 에러를 버리는 코드는 컴파일러가 빌드를 거부합니다. 물론 의도적으로 무시할 수는 있습니다 — 호출 앞에 (void)를 붙이면 됩니다 — 그리고 그것을 타이핑해야 한다는 점이 바로 핵심입니다. [챕터 1](#)을 참조하세요.

대가. if가 늘어납니다. 깊은 곳의 실패에서 한 번에 빠져나오게 해 주는 예외 메커니즘이 없으므로, 에러 경로가 코드의 모양에 그대로 드러납니다. 그 드러남이 바로 기능입니다.

printf는 여러분이 말하는 것을 그대로 믿습니다

```
printf("%d\n", 3.0);       /* wrong: %d with a double. This compiles. */
printf("%s\n", 42);        /* wrong: and this one crashes */
```

포맷 문자열은 런타임에 아무도 검사하지 않습니다. 요즘 컴파일러는 리터럴 포맷에 대해 경고해 주는데, 포맷이 변수가 되는 순간까지만 도움이 됩니다. 그다음부터는 varargs 스택에 마침 들어 있는 바이트를 포맷 문자열이 요구한 모양대로 읽어 대는 함수로 되돌아갑니다.

이 라이브러리는 대신 이렇게 합니다. {}는 타입이 들어 있지 않은 자리표시자이고, 타입은 인자에서 오며 _Generic으로 컴파일 타임에 결정됩니다:

```
proven_println("{} is {}", PROVEN_ARG(name), PROVEN_ARG(count));
```

타입을 두 번 쓰지 않았으므로 %d와 double이 어긋날 여지 자체가 없습니다. 입문용 설명은 [챕터 3 §3](#)을, 전체 문법은 [챕터 8](#)을 보세요.

대가. 인자마다 PROVEN_ARG로 감싸야 하고, 손에 익은 것과는 다른 포맷 언어를 써야 합니다.

이건 누가 free하나요?

```
char *s = build_message(); /* do I free this? the type does not say */
```

함수가 반환한 char *는 갖 할당된 것일 수도, 호출자의 버퍼를 가리킬 수도, 읽기 전용 메모리에 있는 문자열 리터럴일 수도, 다음 호출이 덮어쓸 정적 버퍼를 가리킬 수도 있습니다. 네 경우 모두 타입은 똑같습니다. 답은 문서에 있고, 문서는 코드와 어긋나기 마련입니다.

이 라이브러리는 대신 이렇게 합니다. ownership이 타입 이름과 시그니처에 드러나 있습니다.

proven_u8str_t는 **owned**입니다 — _create에서 받았으니 같은 할당자(allocator)로 _destroy해야 합니다.

proven_u8str_view_t는 **borrowed**입니다 — 다른 누군가가 소유한 바이트를 가리키므로 절대 destroy하지 않고, 소유자가 사라지는 순간 유효하지 않게 됩니다. 할당할 수 있는 함수는 모두 allocator를 파라미터로 받으므로, allocator가 없는 시그니처는 할당할 수 없습니다.

대가. C에 하나뿐이던 타입이 둘이 되고, 모든 경계에서 "이건 누가 소유하지?"를 물어야 하는 규율이 생깁니다 — 어차피 치르던 비용을, 더 늦게 디버거 안에서 치르는 대신 지금 치르는 것입니다.

아무도 타입 검사를 해 줄 수 없는 비교 함수

```
qsort(a, n, sizeof *a, cmp); /* cmp takes const void*; get it wrong and it is UB */
```

qsort는 비교자를 void * 인터페이스로 받으므로, 파라미터 타입이 틀린 비교자도 그대로 컴파일됩니다. 이 버그의 고전적인 형태는 가리키는 대상 대신 포인터 자체를 비교하는 것인데, 그러면 잘 실행되고 아무것도 제대로 정렬하지 않으면서 절대 죽지 않는 프로그램이 나옵니다.

이 라이브러리는 대신 이렇게 합니다. 모양은 똑같은 void *입니다 — 여기는 C이고, 다른 방법은 없습니다

— 대신 라이브러리가 계약을 정확히 문서화하고, 복사해 쓸 수 있는 동작하는 비교자를 제공하며,

proven_array_sort는 공격자가 고른 입력에서 $O(n^2)$ 으로 퇴화하는 퀵소트가 아니라 $O(n \log n)$ 을 보장하는 introsort입니다. [챕터 4](#)를 참조하세요.

바이트에는, 여러분이 정해 주지 않아도, 타입이 있습니다

여러분은 메모리를 바이트로 생각합니다. C의 추상 기계는 그렇지 않습니다. 메모리를 타입이 있는 것으로 다루며, 같은 바이트를 서로 다른 두 타입의 포인터로 읽는 것은 미정의 동작입니다 — 그리고 컴파일러는 그것을, 최적화가 커졌을 때만, 조용히 이용해도 됩니다. 이것을 **strict aliasing**이라 부르며, 바이트 버퍼를 폭이 다른 포인터로 읽는 손수 짠 모든 파서 아래에 깔린 함정입니다:

```
void *buf = malloc(8);
uint32_t *w = buf; /* 같은 메모리를 32비트로 봄 - 캐스트도, 경고도 없음 */
uint16_t *h = buf; /* 같은 메모리를 16비트로 봄 */
*w = 0xAAAAAAAAu;
*h = 0x1234; /* 하위 절반을 바꿈 */
printf("%08x\n", *w); /* aaaa1234를 기대하면 틀림: -02에서는 aaaaaaaa가 찍힘 */
```

-00으로 컴파일하면 aaaa1234가, -02에서는 aaaaaaaa가 찍힙니다. 컴파일러가 uint16_t 쓰기와 uint32_t 읽기는 같은 메모리를 건드릴 수 없다고 가정하고 그 쓰기를 지워 버렸기 때문입니다. 경고는 없고, 프로그램은 디버거 빌드에서 돌린 모든 테스트를 통과했습니다. 이것이 리눅스 커널이 -fno-strict-aliasing으로 컴파일해 피하는 부류의 버그입니다 — 규칙 하나 때문에 플래그 하나를 통째로.

이 라이브러리는 대신 이렇게 합니다. 원시 메모리는 unsigned char의 별칭인 proven_byte_t입니다 — 이 규칙이 명시적으로 면제하는 유일한 타입입니다. 표준이 어떤 객체의 바이트든 그 타입으로 들여다보는 것을 허용하기 때문입니다. 평범한 API는 여러분의 바이트를 몰래 더 넓은 타입으로 재해석하지 않으므로, 위 버그는 그것을 통해서는 쓸 수가 없습니다. (strict aliasing에는 더 미묘한 형제 *provenance*가 있고, 라이브러리 이름이 거기서 왔습니다. [챕터 6 §3](#)과 프로젝트 README가 그것을 다룹니다.)

이 라이브러리가 아닌 것

이것은 프레임워크가 아니며, 여러분의 `main`을 차지할 생각이 없습니다. 초기화해야 할 전역 상태가 없고, 스레드를 띄우지 않으며, `atexit` 핸들러를 등록하지 않고, 여러분이 `allocator`를 건네주지 않은 것은 아무것도 할당하지 않습니다. 모든 모듈은 단독으로 쓸 수 있습니다. 대부분은 운영체제가 전혀 없어도 돌아갑니다 — [프리스탠딩\(freestanding\) 모드](#)를 보세요.

문제	C가 주는 것	proven이 주는 것	대가
버퍼 오버런	<code>strcpy</code> , <code>strcat</code> — 어디에도 크기가 없음	<code>view</code> 가 길이를 지님; 쓰기는 자르는 대신 거부	문자열당 두 워드
확인되지 않은 실패	<code>NULL</code> 반환과 <code>errno</code>	에러를 값으로 반환, 버려서는 안 되는 것에는 <code>[[nodiscard]]</code>	<code>if</code> 가 늘어남
포맷 불일치	<code>printf</code> 는 포맷 문자열을 믿음	타입을 인자에서 가져오는 <code>{}</code>	호출마다 <code>PROVEN_ARG</code>
불분명한 ownership	<code>char *</code> 가 네 가지 서로 다른 것을 의미	<code>owned</code> 와 <code>borrowed</code> 가 서로 다른 타입	타입이 하나에서 둘로
숨은 할당	무엇이든 <code>malloc</code> 을 부를 수 있음	<code>allocator</code> 를 받는 함수만 할당 가능 (한정된 예외 하나: 과도하게 긴 줄에 서의 <code>proven_println</code>)	시그니처에 파라미터가 하나 더
숨은 타입을 가진 바이트	메모리를 더 넓은 포인터로 재해석하면 최적화기가 이용하는 UB	원시 바이트는 규칙이 면제하는 타입 <code>proven_byte_t</code> 를 거침	—

3. 첫 프로그램

이것이 전부입니다. 모든 줄이 §5의 계약 중 하나이고, 빌드가 바로 이 파일을 컴파일하고 실행하므로 조용히 사실이 아니게 될 수 없습니다.

```
/*
 * 첫 프로그램. 일부러 작게 만들었고, 여기 있는 줄 하나하나가 이 매뉴얼의 모든
 * 쪽에서 만나게 될 다섯 계약 가운데 하나다.
 *
 * 여러분이 이미 아는 C 와 견주어 보라.
 *
 *     char buf[64];
 *     strcpy(buf, name);           <- name 은 얼마나 긴가? strcpy 는 묻지 않는다.
 *     strcat(buf, ", welcome!");  <- 이제 남은 자리는? strcat 도 묻지 않는다.
 *     printf("%s\n", buf);
 *
 * 저 프로그램은 `name` 이 여러분의 짐작보다 길어지는 날까지만 옳고, 그날부터는
 * 보안 권고문이 된다. 아래 판은 그럴 수 없다. 모든 쓰기가 목적지의 크기를 알고,
 * 실패할 수 있는 모든 연산이 조용히 무시할 수 없는 오류를 돌려준다.
 */

int main(void) {
    /* (1) 할당자를 여러분이 건넨다. 라이브러리가 등 뒤에서 전역 malloc 에 손을
     * 뻗는 일이 없으므로, 무엇을 누가 할당했는지 언제나 알 수 있다. */
    proven_allocator_t alloc = proven_heap_allocator();

    /* (2) 실패할 수 있는 것은 오류를 값과 *함께* 돌려준다. 확인해야 한다고 기억할
     * errno 같은 것이 없고, `greeting.err` 를 보기 전에는 `greeting.value` 는
     * 아무 뜻도 없다. */
    proven_result_u8str_t greeting = proven_u8str_create(alloc, 64);
```

```

if (!proven_is_ok(greeting.err)) return 1;

/* (3) 뷰는 자기 길이를 아는, 빌려 쓰는 텍스트다. PROVEN_LIT 은 리터럴에서
 * 컴파일 때 그것을 만든다 - 여기서 strlen 처럼 훑는 일은 없다. */
proven_u8str_view_t name = PROVEN_LIT("world");

/* (4) append 는 자르는 대신 거부한다. "hello, " 와 이름이 위에서 요청한 64
 * 바이트에 들어가지 못하면 PROVEN_ERR_OUT_OF_BOUNDS 를 돌려주고 아무것도
 * 쓰지 않는다 - 낱말의 반쪽을 조용히 담아 두고 넘어가는 일이 없다. */
proven_err_t err = proven_u8str_append(&greeting.value, PROVEN_LIT("hello, "));
if (proven_is_ok(err)) err = proven_u8str_append(&greeting.value, name);
if (proven_is_ok(err)) err = proven_u8str_append(&greeting.value, PROVEN_LIT("!"));

if (!proven_is_ok(err)) {
    proven_u8str_destroy(alloc, &greeting.value);
    return 1;
}

EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&greeting.value),
    PROVEN_LIT("hello, world!")),
    "the three appends should have built the whole greeting");

proven_println("{ }", PROVEN_ARG(proven_u8str_as_view(&greeting.value)));

/* (5) `alloc` 으로 만들었으니 *같은* `alloc` 으로 지운다. 소유하는 것은 정확히
 * 한 번 지우고, 빌린 것은 - 위의 `name` 같은 - 아예 지우지 않는다. */
proven_u8str_destroy(alloc, &greeting.value);

return EXAMPLE_OK();
}

```

EXAMPLE_REQUIRE와 EXAMPLE_OK는 라이브러리의 일부가 아닙니다 — manual/examples/example.h에서 오며, 이 매뉴얼의 모든 예제가 본문이 말하는 대로 여전히 동작하는지 빌드가 확인할 수 있게 하려고 존재합니다. 여러분의 프로그램에서는 둘 다 쓰지 않습니다.

여기저기서 계속 나오므로, 잠시 멈춰서 볼 만한 세부 사항 셋:

- **proven_u8str_create(alloc, 64)는 64바이트의 용량을 요청합니다.** 그리고 그 용량에는 NUL 종단 문자가 포함됩니다. 이 문자열은 스스로 커지지 않습니다. proven_u8str_append는 고정 용량 형태라서, 텍스트가 들어가지 않으면 실패합니다. 성장을 원할 때는 proven_u8str_append_grow를 부르면서 allocator를 넘기며, 어느 쪽을 쓰고 있는지는 시그니처만 봐도 한눈에 드러납니다.
- **PROVEN_LIT("hello, ")는 런타임 비용이 0입니다.** 문자열 리터럴에 대한 컴파일 타임 sizeof일 뿐입니다. 텍스트가 다른 곳에서 오는 경우를 위해 proven_u8str_view_from_cstr(s)가 있고, 그쪽은 실제로 NUL을 찾아 스캔합니다 — 하나는 공짜이고 다른 하나는 $O(n)$ 이기 때문에 이름을 다르게 지은 것입니다.
- **destroy는 allocator를 다시 받습니다.** 문자열은 자신을 만든 allocator를 기억하지 않으므로, 같은 것을 넘겨야 합니다. 덕분에 문자열이 작게 유지되고 의존 관계가 눈에 보이지만, 실제 위험 요소이기도 합니다 — § 5를 보세요.

4. 빌드와 include

전부를 위한 헤더는 하나입니다:

```
#include "proven.h"
```

이것이 공개 API 전체를 끌어옵니다. 쓰는 것만 include하고 싶다면 모든 모듈이 include/proven/ 아래에 자기 헤더를 가지고 있고 — #include "proven/u8str.h", #include "proven/fs.h" 하는 식입니다 — 아래 § 14가 모든 헤더를 그것을 문서화한 챕터로 연결해 줍니다.

이 라이브러리에 필요한 전부인, 손으로 컴파일하기:

```
gcc -std=c2x -Iinclude -Iplatform your_program.c src/proven/*.c platform/*.c -o your_program
```

src/proven/은 이식 가능한 라이브러리 본체입니다. platform/은 운영체제와 이야기하는 얇은 계층으로, PAL(platform abstraction layer, 플랫폼 추상화 계층)입니다. 시스템 호출을 하는 것은 전부 거기에 있고 다른 어디에도 없으며, 그래서 나머지 라이브러리를 운영체제가 전혀 없는 대상으로 컴파일할 수 있습니다.

이 저장소는 빌드 시스템 대신 C 프로그램으로 스스로를 빌드합니다:

```
cc -std=c2x -o nob nob.c      # build the build driver, once
./nob build                  # compile everything and run the whole test suite
./nob release                # the same, optimised
```

인자 없이 ./nob을 실행하면 나머지 모드를 보여줍니다 — 새니타이저, freestanding, 크로스 컴파일, 벤치마크입니다. make도 CMake도 없고, 설치할 것도 없습니다.

C23 컴파일러가 필요합니다. 이 라이브러리는 C23 기능을 의도적으로 사용하며, 가장 눈에 띄는 것이 `[[nodiscard]]`입니다. GCC 13+, Clang 16+, 최근 MSVC 모두 동작합니다. 빌드 드라이버는 `-std=c23`을 먼저 시험해 보고, 조금 오래된 컴파일러를 위해 `-std=c2x`로 물려섭니다.

5. 모든 페이지에서 만나게 될 다섯 가지 계약

이 다섯 규칙이 이 라이브러리의 생김새 대부분을 설명합니다. 각각은 여기서 한 번 진술되고, 다른 모든 곳에서는 전제로 깔립니다. 형식적인 버전은 아래 § 8에 있고, 여기 있는 것은 일상 언어판입니다.

#	계약	한 줄 요약	챕터
1	에러는 값이다	실패 가능한 호출은 <code>proven_err_t</code> 나 <code>{ err, value }</code> struct를 반환한다	1
2	view는 borrowed다	view는 다른 누군가가 소유한 메모리를 가리키는 포인터 + 길이다	3
3	할당은 파라미터다	할당할 수 있는 함수는 allocator를 받고, 할 수 없는 함수는 받지 않는다	2
4	호출자 소유 상태는 복사하면 안 된다	어떤 struct는 자기 자신을 가리킨다; 복사하면 dangling 포인터가 생긴다	5
5	거부하되 절대 자르지 않는다	들어가지 않는 연산은 실패하고 아무것도 쓰지 않는다	3

1. 에러는 값이다

실패 가능한 함수는 모두 에러를 돌려줍니다. 그 외에 돌려줄 것이 없으면 맨(bare) `proven_err_t`이고, 값이 있으면 값과 에러가 함께 이동하며 에러를 확인하기 전까지 값은 무의미합니다. `PROVEN_OK`은 0이며, `proven_is_ok(err)`가 `err == 0`보다 잘 읽힙니다.

잘못된 예 — 에러보다 값을 먼저 읽기:

```
proven_result_u8str_t s = proven_u8str_create(alloc, 64);
proven_u8str_append(&s.value, text); /* wrong: s.value is garbage if create failed */
```

둘을 짝지어 놓은 이유가 바로 값이 성공 경로에서만 유효하다는 데 있습니다. 매번, 먼저 확인하세요.

2. view는 borrowed다

`proven_u8str_view_t`는 포인터와 크기입니다. 아무것도 소유하지 않고, 아무것도 할당하지 않으며, 마음대로 복사해도 됩니다 — 하지만 가리키는 대상이 파괴되거나 이동하는 순간 유효하지 않게 됩니다.

잘못된 예 — 가리키는 대상보다 오래 사는 view:

```
proven_u8str_view_t name;
{
    proven_result_u8str_t owned = proven_u8str_create(alloc, 16);
    (void)proven_u8str_append(&owned.value, PROVEN_LIT("temp"));
    name = proven_u8str_as_view(&owned.value);
}
```

```

    proven_u8str_destroy(alloc, &owned.value); /* the bytes are gone */
}
use(name); /* wrong: dangling view */

```

이것은 dangling 포인터와 똑같은 수명 버그이지만, view가 값처럼 보이기 때문에 따로 말해 둘 가치가 있습니다. view는 값이 아닙니다. struct를 뒤집어쓴 포인터입니다.

3. 할당은 파라미터다

시그니처를 읽으세요. `proven_u8str_append(str, data)`는 할당할 수 없으므로 텍스트가 들어가지 않으면 실패합니다. `proven_u8str_append_grow(alloc, str, data)`는 allocator를 받으므로 성장할 수 있습니다. 이 라이브러리의 어떤 것도 여러분 몰래 `malloc`을 부르지 않으며, 그래서 arena에서, pool에서, 또는 힙이 없는 장치에서도 쓸 수 있습니다.

여기서 따라 나오는 것이 바로 위험 요소입니다: **생성할 때 쓴 allocator로 파괴해야 합니다.** 객체는 그것을 기억하지 않습니다.

잘못된 예 — 짝이 맞지 않는 allocator:

```

proven_result_u8str_t s = proven_u8str_create(arena_alloc, 64);
proven_u8str_destroy(heap_alloc, &s.value); /* wrong: heap free on arena memory */

```

현재로서는 아무것도 이것을 검사하지 않습니다. 나중에, 다른 어딘가에서 드러나는 힙 손상입니다.

4. 호출자 소유 상태는 복사하면 안 된다

어떤 객체는 여러분이 소유하고 포인터로 넘기는 struct입니다 — 버퍼드 쓰기 스트림(writer), line 읽기 스트림(reader), 디렉터리 이터레이터 같은 것들입니다. 그중 여럿은 자기 자신의 필드를 가리키는 포인터를 품고 있어서, struct를 복사하면 여전히 *원본*을 가리키는 포인터가 함께 복사됩니다.

잘못된 예 — state struct 복사:

```

proven_writer_buf_t a = ...;
proven_writer_buf_t b = a; /* wrong: b's internals still point into a */

```

아래 § 9.2에 이런 타입 열여섯 개가 모두 나열되어 있습니다. 규칙은 간단합니다. 살아 있을 자리에서 만들고, &it을 넘기고, 대입하지 마세요.

5. 거부하되 절대 자르지 않는다

결과가 들어가지 않을 때, 이 라이브러리는 연산을 실패시키고 목적지를 그대로 둡니다. "들어가는 만큼"을 쓰지 않습니다. 잘린 경로는 엉뚱한 파일을 열고, 잘린 명령은 엉뚱한 명령을 실행하며, 잘린 숫자는 다른 숫자입니다 — 그리고 그 하나하나가 눈에 보이는 에러보다 나쁩니다.

정말로 자르는 것이 원하는 동작인 경우를 위해서는, 얼마나 썼는지 알려 주는 별도의, 이름이 다른 함수가 있습니다:

```

proven_result_size_t r = proven_u8str_append_partial(&s, huge);
/* r.value is how many bytes were actually appended. Reading it is the point. */

```

잘못된 예 — 둘이 똑같이 동작한다고 가정하기:

```

(void)proven_u8str_append_partial(&s, huge); /* wrong: the count WAS the result */

```

여기까지가 일상 언어로 쓴 입문입니다. 이 장의 나머지는 다른 장들이 기대는 레퍼런스입니다: 라이브러리의 의도와 빌드 모델을 형식적으로 적은 것, 전역 계약, 소유권 매트릭스, 연산 동작 클래스, 읽는 순서, 플랫폼 지원, 그리고 부록들. 예전에는 매뉴얼의 첫 페이지인 별도 문서였는데, 첫 페이지를 차례만으로 두기 위해 이곳으로 옮겼습니다.

6. 의도와 설계 철학

`proven`은 컴팩트한 C23 시스템 기반 라이브러리입니다. 메모리 ownership, 에러 제어 흐름, 플랫폼 접근을 전역 상태 뒤에 숨기지 않으면서 실용적인 인프라를 원하는 C 프로그램을 위한 것입니다.

libc 대체품이 아닙니다. 할당자(allocator) 기반 메모리 도구, 바이트 뷰(view), 컨테이너, 문자열, 형식화(formatting), 파싱(scanning), 해싱(FNV, SipHash, CRC-32, SHA-256), hex/Base64 인코딩, OS 강도 난수, 파일시스템 헬퍼, 버퍼드 스트림, 시간 헬퍼, 메모리 매핑, 스택리스 코루틴 매크로, bounded 작업(job) system을 집중적으로 제공합니다.

핵심 설계 원칙:

- C23 우선: 빌드 드라이버는 `-std=c23`를 사용합니다.
- 명시적 에러: 실패 가능한 함수는 `proven_err_t` 또는 `proven_result*_t`를 반환합니다.
- 명시적 ownership: 소유(owned) 객체는 분명한 `destroy` 함수와 allocator 규칙을 가집니다.
- Failure atomicity(실패 원자성): `grow/realloc` 계열 API는 문서에 달리 적지 않는 한 할당 실패 시 기존 객체를 보존합니다.
- 포인터 provenance 규율: 원시 객체 접근은 `proven_byte_t`와 경계 있는 view를 사용합니다.
- PAL 격리: 호스트드(hosted) OS 서비스는 `platform/` 아래에 있고 공개 wrapper를 통해 호출됩니다.
- 핵심 컨테이너는 숨겨진 락을 추가하지 않습니다. 공유 변경(shared mutation)은 호출자의 동기화가 필요합니다.
- 빌드 시스템은 저장소에 체크인된 단일 `nob.c` 하나이며, 테스트는 평범한 C 실행 파일입니다.

7. 빌드와 include 모델

설치할 것이 없습니다

이 라이브러리에는 `configure`도, `CMake`도, 받아올 패키지도, 어딘가에 놓아야 할 공유 오브젝트도 없습니다. 그냥 C 소스입니다. 여러분의 파일과 나란히 놓고 프로그램에 함께 컴파일하면 됩니다.

이는 의도한 선택이고, 대가가 있습니다. 시스템 패키지를 얻지 못하고, 업데이트는 버전 제약을 올리는 것이 아니라 새 소스를 받아오는 일이 됩니다. 대신 얻는 것은 이렇습니다. 라이브러리가 지금 보고 있는 것과 다른 버전일 수 없고, 여러분이 모르지 않은 빌드 플래그를 주워줄 수 없으며, 배포판이 다른 옵션으로 빌드했다는 이유로 링크에 실패할 수 없습니다. hosted 시스템과 베어메탈 양쪽에서 돌아가야 하는 라이브러리에겐 "프로그램과 함께 컴파일한다"는 양쪽 모두에서 통하는 유일한 모델입니다.

중요한 디렉터리는 둘입니다. `src/proven/`은 이식 가능한 라이브러리 본체로, 어디에서도 OS를 호출하지 않습니다. `platform/`은 시스템 호출을 하는 얇은 계층이며, 새 타깃이 교체해야 하는 유일한 부분입니다. 프리스탠딩(freestanding) 빌드는 hosted 파일들을 그냥 빼고 빌드합니다. [freestanding 가이드](#)를 보십시오.

빌드 드라이버 `nob.c`는 빌드 시스템이 아니라 C 프로그램이며, 여기의 다른 모든 것과 똑같은 방식으로 컴파일합니다. 저장소에 함께 들어 있으므로 부트스트랩 단계도, 따로 설치할 버전도 없습니다.

C23 컴파일러가 필요합니다 — GCC 13+, Clang 16+, 또는 최근 MSVC. 드라이버가 `-std=c23`을 시험해 보고, 아직 과도기 철자를 쓰는 컴파일러에는 `-std=c2x`로 물려납니다.

hosted 테스트 스위트를 빌드하고 실행:

```
cc nob.c -o nob
./nob build
```

자주 쓰는 검증 명령:

```
./nob release
./nob strict
./nob strict-error
./nob asan
./nob ubsan
./nob tsan
./nob regression
./nob regression-asan
./nob regression-ubsan
./nob freestanding
./nob cross -build-root build-out/proven_c_lib
```

전체 hosted API가 필요하면 우산(umbrella) 헤더를 사용합니다:

```
#include "proven.h"
```

더 좁은 translation unit을 원하면 작은 include를 사용합니다:

```
#include "proven/heap.h"
#include "proven/u8str.h"
#include "proven/fmt.h"
```

직접 hosted 애플리케이션 빌드는 nob.c와 같은 소스 레이아웃을 따르면 됩니다:

```
cc -std=c23 -D_DEFAULT_SOURCE -D_POSIX_C_SOURCE=200809L \
  -Iinclude -Iplatform \
  app.c src/proven/*.c platform/proven_sys/*.c \
  -pthread -o app
```

8. 전역 계약(Global contracts)

8.1 Result 계약

result 값은 err == PROVEN_OK일 때만 사용할 수 있습니다.

올바른 예:

```
proven_result_mem_mut_t r = alloc.alloc_fn(alloc.ctx, 128, PROVEN_DEFAULT_ALIGNMENT);
if (proven_is_ok(r.err)) {
    /* only now does r.value mean anything */
    proven_mem_mut_t mem = r.value;
    (void)proven_mem_copy(mem.ptr, mem.size, proven_mem_view_from_u8(PROVEN_LIT("hi")));
    alloc.free_fn(alloc.ctx, mem.ptr);
}
```

잘못된 예 (보이는 그대로는 컴파일되지 않음 — 에러를 확인하기 전에 값을 읽음):

```
proven_result_mem_mut_t r = alloc.alloc_fn(alloc.ctx, 128, PROVEN_DEFAULT_ALIGNMENT);
use_bytes(r.value.ptr, r.value.size); /* wrong: r.err was not checked */
```

8.2 공개 struct 계약

많은 공개 struct가 C 사용성을 위해 레이아웃을 노출합니다. 그렇다고 임의 필드 변경이 지원된다는 뜻은 아닙니다. 헤더가 명시적으로 허용하지 않는 한, 내부 필드를 직접 변경하는 것은 호출자 오용으로 취급하세요.

잘못된 예:

```
arr.len = 999; /* wrong: breaks array invariants */
str.internal.cap=0; /* wrong: breaks string invariants */
```

불변식(invariant)을 유지하는 함수와 매크로를 사용하세요.

8.3 Borrowed view 계약

view는 메모리를 소유하지 않습니다. proven_u8str_view_t, proven_u16str_view_t, proven_mem_view_t, proven_mem_mut_t는 참조된 저장소가 살아 있고 이동되지 않은 동안에만 유효합니다.

잘못된 예:

```
proven_u8str_view_t v = proven_u8str_as_view(&s);
proven_u8str_append_grow(alloc, &s, PROVEN_LIT("more"));
use_view(v); /* wrong: growth may reallocate s */
```

8.4 Allocator 계약

proven_allocator_t는 세 함수 포인터가 모두 있을 때만 유효합니다. Reallocation은 failure-atomic이어야 합니다: 실패하면 기존 할당이 그대로 유효해야 합니다.

```
proven_allocator_t heap = proven_heap_allocator();
if (!proven_alloc_is_valid(heap)) {
    /* no usable allocator here: e.g. the freestanding heap stub */
    proven_panic("no heap allocator on this target");
}
```

8.5 PAL 경계 계약

src/proven/ 아래 코드는 OS 헤더에 직접 의존하면 안 됩니다. hosted 서비스는 platform/proven_sys_*.ch와 proven_fs_*, proven_sysio_*, proven_time_*, proven_mmap_* 같은 공개 wrapper를 통해야 합니다.

애플리케이션 코드는 공개 API를 우선하세요. 직접 PAL 호출은 포팅과 플랫폼 통합용입니다.

9. 소유권과 파괴 매트릭스

이 라이브러리에는 두 종류의 객체가 있고, 그 차이가 여러분이 무엇을 해야 하는지를 결정합니다.

Owning 객체는 자신이 할당한 저장소를 쥐고 있으며, 반드시 destroy해야 합니다. 바로 아래 표가 그것입니다.

Caller-owned state 객체(호출자 소유 상태)는 아무것도 할당하지 않습니다. 여러분이 — 보통 스택에 — 선언해 생성자에 넘기는 임시 작업용(scratch) struct이고, 생성자는 그 안을 가리키는 작은 값 핸들을 돌려줍니다. **destroy 함수가 없습니다.** 해제할 것이 없기 때문입니다. 대신 owning 객체에는 없는 규칙 하나가 있고, 그게 바로 발목을 잡습니다:

핸들이 만들어진 뒤에는 **caller-owned state 객체를 복사하거나 이동해서는 안 됩니다.** 핸들은 그 struct 내부를 가리키는 포인터를 쥐고 있습니다. struct를 복사하면 핸들은 여전히 원본을 가리키고, 원본이 스코프를 벗어나면 핸들은 죽은 메모리를 가리킵니다.

이들은 § 4.2에 정리되어 있습니다.

9.1 Owning 객체 — 반드시 destroy해야 함

객체	저장소 소유	allocator 보관	Destroy 함수	비고
proven_arena_t	아니오 — 호출자가 backing 슬라이스(slice) 소유	아니오	proven_arena_destroy(&arena) (no-op)	호출자 메모리 위의 bump pointer: alloc은 오프셋을 전진시키고, free는 no-op, reset은 빈 상태로 되감음. backing 블록은 호출자가 소유·해제.
proven_pool_t	예 — 아이템 + recycle bin	예 (base_alloc)	proven_pool_destroy(&pool)	고정 아이템 크기. allocator 트레이를 통해 (proven_pool_as_allocator) 사용합니다: 트레이의 free_fn은 슬롯을 해제하는 대신 재사용을 위해 recycle bin으로 되돌립니다. proven_pool_free는 없습니다 — 다른 allocator와 마찬가지로 해제는 트레이를 거칩니다.
proven_buf_t	예	아니오	proven_buf_destroy(&buf)	호출자가 일치하는 allocator를 넘겨야 함.

proven_u8str_t	예	아니오	proven_u8str_destroy(&str) 또는 PROVEN_U8STR_DESTROY(&str)	유지할 때 항상 NUL 종단.
proven_u16str_t	예	아니오	proven_u16str_destroy(&str) 또는 PROVEN_U16STR_DESTROY(&str)	API 길이는 바이트로 추적; API 길이는 proven_u16 단위.
proven_array_t	예	예	proven_array_destroy(&arr) 또는 PROVEN_ARRAY_DESTROY(&arr)	원소를 가리키는 포인터는 성장으로 무효화될 수 있음.
proven_ring_t	예	예	proven_ring_destroy(&ring) 또는 PROVEN_RING_DESTROY(&ring)	고정 용량; 성장 없음.
proven_map_t	예	예	proven_map_destroy(&map) 또는 PROVEN_MAP_DESTROY(&map)	Borrowed U8 키는 복사되지 않음.
proven_fs_list()가 반환한 proven_array_t	예	예 + owned 엔트리 이름	proven_fs_list_destroy(&list)	예비한 array destroy를 쓰지 말 것; 엔트리 이름 정리가 필요.
proven_mmap_t	OS 매핑	OS 핸들 상태	proven_mmap_destroy(&map)	매핑을 가리키는 view는 매핑과 함께 죽음.
proven_job_sys_t *	예	내부	proven_job_system_close(&sys) 후 proven_job_system_destroy(&sys)	destroy는 producer와 경쟁(race)하면 안 됨.

9.2 Caller-owned state — destroy 없음, 복사 금지

이들은 아무것도 할당하지 않고 아무것도 해제하지 않습니다. 하나 선언해 그 주소를 생성자에 넘기고, 돌려받은 작은 핸들을 사용합니다. struct는 핸들보다 **오래 살아야** 하며, 핸들이 살아 있는 동안 **복사·이동**해서는 안 됩니다.

State 객체	생성 함수	뒷받침하는 핸들	비고
proven_sha256_t	proven_sha256_init	(직접 사용)	해싱 컨텍스트. 시작 *전*에는 복사해도 안전하고, 스트림 도중 복사는 해시를 fork하려는 의도가 아닌 한 무의미.
proven_xoshiro256ss_t	proven_xoshiro256ss_seed	proven_rng_t	복사하면 수열이 복제됩니다 — 재현(replay)에는 의도적이고 유용하지만, 그 외에는 버그.
proven_chacha_rng_t	proven_chacha_rng_seed / _seed_from_entropy	proven_rng_t	복사하면 키스트림이 복제됩니다: "독립적인" 두 토큰이 같은 토큰이 됨.
proven_writer_buf_t	proven_writer_from_buffer	proven_writer_t	복사 금지.
proven_writer_u8str_t	proven_writer_from_u8str	proven_writer_t	복사 금지. 문자열과 allocator도 그보다 오래 살아야 함.

proven_writer_buffered_t	proven_writer_buffered	proven_writer_t	복사 금지. 그것이나 그 버퍼가 죽기 전에 반드시 proven_writer_flush 해야 함.
proven_reader_view_t	proven_reader_from_view	proven_reader_t	복사 금지.
proven_reader_buffered_t	proven_reader_buffered	proven_reader_t	복사 금지. proven_reader_read_line이 반환한 view는 이 버퍼 안을 가리킴.
proven_sysio_std_t	proven_sysio_stdout_writer / _stderr_writer / _stdin_reader	proven_writer_t / proven_reader_t	복사 금지. 쓰기 스트림 (writer)가 가리키는 표준 핸들을 보관.
proven_sysio_out_t	proven_sysio_stdout_buffered / _file_buffered	proven_writer_t	복사 금지. 반드시 flush 필요.
proven_sysio_lines_t	proven_sysio_lines_open / _stdin_lines	(proven_sysio_read_line으로 사용)	유일한 예외: proven_sysio_read_line이 매 호출마다 재바인딩하므로, 이것은 이동해도 됨.
proven_sysio_scanner_t	proven_sysio_scanner_init	(직접 사용)	반대 방향의 예외: 이것은 버퍼를 소유 하므로 proven_sysio_scanner_deinit을 호출해야 함.

잘못된 예 — 이 복사는 무해해 보이지만 use-after-free입니다:

```
proven_sysio_out_t out;
proven_writer_t w = proven_sysio_stdout_buffered(&out, buf);

proven_sysio_out_t saved = out; /* wrong: `w` still points into `out`, not `saved` */
use_elsewhere(&saved);          /* and if `out` goes out of scope, `w` is dangling */
```

잘못된 예 — 팩토리 함수에서 state를 값으로 반환해도 같은 일이 벌어집니다:

```
proven_sysio_out_t make_logger(void) {
    proven_sysio_out_t out;
    proven_writer_t w = proven_sysio_stdout_buffered(&out, buf);
    (void)w;
    return out; /* wrong: any writer made from `out` addresses this dead frame */
}
```

올바른 예 — state는 제자리에 두고, 핸들이 이동합니다:

```
proven_byte_t buf[512];
proven_sysio_out_t out; /* lives as long as `w` */
proven_writer_t w = proven_sysio_stdout_buffered(&out,
    (proven_mem_mut_t){ .ptr = buf, .size = sizeof buf });

(void)proven_fprintln(w, "one syscall, not {} ", PROVEN_ARG(100));
(void)proven_writer_flush(w); /* or the bytes never happened */
```

10. 연산 동작 클래스

한 연산, 세 가지 정직한 대답

"이 텍스트를 저 문자열에 덧붙여라"는 텍스트가 들어가지 않을 때 옹호할 만한 동작이 셋 있고, 대부분의 라이브러리는 그중 하나를 골라 그 선택을 숨깁니다. 이 라이브러리는 셋을 모두 드러내고, 각각에 다른 이름을 주며, 그 차이를 시그니처에 둡니다 — 어느 것을 원하는지는 그 텍스트가 *무엇인지*에 달려 있고, 그것은 호출자만 알기 때문입니다.

파일 경로에 덧붙이는 경우를 생각해 보십시오.

- 경로가 들어가지 않는데 **자르는 것은 치명적입니다**. `documents/report.pdf`가 `documents/rep`가 되는데, 이는 다른 파일이고 실제로 존재할 수도 있는 파일입니다. 이 연산은 실패하고 아무것도 바꾸지 않아야 합니다.
- 이제 로그 한 줄에 덧붙이는 경우를 생각해 보십시오. 들어가지 않으면 **자르는 것이 괜찮습니다** — 메시지 전체를 잃느니 대부분이라도 남는 편이 낫습니다 — 얼마나 썼는지 알려주기만 한다면요.
- 그리고 쌓아 올리는 중인 버퍼에 덧붙이는 경우, 그냥 **자라기를** 원할 것입니다.

아래 세 클래스가 그 세 대답입니다. 어느 것을 받을지는 함수 이름과 `allocator` 매개변수의 유무가 결정하며, 플래그나 전역 변수가 결정하는 일은 결코 없습니다.

- `proven_u8str_append(str, data)` — `allocator`가 없으므로 자랄 수 없습니다: **거부합니다**.
- `proven_u8str_append_partial(str, data)` — 개수를 반환합니다: **자르고 알려줍니다**.
- `proven_u8str_append_grow(alloc, str, data)` — `allocator`를 받습니다: **자랍니다**.

라이브러리 전반의 기본값은 첫 번째이며, [챕터 0 §5](#)가 그 이유를 설명합니다. 잘린 경로는 엉뚱한 파일을 열고, 잘린 명령은 엉뚱한 명령을 실행하며, 잘린 숫자는 다른 숫자입니다.

잘못된 예 — truncating 형태가 돌려준 개수를 무시:

```
(void)proven_u8str_append_partial(&s, huge); /* wrong: the count WAS the answer */
```

여러 API가 의도적으로 세 가지 동작 클래스를 드러냅니다:

클래스	예	용량 부족 시 동작
Atomic 고정 용량	<code>proven_u8str_append</code> , <code>proven_u16str_append</code> , <code>proven_u8str_append_fmt</code>	에러를 반환하고 기존 객체를 그대로 둡니다.
Best-effort truncating	<code>proven_u8str_append_partial</code> , <code>proven_u16str_append_partial</code> , <code>proven_u8str_append_fmt_trunc</code>	들어가는 만큼 쓰고, 유효한 객체를 보존하며, 얼마를 썼는지 보고.
Atomic growable	<code>proven_u8str_append_grow</code> , <code>proven_u16str_append_grow</code> , <code>proven_u8str_append_fmt_grow</code>	<code>allocator</code> 로 성장; 할당 실패 시 기존 객체를 그대로 둡니다.
거부하되 절대 자르지 않음	<code>proven_hex_encode</code> , <code>proven_base64_encode</code> , <code>proven_base64_decode</code> , <code>proven_reader_read_line</code>	아무것도 쓰지 않고 <code>PROVEN_ERR_OUT_OF_BOUNDS</code> 를 반환. 반쯤 인코딩된 문자열이나 짧아진 줄은 옳아 보이는 오답이므로, 이들은 그런 것을 만들지 않음. 버퍼 크기는 눈대중이 아니라 모듈의 크기 함수(<code>proven_base64_encoded_size</code> 등)로 정할 것.

클래스를 의도적으로 고르세요. truncating 함수를 all-or-nothing 함수처럼 다루지 마세요.

잘못된 예 — truncating과 atomic 형태가 똑같이 동작한다고 가정:

```
/* `_partial` wrote what fit and told you so; the error you did not read is the
   difference between "all of it" and "some of it". */
(void)proven_u8str_append_partial(&s, huge); /* wrong: the result was the point */
```

11. 매뉴얼 챕터

상세 레퍼런스는 읽기 쉽고 소스에 근거를 두도록 챕터별로 나눕니다.

입문용 C 책을 막 뒀다면 따라 하며 익히기부터 보세요. 짧은 프로그램 아홉 개로 개념을 하나씩만 더해 갑니다. 여섯 번째에서 이 장의 인사말 프로그램을 한 줄씩 읽고, 마지막 셋은 아레나·컨테이너·파일로 나아갑니다.

그 밖의 경우 이 장부터 읽으십시오. 아무것도 전제하지 않는 유일한 장입니다: 이 라이브러리가 왜 존재하는지를 이 라이브러리가 답하려는 C의 버그들로부터 논증하고, hello world 프로그램, 빌드 방법, 나머지 장들이 당연히 여기는 다섯 가지 계약, 그리고 용어집과 libc 대응표를 담고 있습니다.

읽는 순서

챕터들은 부(Part)로 묶여 있고, 각 부는 그 앞의 부들만 필요로 하도록 배열되어 있습니다. 이 순서는 헤더의 존 그래프와 같지 않으며, 임의로 정한 것도 아닙니다. 문자열은 allocator가 필요하고, 컨테이너도 allocator가 필요하며, hosted 서비스는 문자열이 필요합니다. 아래 순서는 자료 자체가 요구하는 순서입니다.

챕터 *번호*는 안정적인 식별자이지 읽는 순서가 아닙니다. 그중 둘은 일부러 순서를 벗어나 있습니다. alias 인덱스는 찾아보는 부록이고, 8장은 3장이 주제를 소개한 뒤에 읽는 레퍼런스입니다.

부	읽을 것	선행 조건	그러면 할 수 있는 것
I — 여기서부터	따라 하며 익히기 → 0	입문용 C 책 한 권	라이브러리를 붙여 빌드하고, 아래 무엇이든 읽기
II — 모든 프로그램이 쓰는 어휘	1 → 2 → 3	0장	에러를 값으로 다루고, 메모리를 의도적으로 소유하고, 텍스트를 안전하게 담기
III — 자료구조	4	II부	array, map, list, ring, 정렬, 검색, 해싱, 인코딩
IV — 텍스트 입출력	8	3장 § 3–§ 4	무엇이든 형식화하고 파싱하며, 포매터에 내 타입 가르치기
V — 운영체제와 대화하기	5	II부	파일, 디렉터리, 스트림, 표준 I/O, 시간, 난수, 매핑
VI — 더 나아가기	6 → freestanding	II–V부	코루틴, job, 스레드 안전성, 베어메탈, 크로스 빌드
부록	A, B, C, D	—	찾아보기

챕터 목록

• [따라 하며 익히기](#): 짧은 프로그램 아홉 개, 한 번에 개념 하나 — Part I, 선택 입구

1. [여기서부터 시작](#): 왜 존재하는가, hello world, 다섯 계약, 용어집, libc 대응표 — I부
2. [Foundation](#): 타입, 에러, 메모리 view, 정렬, 버전, 패닉(panic) — II부
3. [Allocation](#): 힙(heap), 아레나(arena), 풀(pool), byte buffer, 그리고 [allocator 트레잇](#) — II부
4. [문자열과 텍스트](#): U8, U16, 그리고 형식화와 파싱 입문 — II부; 텍스트 자료의 튜토리얼 절반
5. [컨테이너와 알고리즘](#): array, list, ring, map, 정렬/검색, 해싱, 인코딩 — III부
6. [Hosted 서비스](#): 파일시스템, 트리 순회, 스트림, sysio, 환경변수, 난수, mmap, 시간 — V부
7. [실행과 플랫폼](#): 코루틴, job, 스레드 안전성, alias, PAL, 크로스 빌드 — VI부
8. [부록 A — Alias 인덱스](#): [alias_xcv.h](#)의 모든 철자 — 참조 전용; 읽는 자료가 아님

9. 형식화와 파싱: 전체 `fmt.h`와 `scan.h` 레퍼런스 — IV부; 텍스트 자료의 레퍼런스 절반

3장과 8장은 둘 다 포맷터와 스캐너를 다루며, 이 분담은 의도적입니다. 3장은 문자열과 나란히 이들을 소개하며, 일상적인 경우와 생산성을 내기에 충분한 만큼을 담습니다. 8장은 완전한 레퍼런스입니다. 전체 형식 문법, 모든 인자 생성자, 스캐너의 에러 코드와 복구 규칙, 그리고 포맷터에 내 타입을 가르치는 방법을 담습니다. 3장을 먼저 읽고, 어떤 지정자나 실패의 정확한 동작이 필요할 때 8장을 펴십시오.

12. 플랫폼 지원과 검증

주요 검증 hosted 대상:

- C23 모드의 GCC 또는 Clang을 쓰는 Linux x86_64.

해당 toolchain이 설치되어 있으면 compile-only 크로스 커버리지가 존재합니다:

- Linux AArch64.
- Linux ARM hard-float.
- i686-linux-gnu-gcc 또는 gcc -m32 multilib을 통한 Linux i686.
- MinGW/WinAPI 경로를 통한 Windows x86_64와 i686.
- ARM Cortex-M freestanding.
- RISC-V ELF freestanding.

크로스 매트릭스는 컴파일, 공개 헤더 가시성, 대상 ABI 가정을 확인합니다. 대상 플랫폼에서의 런타임 검증을 대체하지는 않습니다.

Freestanding 모드는 OS 기반 서비스를 제거한 축소된 서브셋을 빌드합니다. 자세한 내용은 [manual-freestanding-ko.md](#)와 챕터 6을 참조하세요.

13. 부록 B: 용어집

이 매뉴얼이 평범한 단어인 양 쓰는 용어들입니다. 평범한 C 단어가 아니며, 하나하나가 무게를 지고 있습니다.

본문이 지키는 규칙이 셋 있습니다. 덕분에 여기 나오는 어떤 말도 짐작할 필요가 없습니다.

- **저희끼리만 쓰는 은어를 만들지 않습니다.** 어떤 생각에 널리 쓰이는 말이 이미 있으면 그 말을 씁니다. 이 라이브러리가 새로 만든 것만 여기에서 정의합니다.
- **영어 용어는 각 장에서 처음 나올 때 괄호로 원문을 함께 적습니다** — 뷰(view)처럼. 영어로 개념을 익힌 분은 원래 이름을 찾을 수 있고, 그렇지 않은 분은 번역되지 않은 낱말을 마주치지 않습니다.
- **줄임말은 처음 나올 때 풀어 씁니다.** 예를 들어 PAL은 플랫폼 추상화 계층(Platform Abstraction Layer, PAL)으로 적습니다. 모든 줄임말은 아래 표에도 들어 있습니다.

용어	여기서의 뜻
owned	파괴할 책임이 여러분에게 있습니다. <code>_create</code> 에서 와서 <code>_destroy</code> 로 가며, 정확히 한 번입니다.
borrowed	다른 누군가가 소유한 메모리를 가리킵니다. 절대 파괴하지 않습니다. 소유자가 살아 있는 동안에만 유효합니다.
view	borrowed 포인터 + 길이 쌍, 예를 들어 <code>proven_u8str_view_t</code> . 복사 가능하고, 소유하지 않으며, 할당하지 않습니다.
allocator	네 가지를 담은 값: 컨텍스트 포인터 하나와 함수 포인터 셋(<code>alloc</code> , <code>realloc</code> , <code>free</code>). 할당할 수 있는 모든 것에 값으로 전달됩니다.
arena	하나의 블록 안에서 포인터를 밀어 가며 메모리를 나눠 주는 allocator. 개별 <code>free</code> 는 아무 일도 하지 않

	고, arena 전체를 한 번에 reset하거나 파괴합니다. 빠르며, "수명이 같은 작은 것 여럿"에 딱 맞습니다.
pool	고정 크기 하나짜리 객체를 여럿 다루는 allocator로, free 리스트를 두어 해제가 실제로 슬롯을 재활용합니다.
trait(트레잇)	인터페이스로 쓰이는 함수 포인터 struct — 가상 함수 테이블에 대한 C의 답입니다. proven_allocator_t, proven_writer_t, proven_rng_t가 트레잇입니다. C 키워드가 아니라 빌려 온 용어입니다.
PAL	플랫폼 추상화 계층(platform abstraction layer): 실제 시스템 호출을 하는 platform/ 아래의 코드. OS에 의존하는 유일한 부분입니다.
freestanding	운영체제도 libc도 없는 빌드 — 베어메탈입니다. PROVEN_FREESTANDING으로 선택합니다.
failure atomicity(실패 원자성)	연산이 실패하면 아무것도 바꾸지 않습니다. 실패한 grow는 기존 데이터를 온전하고 유효하게 남겨둡니다.
provenance	포인터가 어느 할당에서 왔는가. C의 최적화기는 서로 다른 할당에서 온 포인터가 절대 겹치지 않는다고 가정하며, 이를 어기는 것은 단지 놀라운 일이 아니라 미정의 동작입니다. 챕터 6에서 다룹니다.
UB(undefined behaviour, 미정의 동작)	"예측할 수 없는 출력"이 아닙니다 — 표준이 아무 요구도 하지 않으며, 최적화기는 그런 일이 결코 일어나지 않는다고 가정해도 됩니다. UB가 여러분의 if를 지워 버릴 수 있는 이유입니다.
[[nodiscard]]	C23 어트리뷰트. 반환값을 버리면 컴파일러가 에러를 냅니다. 에러를 놓쳐서는 안 되는 모든 함수에 붙어 있습니다.
fixed-capacity(고정 용량)	성장하지 않습니다. 가득 차면 실패합니다. allocator를 받지 않습니다.
growable(성장 가능)	가득 차면 재할당합니다. allocator를 받습니다. 이름에 항상 _grow가 붙습니다.
CSPRNG	암호학적으로 안전한 의사난수 생성기: 앞선 출력을 본 뒤에도 공격자가 예측할 수 없는 출력.
intrusive(침입형)	리스트의 링크가 따로 할당된 노드가 아니라 여러분의 struct 안에 들어 있습니다. 원소마다 할당이 발생하지 않습니다.
code unit(코드 유닛)	인코딩의 원소 하나: UTF-8에서는 한 바이트, UTF-16에서는 16비트 값. 문자가 아닙니다 — 한 문자가 여러 개를 차지할 수 있습니다.
code point(코드 포인트)	유니코드에서 한 글자에 매겨진 번호입니다. 한 코드 포인트는 UTF-8에서 1~4바이트, UTF-16에서 코드 유닛 1~2개를 차지합니다. 둘 중 무엇을 세어도 글자 수가 되지 않는 이유입니다.

API(Application Programming Interface, 응용 프로그래밍 인터페이스)	라이브러리가 다른 프로그램이 부르라고 내놓은 함수와 타입의 집합입니다. 이 라이브러리에서는 <code>include/proven/</code> 에 선언된 모든 것입니다.
hosted(호스팅)	밑에 운영체제와 C 표준 라이브러리가 있는 빌드입니다. <i>*freestanding*</i> 의 반대입니다.
heap(힙)	<code>malloc</code> 이 메모리를 나눠 주는 범용 메모리 영역입니다. <code>proven_heap_allocator()</code> 가 이것을 쓰는 allocator입니다.
slice(슬라이스)	쓸 수 있는 borrowed 포인터 + 길이 쌍입니다. 예를 들어 <code>proven_mem_mut_t</code> . 같은 생각의 읽기 전용 형태가 <i>*view*</i> 입니다.
result(결과 구조체)	여러 코드와 값을 함께 담은 작은 struct입니다. 예를 들어 <code>proven_result_u8str_t</code> . 옆에 있는 여러를 확인하기 전까지 값은 아무 뜻도 없습니다.
panic(패닉)	실패를 돌려줄 호출자가 없을 때 택하는 의도적인 정지입니다. <code>proven_set_panic_handler()</code> 로 그때 할 일을 고르며, 기본값은 프로그램을 멈춥니다.
reserve(예약)	컨테이너의 용량을 지금 올려서 이후의 증가가 재할당하지 않게 하는 일입니다. 힙에서는 복사를, arena에서는 죽은 저장 공간을 아깁니다.
dangling(허공을 가리키는)	이미 해제되었거나 옮겨진 메모리를 가리키는 포인터입니다. 그것을 쓰는 것은 미정의 동작이며, 여기서 흔한 원인은 재할당할 수 있는 호출을 사이에 두고 포인터를 붙들고 있는 것입니다.
use-after-free(해제 후 사용)	dangling 포인터로 읽거나 쓰는 일입니다. 아래의 새니타이저가 잡아냅니다.
sanitizer(새니타이저)	실행 시 검사를 덧붙이는 컴파일러 모드입니다. ASan (AddressSanitizer)은 메모리 오류를, UBSan (UndefinedBehaviorSanitizer)은 미정의 동작을, TSan (ThreadSanitizer)은 데이터 경쟁을 찾습니다. <code>./nob asan, ./nob ubsan, ./nob tsan</code> .
partial write / short read(부분 쓰기 / 짧은 읽기)	요청한 것보다 적은 바이트를 옮긴 한 번의 쓰거나 읽기입니다. 예러가 아니라 정상이며, 짧은 읽기를 입력의 끝으로 착각하는 것이 파일 뒤끝을 읽는 고전적인 방법입니다.
EOF(End Of File, 파일의 끝)	더 읽을 입력이 없다는 뜻입니다. 0바이트 성공이 아니라 <code>PROVEN_ERR_EOF</code> 로 보고되므로 "아직 아무것도 안 왔다"와 혼동될 수 없습니다.
flush(비우기)	버퍼드 writer에 쌓인 바이트를 다음 단계로 밀어 보내는 일입니다. 이 라이브러리의 어떤 것도 종료 시에 대신 flush해 주지 않습니다.
back-pressure(배압)	소비자가 따라오지 못해 생산자를 늦추는 일입니다. <code>proven_writer_write_partial()</code> 이 그것을 알아채고 대응하게 해 주는 호출입니다.
durability(지속성)	정전을 겪고도 데이터가 남는다는 보장입니다. 운영 체제에 도달한 것만으로는 모자랍니다.

	proven_fs_sync()가 파일의 바이트를 저장 장치에 올리고, proven_fs_sync_dir()이 이름 바꾸기에 대해 같은 일을 합니다.
atomic rename(원자적 이름 바꾸기)	완성된 임시 파일을 원래 이름 위로 rename해 파일을 교체하는 방식입니다. 읽는 쪽은 옛 파일 전체나 새 파일 전체를 보며, 반쯤 쓰인 혼합물은 보지 않습니다.
advisory lock(권고 잠금)	마찬가지로 잠금을 요청하는 프로세스만 막는 잠금입니다(proven_fs_lock()). 접근 권한 통제가 아니라 협력하는 프로그램들 사이의 약속입니다.
hard link(하드 링크)	같은 파일에 대한 두 번째 이름입니다. 원본이라는 것이 없고, 마지막 이름이 지워질 때까지 데이터가 삽니다. 같은 파일 시스템 안에서만 됩니다.
symbolic link(심볼릭 링크)	경로를 담은 작은 파일입니다. 파일 시스템을 건너뛸 수 있고, 아무것도 가리키지 않을 수도 있습니다. 그 때는 따라가기가 실패합니다.
memory mapping(메모리 매핑)	파일 내용을 어떤 주소에 나타나게 해서 프로세서가 메모리처럼 읽게 하는 일입니다(mmap.h). SHARED 매핑의 쓰기는 파일로 돌아가고, PRIVATE 매핑은 *기록 시 복사*라서 쓰기가 이 프로세스에만 남습니다.
copy-on-write(기록 시 복사)	누군가 쓰기 전까지 메모리를 공유하다가, 쓰는 쪽에 게만 사적인 사본을 주는 방식입니다. PROVEN_MMAP_PRIVATE가 하는 일입니다.
cursor(커서)	스캐너가 읽고 있는 텍스트에서의 현재 위치입니다(proven_scan_t.cursor). 스캔은 이것을 앞으로 밀고, proven_scan_skip_*은 의도적으로 옮깁니다.
locale(로케일)	소수점이 .인지, 인지를 포함한, 지역 관습에 대한 시스템의 설정입니다. 이 라이브러리의 수 파싱은 로케일에서 자유롭습니다 . 어떤 기계에서도 심표는 소수점이 아닙니다.
entropy(엔트로피)	운영체제나 하드웨어에서 오는, 진짜로 예측할 수 없는 비트입니다. 생성기는 엔트로피로 씨앗을 받고, 시계나 계수기는 아무리 무작위처럼 보여도 엔트로피가 아닙니다.
seed(씨앗)	생성기의 시작값입니다. 같은 씨앗은 같은 수열을 다시 냅니다. 재현 가능한 시험에는 꼭 필요하고, 키에는 치명적입니다.
PRNG / CSPRNG	의사난수 생성기(Pseudo-Random Number Generator)는 씨앗에서 수열을 계산합니다. 암호학적으로 안전한 (Cryptographically Secure) 것은 앞선 출력을 본 공격자에게도 예측 불가능하다는 성질이 더 있습니다.
HashDoS(해시 서비스 거부 공격)	충돌하도록 고른 키를 해시 테이블에 먹여 O(1) 조회를 *O(n ²)*으로 만드는 공격입니다. proven_map_create()는 키가 있는 해시로 이를 막고,

	proven_map_create_trusted()는 내가 고른 키에 한해 그 방어를 포기합니다.
open addressing(개방 주소법)	map의 배치 방식입니다. 항목들이 평평한 버킷 배열 하나에 살고, 충돌하면 따로 할당된 노드 사슬을 따라가는 대신 다음 칸으로 옮겨 갑니다.
tombstone(묘비)	map에서 항목이 지워진 자리에 남기는 표시입니다. 조희가 그 자리를 지나 계속 탐색하게 해 줍니다. 묘비도 적재물에 셈되므로, 삭제가 잦으면 여전히 리해시가 일어납니다.
rehash(리해시)	버킷 배열을 새 크기로 다시 만드는 일입니다. 앞서 get_mut이 돌려준 모든 포인터를 무효로 만들며, 그래서 포인터가 아니라 키를 붙들어야 합니다.
checksum(검사합)	우연한 손상을 알아내는 짧은 값입니다 — proven_crc32(). 사고는 잡아내지만 변조는 결코 잡지 못합니다.
digest / hash(다이제스트 / 해시)	데이터에서 계산한 고정 크기 값입니다. SHA-256은 변조를 드러내는 암호학적 다이제스트이고, FNV-1a와 SipHash-2-4는 테이블용 해시이며 이중 키를 쓰는 것은 SipHash뿐입니다.
hex / Base64 / Base64URL	바이트를 텍스트로 적는 방법들입니다. hex는 바이트당 두 글자, Base64는 3바이트를 + / =를 쓰는 네 글자로 묶고, Base64URL은 - _를 쓰고 패딩이 없어 URL이나 파일 이름에 안전합니다.
padding(패딩)	출력 길이를 4의 배수로 맞추려고 Base64가 붙이는 = 문자입니다. Base64URL은 붙이지 않습니다.
BMP(Basic Multilingual Plane, 기본 다국어 평면)	유니코드의 첫 65,536개 코드 포인트입니다. 그 밖의 글자 — 이모지, 드문 한중일 한자 다수 — 는 UTF-16 코드 유닛 두 개가 필요합니다.
NUL terminator(NUL 종결자)	C 문자열의 끝을 표시하는 0 바이트입니다. *view*에는 이것이 없으며, 그래서 대신 길이를 지니고 다닙니다.
shortest round trip(최단 왕복 표기)	다시 읽었을 때 정확히 같은 부동소수점 값이 되는 가장 짧은 자릿수로 찍는 것입니다. PROVEN_FLOAT_FORMAT_MODE_SHORTEST가 이것을 요청하며, 직렬화기가 원하는 성질입니다.
dispatch macro(디스패치 매크로)	인자의 타입에서 함수를 골라 주는, C11 _Generic 위에 만든 매크로입니다 — PROVEN_ARG(x)와 PROVEN_SCAN_ARG(&x). 이름 있는 생성자들 중에서 고를 뿐, 그 자신이 생성자는 아닙니다.
identity constructor(항등 생성자)	proven_arg_identity() / proven_scan_arg_identity(). 이미 만들어진 인자를 받아 그대로 통과시켜, 매크로로 굴러가는 코드가 그것을 받아들일 수 있게 합니다.
scratch(임시 작업용) allocator	결과를 소유하는 allocator와 따로, 임시 작업 메모리만을 위해 넘기는 allocator입니다. 예를 들어 proven_map_set_with_scratch()의 scratch 매개변수.

stackless coroutine(스택 없는 코루틴)	자기 스택 없이 멈췄다 이어 갈 수 있는 함수입니다. 상태가 여러분이 들고 있는 struct 안에 삽니다. coro.h는 이것을 switch로 구현하므로 스레드도 할당도 들지 않습니다.
bounded queue(상한 있는 큐)	용량이 정해져 있어 가득 차면 늘어나는 대신 거절하는 큐입니다. job 시스템이 이것을 쓰므로, 일꾼보다 빠른 생산자는 메모리를 소진하는 대신 거절을 듣습니다.
monotonic clock(단조 시계)	앞으로만 가는 시계로, 무언가에 걸린 시간을 재는데 씁니다. 시스템 시각이 교정될 때 뛸 수 있는 벽시계(wall clock)와 다릅니다. 벽시계로 소요 시간을 재면 음수가 나오는 일이 그래서 생깁니다.

14. 부록 C: 공개 헤더 맵

찾는 방법

공개 헤더는 35개이고 우산 헤더가 하나 있습니다. `#include "proven.h"`는 전부를 끌어오며, 이 매뉴얼의 예제들이 하는 방식입니다. 개별 헤더를 include하는 것은 컴파일 시간이 신경 쓰이거나 의존 관계를 파일에 드러내고 싶을 때 씁니다.

이 표가 파일 이름만으로는 알 수 없는 두 가지를 알려줍니다.

- **어느 챕터가 그것을 문서화하는가.** 모든 헤더에는 그것을 설명하는 챕터가 정확히 하나 있고, 빌드는 모든 공개 함수가 `manual/` 어딘가에 이름을 올리도록 강제합니다 — 그러니 어떤 심벌이 기대한 챕터에 없더라도 매뉴얼 어딘가에는 있고, 이 맵이 그 위치를 알려줍니다.
- **freestanding 빌드에서 살아남는가.** 챕터 5에 배정된 헤더들이 hosted 쪽입니다. 파일시스템, 표준 스트림, 시계, 가상 메모리 또는 스레드가 필요합니다. 그 밖의 모든 것은 운영체제 없이 컴파일됩니다. 모듈별 권위 있는 표는 [freestanding 가이드](#)에 있습니다.

헤더	주요 용도	챕터
<code>proven.h</code>	우산 include	이 파일
<code>types.h</code>	고정 폭 별칭, 검사된 산술, 에리 enum	챕터 1
<code>error.h</code>	에리 술어(predicate) 헬퍼	챕터 1
<code>memory.h</code>	바이트 view, 슬라이싱, 범위 검사, memcmp	챕터 1
<code>align.h</code>	정렬 상수와 align-up 헬퍼	챕터 1
<code>version.h</code>	버전 매크로	챕터 1
<code>panic.h</code>	등록 가능한 panic 핸들러	챕터 1
<code>config.h</code>	컴파일 타임 기능 토글 (PROVEN_FREESTANDING, PROVEN_FMT_NO_FLOAT, PROVEN_NO_U16STR 등)	챕터 1, 6
<code>allocator.h</code>	Allocator 트레잇	챕터 2
<code>heap.h</code>	PAL 기반 heap allocator	챕터 2
<code>arena.h</code>	Bump allocator	챕터 2
<code>pool.h</code>	고정 크기 recycler allocator	챕터 2
<code>buffer.h</code>	고정 용량 byte buffer	챕터 2
<code>u8str.h</code>	Owned U8 문자열과 빌려 쓰는(borrowed) U8 view	챕터 3
<code>u16str.h</code>	Owned U16 문자열과 borrowed U16 view	챕터 3
<code>fmt.h</code>	구조적 formatter와 format 인자	챕터 3

scan.h	구조적 scanner와 타입 있는 scan 목적지	챕터 3
float_parse.h	로케일 없는 십진수 → double/float 파서 (proven_strtod, proven_parse_double_ascii)	챕터 8
float_format.h	double/float → 십진수 formatter (고정 %f/%e, shortest)	챕터 8
float_config.h	Float 엔진 튜닝 (PROVEN_FLOAT_BIGINT_LIMBS, 정밀도 상한)	챕터 6, 8
array.h	제네릭 growable 벡터	챕터 4
list.h	침입형(intrusive) 이중 연결 리스트	챕터 4
ring.h	고정 용량 FIFO ring	챕터 4
map.h	Open-addressing map	챕터 4
algorithm.h	Array 정렬·검색 헬퍼	챕터 4
hash.h	FNV-1a, SipHash-2-4, CRC-32, SHA-256, 용도별	챕터 4
encode.h	Hex와 Base64 (표준 + URL-safe), 바이트↔텍스트	챕터 4
fs.h	파일, 디렉터리, 메타데이터, 링크, 락, read-all, 트리 순회	챕터 5
stream.h	버퍼드 writer·읽기 스트림(reader)와 line reader — 그리고 sysio.h를 통해 표준 스트림 (hosted 전용)	챕터 5
sysio.h	표준 스트림을 writer/reader로, stdin 줄 입력, 버퍼드 출력, 출력, 파싱, 환경 변수 접근	챕터 5
random.h	용도별 난수: xoshiro256** (재현 가능), ChaCha20 (암호학적), OS CSPRNG, 무편향 range/shuffle 헬퍼. 생성기는 freestanding에서 동작하고 OS 소스만 hosted.	챕터 5
mmap.h	메모리 매핑 파일 영역	챕터 5
time.h	타임스탬프, datetime, sleep, datetime 형식화	챕터 5
coro.h	스택리스 코루틴 매크로	챕터 6
job.h	Bounded worker-thread job system	챕터 6
alias_xcv.h	선택적 짧은 alias 계층과 생성된 철자 맵	챕터 6, 7

15. 부록 D: libc 대응표

이미 C를 쓰고 있다면 이것이 가장 빠른 입구입니다. 각 행은 그 맛바꿈을 설명하는 챕터로 연결됩니다.

원래 쓰던 것	대신 쓸 것	무엇이 다른가
malloc / free	proven_heap_allocator() + _create / _destroy	allocator가 파라미터라서, 같은 코드가 arena나 pool 위에서도 돌아갑니다. 챕터 2
strcpy, strcat	proven_u8str_append, _append_grow	크기를 알고 있으므로, 오버런을 저지르는 대신 거부합니다. 챕터 3
strlen	view.size	길이가 이미 거기 있습니다. 아무것도 스캔하지 않습니다. 챕터 3
동등 비교용 strcmp	proven_u8str_view_eq	중간에 NUL이 박힌 텍스트에서도 동작하고, 끝을 넘어가지 않습니다. 챕터 3
strstr	proven_u8str_view_find	인덱스 또는 PROVEN_INDEX_NOT_FOUND를 반환하며, 탐색이 단순 무식하지 않습니다. 챕터 3

strtok	(아직 대응물 없음)	strtok은 입력을 변경하고 중첩해서 쓸 수 없습니다. view 기반 분할기가 docs/RFC-0002에 설계되어 있습니다.
printf	proven_println("{", PROVEN_ARG(x))	타입이 포맷 문자열이 아니라 인자에서 옵니다. 챕터 8
sprintf	proven_u8str_append_fmt	크기가 정해진 목적지에 쓰고, 그것을 넘기기를 거부합니다. 챕터 8
sscanf	proven_scan_*, proven_scan_fmt	어느 필드가 실패했고 커서가 어디서 멈췄는지 알려 줍니다. 챕터 8
strtod	proven_parse_f64_ascii	올바르게 반올림하고, 로케일에 영향받지 않으며, errno를 쓰지 않습니다. 챕터 8
fopen / fread / fclose	proven_fs_open, _read, _close, 또는 proven_fs_read_all_u8str	명시적인 에러, 숨은 버퍼링 없음, 파일 전체를 한 번의 호출로. 챕터 5
fgets	proven_sysio_read_line, proven_reader_read_line	버퍼를 정확히 꽉 채우는 줄도 잃어버리지 않고 반환합니다. 챕터 5
qsort	proven_array_sort	introsort입니다: 퀵소트의 최악 경우가 아니라 $O(n \log n)$ 보장. 챕터 4
bsearch	proven_array_binary_search	같은 모양, 같은 비교자 계약. 챕터 4
rand	proven_xoshiro256ss_* 또는 proven_random_bytes	재현 가능하고 빠르거나, 예측 불가능하고 안전하거나 — 여러분의 의도적으로 고릅니다. 챕터 5
time / clock	proven_time_now, proven_time_breakdown	나노초 단위이고, 벽시계와 monotonic을 명시적으로 구분합니다. 챕터 5
assert	proven_panic + panic 혹은	freestanding 빌드에서도 동작하고, 교체할 수 있습니다. 챕터 1

16. 다음에 읽을 것

각 장이 그 앞의 장들만 필요로 하도록 순서를 잡았습니다.

파트	읽을 것	무엇을 위해
I	이 장	계약과 어휘
II	1 → 2 → 3	에러, 메모리, 텍스트: 모든 프로그램이 쓰는 것
III	4	배열, 맵, 리스트, 링, 정렬, 해싱, 인코딩
IV	8	챕터 3이 소개한 뒤의, 형식화와 파싱 전체
V	5	파일, 스트림, 표준 I/O, 시간, 난수, 매핑
VI	6 → freestanding	코루틴, job, 스레드 안전성, 베어메탈, 크로스 빌드
부록	A: alias 인덱스 , 위의 B와 D	찾아보기

숙련된 C 프로그래머이고 시간이 없다면, 위의 § 15를 읽고, 그다음 [챕터 1](#)을 읽고, 그다음 필요한 것을 다루는 [챕터](#)를 읽으세요. 더 초심자라면 파트 I과 II를 순서대로 읽으세요 — 짧고, 뒤의 모든 것이 그것을 전제로 합니다.

1장: 기초 — 에러, 타입, 메모리 view

2부 — 모든 프로그램이 사용하는 어휘. 선행 조건: 0장. 이 장을 마치면 이 라이브러리가 보고하는 모든 실패를 처리할 수 있고, 크기를 잃어버리지 않으면서 메모리 영역을 기술할 수 있으며, 조용히 감기지 않는 크기 산술을 할 수 있다.

이 장은 `types.h`, `error.h`, `memory.h`, `align.h`, `version.h`, `panic.h`를 다룬다 — 다른 모든 장이 그 위에 세워지는 조각들이다. 여기에는 할당하는 것이 아무것도 없고, 운영체제와 이야기하는 것도 아무것도 없다. 그러면서도 진짜로 반드시 읽어야 하는, 가장 짧은 장이다.

목차

1. 에러는 값이다
2. 타입, 그리고 왜 표기가 다른가
3. 메모리 view: 포인터와 길이를 하나로
4. 감길 수 없는 크기 산술
5. 정렬
6. Panic: 에러를 돌려줄 상대가 남지 않았을 때
7. 버전 매크로
8. 예제와 오용 사례

1. 에러는 값이다

왜 이것이 맨 앞에 오는가

이 매뉴얼의 나머지 모든 페이지는 여기서 실패가 어떻게 보고되는지 여러분이 안다고 가정한다. 그러니 이 절은 건너뛸 수 없는 유일한 절이다.

C는 함수가 실패를 어떻게 보고해야 하는지에 대해 스스로와 합의한 적이 한 번도 없다. `malloc`은 `NULL`을 반환한다. `fopen`도 `NULL`을 반환한다. `read`는 `-1`을 반환한다. `strtol`은 `0`을 반환하고 어쩌면 `errno`를 설정하는데, 그것도 여러분이 먼저 `errno`를 지웠을 때에만 그렇다. 성공한 호출도 거기에 쓰레기를 남겨 두는 것이 허용되기 때문이다. `printf`는 거의 아무도 확인하지 않는 음수를 반환한다. 네 가지 관례가 있고, 이들이 공유하는 유일한 성질은 무시해도 조용하고 합법이라는 것이다:

```
char *p = malloc(n);
p[0] = 'x';                /* wrong: malloc returns NULL on failure */
```

```
FILE *f = fopen(path, "r");
fread(buf, 1, n, f);       /* wrong: f may be NULL */
```

```
long v = strtol(s, NULL, 10); /* wrong: 0 could be the value or the failure */
```

더 깊은 문제는 이것들이 틀리기 쉽다는 데 있지 않다. 타입이 무언가 일어날 수 있다는 사실을 전혀 말해 주지 않는다는 것이 문제다. `char *`는 `NULL`이 될 수 있든 없든 똑같은 타입이므로, 그 무엇도 — 컴파일러도, 대충 훑어보는 리뷰어도, 새벽 두 시의 여러분도 — 확인하라는 신호를 받지 못한다.

`errno`는 에러를 전역이면서 일시적인 것으로 만들어 상황을 더 나쁘게 한다. 정확히 알맞은 순간에 읽어야 하고, 중간에 끼어든 어떤 라이브러리 호출이든 그것을 덮어쓸 수 있으며, 스레드마다 따로 존재하므로 한 부류의 버그를 고치려다 또 다른 부류를 열어젖혔다.

이 라이브러리가 대신 하는 일

에러는 반환값이며, 자기만의 타입을 가진다.

함수의 결과가 성공 아니면 실패뿐일 때에는 `proven_err_t`를 반환한다 — `PROVEN_OK`가 `0`인 평범한 열거형이다:

```
proven_err_t err = proven_u8str_append(&s, PROVEN_LIT("hello"));
if (!proven_is_ok(err)) { /* handle it */ }
```

함수가 돌려줄 값이 있을 때에는 값과 에러가 하나의 구조체 안에 함께 실려 오며, 에러를 확인하기 전까지 그 값은 아무 의미도 없다:

```
proven_result_u8str_t s = proven_u8str_create(alloc, 64);
if (!proven_is_ok(s.err)) return;      /* s.value is not a string; do not touch it */
```

이는 Rust의 Result나 C++23의 std::expected와 같은 발상을, C가 허용하는 유일한 방식 — 값으로 반환되는 구조체 — 으로 표현한 것이다. 할당도, 간접 참조도, 숨은 제어 흐름도 없다. 여러분이 포기하는 것은 에러 처리를 조용히 건너뛸 수 있는 능력인데, 그것이 바로 요점이다.

라이브러리의 나머지가 이 세 가지 성질에 의존하므로 명시해 둘 가치가 있다:

- **PROVEN_OK은 0이다.** 그래서 {0}으로 영초기화한 result 구조체는 "성공, 비어 있음"을 뜻하는 상태로 시작한다.
- **실패 가능한 함수는 [[nodiscard]]다** — 반환값을 버리는 코드를 컴파일러가 거부하게 만드는 C23 속성이다. 여전히 에러를 무시할 수는 있지만, 호출 앞에 (void)를 써서 그것이 의도였음을 밝혀야 한다.
- 함수가 달리 문서화하지 않는 한 **실패는 failure-atomic이다**: 연산이 실패했다면 아무것도 바꾸지 않았다. 실패한 append는 여러분의 문자열을 있던 그대로 남겨 둔다.

레퍼런스

```
typedef enum {
    PROVEN_OK = 0,
    PROVEN_ERR_NOMEM,
    PROVEN_ERR_OUT_OF_BOUNDS,
    PROVEN_ERR_INVALID_ENCODING,
    PROVEN_ERR_INVALID_ARG,
    PROVEN_ERR_IO,
    PROVEN_ERR_NOT_FOUND,
    PROVEN_ERR_INVALID_STATE,
    PROVEN_ERR_OVERFLOW,
    PROVEN_ERR_UNSUPPORTED,
    PROVEN_ERR_AGAIN,
    PROVEN_ERR_EOF,
    PROVEN_ERR_BUSY,
    PROVEN_ERR_PERMISSION,
    PROVEN_ERR_INVALID_FORMAT
} proven_err_t;
```

에러	일반적인 의미	보통의 대응
PROVEN_OK	성공.	계속 진행한다.
PROVEN_ERR_NOMEM	allocator가 메모리를 제공하지 못했다.	이 연산을 포기한다. 객체는 바뀌지 않았다.
PROVEN_ERR_OUT_OF_BOUNDS	인덱스, 크기, 용량, 또는 범위가 유효하지 않다 — "들어가지 않는다"도 포함한다.	대상을 키우거나, 입력을 거부한다.
PROVEN_ERR_INVALID_ENCODING	인코딩된 텍스트가 검증에 실패했다 (hex, Base64).	입력을 잘못된 형식으로 보고 거부한다.
PROVEN_ERR_INVALID_ARG	널 포인터, 유효하지 않은 allocator, 불가능한 모드, 잘못된 인자 개수.	호출을 고친다 — 이것은 보통 잘못된 데이터가 아니라 여러분 코드의 버그를 뜻한다.
PROVEN_ERR_IO	OS 또는 장치가 연산을 실패시켰다.	재시도하거나, 사용자에게 보고한다.
PROVEN_ERR_NOT_FOUND	파일, 키, 부분 문자열, 또는 리소스가 존재하지 않는다.	"없음" 분기를 탄다. 흔히 에러조차 아니다.
PROVEN_ERR_INVALID_STATE	객체의 상태가 이 연산을 허용하지 않는다.	호출 순서를 고친다.

PROVEN_ERR_OVERFLOW	정수 변환이나 크기 산술이 오버플로 했다.	그 크기를 거부한다. § 4를 보라.
PROVEN_ERR_UNSUPPORTED	이 플랫폼이나 빌드 프로파일에서는 사용할 수 없다.	다른 경로를 택한다(예: 파이프는 seek할 수 없다).
PROVEN_ERR_AGAIN	지금은 안 된다. 나중에 다시 시도하라.	보통 기다린 뒤 재시도한다.
PROVEN_ERR_EOF	입력의 끝.	읽기 루프를 멈춘다 — 예외가 아니라 예상된 일이다.
PROVEN_ERR_BUSY	큐, 락, 또는 리소스가 사용 중이다.	물러나서 기다린다.
PROVEN_ERR_PERMISSION	접근이 거부되었다.	보고한다. 재시도해도 소용없다.
PROVEN_ERR_INVALID_FORMAT	포맷이나 스캔 템플릿 자체가 잘못되었다.	포맷 문자열을 고친다 — 잘못된 데이터가 아니라 버그다.

```
static inline int proven_is_ok(proven_err_t err);
#define PROVEN_IS_OK(err) proven_is_ok(err)
```

proven_is_ok은 err == PROVEN_OK일 때 0이 아닌 값을 반환한다. PROVEN_IS_OK은 매크로처럼 보이기를 원하는 자리를 위한 매크로 표기다. 둘은 같은 일을 한다.

올바른 예:

```
proven_result_u8str_t s = proven_u8str_create(alloc, 32);
if (!proven_is_ok(s.err)) {
    return; /* nothing was created, so there is nothing to destroy */
}
(void)proven_u8str_append(&s.value, PROVEN_LIT("ready"));
proven_u8str_destroy(alloc, &s.value);
```

반례 — 에러를 맨 진리값처럼 검사하기:

```
proven_result_u8str_t s = proven_u8str_create(alloc, 32);
if (s.err) {
    /* wrong: works today only because PROVEN_OK is 0. It reads as "if there is
       an error" to someone who already knows that, and as nothing at all to
       everyone else. Say what you mean: !proven_is_ok(s.err). */
    return s.err;
}
```

반례 — 이 설계 전체가 막으려고 존재하는 바로 그 실수:

```
proven_result_u8str_t s = proven_u8str_create(alloc, 32);
(void)proven_u8str_append(&s.value, text); /* wrong: s.value may be garbage */
proven_u8str_destroy(alloc, &s.value); /* wrong: destroying what was never created */
```

.err를 확인하기 전에 .value를 읽는 것은 이 관례가 여러분 대신 잡아 줄 수 없는 유일한 예러다. 구조체는 어느 쪽이든 존재하므로 컴파일러는 불평하지 않는다 — 그러니 호출 바로 다음 줄에 확인을 적는 습관을 들일 가치가 있다.

실전 예제: 값으로서의 에러

이 프로그램은 테스트 스위트가 컴파일하고 실행하므로 낚을 수 없다. 실패 가능한 호출이 취하는 두 가지 형태 — 돌려줄 것이 없을 때의 맨 proven_err_t와, 돌려줄 것이 있을 때의 proven_result_*_t — 을 보여주고, result 안의 값이 그 옆의 에러를 확인하기 전까지는 아무 의미도 없는 이유를 보여준다.

```
/*
 * proven 에서 오류는 값이다. 실패할 수 있는 호출은 오류나 result 를 돌려주고,
 * 컴파일러가 그것을 보게 만든다. 풀어 되돌릴 것도 없고, 찾아볼 전역도 없다.
 */
```

```
/* 돌려줄 값이 없는 실패 가능 연산은 proven_err_t 를 돌려준다. */
```

```

static proven_err_t write_greeting(proven_u8str_t *out) {
    return proven_u8str_append(out, PROVEN_LIT("hello"));
}

/* 돌려줄 값이 *있으면* 그것을 지키는 오류와 함께 싸여 온다. 값은 `err` 를 확인한
 * 뒤에야 뜻을 갖는다. */
static proven_result_size_t half(proven_size_t n) {
    proven_result_size_t res = {0};
    if (n % 2 != 0) {
        res.err = PROVEN_ERR_INVALID_ARG; /* res.value 는 0 그대로 둔다: 아무 뜻도 없다 */
        return res;
    }
    res.err = PROVEN_OK;
    res.value = n / 2;
    return res;
}

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* --- 맨 proven_err_t 를 확인하기 ----- */
    proven_result_u8str_t s = proven_u8str_create(alloc, 32);
    EXAMPLE_REQUIRE(proven_is_ok(s.err), "creating a 32-byte string should succeed");

    proven_err_t err = write_greeting(&s.value);
    if (!proven_is_ok(err)) {
        /* 아무것도 볼지 않았고 문자열은 여전히 온전하다. proven 의 늘리는 연산은
         * 실패 원자적이다. */
        proven_u8str_destroy(alloc, &s.value);
        return 1;
    }
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&s.value), PROVEN_LIT("hello")),
        "the greeting should have been appended");

    /* --- result 구조체를 확인하기 ----- */
    proven_result_size_t ok = half(10);
    EXAMPLE_REQUIRE(proven_is_ok(ok.err), "10 is even, so halving it must succeed");
    EXAMPLE_REQUIRE(ok.value == 5, "half of 10 is 5");

    proven_result_size_t bad = half(7);
    EXAMPLE_REQUIRE(bad.err == PROVEN_ERR_INVALID_ARG, "7 is odd, so halving it must fail");
    /* bad.value 는 읽으면 안 된다. 여기서는 0 이지만 그것은 이 함수의 구현 사정일
     * 뿐, result 타입의 약속이 아니다. */

    /* --- 오류를 실수로 흘릴 수는 없다 ----- */
    /* proven_u8str_append 에는 [[nodiscard]] 가 붙어 있어 이렇게 쓰면 컴파일 오류다.
     *
     * proven_u8str_append(&s.value, PROVEN_LIT("!"));
     *
     * 실패를 정말로 무시하겠다면, 무시한다고 적어야 한다. */
    (void)proven_u8str_append(&s.value, PROVEN_LIT("!"));

    printf("greeting: %s\n", proven_u8str_as_cstr(&s.value));

    proven_u8str_destroy(alloc, &s.value);
    return EXAMPLE_OK();
}

```

2. 타입, 그리고 왜 표기가 다른가

그냥 int와 char를 쓰면 안 되는가?

C의 내장 타입은 모호하기로 유명하다. int는 최소 16비트, long은 최소 32비트이고, char는 컴파일러와 대상 플랫폼에 따라 부호가 있을 수도 없을 수도 있다. 그렇지 않다고 가정한 코드는 여러분의 기계에서는 동작하고 남의 기계에서는 깨지며, 그 깨짐은 대개 조용하다.

C99는 <stdint.h>(int32_t, uint64_t 등)로 폭 문제를 해결했다. 이 라이브러리는 그 아래에서 그것들을 사용하되 자기만의 이름을 붙인다. 이유는 하나다: **라이브러리가 지원하는 모든 대상에서 같은 것을 뜻하는 단 하나의 표기가 필요하기 때문이다.** <stdint.h>가 유일하게 쓸 수 있는 헤더일지도 모르는 프리스탠딩(freestanding) 환경까지 포함해서 말이다.

두 번째의, 더 미묘한 이유가 있고 실전에서 중요한 것은 그쪽이다: proven_u8과 proven_byte_t는 둘 다 8비트지만 서로 다른 것을 뜻한다.

- proven_u8은 0에서 255 사이의 수다. 더하고, 비교하고, 출력하라.
- proven_byte_t는 **어떤 객체 표현의 바이트**다. 이는 unsigned char의 별칭이며, 무엇이든 그 원시 바이트를 미정의 동작 없이 들여다볼 수 있도록 C가 허용하는 유일한 타입이다.

원시 메모리에 proven_byte_t를 쓰는 것은 취향의 문제가 아니다. C의 strict-aliasing 규칙은 잘못된 타입의 포인터로 객체를 읽는 것이 미정의라고 말하며, unsigned char가 명시적인 예외다. 이 라이브러리의 모든 버퍼, view, 해시 입력이 proven_byte_t인 것은 정확히 그 이유 때문이다.

레퍼런스

이름	의미	비고
proven_i8, proven_i16, proven_i32, proven_i64	부호 있는 고정 폭 정수	표준 고정 폭 타입의 별칭.
proven_u8, proven_u32, proven_u64	부호 없는 고정 폭 정수	proven_u8은 수치값 이다. 원시 객체 바이트에는 proven_byte_t를 쓰라.
proven_u16	16비트 코드 유닛 타입	<uchar.h>가 있으면 char16_t를, 없으면 uint_least16_t를 쓴다. U16 문자열 API가 UTF-16 코드 유닛에 이 타입을 쓴다.
proven_byte_t	바이트 단위 객체 표현	unsigned char의 별칭. 어떤 객체의 바이트든 합법적으로 들여다볼 수 있는 타입이다.
proven_size_t	크기 및 인덱스 타입	size_t의 별칭. 부호 없음 — 둘을 빼기 전에 § 4를 보라.
proven_ptrdiff_t	포인터 차이, 부호 있는 오프셋	ptrdiff_t의 별칭.
proven_intptr_t, proven_uintptr_t	포인터 크기 정수	arena의 범위 검사처럼 명시적인 포인터-정수 작업에만 쓴다.

```
typedef struct {
    proven_err_t err;
    proven_size_t value;
} proven_result_size_t;
```

proven_result_size_t는 크기 또는 에러를 반환한다. value는 err == PROVEN_OK일 때만 유효하다. 부분 append, 파일 읽기와 쓰기, 크기 조회에서 돌려받는 것이 이것이다 — 답이 "몇 개인가"이면서 연산이 실패할 수 있는 모든 자리에서 쓰인다.

반례 — 바이트를 작은 수의 저장소처럼 다루기:

```
proven_u8 *raw = (proven_u8 *)&some_struct; /* wrong type for raw inspection */
for (size_t i = 0; i < sizeof some_struct; i++) hash ^= raw[i];
```

주류 컴파일러 전부에서 우연히 동작하지만, 그래도 이것은 잘못 쓴 타입이다. aliasing 보장이 뒤에 있는 쪽은 proven_byte_t이고, 그것을 쓰는 것은 여러분이 수가 아니라 표현을 들여다보고 있음을 문서화한다.

3. 메모리 view: 포인터와 길이를 하나로

문제: 포인터는 잊어버린다

포인터는 무언가가 어디서 시작하는지만 말하고 그 외에는 아무것도 말하지 않는다. "어디서 끝나는가"라는 나머지 절반에 대해 C가 지금까지 해 온 모든 것은 그 위에 얹은 관례였다:

- 문자열을 위한 **NUL 종료자** — 길이를 알고 싶을 때마다 $O(n)$ 스캔이 들고, 0 바이트를 포함한 텍스트를 표현할 수 없으며, 종료자가 없어지는 순간 버퍼 오버런을 낸다.
- 그 밖의 모든 것을 위한 **별도의 길이 매개변수** — 누군가 잘못된 값을 넘기기 전까지만 동작한다. 두 인자를 서로 묶어 주는 것이 아무것도 없기 때문이다.

```
void process(const unsigned char *data, size_t len);
process(buf, sizeof buf);           /* fine */
process(buf, sizeof(buf) - 1);      /* also compiles; now the last byte is invisible */
process(other, sizeof buf);         /* also compiles; now it reads past the end */
```

저것 하나하나가 모두 합법적인 호출이다. 포인터와 길이 사이의 관계는 오직 프로그래머의 머릿속에만 존재한다.

이 라이브러리가 대신 하는 일

view는 포인터와 크기를 한 구조체에 담아 값으로 전달한 것이다. 둘은 하나이기 때문에 떨어질 수 없다.

```
typedef struct { const proven_byte_t *ptr; proven_size_t size; } proven_mem_view_t; /* read-only */
typedef struct {      proven_byte_t *ptr; proven_size_t size; } proven_mem_mut_t;  /* writable */
typedef struct {      proven_byte_t *ptr; proven_size_t size; } proven_mem_t;      /* owned */
```

이들은 두 워드다. 복사는 공짜이고, 근처 어디에도 할당이 없다. view가 하지 않는 일은 소유다. view는 남의 것인 바이트를 가리키며, 그 바이트가 무효가 되는 순간 함께 무효가 된다. 이 규칙이 0장의 계약 2다.

세 가지 표기는 의도만 다르며, 그 의도가 핵심이다:

- proven_mem_view_t — **빌려 쓰는(borrowed), 읽기 전용**. const가 가리켜지는 바이트에 붙어 있으므로 컴파일러가 강제한다.
- proven_mem_mut_t — **borrowed, 쓰기 가능**. 이를 통해 쓸 수 있지만, 여전히 소유하지는 않는다.
- proven_mem_t — **owned**. 누군가는 이것을 만들어 낸 할당자(allocator)로 해제해야 한다. 타입이 그것을 강제하지는 않는다. 알릴 뿐이다.

그리고 실패할 수 있는 연산을 위한 result 래퍼 두 개가 있다:

```
typedef struct { proven_err_t err; proven_mem_mut_t value; } proven_result_mem_mut_t;
typedef struct { proven_err_t err; proven_mem_view_t value; } proven_result_mem_view_t;
```

proven_result_mem_mut_t는 allocator가 돌려주는 것이다. proven_result_mem_view_t는 검사된 슬라이스가 돌려주는 것이다.

슬라이싱, 검사된 것과 검사되지 않은 것

view의 부분 범위를 취하는 것은 view로 하게 될 가장 흔한 일이며, 의도적으로 두 가지 형태로 제공된다:

API	목적	반환
proven_mem_view_from_owned(mem)	소유(owned) 블록에 대한 읽기 전용 view.	proven_mem_view_t.
proven_mem_mut_from_owned(mem)	owned 블록에 대한 쓰기 가능 view.	proven_mem_mut_t.
proven_mem_view_slice_checked(view, offset, size)	서브뷰, 검증함.	proven_result_mem_view_t.

proven_mem_view_slice_unchecked(view, offset, size)	서브뷰, 검증하지 않음 .	proven_mem_view_t.
proven_mem_mut_slice_checked(mut, offset, size)	쓰기 가능 서브슬라이스, 검증함.	proven_result_mem_mut_t.
proven_mem_mut_slice_unchecked(mut, offset, size)	쓰기 가능 서브슬라이스, 검증하지 않음 .	proven_mem_mut_t.
proven_range_contains_ptr(base, cap, ptr, size, out_offset)	이 포인터 범위가 저 할당 안에 있는가? 정수 주소 비교이며, 무관한 포인터끼리 포인터 산술을 하지 않는다.	_Bool.
proven_memcmp(s1, s2, size)	원시 메모리를 비교한다.	같으면 0. 부호는 바이트 순서(부호 없음)에 따른다.
proven_mem_copy(dst, dst_cap, src)	view를 dst로 경계 검사하며 복사한다.	PROVEN_OK. 들어가지 않으면 PROVEN_ERR_OUT_OF_BOUNDS이며 — 아무것도 기록되지 않는다 . 크기가 0이 아닌데 널 포인터이면 PROVEN_ERR_INVALID_ARG. 영역은 겹쳐서는 안 된다.
proven_mem_move(dst, dst_cap, src)	proven_mem_copy와 같지만 영역이 겹쳐도 된다.	위와 같다.

검사된 형태의 정확한 동작:

- `view.size > 0 && view.ptr == NULL` → `PROVEN_ERR_INVALID_ARG`(뒤에 메모리가 없는 크기는 빈 view가 아니라 버그다).
- 요청한 범위가 view 밖으로 벗어남 → `PROVEN_ERR_OUT_OF_BOUNDS`.
- `size == 0` → `PROVEN_OK`와 함께 빈 view(`ptr == NULL, size == 0`).

기본적으로 검사된 형태를 쓰라. 검사되지 않은 형태는 바로 윗줄들에서 이미 범위를 증명한 경우 — 예컨대 여러분이 직접 계산한 경계를 가진 루프 안 — 를 위해 존재하며, 그때 두 번째 검사는 순수 비용일 뿐이다. 그런 경우는 실제로 있고, 느낌보다 좋다.

반례 — 바깥에서 온 수에 검사 없는 슬라이싱을 쓰기:

```
proven_mem_view_t part = proven_mem_view_slice_unchecked(view, user_offset, user_size);
/* wrong: user_offset and user_size were never validated. This is the buffer
   overrun from the top of section 3, reintroduced through a safer-looking API. */
```

반례 — 빈 view를 거부하기:

```
if (!view.ptr) return PROVEN_ERR_INVALID_ARG; /* wrong: {NULL, 0} is legal */
```

빈 view는 정상적인 값이다. 길이 0인 슬라이스, 내용이 없는 파일, 방금 초기화된 문자열이 그렇다. 잘못된 view를 가려내는 검사는 크기가 0이 아닌데 포인터가 널인 경우다:

```
proven_mem_view_t view = {0}; /* size 0, ptr NULL: a legal empty view */
proven_err_t err = PROVEN_OK;

if (view.size > 0 && !view.ptr) {
    err = PROVEN_ERR_INVALID_ARG; /* only a null pointer with bytes behind it is a bug */
}
(void)err;
```

4. 감길 수 없는 크기 산술

왜 곱셈 하나에 함수 호출이 필요한가

proven_size_t는 부호가 없고, C의 부호 없는 산술은 오버플로하지 않는다 — 조용하고 합법적으로 2^{64} 를 법으로 감긴다. 그 규칙 하나가 업계 할당 버그의 상당 부분 뒤에 있다:

```
proven_size_t total = count * sizeof(item_t); /* wrong: wraps for large count */
void *p = malloc(total); /* succeeds, and is far too small */
items[count - 1] = x; /* writes way past the end */
```

count = 2^{61} 이고 항목이 16바이트라면 total은 0이다. malloc(0)은 성공한다. 그 뒤의 모든 쓰기는 범위 밖이고, 어디에서도 에러를 보고하지 않았다. 크기가 프로그램 바깥에서 온 데이터로부터 계산될 때마다 — 파일 헤더, 네트워크 패킷, 사용자가 준 개수 — 같은 모양이 나타난다.

빨셈에는 거울상 문제가 있고, 이쪽이 더 자주 걸린다:

```
proven_size_t remaining = view.size - offset; /* wrong when offset > size: a huge number */
```

이 라이브러리가 대신 하는 일

산술을 수행하고 그 결과가 들어맞았는지 알려 주는 매크로 세 개다.

```
#define PROVEN_CKD_ADD(res, a, b) /* true on overflow */
#define PROVEN_CKD_SUB(res, a, b)
#define PROVEN_CKD_MUL(res, a, b)
```

매크로	계산	반환	*res 기록
PROVEN_CKD_ADD(res, a, b)	$a + b$	오버플로 시 true, 성공 시 false	들어맞을 때만
PROVEN_CKD_SUB(res, a, b)	$a - b$	오버플로 시 true(부호 없는 언더플로 포함)	들어맞을 때만
PROVEN_CKD_MUL(res, a, b)	$a * b$	오버플로 시 true	들어맞을 때만
PROVEN_SIZE_MAX	—	proven_size_t가 담을 수 있는 가장 큰 값	—

- res는 답이 들어갈 곳을 가리키는 포인터다.
- 이들은 **오버플로 시 true**, 성공 시 false를 반환한다. 처음에는 거꾸로 읽힌다. 이 관례는 C23의 <stdckdint.h>에서 왔고(컴파일러가 제공하면 이 매크로들이 그것을 사용한다), 그렇지 않으면 GCC/Clang의 __builtin*_overflow 계열에서 왔다. 호출을 `"이거 오버플로했나?"`로 읽으면 말이 맞아떨어진다.
- PROVEN_SIZE_MAX는 계산한 뒤 검사하는 대신 한계와 직접 비교하고 싶을 때를 위해 있다.

올바른 예:

```
typedef struct { int id; double weight; } item_t;

proven_size_t count = 1024;
proven_size_t total = 0;
proven_err_t err = PROVEN_OK;

if (PROVEN_CKD_MUL(&total, count, sizeof(item_t))) {
    err = PROVEN_ERR_OVERFLOW; /* refuse to allocate a size that wrapped */
} else {
    proven_result_mem_mut_t block = alloc.alloc_fn(alloc.ctx, total, alignof(item_t));
    err = block.err;
    if (proven_is_ok(err)) {
        alloc.free_fn(alloc.ctx, block.value.ptr);
    }
}
(void)err;
```

반례:

```
proven_size_t total = count * sizeof(item_t); /* wrong: may wrap silently */
```

프로그램의 모든 산술 식에 이것들이 필요하지는 않다. 필요한 곳은 크기가 **여러분이 직접 고르지 않은 값으로부터 계산되는** 자리이며, 버그가 있는 곳이 정확히 거기다.

5. 정렬

정렬이란 무엇이고, 라이브러리가 왜 이를 언급하는가

C의 모든 타입에는 정렬이 있다. 하드웨어가 효율적으로, 또는 아예 로드할 수 있으려면 그 값이 시작해야 하는 주소다. `double`은 보통 8로 나누어떨어지는 주소를 원한다. 어떤 아키텍처는 정렬되지 않은 로드에서 폴트를 내고, x86은 그저 느려질 뿐이다. 어느 쪽이든 C는 정렬되지 않은 포인터로 객체에 접근하는 것을 미정의 동작이라고 말한다.

여러분이 이것을 생각할 필요가 없었던 이유는 `malloc`이 무엇에나 알맞게 정렬된 메모리를 반환하기 때문이다. 그 보장은 여러분이 직접 메모리를 나눠 주기 시작하는 순간 정확히 사라진다 — 그리고 직접 메모리를 나눠 주는 일이 바로 [2장](#)의 아레나(arena)와 풀(pool)이 하는 일이다. 포인터를 13바이트 밀고 그 결과를 건네 주는 arena는 방금 여러분에게 정렬되지 않은 `double`을 준 것이다.

그래서 여기의 헬퍼들은 allocator를 위해, 그리고 allocator를 직접 쓰는 여러분을 위해 존재한다.

레퍼런스

매크로	의미
<code>PROVEN_DEFAULT_ALIGNMENT</code>	여러분이 달리 말하지 않을 때 라이브러리가 쓰는 기본값. 현재 8이다.
<code>PROVEN_MAX_ALIGN</code>	<code>alignof(max_align_t)</code> — 어떤 내장 타입에도 충분하다.

함수	목적	반환
<code>proven_is_pow2(x)</code>	x가 0이 아닌 2의 거듭제곱인가? 정렬값은 그해야 한다.	bool.
<code>proven_mem_align_up(addr, align)</code>	크기나 주소를 <code>align</code> 의 다음 배수로 올림한다.	정렬된 값. <code>align</code> 이 유효하지 않거나 올림이 오버플로하면 0.
<code>proven_uintptr_align_up(addr, align)</code>	같은 일을 <code>proven_uintptr_t</code> 주소에 대해 한다.	위와 같다.

에러 관례에 주의하라: 이들은 에러 코드를 담을 자리가 없으므로 **실패 시 0**을 반환한다. 0은 결코 유효한 정렬 주소가 아니므로, 0인지 검사하는 것은 모호하지 않다.

올바른 예:

```
typedef struct { int id; double weight; } item_t;

proven_size_t size = 100;
proven_size_t aligned = proven_mem_align_up(size, alignof(item_t));
proven_err_t err = (aligned == 0) ? PROVEN_ERR_OVERFLOW : PROVEN_OK;
(void)err;
```

반례 — 고전적인 비트 조작 버전:

```
proven_size_t aligned = (size + align - 1) & ~(align - 1); /* wrong: may overflow */
```

저 줄은 수많은 프로덕션 코드에 들어 있고, 표현 범위 꼭대기 근처의 입력을 빼면 모든 입력에 대해 옳다. 그 근처에서는 `size + align - 1`이 작은 수로 감기고, 결과는 시작한 곳보다 낮은 주소가 된다.

실전 예제: view·슬라이싱·정렬을 한자리에서

이 장의 세 가지 생각 — 길이를 스스로 지니고 다니는 뷰(view), 잘라 내기 전에 범위를 검사하는 슬라이스(slice), 2의 거듭제곱이어야만 하는 정렬 경계(alignment boundary) — 은 프로그램이 이들을 한꺼번에 쓸 때 비로소 요점이 드러난다. 아래 프로그램은 고정 크기 레코드(record, 같은 크기의 자료 한 칸) 표를 자기가 소유한 메모리 한 덩이에 담고, 그런 프로그램이라면 반드시 해야 하는 네 가지 일을 한다. 표를 고쳐 쓰

면 안 되는 코드에 넘기기, 한 행(row)만 고칠 수 있게 넘기기, 뒤쪽 행들을 앞으로 당겨 한 행을 지우기, 그리고 다른 데서 건네받은 포인터가 애초에 이 표를 가리키기는 하는지 판단하기.

이 중 둘은 C가 조용히 틀리기 가장 쉬운 자리다. 겹치는(overlapping) 영역을 옮기는 것은 proven_mem_copy가 약속한 일이 아니어서 proven_mem_move라는 별도의 호출이 있다. 그리고 서로 다른 객체에 속할 수도 있는 두 포인터를 <로 비교하는 것은 미정의 동작(undefined behaviour, 컴파일러가 "그런 일은 절대 없다"고 가정해도 되는 상황)이므로, "이 포인터가 내 버퍼 안에 있는가"는 >=와 <가 아니라 proven_range_contains_ptr로 묻는다.

이 프로그램은 테스트 스위트가 컴파일하고 실행하므로 낚을 수 없다.

```
#include <string.h>

/*
 * 소유한 버퍼 하나를, 세 가지 방식으로 이야기하기.
 *
 * 이 프로그램은 고정 크기 레코드로 된 작은 표를 자기가 소유한 기억 덩이 하나에 담고,
 * 그런 프로그램이면 반드시 하게 되는 네 가지를 한다.
 *
 * 1. 고치면 안 되는 읽는 쪽에 표를 건네고(읽기 전용 뷰),
 * 2. 한 행만 고쳐도 되는 쪽에 그 행을 건네고(슬라이스),
 * 3. 뒤 행들을 앞으로 덮어 옮겨 한 행을 지우고(겹치는 복사 - 보통의 복사에는
 *    허용되지 않는 일이다),
 * 4. 남이 돌려준 포인터를 받아, 행 번호로 쓰기 전에 애초에 이 표를 가리키기는
 *    하는지 가린다.
 *
 * 네 걸음 모두 맨 `char *` 가 길이를 잃어버리고 프로그램이 나중에야 그것을 알게 되는
 * 자리다. 여기 쓰인 타입들은 길이를 포인터와 함께 들고 다니므로, 경계에 대한 물음이
 * 실수가 벌어질 바로 그 자리에서 답해진다.
 */

#define ROW_SIZE 8u
#define ROW_COUNT 4u

/* 읽는 쪽은 뷰를 받는다. 모든 바이트를 읽을 수 있고 하나도 쓸 수 없다. 타입에 붙은
 * const 는 권고가 아니다 - 그것을 통해 대입하면 컴파일되지 않는다. */
static proven_size_t count_nonzero(proven_mem_view_t table) {
    proven_size_t n = 0;
    for (proven_size_t i = 0; i < table.size; ++i) {
        if (table.ptr[i] != 0) {
            ++n;
        }
    }
    return n;
}

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* 표를 할당한다. 할당자는 읽고 쓸 수 있는 슬라이스를 돌려준다. 소유한 쪽이 쥐는
     * 것이 그것이다 - 포인터와 그에 딸린 크기. */
    proven_result_mem_mut_t got = alloc.alloc_fn(alloc.ctx, ROW_SIZE * ROW_COUNT, alignof(proven_u32));
    EXAMPLE_REQUIRE(proven_is_ok(got.err), "allocating the record table must succeed");
    if (!proven_is_ok(got.err)) {
        return 1;
    }

    /* proven_mem_t 가 "내가 이것을 소유한다" 는 타입이다. 소유한 덩이를 변수 하나에
     * 두고 거기서 파생한 뷰와 슬라이스를 나눠 주는 것, 그것이 이 라이브러리가 요구하는
     * 소유 규율의 전부다. */
```

```

proven_mem_t owned = { .ptr = got.value.ptr, .size = got.value.size };
proven_mem_mut_t table = proven_mem_mut_from_owned(owned);

/* i 번 행을 i + 1 값의 바이트로 채운다. 그래야 옮겨진 행을 알아볼 수 있다. */
for (proven_u32 row = 0; row < ROW_COUNT; ++row) {
    proven_result_mem_mut_t slot = proven_mem_mut_slice_checked(table, row * ROW_SIZE, ROW_SIZE);
    EXAMPLE_REQUIRE(proven_is_ok(slot.err), "every row of the table must be in bounds");
    if (!proven_is_ok(slot.err)) {
        alloc.free_fn(alloc.ctx, owned.ptr);
        return 1;
    }
    memset(slot.value.ptr, (int)(row + 1), slot.value.size);
}

/* 1. 읽기 전용으로 쓰기. 읽는 쪽은 표를 고칠 수도, 끝을 넘어갈 수도 없다.
 * 두 사실을 한꺼번에 건네받았기 때문이다. */
proven_mem_view_t read_only = proven_mem_view_from_owned(owned);
EXAMPLE_REQUIRE(count_nonzero(read_only) == ROW_SIZE * ROW_COUNT,
    "every byte of every row was written");

/* 2. 없는 행을 달라고 하는 것은 디버깅할 사고가 아니라 다룰 수 있는 오류다.
 * 행이 넷인 표의 4번 행은 끝에서 한 행 지난 자리에서 시작한다. */
proven_result_mem_mut_t past_end = proven_mem_mut_slice_checked(table, ROW_COUNT * ROW_SIZE, ROW_SIZE);
EXAMPLE_REQUIRE(past_end.err == PROVEN_ERR_OUT_OF_BOUNDS,
    "slicing past the end must report OUT_OF_BOUNDS, not return a bad slice");

/* 3. 2번과 3번 행을 한 행씩 앞으로 옮겨 1번 행을 지운다. 원본과 목적지가
 * 겹치므로, 보통의 복사가 다루면 안 되는 바로 그 경우다. proven_mem_copy 는
 * 겹치지 않는 구역을 계약으로 적고 있고, 겹쳐도 받는 것은 proven_mem_move 다. */
proven_mem_view_t tail = {
    .ptr = table.ptr + 2 * ROW_SIZE,
    .size = 2 * ROW_SIZE
};
proven_err_t moved = proven_mem_move(table.ptr + 1 * ROW_SIZE, table.size - 1 * ROW_SIZE, tail);
EXAMPLE_REQUIRE(proven_is_ok(moved), "moving the tail of the table down must succeed");
EXAMPLE_REQUIRE(table.ptr[1 * ROW_SIZE] == 3, "row 2 moved into row 1's place");
EXAMPLE_REQUIRE(table.ptr[2 * ROW_SIZE] == 4, "row 3 moved into row 2's place");

/* 옮기기도 다른 모든 것처럼 경계가 있다. 원본을 담기에 목적지가 작으면 거부되고,
 * 아무것도 쓰이지 않는다. */
proven_err_t refused = proven_mem_move(table.ptr, ROW_SIZE, tail);
EXAMPLE_REQUIRE(refused == PROVEN_ERR_OUT_OF_BOUNDS,
    "a move that would not fit must be refused before it writes");

/* 4. 다른 데서 온 포인터. 서로 무관한 포인터를 < 로 비교하는 것은 C 에서 미정의
 * 동작이고 - 컴파일러는 그런 일이 없다고 가정해도 된다 - 그래서 "이 포인터가
 * 내 버퍼를 가리키는가" 는 비교를 옮겨 해 주는 함수로 물어야 한다. */
proven_size_t offset = 0;
const proven_byte_t *from_elsewhere = table.ptr + 2 * ROW_SIZE;
bool inside = proven_range_contains_ptr(table.ptr, table.size, from_elsewhere, ROW_SIZE, &offset);
EXAMPLE_REQUIRE(inside, "a pointer to row 2 is inside the table");
EXAMPLE_REQUIRE(offset / ROW_SIZE == 2, "and its offset recovers the row index");

proven_u8 unrelated[ROW_SIZE] = { 0 };
EXAMPLE_REQUIRE(!proven_range_contains_ptr(table.ptr, table.size, unrelated, ROW_SIZE, NULL),
    "a pointer into a different object is not inside the table");

/* 안에서 시작하지만 밖에서 끝나는 행도 밖이다. 검사는 시작 자리가 아니라 구간
 * 전체에 대한 것이다. */
EXAMPLE_REQUIRE(!proven_range_contains_ptr(table.ptr, table.size,

```

```

        table.ptr + table.size - 1, ROW_SIZE, NULL),
    "a range that starts inside but runs past the end is refused");

/* 5. 구간 검사로 그 행이 경계 안임을 증명했다면, 검사 없는 슬라이스가 옳은
 * 호출이다. 하는 일이 없고, 증명은 주석이 아니라 바로 뒷줄이다. 그 증명 없이는
 * 절대 쓰지 말 것. */
proven_mem_mut_t row2 = proven_mem_mut_slice_unchecked(table, offset, ROW_SIZE);
EXAMPLE_REQUIRE(row2.size == ROW_SIZE && row2.ptr[0] == 4,
    "the unchecked slice names the row the checked test just proved");

/* 6. 정렬 - 부르는 쪽이 그것을 만나는 유일한 자리다. 정렬이 다른 두 가지를 한
 * 덩어리 안에 나란히 놓는 일. 둘째 것의 주소는 올림해야 하고, 올림은 2의 거듭제곱
 * 경계에서만 정의된다 - 그래서 경계를 먼저 검사한다. */
proven_size_t want_align = alignof(proven_u32);
EXAMPLE_REQUIRE(proven_is_pow2(want_align), "an alignment boundary must be a power of two");
EXAMPLE_REQUIRE(!proven_is_pow2(24u), "24 is not a power of two, so it is not a valid boundary");

proven_uintptr_t raw = (proven_uintptr_t)(table.ptr + 1); /* 일부러 홀수 */
proven_uintptr_t aligned = proven_uintptr_align_up(raw, want_align);
EXAMPLE_REQUIRE(aligned >= raw, "aligning up never moves backwards");
EXAMPLE_REQUIRE(aligned % want_align == 0, "and the result is on the boundary asked for");
EXAMPLE_REQUIRE(aligned - raw < want_align, "the padding is never more than one boundary");

/* 2의 거듭제곱이 아닌 경계는 그럴듯한 틀린 주소 대신 0 을 돌려준다. 그래서 실수가
 * 아래로 흘러가지 않고 여기서 멈춘다. */
EXAMPLE_REQUIRE(proven_uintptr_align_up(raw, 24u) == 0,
    "aligning to a non-power-of-two returns 0, not a wrong address");

alloc.free_fn(alloc.ctx, owned.ptr);
return EXAMPLE_OK();
}

```

반례 — 겹침을 금지하는 복사로 같은 삭제를 쓴 경우:

```

/* rows 2..3 moved down onto row 1: source and destination overlap */
proven_mem_copy(table.ptr + ROW_SIZE, table.size - ROW_SIZE, tail); /* wrong */

```

proven_mem_copy는 겹치지 않는 영역을 문서로 약속한다. 이 호출은 그 위반을 보고하지 않는다. 그저 가장 빠른 순서로 복사할 자유를 가질 뿐이고, 그 순서가 바뀌는 날 표에는 같은 행이 조용히 두 번 들어간다.

반례 — 범위 질문을 평범한 포인터 비교로 물은 경우:

```

if (ptr >= table.ptr && ptr + size <= table.ptr + table.size) { /* wrong */ }

```

ptr이 다른 할당에서 왔다면 이 비교는 "답이 틀린 비교"가 아니라 미정의 동작이다.

6. Panic: 에러를 돌려줄 상대가 남지 않았을 때

에러를 반환하는 라이브러리에 panic이 왜 있는가

이 장의 모든 내용은 실패가 들여다볼 수 있는 값이어야 한다고 주장해 왔다. 패닉(panic)은 그것이 항상 가능하지는 않다는 인정이다.

어떤 API는 구조상 에러 채널이 없다. proven_arena_alloc_or_panic()은 result가 아니라 메모리 블록을 반환하는데, 이는 할당 실패가 프로그램이 아예 실행될 수 없다는 뜻이고 돌려줄 만한 것이 없는, 프로그램의 초기 설정 단계를 위해 존재하기 때문이다. freestanding 빌드에서는 abort()도, stderr도, 되돌아갈 곳도 없을 수 있다.

그래서 라이브러리는 proven_panic()을 호출해 panic을 일으키며, 이는 여러분이 교체할 수 있는 핸들러로 디스패치한다. exit를 호출하지도, stderr에 출력하지도, 운영체제를 가정하지도 않는다.

```

typedef void (*proven_panic_handler_t)(const char *msg);

```

```
void proven_panic(const char *msg);
void proven_set_panic_handler(proven_panic_handler_t handler);
```

API	목적	비고
proven_panic(msg)	panic을 일으킨다. 설치된 핸들러로 디스패치한다.	라이브러리의 _or_panic API가 호출한다. 여러분이 직접 호출해도 된다.
proven_set_panic_handler(handler)	핸들러를 설치한다.	기본값으로 되돌리려면 NULL을 넘겨라. 스레드 안전하지 않다 — 다른 스레드가 생기기 전, 시작 단계에서 설치하라.
proven_panic_handler_t	void (*)(const char *msg)	핸들러는 반환해서는 안 된다.

기본 핸들러는 trap한다: GCC와 Clang에서는 __builtin_trap()이고, 그 밖의 컴파일러에서는 무한 루프다. 디스패치는 weak symbol이 아니라 함수 포인터를 거치므로, ELF와 PE/COFF(Windows, mingw-w64) 툴체인에서 같은 방식으로 링크된다.

직접 설치하기(파일 스코프 함수이므로, 이는 다른 함수 안에 붙여넣는 것이 아니라 리스팅이다):

```
static void my_panic(const char *msg) {
    log_critical(msg);          /* your logger */
    for (;;) {
        /* reset, halt, or wait for a debugger */
    }
}
```

```
proven_set_panic_handler(my_panic); /* pass NULL to restore the default */
```

panic 핸들러는 반환해서는 안 된다. 반환하면 _or_panic 계열은 돌려줄 것이 없고, 그 result의 유효성은 보장되지 않는다.

반례 — 반환하는 핸들러:

```
static void my_panic(const char *msg) {
    fprintf(stderr, "%s\n", msg); /* wrong: this returns, and then the caller
                                   proceeds with a block that was never allocated */
}
```

반례 — 평범한 코드에서 _or_panic에 손을 뻗기:

```
proven_mem_mut_t block = proven_arena_alloc_or_panic(&arena, n);
/* wrong for anything that handles input: an arena that is merely full has just
   killed the process. Use proven_arena_alloc and read the error. */
```

7. 버전 매크로

진단용으로, 그리고 라이브러리 버전에 맞춰 적응해야 하는 코드를 위한 컴파일 타임 식별자다.

```
#define PROVEN_VERSION_MAJOR 0
#define PROVEN_VERSION_MINOR 1
#define PROVEN_VERSION_PATCH 1
#define PROVEN_VERSION_STRING "proven_c_lib-v0.1.1"
#define PROVEN_VERSION_ENCODE(major, minor, patch) ((major) * 1000000L + (minor) * 1000L + (patch))
#define PROVEN_VERSION_NUM PROVEN_VERSION_ENCODE(PROVEN_VERSION_MAJOR, PROVEN_VERSION_MINOR,
PROVEN_VERSION_PATCH)
```

버전은 시맨틱 버전 — MAJOR.MINOR.PATCH — 이고 v0.0.1부터 시작한다. PATCH 릴리스는 고치거나 문서화할 뿐 호출자가 바뀌어야 할 것이 없다. MINOR 릴리스는 기존 호출을 깨지 않고 더한다. MAJOR 릴리스는 기존 호출을 깰 수 있다. MAJOR가 0인 동안은 MINOR 사이에서도 API가 움직일 수 있다.

PROVEN_VERSION_STRING은 빌드 보고서나 --version 플래그에서 출력하는 것이다. 숫자 셋은 라이브러리 버전에 맞춰 적응해야 하는 코드를 위한 것이고, PROVEN_VERSION_NUM은 그 셋을 #if용 정수 하나로 접은 것이다 — 리터럴이 아니라 PROVEN_VERSION_ENCODE와 비교한다: #if PROVEN_VERSION_NUM >= PROVEN_VERSION_ENCODE(0, 1, 0).

v0.0.1 이전의 릴리스는 날짜로 번호를 붙였고(v26.MM.DDx), PROVEN_VERSION_NUM은 그 날짜였다. 번호는 체계와 함께 처음부터 다시 시작했으므로, 옛 날짜 값과의 비교는 지금의 모든 릴리스에서 틀린다.

PROVEN_VERSION_ENCODE로 다시 쓰라.

숫자와 문자열은 빌드가 include/proven/version.h와 대조해 검사하고, 문자열은 숫자와도 대조하므로, 매뉴얼이 헤더로부터 어긋날 수 없다. (한 번 어긋난 적이 있고, 이 문장이 다시는 그러지 않을 이유다: 문자열에는 게이트가 걸려 있었고 나머지는 없었다.)

8. 예제와 오용 사례

안전한 result 처리

```
proven_byte_t record[16] = {0};
proven_mem_view_t view = { .ptr = record, .size = sizeof record };

proven_result_mem_view_t part = proven_mem_view_slice_checked(view, 4, 8);
if (!proven_is_ok(part.err)) {
    return; /* the range was not inside the view */
}
```

```
proven_byte_t payload[8];
proven_err_t err = proven_mem_copy(payload, sizeof payload, part.value);
(void)err;
```

증명한 뒤에만 검사 없는 슬라이싱

호출자가 이미 범위를 증명했을 때 올바른 예:

```
proven_byte_t record[16] = {0};
proven_mem_view_t view = { .ptr = record, .size = sizeof record };
proven_size_t offset = 4;
proven_size_t size = 8;

if (offset <= view.size && size <= view.size - offset) {
    proven_mem_view_t part = proven_mem_view_slice_unchecked(view, offset, size);
    (void)part; /* proved in range: safe to read */
}
```

저 검사의 모양에 주목하라. 더 자연스러운 `offset + size <= view.size` 대신 `size <= view.size - offset`으로 쓴 것은 정확히 § 4 때문이다. 앞서 나온 `offset <= view.size` 검사가 있으면 합은 감길 수 있지만 차는 감길 수 없다.

반례:

```
proven_mem_view_t part = proven_mem_view_slice_unchecked(view, user_offset, user_size);
/* wrong: user_offset and user_size were not validated */
```

빈 view는 허용된다

크기가 0인 view는 널 포인터를 가질 수 있다. 그것을 무턱대고 거부하지 마라.

반례:

```
if (!view.ptr) return PROVEN_ERR_INVALID_ARG; /* wrong for empty views */
```

더 나은 예:

```
proven_mem_view_t view = {0}; /* size 0, ptr NULL: a legal empty view */
proven_err_t err = PROVEN_OK;

if (view.size > 0 && !view.ptr) {
```

```
    err = PROVEN_ERR_INVALID_ARG;    /* only a null pointer with bytes behind it is a bug */
}
(void)err;
```

다음으로 갈 곳

2장은 이 타입들이 일을 시작하는 곳이다. allocator는 proven_mem_mut_t를 만들어 내고, arena는 § 5의 정렬 헬퍼를 사용하며, 라이브러리의 모든 확장 가능 컨테이너는 allocator를 매개변수로 받는다. § 1이 에러를 반환값으로 받는 이유로 든 것과 같은 이유 때문이다 — 비용은 시그니처에 보여야 한다.

2장: 할당 — 힙, 아레나, 풀, 버퍼

Part II — 모든 프로그램이 쓰는 어휘. 선행 장: 1장. 이 장을 마치면 반사적으로 malloc을 집어드는 대신 할당 전략을 의도적으로 고를 수 있고, 세 가지 비용 중 어느 것을 치르고 있는지 알게 된다.

이 장은 heap.h, arena.h, pool.h, allocator.h, buffer.h를 다룬다. 예전에 이 장의 7절이었던 스레드 안전성과 포인터 provenance 자료는 이제 나머지 동시성 주제와 함께 6장에 있다 — 책에서 가장 어려운 자료가 맨 앞쪽 장 중 하나에 들어앉아 있었기 때문이다.

목차

1. 할당이 매개변수인 이유, 그리고 heap allocator
2. Arena: 여러 객체, 하나의 수명
3. Pool: 여러 객체, 하나의 크기
4. Allocator trait
5. 원시 바이트 버퍼
6. 예제와 오용 사례

1. 할당이 매개변수인 이유, 그리고 heap allocator

malloc의 문제

malloc은 전역이다. 어떤 함수든 호출할 수 있고, 시그니처의 그 무엇도 함수가 호출하는지 아닌지를 말해 주지 않으며, 메모리가 무엇에 쓰이든 모든 호출은 똑같은 범용 allocator로 간다.

여기서 따라 나오는 결과가 넷이고, 아마 이미 겪어 봤을 것이다:

- **시그니처만 보고는 함수가 할당하는지 알 수 없다.** char *build_message(int n)은 할당할 수도, 정적 버퍼 안의 포인터를 반환할 수도, 리터럴을 반환할 수도 있다. 세 경우 모두 타입이 같으므로 호출자는 해제해야 하는지 알 수 없고, 답은 틀렸을지도 모르는 주석 속에 있다.
- **프로그램의 한 부분만 전략을 바꿀 수 없다.** 파서가 파싱이 끝날 때 전부 죽는 작은 할당을 만 번 한다면, malloc과 free는 만 번의 범용 할당과 만 번의 해제를 한다. bump allocator라면 한 번이면 된다. 모든 호출 지점을 다시 쓰지 않고는 그렇게 하겠다고 말할 방법이 없다.
- **실패 경로를 테스트할 수 없다.** malloc을 원할 때 실패시키려면 전역으로 가로채야 한다 — LD_PRELOAD, 링커 트릭, 라이브러리 내부 호출까지 잡아채는 #define malloc my_malloc. 그런데 정작 가장 테스트하고 싶은 분기는 메모리가 바닥났을 때 실행되는 그 분기다.
- **힙이 없는 곳에서는 아예 쓸 수 없다.** 펌웨어, 커널, 부트로더: malloc이 없고, 그것을 전제하는 모든 라이브러리는 쓸 수 없다.

malloc은 잘못 설계된 것이 아니다. 그것은 고정된 정책이며, 다르게 하라고 말할 매개변수를 받지 않는다는 사실 때문에 모든 호출 지점에 하드코딩되어 있는 것이다.

이 라이브러리가 대신 하는 것

allocator는 값이고, 매개변수다. 할당할 수 있는 함수는 proven_allocator_t를 받고, 할당할 수 없는 함수는 받지 않는다. 발상은 그게 전부이고, 모든 결과가 여기서 따라 나온다:

```
proven_err_t proven_u8str_append(proven_u8str_t *str, proven_u8str_view_t data);
proven_err_t proven_u8str_append_grow(proven_allocator_t alloc, proven_u8str_t *str, proven_u8str_view_t data);
```

시그니처를 읽어 보라. 첫 번째는 문자열을 키울 수 없으므로 텍스트가 들어가지 않으면 실패한다. 두 번째는 키울 수 있고, allocator를 받음으로써 그렇다고 말한다. 어느 쪽을 호출했는지 궁금해할 일이 결코 없다.

이제 같은 코드가 호출자가 고르는 세 가지 서로 다른 전략과 함께 동작한다:

Allocator	얻는 방법	개별 해제?	쓸 때
Heap	proven_heap_allocator()	예	일반적인 경우. 서로 무관한 수명을 가진 객체들.

Arena	proven_arena_create(backing) 후 proven_arena_as_allocator(&a)	아니오 — free는 no-op이다; 전체를 reset하거나 destroy한다	전부 같은 순간에 죽는 많은 할당: 요청 하나, 프레임 하나, 파싱 하나.
Pool	proven_pool_init(&p, base, size, align, bin_cap) 후 proven_pool_as_allocator(&p)	예, free list로 들어간다	하나의 고정 크기인 객체를 반복해서 할당하고 해제할 때.

힙(heap)에서 시작하라. 이유가 있을 때 나머지 둘로 손을 뻗되, 그 이유는 대개 측정이다.

Heap allocator(힙 할당자)

```
proven_allocator_t proven_heap_allocator(void);
```

이것은 라이브러리의 인터페이스를 입은 malloc, realloc, free다. 그러지 않을 구체적인 이유가 없다면 이것을 써야 하며, 여기에 영리한 구석은 전혀 없다 — 그게 요점이다. 이 매뉴얼에서 달리 할 이유가 없는 모든 예제는 이것을 쓴다:

```
proven_allocator_t heap = proven_heap_allocator();
```

```
proven_result_u8str_t s = proven_u8str_create(heap, 64);
if (!proven_is_ok(s.err)) {
    return; /* nothing was created, so there is nothing to destroy */
}
(void)proven_u8str_append(&s.value, PROVEN_LIT("ready"));
proven_u8str_destroy(heap, &s.value); /* the SAME allocator */
```

값은 네 워드 — 컨텍스트 포인터 하나와 함수 포인터 셋 — 이고 값으로 전달된다. 복사는 공짜이고, 자신의 struct에 저장해도 되며, 파괴할 필요도 없다.

프리스탠딩(freestanding) 빌드에서는 proven_heap_allocator가 존재하지 않는다. 감쌀 malloc이 없기 때문이다. 이는 우회해야 할 제약이 아니라, 라이브러리 전체가 allocator를 매개변수로 받는 바로 그 이유이며, 정적 배열 위의 아레나(arena)가 같은 코드를 마이크로컨트롤러에서 돌아가게 하는 이유다. [freestanding 모드를](#) 참고하라.

잘못된 예 — 생성할 때와 다른 allocator로 파괴하기:

```
proven_result_u8str_t s = proven_u8str_create(arena_alloc, 64);
proven_u8str_destroy(heap_alloc, &s.value); /* wrong: heap free on arena memory */
```

객체는 자신을 만든 allocator를 기억하지 않는다 — 그것이 객체를 작게 유지하는 요인이다. 오늘날 이것을 검사하는 것은 없으며, 실패는 다른 어딘가에서 나중에 드러나는 힙 손상이다. 구조적으로 짝을 지어라: allocator를 객체 옆에 두거나, 둘을 함께 전달하라.

2. Arena: 여러 객체, 하나의 수명

문제: 사망 시점이 같은 수많은 작은 할당

설정 파일을 파싱한다고 하자. 키마다 문자열 하나, 값마다 문자열 하나, 섹션마다 노드 하나를 할당한다 — 작은 할당 수천 개다. 그리고 파싱이 끝나면 결과를 만들고, 그 할당 하나 하나는 모두 같은 순간에 쓰레기가 된다.

malloc으로는 그 대가를 두 번 치른다. 할당마다 free list를 검색하고, 장부를 갱신하고, 헤더가 붙은 메모리를 반환한다; free마다 메모리를 돌려주고 어찌면 이웃과 병합한다. 그리고 하나도 빠뜨려선 안 된다. 잊어버린 free 하나면 누수이기 때문이다.

arena가 하는 관찰은 **그 객체들이 전부 같은 수명을 가진다**는 것이며, 따라서 개별 해제는 무의미한 작업이라는 것이다. 함께 죽는다면, 함께 해제될 수 있다.

Arena가 동작하는 방식

arena는 메모리 블록 하나와 오프셋 하나다. 할당한다는 것은 요청받은 정렬에 맞게 오프셋을 올림하고, 그 주소를 반환하고, 오프셋을 앞으로 옮기는 것이다. 알고리즘 전체가 이게 전부다:

```
[####used####|           free           ]
      ^ offset
```

- 할당은 명령어 몇 개다. 검색도, free list도, 할당마다 붙는 헤더도 없다.
- free는 아무 일도 하지 않는다. 의도적으로 no-op이다.
- 회수는 arena 전체를 reset하거나 destroy해서 한다. 오프셋을 0으로 되돌리는 것이다. 연산 하나가 객체만 개를 해제한다.

메모리는 여러분이 준다. proven_arena_create는 proven_mem_mut_t를 받는다 — 힙에서 받아 온 블록이든, static 배열이든, 스택 위의 영역이든 상관없다. arena는 스스로 할당하지 않으며, 그것이 밑에 힙이 없어도 동작하게 하는 요인이다.

포기하는 것

arena는 범용 allocator가 아니며, 그 맛바꿈은 실제적이다:

- 객체 하나를 해제할 수 없다. 수명이 실제로 공유되지 않는다면, arena는 설계상 누수한다 — 메모리는 reset에서만 회수된다.
- 바닥나는 것은 단단한 한계다. backing 블록은 고정되어 있다. 여기서의 PROVEN_ERR_NOMEM은 기계의 메모리가 바닥났다는 뜻이 아니라, 이 arena가 바닥났다는 뜻이다.
- arena를 가리키는 모든 포인터는 reset에서 죽는다. 한꺼번에, 아무 경고도 없이. reset보다 오래 사는 뷰(view)는 dangling view다. 0장의 계약 2를 보라.

수명 공유가 프로그램에 대한 사실일 때 arena를 쓰라. 희망 사항일 때가 아니라.

```
proven_arena_t
typedef struct {
    proven_mem_mut_t backing;
    proven_size_t offset;
} proven_arena_t;
```

필드:

- backing: 호출자가 소유하는 가변 메모리 범위.
- offset: backing 내에서 다음 할당 위치.

Arena 함수와 헬퍼

API	의도	매개변수	반환
proven_arena_create(backing)	backing 슬라이스 위에 arena를 초기화한다.	backing: 호출자 소유 메모리.	proven_arena_t.
proven_arena_reset(arena)	arena의 모든 할당을 버린다.	arena.	void.
proven_arena_destroy(arena)	형식적 정리.	arena.	void; 호출자 backing arena에는 no-op.
proven_arena_alloc_aligned(arena, size, align)	정렬로 할당한다.	arena, 바이트 size, 2의 거듭제곱 align.	proven_result_mem_mut_t.
proven_arena_realloc_aligned(arena, old_ptr, old_size, new_size, align)	재할당; 꼬리(tail) 할당을 제자리에서 확장/축소할 수 있다.	이전 할당 세부 정보와 새 크기.	proven_result_mem_mut_t.
proven_arena_alloc(arena, size)	PROVEN_DEFAULT_ALIGNMENT로 할당한다.	arena, 바이트 size.	proven_result_mem_mut_t.
proven_arena_alloc_aligned_or_panic(arena, size, align)	할당하거나 패닉(panic) 호출을 호출한다.	arena, size, align.	proven_mem_mut_t.
proven_arena_alloc_or_panic(arena, size)	정렬 panic 할당.	arena, size.	proven_mem_mut_t.

proven_arena_as_allocator(arena)	arena를 proven_allocator_t로 노출한다.	arena.	allocator trait, 또는 널 arena에 대해 제로 allocator.
----------------------------------	----------------------------------	--------	---

Trait 어댑터 헬퍼:

- proven_arena_alloc_trait
- proven_arena_realloc_trait
- proven_arena_free_trait

이들은 allocator 트레이트(trait)가 함수 포인터를 필요로 하기 때문에 노출된다. 애플리케이션 코드는 보통 대신 proven_arena_as_allocator()를 호출한다.

예제:

```
alignas(max_align_t) proven_byte_t storage[4096];
proven_arena_t arena = proven_arena_create((proven_mem_mut_t){
    .ptr = storage,
    .size = sizeof storage,
});

proven_result_mem_mut_t r = proven_arena_alloc(&arena, 64);
if (!proven_is_ok(r.err)) {
    return; /* the arena cannot grow: it reports NOMEM instead */
}

proven_arena_reset(&arena); /* reclaims r and everything else at once */
proven_arena_destroy(&arena);
```

성장 가능한 컨테이너와 함께 쓰는 arena

성장 가능한 컨테이너는 arena allocator를 사용할 수 있지만, arena의 free가 no-op이므로 성장이 이전 블록을 버릴 수 있다. 가능하면 용량을 미리 확보(reserve)하라.

올바른 예:

```
alignas(max_align_t) proven_byte_t storage[4096];
proven_arena_t arena = proven_arena_create((proven_mem_mut_t){
    .ptr = storage,
    .size = sizeof storage,
});
proven_allocator_t a = proven_arena_as_allocator(&arena);

/* Ask for the capacity up front, so growth never has to abandon a block. */
proven_result_array_t ar = PROVEN_ARRAY_INIT(a, int, 128);
if (!proven_is_ok(ar.err)) {
    return;
}
(void)PROVEN_ARRAY_PUSH(&ar.value, int, 10);
PROVEN_ARRAY_DESTROY(&ar.value); /* correct, but arena free reclaims nothing */
proven_arena_reset(&arena); /* this is what gives the bytes back */
```

잘못된 예 — 아주 작은 초기 용량에서 arena 안으로 성장시키기:

```
proven_result_array_t ar = PROVEN_ARRAY_INIT(a, int, 1);
for (int i = 0; i < 10000; ++i) {
    PROVEN_ARRAY_PUSH(&ar.value, int, i); /* wrong: every regrow abandons the old block */
}
```

재성장할 때마다 arena에 더 큰 블록을 요청하고 이전 것을 "해제"하는데 — arena에서 그것은 아무 일도 하지 않는다. 배열 자체는 올바르게 끝나지만, arena는 지금까지 할당한 모든 중간 크기를 그대로 쥔 채 끝난다:

원하던 버퍼 하나 위에 1, 2, 4, 8 ... 8192개 원소어치의 버려진 공간이 없린다. 위의 올바른 예처럼 용량을 미리 확보하라.

잘못된 예 — reset보다 오래 사는 view:

```
proven_u8str_view_t name = /* ... built in the arena ... */;
proven_arena_reset(&arena);
use(name); /* wrong: those bytes are now free space */
```

이것이 arena의 가장 날카로운 모서리다. reset은 회수하는 메모리를 건드리지 않으므로 바이트는 보통 아직 그대로 있고, 버그는 보통 테스트에서 드러나지 않는다 — 다음 할당이 그 위에 덮어쓰기 직전까지는.

3. Pool: 여러 객체, 하나의 크기

arena가 풀지 못하는 문제

arena는 수명이 공유된다고 가정한다. 정반대 모양을 한 작업 부하도 많다: 한 타입의 객체가 특별한 순서 없이 계속 만들어지고 파괴되는 경우다. 이벤트의 연결 리스트. 커졌다 작아졌다 하는 트리의 노드들. 생겼다 사라지는 연결 레코드들.

arena는 이것을 할 수 없다 — 객체 하나를 결코 회수하지 않으므로, 오래 도는 프로그램은 한없이 커질 것이다. 힙은 할 수 있고, 그것이 정확히 그 비용이다: 할당마다 검색하고, 해제마다 장부를 갱신하며, 범용 allocator는 이미 천 번이나 처리한 요청에 대해 범용적인 일을 한다.

풀(pool)이 하는 관찰은 **이 객체들이 전부 같은 크기**라는 것이다. 크기가 같다면, 해제된 것 하나가 앞으로의 요청에 정확히 들어맞으므로, 아무런 검색 없이 곧바로 다시 건네줄 수 있다.

Pool이 동작하는 방식

pool은 해제된 블록들의 작은 스택 — bin — 을 유지하고, 뻔한 일을 한다:

- **할당:** bin에 뭔가 있으면 하나를 pop해서 반환한다. 포인터 읽기 하나와 감소 하나다. bin이 비어 있을 때만 하위 allocator로 간다.
- **해제:** bin에 자리가 있으면 재사용을 위해 블록을 push한다. bin이 꽉 찼으면 pool이 메모리를 영원히 세워 두지 않도록 하위 allocator로 되돌려준다.

따라서 bin_cap은 다이얼이다: 이 pool이 예비로 쥐고 있어도 되는 메모리의 양이다.

포기하는 것

- **하나의 pool은 정확히 하나의 크기와 정렬만 처리한다.** 그 밖의 요청은 다른 데서 처리되는 것이 아니라 `PROVEN_ERR_INVALID_ARG`로 거부된다. pool이 만들어질 때보다 더 엄격한 정렬도 거부되고, 더 느슨한 것은 블록이 이미 그것을 충족하므로 괜찮다.
- **pool은 살아 있는 객체를 추적하지 않는다.** `proven_pool_destroy`는 bin에 있는 것을 해제하지, 여러분이 아직 쥐고 있는 것을 해제하지 않는다. 할당한 것을 먼저 전부 해제하라. 그러지 않으면 누수인 데다가 이제 dangling이다.

arena 대 pool을 한 줄씩으로: arena는 *많이 할당하고 한 번에 전부 해제*를 위한 것이고, pool은 *같은 크기를 몇 번이고 저렴하게 할당하고 해제*하기 위한 것이다.

```
proven_pool_t
typedef struct {
    proven_allocator_t base_alloc;
    proven_size_t item_size;
    proven_size_t item_align;
    void **bin;
    proven_size_t bin_cap;
    proven_size_t bin_len;
} proven_pool_t;
```

필드:

- `base_alloc`: 새 블록과 재활용 bin에 사용되는 allocator.

- `item_size`: pool이 받아들이는 정확한 객체 크기. 그 외 크기는 `PROVEN_ERR_INVALID_ARG`로 거부된다.
- `item_align`: 모든 블록이 할당되는 정렬. 더 엄격한 정렬 요청은 `PROVEN_ERR_INVALID_ARG`로 거부되고, 더 느슨한 요청은 블록이 이미 그것을 충족하므로 처리된다.
- `bin`: 캐시된 free-list 배열.
- `bin_cap`: 최대 캐시 블록 개수.
- `bin_len`: 현재 캐시 블록 개수.

재활용이 작동하는 방식

pool은 하나의 큰 슬랩(slab)을 소유하지 않는다. 각 항목을 `base_alloc`에서 개별적으로 할당하고, 해제된 블록들의 작은 스택(bin, 최대 `bin_cap`개의 포인터로 이루어진 배열)을 유지한다:

- **할당**. `bin_len > 0`이면 맨 위 포인터를 pop한다($O(1)$, `base_alloc` 호출 없음) — 이것이 pool의 전체 요점이다. 그렇지 않으면 `base_alloc`로 넘어간다.
- **해제**. `bin_len < bin_cap`이면 재사용을 위해 포인터를 bin에 push한다; 그렇지 않으면(bin이 꽉 참) `base_alloc`로 곧바로 되돌려 해제한다. 따라서 bin은 pool이 재사용을 위해 세워둘 수 있는 메모리의 양을 제한한다.
- **파괴**. `proven_pool_destroy`는 bin에 아직 남아 있는 모든 포인터와 bin 배열 자체를 해제한다 — 하지만 살아 있는(넘겨준) 항목은 추적하지 **않으므로**, pool을 파괴하기 전에 할당한 모든 것을 해제해야 한다.

이는 pool을 지역(region) allocator가 아니라 수명이 짧은 동일 타입 객체(노드, 이벤트)를 위한 churn 최적화기(optimizer)로 만든다. "많이 할당하고 한 번에 전부 해제"하고 싶으면 arena를 사용하라; "같은 크기를 몇 번이고 저렴하게 할당/해제"하고 싶으면 pool을 사용하라.

반례

```
/* WRONG: the pool only handles its configured size/alignment. */
proven_allocator_t a = proven_pool_as_allocator(&pool); /* item_size == sizeof(Node) */
a.alloc_fn(a.ctx, sizeof(BigThing), alignof(BigThing)); /* not the pool's item -> rejected */

/* WRONG: destroying while items are still live leaks (and dangles) them. */
void *p = a.alloc_fn(a.ctx, sizeof(Node), alignof(Node)).value.ptr;
proven_pool_destroy(&pool); /* `p` is NOT freed and is now dangling */
/* RIGHT: free every handed-out item first, then destroy. */
```

Pool 함수

API	의도	반환
<code>proven_pool_init(pool, base_alloc, item_size, item_align, bin_cap)</code>	고정 크기 pool을 초기화한다.	<code>PROVEN_OK</code> 또는 에러.
<code>proven_pool_as_allocator(pool)</code>	pool이 뒷받침하는 allocator trait를 반환한다.	<code>proven_allocator_t</code> .
<code>proven_pool_destroy(pool)</code>	캐시된 블록과 bin 저장소를 해제한다.	<code>void</code> .

예제:

```
typedef struct Node { int value; } Node;

proven_pool_t pool = {0};
proven_err_t e = proven_pool_init(&pool, alloc, sizeof(Node), alignof(Node), 64);
if (!proven_is_ok(e)) {
    return;
}

proven_allocator_t node_alloc = proven_pool_as_allocator(&pool);
proven_result_mem_mut_t n = node_alloc.alloc_fn(node_alloc.ctx, sizeof(Node), alignof(Node));
if (proven_is_ok(n.err)) {
    /* Hand it back before destroy: the pool does not track live items. */
    node_alloc.free_fn(node_alloc.ctx, n.value.ptr);
}
```

```
}
```

```
proven_pool_destroy(&pool);
```

잘못된 예:

```
node_alloc.alloc_fn(node_alloc.ctx, sizeof(LargerObject), alignof(LargerObject));  
/* wrong: one pool is for one fixed object size and alignment */
```

4. Allocator 트레이트

이 절이 첫 번째가 아니라 네 번째인 이유

여러분은 세 가지 allocator를 그들이 공유하는 인터페이스를 보지 않고 이미 써 보았고, 그것은 의도적이였다. 인터페이스는 이 발상에서 가장 재미없는 부분이고 아무 맥락 없이 읽기에 가장 어려운 것이다: 함수 포인터 typedef 셋과 정렬 계약은, 그 뒤에 있는 heap과 arena와 pool을 보기 전까지는 거의 아무 의미도 없다.

이 절이 필요한 이유는 둘이다: 자신만의 allocator를 작성하는 것, 그리고 이 라이브러리의 모든 allocator가 지키기로 약속한 규칙을 이해하는 것. 위의 셋을 쓰기만 할 생각이라면 proven_heap_allocator(), proven_arena_as_allocator(), proven_pool_as_allocator()가 API 전부이니 건너뛰어도 된다.

여기서 trait란 인터페이스로 쓰이는 함수 포인터 struct를 말한다 — 손으로 써 낸, C 버전의 가상 테이블이다. proven_allocator_t는 라이브러리에서 가장 중요한 trait이고, stream.h와 random.h도 같은 모양을 쓴다.

함수 포인터 타입

```
typedef proven_result_mem_mut_t (*proven_alloc_fn_t)(  
    void *ctx,  
    proven_size_t size,  
    proven_size_t align  
);
```

```
typedef proven_result_mem_mut_t (*proven_realloc_fn_t)(  
    void *ctx,  
    void *old_ptr,  
    proven_size_t old_size,  
    proven_size_t new_size,  
    proven_size_t align  
);
```

```
typedef void (*proven_free_fn_t)(void *ctx, void *ptr);
```

의도:

- alloc_fn은 새로운 바이트 슬라이스를 할당한다.
- realloc_fn은 할당 크기를 변경하며 failure-atomic이어야 한다: 실패 시 이전 블록은 여전히 유효하고 변경되지 않으므로, 호출자는 아무것도 잃지 않는다.
- 블록은 할당될 때와 같은 align으로 재할당되고 해제되어야 한다. allocator는 과도하게 정렬된(over-aligned) 요청에 대해 일반 요청과는 다른 하위 메커니즘을 선택할 수 있으며, heap allocator가 실제로 그렇게 한다: align <= alignof(max_align_t) — 이 라이브러리의 모든 문자열, 버퍼, 바이트 배열이 여기에 해당한다 — 는 제자리(in place)에서 성장이 일어날 수 있도록 malloc/realloc을 거치고, 그보다 더 엄격하게 정렬된 것은 정렬된 allocator를 거친다. 블록을 다른 정렬 클래스로 되돌려주는 것은 정의되지 않은 동작이다.
- free_fn은 메모리를 해제한다. allocator가 크기 메타데이터를 필요로 한다면, 내부적으로 그것을 추적해야 한다.

```
proven_allocator_t  
typedef struct {  
    void *ctx;  
    proven_alloc_fn_t alloc_fn;  
    proven_realloc_fn_t realloc_fn;
```

```
    proven_free_fn_t free_fn;
} proven_allocator_t;
```

필드:

- ctx: allocator별 상태.
- alloc_fn: 할당 함수.
- realloc_fn: 재할당 함수.
- free_fn: 해제 함수.

레퍼런스

멤버 / API	형태	계약
ctx	void *	allocator 자신의 상태. 호출자에게는 불투명하며, 첫 번째 인자로 되돌아온다.
alloc_fn	proven_alloc_fn_t	align 정렬로 size 바이트를 할당한다. proven_result_mem_mut_t를 반환한다.
realloc_fn	proven_realloc_fn_t	크기를 바꾼다. failure-atomic : 실패 시 이전 블록은 손대지 않은 채 여전히 유효하다.
free_fn	proven_free_fn_t	해제한다. 블록이 할당될 때와 같은 align 클래스를 주어야 한다.
proven_alloc_is_valid(alloc)	static inline bool	모든 함수 포인터가 널이 아닐 때, 즉 trait를 호출할 수 있을 때 true.

이 라이브러리의 모든 allocator가 지키며, 여러분의 것도 지켜야 하는 세 가지 규칙:

1. 같은 **allocator**, 전체 수명 동안. 한 블록의 할당, 재할당, 해제는 하나의 allocator로 하라.
2. 같은 **정렬 클래스**. 어떤 정렬로 할당된 블록은 그 정렬로 재할당되고 해제되어야 한다. allocator는 과도하게 정렬된 요청에 다른 메커니즘을 쓸 수 있다.
3. 실패는 아무것도 바꾸지 않는다. 실패한 realloc_fn은 이전 블록을 유효하고 변경되지 않은 채로 남긴다.

```
static inline bool proven_alloc_is_valid(proven_allocator_t alloc);
```

목적: 모든 함수 포인터가 널이 아닌지 확인한다.

반환: trait를 호출할 수 있으면 true.

올바른 예:

```
proven_allocator_t heap = proven_heap_allocator();
proven_err_t err = proven_alloc_is_valid(heap) ? PROVEN_OK : PROVEN_ERR_UNSUPPORTED;
(void)err; /* a freestanding build hands back a zero allocator, not a valid one */
```

잘못된 예:

```
proven_allocator_t alloc = {0};
proven_result_mem_mut_t r = alloc.alloc_fn(alloc.ctx, 64, 8); /* wrong: null call */
```

5. 원시 바이트 버퍼

무엇을 위한 것인가

proven_buf_t는 이 장에서 가장 단순한 것이다: 포인터 하나, 길이 하나, 용량 하나. 자기 바이트를 소유하고, 결코 성장하지 않으며, proven_u8str_t와 proven_u16str_t가 이것으로 만들어져 있다.

*텍스트*가 아니라 *바이트*를 원할 때 — 조립 중인 레코드, 소켓에 곧 써 넣을 프레임 — 그리고 최대 크기를 미리 알 때 직접 손을 뻗어라. 내용이 텍스트라면 대신 3장의 문자열 타입을 쓰라. 같은 저장소에 더해 NUL 종료, view, 검색, 포매팅까지 준다.

여기 있는 다른 모든 것과 마찬가지로 allocator를 저장하지 않으므로, proven_buf_destroy는 proven_buf_create에 주었던 것과 같은 것을 받는다.

```
proven_buf_t
typedef struct {
    proven_byte_t *ptr;
    proven_size_t len;
    proven_size_t cap;
} proven_buf_t;
```

필드:

- ptr: 저장소 포인터.
- len: 현재 사용 중인 바이트.
- cap: 할당된 바이트.

```
proven_result_buf_t
typedef struct {
    proven_err_t err;
    proven_buf_t value;
} proven_result_buf_t;
```

버퍼 함수

API	의도	반환
proven_buf_create(alloc, cap)	용량이 0이 아닌 버퍼를 할당한다.	proven_result_buf_t; 유효하지 않은 allocator 또는 용량 0은 PROVEN_ERR_INVALID_ARG를 반환한다.
proven_buf_append(buf, data)	원시 바이트가 들어맞으면 추가한다.	PROVEN_OK 또는 에러.
proven_buf_destroy(alloc, buf)	일치하는 allocator로 버퍼 저장소를 해제한다.	void.

예제:

```
proven_result_buf_t r = proven_buf_create(alloc, 64);
if (!proven_is_ok(r.err)) {
    return;
}
proven_buf_t buf = r.value;

proven_err_t e = proven_buf_append(&buf, (proven_mem_view_t){
    .ptr = (const proven_byte_t *)"abc",
    .size = 3,
});
(void)e; /* PROVEN_ERR_OUT_OF_BOUNDS if it would not fit: the buffer never grows */

proven_buf_destroy(alloc, &buf);
```

6. 예제와 오용 사례

파괴 시 allocator 일치시키기

올바른 예:

```
proven_allocator_t heap = proven_heap_allocator();

proven_result_buf_t r = proven_buf_create(heap, 128);
if (!proven_is_ok(r.err)) {
    return;
}
proven_buf_destroy(heap, &r.value); /* the same allocator that created it */
```

잘못된 예:

```
proven_result_buf_t r = proven_buf_create(heap, 128);
proven_buf_destroy(other_alloc, &r.value); /* wrong: allocator mismatch */
```

Arena의 free는 no-op이다

```
alignas(max_align_t) proven_byte_t storage[1024];
proven_arena_t arena = proven_arena_create((proven_mem_mut_t){
    .ptr = storage,
    .size = sizeof storage,
});
proven_allocator_t a = proven_arena_as_allocator(&arena);

proven_result_mem_mut_t r = proven_arena_alloc(&arena, 32);
if (proven_is_ok(r.err)) {
    /* Legal, and correct to write - but it intentionally reclaims nothing. */
    a.free_fn(a.ctx, r.value.ptr);
}
proven_arena_reset(&arena); /* only this gives the 32 bytes back */
```

Pool 크기 계약

올바른 예:

```
typedef struct Node { int value; } Node;

proven_pool_t pool = {0};
if (!proven_is_ok(proven_pool_init(&pool, alloc, sizeof(Node), alignof(Node), 8))) {
    return;
}
proven_allocator_t node_alloc = proven_pool_as_allocator(&pool);

/* The pool serves exactly the size and alignment it was configured with. */
proven_result_mem_mut_t n = node_alloc.alloc_fn(node_alloc.ctx, sizeof(Node), alignof(Node));
if (proven_is_ok(n.err)) {
    node_alloc.free_fn(node_alloc.ctx, n.value.ptr);
}

/* Anything else is refused with PROVEN_ERR_INVALID_ARG rather than served. */
proven_result_mem_mut_t wrong = node_alloc.alloc_fn(node_alloc.ctx, sizeof(Node) * 2, alignof(Node));
(void)wrong;

proven_pool_destroy(&pool);
```

잘못된 예:

```
node_alloc.alloc_fn(node_alloc.ctx, sizeof(Other), alignof(Other));
/* wrong: PROVEN_ERR_INVALID_ARG - a pool is for one size and one alignment */
```

버퍼 append는 성장하지 않는다

잘못된 예:

```
proven_buf_append(&buf, huge_data); /* returns out-of-bounds; no automatic growth */
```

성장이 필요하면 proven_u8str_append_grow()나 배열을 사용하라.

버퍼 append는 겹침을 허용한다

올바른 예:

```
proven_result_buf_t r = proven_buf_create(alloc, 64);
if (!proven_is_ok(r.err)) {
    return;
}
proven_buf_t buf = r.value;

proven_err_t e = proven_buf_append(&buf, (proven_mem_view_t){
```

```

        .ptr = (const proven_byte_t *)"abcdefgh",
        .size = 8,
    });
    if (proven_is_ok(e)) {
        /* The source view points back into the buffer's own bytes. That is allowed:
         * append moves rather than copies, so the overlap is well defined. */
        e = proven_buf_append(&buf, (proven_mem_view_t){
            .ptr = buf.ptr + 2,
            .size = 4,
        });
    }
}
(void)e;

```

```
proven_buf_destroy(alloc, &buf);
```

잘못된 예:

```
proven_sys_mem_copy(buf.ptr + buf.len, buf.ptr + 2, 4); /* copy is not the public contract here */
```

실전 예제: 호출자가 제공한 메모리 위의 arena

테스트 스위트에서 컴파일되고 실행된다. arena를 손 뺄 수 있게 만드는 bump-and-drop 수명을 보여준다: 할당은 거의 공짜이고, 개별적으로 해제되는 것은 없으며, proven_arena_reset이 전체 영역을 한 번에 회수한다.

```

/*
 * 아레나는 기억을 소유하지 않는다. *여러분* 가진 기억 위로 포인터를 밀고 갈 뿐이다.
 * 거래는 그것이 전부다. 할당은 덧셈 한 번이고, 날개 해제는 아예 없으며, reset 한 번에
 * 전부 되돌려 받는다.
 *
 * 쓸 값어치가 나는 모양은 "밀고 나서 버리기"다. 한 단계 동안 마음껏 할당하고, 단계가
 * 끝나면 reset 하나로 전부 회수한다. 물건마다 장부를 적을 일이 없으니 틀릴 일도 없고,
 * 셀 것도 없다 - 아래의 뒷받침 저장소는 자동 수명을 가진 평범한 배열이다.
 */

int main(void) {
    /* 뒷받침 저장소는 부르는 쪽의 것이다. 넉넉히 정렬해 두어, 첫 바이트부터도 어떤
     * 정렬 요구든 아레나가 맞춰 줄 수 있게 한다. */
    alignas(max_align_t) static proven_byte_t storage[4096];

    proven_arena_t arena = proven_arena_create((proven_mem_mut_t){
        .ptr = storage,
        .size = sizeof storage,
    });

    /* --- 밀기 ----- */
    proven_result_mem_mut_t a = proven_arena_alloc(&arena, 64);
    EXAMPLE_REQUIRE(proven_is_ok(a.err), "64 bytes must fit in a 4 KiB arena");
    EXAMPLE_REQUIRE(a.value.ptr == storage, "the first allocation starts at the backing store");

    /* 타입이 PROVEN_DEFAULT_ALIGNMENT 보다 큰 정렬을 요구하면 그것을 명시한다.
     * 아레나는 거기까지 채워 넘어가므로, 건너뛴 바이트는 reset 전까지 그냥 사라진다. */
    proven_result_mem_mut_t b = proven_arena_alloc_aligned(&arena, 32, 64);
    EXAMPLE_REQUIRE(proven_is_ok(b.err), "an over-aligned block must still fit");
    EXAMPLE_REQUIRE(((uintptr_t)b.value.ptr % 64) == 0, "the block must honour the requested alignment");

    /* --- 다른 API 에 건네는 할당자로서의 아레나 ----- */
    /* proven 에서 proven_allocator_t 를 받는 것이면 무엇이든 아레나로 굴릴 수 있다.
     * 그래서 아래 문자열은 `storage` 안에 산다. */
    proven_allocator_t arena_alloc = proven_arena_as_allocator(&arena);
    EXAMPLE_REQUIRE(proven_alloc_is_valid(arena_alloc), "the arena must expose a usable allocator");
}

```

```

proven_result_u8str_t s = proven_u8str_create(arena_alloc, 32);
EXAMPLE_REQUIRE(proven_is_ok(s.err), "the arena should be able to back a 32-byte string");

proven_err_t err = proven_u8str_append_grow(arena_alloc, &s.value, PROVEN_LIT("scratch line"));
EXAMPLE_REQUIRE(proven_is_ok(err), "appending into an arena-backed string must succeed");

/* 지우는 것은 여전히 옳고 소유 규칙이 요구하는 일이다 - 다만 아레나의 free 는
 * 아무 일도 하지 않으므로 되돌리는 것이 없다. 이것은 누수가 아니다. 바이트는
 * `storage` 의 것이고, 아래의 reset 이 그것을 돌려준다. */
proven_u8str_destroy(arena_alloc, &s.value);

proven_size_t used = arena.offset;
EXAMPLE_REQUIRE(used > 64, "every allocation above came out of the same backing store");

/* --- 버리기 ----- */
/* 한 문장이 64바이트 블록과 정렬된 블록과 문자열을 모두 되돌린다. reset 의 값은
 * 열 개를 할당했던 만 개를 할당했던 같다. */
proven_arena_reset(&arena);
EXAMPLE_REQUIRE(arena.offset == 0, "reset must reclaim every allocation at once");

/* 저장소가 정말 다시 쓰인다는 증거다. 다음 할당이 처음 자리로 돌아온다. reset
 * 이전에 나눠 준 포인터는 이제 전부 매달린 포인터다 - reset 이 공짜인 값이다. */
proven_result_mem_mut_t c = proven_arena_alloc(&arena, 64);
EXAMPLE_REQUIRE(proven_is_ok(c.err), "allocation after reset must succeed");
EXAMPLE_REQUIRE(c.value.ptr == storage, "after reset the arena bumps from the beginning again");

/* --- 바닥나면 죽는 것이 아니라 오류다 ----- */
proven_result_mem_mut_t too_big = proven_arena_alloc(&arena, sizeof storage);
EXAMPLE_REQUIRE(too_big.err == PROVEN_ERR_NOMEM, "an arena cannot grow: it reports NOMEM instead");

printf("arena: %zu bytes used before reset, %zu in use now\n",
       (size_t)used, (size_t)arena.offset);

/* 형식상의 뒷정리. 부르는 쪽이 뒷받침하는 아레나에서는 하는 일이 없지만, 나중에
 * 뒷받침 저장소가 힙이 되더라도 수명이 눈에 보이게 남는다. */
proven_arena_destroy(&arena);
return EXAMPLE_OK();
}

```

실전 예제: 고정 크기 블록을 재활용하는 pool

테스트 스위트에서 컴파일되고 실행된다. 해제된 블록이 재활용 bin에서 곧장 다시 나오는 것과, pool이 무엇을 거부하는지를 보여준다.

```

/*
 * 풀(pool)은 구역이 아니라 갖은 교체를 위한 최적화다. 타입 하나를 위한 것으로,
 * 같은 크기의 블록을 몇 번이고 할당하고 해제하는 자리 - 리스트 노드, 이벤트, 파티클 -
 * 에서 매번 malloc 값을 치르지 않게 한다.
 *
 * 풀은 해제된 블록을 담아 두는 작은 스택("bin")을 갖는다. 해제하면 바탕 할당자로
 * 돌려보내는 대신 bin 에 얹고, 할당하면 거기서 하나를 꺼낸다. 둘 다 O(1) 이고 힙을
 * 건드리지 않는다. 그 재활용이 존재 이유 전부이고, 아래 검사가 그것이 실제로
 * 일어남을 보인다.
 *
 * 소유: 풀은 해제된 블록을 캐시하지만, 자기가 *내준* 블록은 추적하지 않는다. 가져간
 * 블록은 destroy 전에 모두 돌려주어야 한다 - 풀은 자기가 모르는 것을 해제할 수 없다.
 */

```

```

typedef struct {
    int id;
    int score;
}

```

```

} node_t;

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
    EXAMPLE_REQUIRE(proven_alloc_is_valid(heap), "hosted builds have a heap allocator");

    /* 풀은 bin 으로 감당 못 하는 블록을 위한 바탕 할당자를 받고, 자기가 다루는 그 한
     * 타입의 정확한 크기와 정렬을 받는다. 마지막 인자는 재사용을 위해 세워 둘 해제된
     * 블록의 최대 개수다. */
    proven_pool_t pool = {0};
    proven_err_t err = proven_pool_init(&pool, heap, sizeof(node_t), alignof(node_t), 4);
    EXAMPLE_REQUIRE(proven_is_ok(err), "initializing a pool of node_t must succeed");
    if (!proven_is_ok(err)) {
        return 1;
    }

    proven_allocator_t nodes = proven_pool_as_allocator(&pool);

    /* --- 첫 블록: bin 이 비어 있으니 힙에서 온다 ----- */
    proven_result_mem_mut_t first = nodes.alloc_fn(nodes.ctx, sizeof(node_t), alignof(node_t));
    EXAMPLE_REQUIRE(proven_is_ok(first.err), "the pool must be able to serve its own item type");
    if (!proven_is_ok(first.err)) {
        proven_pool_destroy(&pool);
        return 1;
    }

    node_t *n = (node_t *)first.value.ptr;
    *n = (node_t){ .id = 1, .score = 100 };
    void *first_addr = n;

    /* --- 돌려주기: 힙이 아니라 bin 으로 들어간다 ----- */
    nodes.free_fn(nodes.ctx, n);
    EXAMPLE_REQUIRE(pool.bin_len == 1, "a freed block is cached for reuse, not returned to the heap");
    /* 여기서부터 `n` 은 매달린 포인터다. 그 바이트는 다시 풀의 것이다. */

    /* --- 두 번째 블록: 방금 해제한 것이 곧바로 돌아온다 ----- */
    proven_result_mem_mut_t second = nodes.alloc_fn(nodes.ctx, sizeof(node_t), alignof(node_t));
    EXAMPLE_REQUIRE(proven_is_ok(second.err), "allocating from a non-empty bin must succeed");
    EXAMPLE_REQUIRE(second.value.ptr == first_addr, "the recycled block is the one that was freed");
    EXAMPLE_REQUIRE(pool.bin_len == 0, "taking it back out empties the bin");

    /* 재활용된 기억은 0 으로 채워지지 *않는다* - 풀이 남겨 둔 그대로다.
     * 갓 받은 malloc 에 하듯 모든 필드를 초기화할 것. */
    node_t *m = (node_t *)second.value.ptr;
    *m = (node_t){ .id = 2, .score = 50 };
    EXAMPLE_REQUIRE(m->id == 2, "the recycled block is ours to overwrite");

    /* --- 풀 하나는 크기 하나와 정렬 하나만 감당한다 ----- */
    /* 그 밖의 요청은 거부된다. 이것은 범용 할당자가 아니고, 크기가 다른 블록을 조용히
     * 내주지도 않는다. 코드는 PROVEN_ERR_UNSUPPORTED - "내 일이 아니다" - 이지 INVALID_ARG
     * 가 아니다. 후자였다면 "쓰레기를 건넸다"로 읽혀 여러분이 자기 코드에서 버그를 찾아
     * 해매게 만들었을 것이다. */
    proven_result_mem_mut_t wrong = nodes.alloc_fn(nodes.ctx, sizeof(node_t) * 2, alignof(node_t));
    EXAMPLE_REQUIRE(wrong.err == PROVEN_ERR_UNSUPPORTED, "the pool only serves its configured item size");

    /* --- 지우기 전에 살아 있는 블록을 모두 돌려준다 ----- */
    /* proven_pool_destroy 는 bin 에 있는 것과 bin 자체를 해제한다. `m` 은 아직 나가
     * 있으므로, 이 free 를 건너뛰면 그대로 누수가 된다. */
    nodes.free_fn(nodes.ctx, m);

```

```

printf("pool: %zu block(s) cached for reuse at teardown\n", (size_t)pool.bin_len);

proven_pool_destroy(&pool);
return EXAMPLE_OK();
}

```

실전 예제: 시동 시 할당, 제자리에서 늘리기, 할당자 감싸기

앞의 두 예제는 대부분의 코드가 쓰는 방식으로 arena와 pool을 쓴다. 이 예제는 프로그램이 실제 규모가 되었을 때 손이 가는 세 가지를 다룬다.

1. **시동(start-up) 단계의 할당.** 어떤 할당은 실패했을 때 갈 길이 없다. 프로그램이 자기 설정 표조차 얻지 못했다면 남은 할 일이 없다. `proven_arena_alloc_or_panic()`과 `proven_arena_alloc_aligned_or_panic()`은 `result`가 아니라 메모리 블록 자체를 돌려주고, 실패는 패닉 핸들러(panic handler, 회복 불가능한 실패가 났을 때 불리는 함수)로 넘긴다 (1장 6절). 그때 무슨 일이 일어날지는 `proven_set_panic_handler()`로 고른다. 예제는 메시지를 기록하는 핸들러를 설치해 실패를 보여 준 뒤, 기본 핸들러를 되돌린다.
2. **마지막으로 받은 블록 늘리기.** `proven_arena_realloc_aligned()`는 그 블록을 있던 자리에서 늘린다. 가장 최근 블록이 곧 사용 영역의 끝에 있는 블록이기 때문이다. 그보다 앞선 블록은 끝으로 복사된다 — 여전히 옳지만, 더 이상 공짜는 아니다.
3. **할당자(allocator) 감싸기.** 할당자는 함수 포인터 셋과 문맥 포인터 하나이므로, 호출을 세거나 기록하거나 일부러 열 번째 호출을 실패시키는 래퍼(wrapper, 감싸는 코드)는 전달 함수 세 개면 된다. arena 자신의 세 함수 — `proven_arena_alloc_trait()`, `proven_arena_realloc_trait()`, `proven_arena_free_trait()` — 가 공개되어 있는 이유가 바로 이것이다. 여러분의 래퍼는 arena를 다시 구현하는 대신 이들을 부르면 된다. 그러면 `proven_allocator_t`를 받는 라이브러리의 모든 것이, 그 사실을 모른 채 여러분의 래퍼를 지나간다.

```

/*
 * 자기 기억을 소유하는 프로그램이 언젠가 반드시 하게 되는 세 가지, 그리고 그것을 하는
 * 호출들.
 *
 * - 시작할 때 할당하기. 이때 기억이 바닥나는 것은 프로그램이 이어서 갈 수 있는
 *   상황이 아니다. `_or_panic` 계열이 그것을 위한 것이고, 패닉 처리기를 다는 것이
 *   "이어서 갈 수 없다" 가 이 프로그램에서 무슨 뜻인지 정하는 방법이다 - 덮에
 *   걸리는 대신 로그 한 줄과 종료 같은 것으로.
 *
 * - 마지막에 할당한 블록을, 복사 없이 늘리기. 아레나는 그것을 제자리에서 할 수
 *   있다. 마지막에 할당된 블록이 곧 쓰인 구역의 끝에 앉아 있는 그 블록이기 때문이다.
 *
 * - 할당자를 계측하기 - 할당 횟수를 세거나, 시험에서 열 번째를 일부러 실패시키거나 -
 *   재는 대상 코드는 건드리지 않고서. 여기서 할당자는 함수 포인터 셋과 문맥 포인터
 *   하나이므로, 감싸는 일은 전달 함수 셋을 쓰는 일이다. 아레나 자신의 셋이 공개되어
 *   있는 이유가 바로 이것이다 - 여러분의 껍데기가 그것으로 전달한다.
 */

```

```

/* --- 패닉 처리기는 무엇을 위한 것인가 ----- */

```

```

static int g_panics = 0;
static char g_last_panic[128];

```

```

/* 패닉 처리기는 메시지를 받아 프로그램의 운명을 정한다. 기본 처리기는 즉시 덮에
 * 걸리는데, 그것이 운영에서는 옳고 시험에서는 쓸모없다 - 그래서 이 처리기는 메시지를
 * 적어 두고 돌아온다. 돌아오는 것은 패닉 경로를 *일부러 시험할 때만* 허용되고, 그때
 * 패닉한 호출이 돌려준 기억 블록은 써서는 안 된다. */

```

```

static void record_panic(const char *msg) {
    ++g_panics;
    snprintf(g_last_panic, sizeof g_last_panic, "%s", msg);
}

```

```

/* --- 자기를 지나가는 것을 세는 할당자 ----- */

```

```

typedef struct {
    proven_arena_t *arena;
    proven_size_t live_bytes;
    proven_size_t alloc_calls;
    proven_size_t free_calls;
} counting_ctx_t;

/* 셋 각각이 proven_allocator_t 의 필드 하나에 대응한다. 각자 제 장부를 적은 뒤 아레나의
 * 공개된 특성 함수로 전달하므로, 재는 대상이 되는 동작은 아레나를 다시 구현한 것이
 * 아니라 정확히 아레나의 동작이다. */
static proven_result_mem_mut_t counting_alloc(void *ctx, proven_size_t size, proven_size_t align) {
    counting_ctx_t *c = (counting_ctx_t *)ctx;
    proven_result_mem_mut_t r = proven_arena_alloc_trait(c->arena, size, align);
    if (proven_is_ok(r.err)) {
        c->live_bytes += size;
        ++c->alloc_calls;
    }
    return r;
}

static proven_result_mem_mut_t counting_realloc(void *ctx, void *old_ptr, proven_size_t old_size,
                                              proven_size_t new_size, proven_size_t align) {
    counting_ctx_t *c = (counting_ctx_t *)ctx;
    proven_result_mem_mut_t r = proven_arena_realloc_trait(c->arena, old_ptr, old_size, new_size, align);
    if (proven_is_ok(r.err)) {
        c->live_bytes = c->live_bytes - old_size + new_size;
    }
    return r;
}

static void counting_free(void *ctx, void *ptr) {
    counting_ctx_t *c = (counting_ctx_t *)ctx;
    ++c->free_calls;
    proven_arena_free_trait(c->arena, ptr); /* 아레나의 free 는 아무 일도 하지 않는다. 세는 것이 요점이다 */
}

int main(void) {
    alignas(max_align_t) static proven_byte_t storage[1024];
    proven_arena_t arena = proven_arena_create((proven_mem_mut_t){ .ptr = storage, .size = sizeof storage });

    /* --- 1. 실패하면 안 되는 시작 시점 할당 ----- */

    proven_set_panic_handler(record_panic);

    /* 풀어 볼 result 가 없다. 이들은 블록을 곧바로 돌려준다. 부르는 쪽이 손쓸 수 있는
     * 오류가 없기 때문이다. 차이는 그것이 전부다. */
    proven_mem_mut_t table = proven_arena_alloc_or_panic(&arena, 256);
    EXAMPLE_REQUIRE(table.ptr != NULL, "a 256-byte start-up allocation must succeed");
    EXAMPLE_REQUIRE(g_panics == 0, "a successful allocation must not panic");

    /* 기본 경계보다 큰 정렬이 필요한 타입을 위한 정렬 지정 풀 - 여기서는 64바이트
     * 캐시 줄이고, 그것이 흔한 이유다. */
    proven_mem_mut_t cache_line = proven_arena_alloc_aligned_or_panic(&arena, 64, 64);
    EXAMPLE_REQUIRE(((proven_uintptr_t)cache_line.ptr % 64) == 0,
        "the block must start on the boundary that was asked for");
    EXAMPLE_REQUIRE(g_panics == 0, "an over-aligned allocation that fits must not panic either");

    /* 이제 일부러 실패하는 경우다. 아레나가 결코 담을 수 없는 크기를 청한다. 기록하는
     * 처리기를 달아 두었으니 그것을 지켜볼 수 있다. 기본 처리기였다면 프로그램은 여기서
     * 멈췄을 것이고, 그것이 기본 처리기의 목적이다. */

```

```

proven_mem_mut_t impossible = proven_arena_alloc_or_panic(&arena, sizeof storage * 2);
(void)impossible; /* 처리기가 돌아온 뒤, 이 블록은 아무 뜻도 없다 */
EXAMPLE_REQUIRE(g_panics == 1, "exhausting the arena through _or_panic must panic");
EXAMPLE_REQUIRE(g_last_panic[0] != '\0', "the handler receives a message naming the call");
printf("panic handler saw: %s\n", g_last_panic);

/* NULL 을 건네면 덮어 쓰는 기본 처리기가 되돌아온다. 시험용 처리기를 그대로 두면
 * 진짜 실패가 조용한 오염으로 바뀐다. */
proven_set_panic_handler(NULL);

/* --- 2. 가장 최근 블록을 제자리에서 늘리기 ----- */

/* 머리말을 읽고 나서 본문이 짐작보다 길다는 것을 알게 된 파서는, 방금 받은 버퍼를
 * 복사하는 것이 아니라 늘리고 싶어 한다. */
proven_result_mem_mut_t buf = proven_arena_alloc_aligned(&arena, 32, alignof(proven_u32));
EXAMPLE_REQUIRE(proven_is_ok(buf.err), "the initial 32-byte buffer must fit");

proven_size_t before = arena.offset;
proven_result_mem_mut_t grown = proven_arena_realloc_aligned(&arena, buf.value.ptr, 32, 96,
alignof(proven_u32));
EXAMPLE_REQUIRE(proven_is_ok(grown.err), "growing the most recent block must succeed");
EXAMPLE_REQUIRE(grown.value.ptr == buf.value.ptr,
    "the most recent block grows in place: same address, no copy");
EXAMPLE_REQUIRE(arena.offset == before + 64, "only the extra 64 bytes were taken");

/* 제자리 경로는 *마지막에* 할당된 블록에만 열려 있다. 다른 블록을 하나 더 받고 나면
 * 앞의 것은 있던 자리에서 더 늘릴 수 없다 - 아레나는 대신 그것을 끝으로 복사하고,
 * 옛 바이트는 다음 reset 까지 죽은 자리가 된다. 어느 쪽이든 옳다. 다만 공짜가 아닐
 * 뿐이다. */
proven_result_mem_mut_t other = proven_arena_alloc(&arena, 16);
EXAMPLE_REQUIRE(proven_is_ok(other.err), "a second block must fit");
proven_result_mem_mut_t moved = proven_arena_realloc_aligned(&arena, grown.value.ptr, 96, 128,
alignof(proven_u32));
EXAMPLE_REQUIRE(proven_is_ok(moved.err), "growing an older block must still succeed");
EXAMPLE_REQUIRE(moved.value.ptr != grown.value.ptr, "but it is relocated, not extended");

/* --- 3. 세는 꺾대기 뒤의 아레나 ----- */

proven_arena_reset(&arena);

counting_ctx_t counted = { .arena = &arena };
proven_allocator_t alloc = {
    .ctx      = &counted,
    .alloc_fn  = counting_alloc,
    .realloc_fn = counting_realloc,
    .free_fn   = counting_free,
};
EXAMPLE_REQUIRE(proven_alloc_is_valid(alloc), "all three function pointers must be present");

/* 이제 할당자를 받는 라이브러리의 어느 부분이든 그 꺾대기를 지나 돈다. 그것을 알지도
 * 못한 채, 바뀐 것도 없이. */
proven_result_u8str_t s = proven_u8str_create(alloc, 16);
EXAMPLE_REQUIRE(proven_is_ok(s.err), "creating a string through the wrapper must succeed");

proven_err_t err = proven_u8str_append_grow(alloc, &s.value, PROVEN_LIT("a line long enough to need more
room"));
EXAMPLE_REQUIRE(proven_is_ok(err), "appending past the initial capacity must succeed");

proven_u8str_destroy(alloc, &s.value);

```

```

EXAMPLE_REQUIRE(counted.alloc_calls >= 1, "the wrapper saw the string being created");
EXAMPLE_REQUIRE(counted.free_calls >= 1, "and saw it being destroyed");
printf("counting allocator: %zu alloc call(s), %zu free call(s), %zu bytes handed out\n",
       (size_t)counted.alloc_calls, (size_t)counted.free_calls, (size_t)counted.live_bytes);

proven_arena_destroy(&arena);
return EXAMPLE_OK();
}

```

반례 — 시험용 패닉 핸들러를 설치한 채 두는 경우:

```

proven_set_panic_handler(record_panic); /* returns instead of stopping */
/* ... the rest of the program ... */ /* wrong: no restore */

```

돌아오는 핸들러는 이후의 모든 메모리 부족 패닉을, 널 블록을 돌려주고 계속 진행하는 호출로 바꾼다. 시험할 코드 주위에서만 설치하고, proven_set_panic_handler(NULL)로 기본값을 되돌린다.

반례 — 돌아온 핸들러 뒤에서 그 블록을 쓰는 경우:

```

proven_mem_mut_t m = proven_arena_alloc_or_panic(&arena, huge);
memset(m.ptr, 0, huge); /* wrong: m.ptr is null if the handler returned */

```

반례 — 셋 중 하나를 빠뜨린 래퍼:

```

proven_allocator_t alloc = { .ctx = &counted, .alloc_fn = counting_alloc }; /* wrong */

```

proven_alloc_is_valid()는 이것에 false를 돌려주고, 이 할당자를 통한 첫 realloc이나 destroy는 널 포인터를 호출한다. 둘은 그저 전달만 하더라도 셋을 모두 채운다.

3장: 문자열, 포매팅, 스캐닝

Part II — 모든 프로그램이 쓰는 어휘. 선행 장: 1장과 2장. 이 장을 마치면 NUL 종료자가 프로그램의 운명을 결정하지 않는 방식으로 텍스트를 담을 수 있고, 오버플로를 거부하는 문자열을 만들 수 있으며, 일상적인 경우의 포매팅과 파싱을 할 수 있다.

이 장은 `ustr.h`, `u16str.h`, `fmt.h`, `scan.h`를 다룬다. 텍스트 자료의 튜토리얼 절반으로서, 매일 만나는 사례로 포매팅과 스캐너를 소개한다. 8장이 레퍼런스 절반이다 — 전체 문법, 모든 인자 생성자, 스캐너의 예러와 복구 규칙을 담는다. 이 장을 먼저 읽으라.

목차

1. U8 문자열과 view
2. U16 문자열과 view
3. 포매팅
4. 스캐닝
5. 예제와 오용 사례

1. U8 문자열과 view

문제: C 문자열은 자기 길이를 모른다

C 문자열은 포인터이고, 어디서 끝나는지는 메모리 어딘가의 0 바이트가 결정한다. 문자열당 1바이트를 아끼려고 1972년에 내린 그 결정 하나가 놀랄 만큼 많은 피해의 배후에 있다:

- **길이 구하기가 검색이다.** `strlen`은 문자열을 훑는다. 반복마다 `strlen(s)`를 검사하는 루프는 제곱 시간이고, 겹보기에는 평범한 코드처럼 보인다.
- **텍스트가 0 바이트를 담을 수 없다.** 그래서 C 문자열은 UTF-16 버퍼도, 프로토콜 프레임도, 파일의 일부도, 어떤 이진 데이터도 담을 수 없다 — 문자열 타입과 바이트 타입은 다른 것인데도 언어는 아닌 척한다.
- **종료자가 없는 것은 탐지할 수 없다.** 자리가 없는 버퍼로 `strcpy`하면 스택 프레임 어딘가에서 0을 찾을 때까지 써 나간다. 아무것도 그것을 보고하지 않는다. 이는 이 언어 역사상 가장 많이 악용된 버그 부류다.
- **문자열의 일부를 저렴하게 가리킬 수 없다.** "이 줄의 세 번째 필드"라는 말은 그 바이트를 복사해 내거나, 원본 위에 0을 써서 원본을 망가뜨리거나 둘 중 하나를 뜻한다. `strtok`은 두 번째를 골랐고, 그래서 입력을 변형시키며 중첩할 수 없다.

전통적인 땀질인 `strncpy`는 항상 NUL로 종료해 주지는 않는다 — 그러니 그 "안전한" 함수가 문자열이 아닌 문자열을 만들어 낼 수 있다.

이 라이브러리가 대신 하는 것

두 개의 타입이 있고, 그 둘의 차이는 소유권이다:

- **`proven_u8str_view_t` — borrowed.** 포인터와 크기가 함께, 다른 누군가가 소유한 바이트를 가리킨다. 복사는 공짜다. 아무것도 할당하지 않고 아무것도 파괴하지 않으며, 소유자가 사라지면 함께 유효하지 않게 된다. 함수에 전달하는 것이 이것이다.
- **`proven_u8str_t` — owned.** 자기 저장소와 용량을 가지고, NUL 종료를 대신 유지해 주므로 `proven_u8str_as_cstr`이 여전히 NUL을 원하는 `libc` 함수에 바이트를 건네줄 수 있다. 할당자(allocator)로 만들었으니, 같은 것으로 파괴한다.

둘 다 문자가 아니라 **바이트**를 센다. "한"은 UTF-8에서 3바이트이고 1문자인데, 이 라이브러리는 3이라고 말할 것이다. 그것이 라이브러리가 아는 것이기 때문이다. 여기서 텍스트는 바이트의 나열이며, 그 바이트를 문자로 해석하는 일은 이 라이브러리에 없는 Unicode 계층의 몫이다.

뷰(view)는 자기 길이를 지니고 다니므로, NUL 종료가 어렵게 만들었던 모든 것이 평범해진다: 길이는 필드이고, 텍스트가 0 바이트를 담을 수 있으며, 부분 범위는 복사 없이 같은 메모리를 가리키는 view이고, 들어가지 않을 쓰기는 수행되는 대신 거부된다.

잘못된 예 — view를 C 문자열처럼 다루기:

```
proven_u8str_view_t v = proven_u8str_view_slice(line, 4, 8); /* a field inside a line */
printf("%s\n", (const char *)v.ptr); /* wrong: no terminator, prints until it finds a zero */
```

view는 의도적으로 NUL 종료되지 않는다: 보통 다른 누군가의 버퍼 한가운데를 가리키며, 거기에 종료자를 쓰면 다음 필드를 손상시킬 것이다. 크기를 받는 proven_println("{", PROVEN_ARG(v))를 쓰거나, 먼저 소유(owned) 문자열로 복사하라.

구조체

```
typedef struct {
    const proven_byte_t *ptr;
    proven_size_t size;
} proven_u8str_view_t;
```

```
typedef struct {
    proven_byte_t *ptr;
    proven_size_t size;
} proven_u8str_mut_t;
```

```
typedef struct {
    proven_buf_t internal;
    bool        borrowed;
} proven_u8str_t;
```

```
typedef struct {
    proven_err_t err;
    proven_u8str_t value;
} proven_result_u8str_t;
```

```
typedef struct {
    proven_err_t err;
    const char *value;
} proven_result_cstr_t;
```

의도:

- proven_u8str_view_t: 빌려 쓰는(borrowed) 읽기 전용 바이트 문자열 view.
- proven_u8str_mut_t: borrowed 가변 바이트 문자열 view.
- proven_u8str_t: NUL로 종료되는 owned 문자열.
- proven_result_u8str_t: owned 문자열용 result 래퍼.
- proven_result_cstr_t: 할당된 NUL 종료 C 문자열용 result 래퍼.

내부 레이아웃

view는 보이는 그대로다 — 포인터 하나와 바이트 개수 하나:

```
typedef struct { const proven_byte_t *ptr; proven_size_t size; } proven_u8str_view_t;
typedef struct { proven_byte_t *ptr;          proven_size_t size; } proven_u8str_mut_t;
```

owned 문자열은 proven_buf_t(2장의 고정 용량 바이트 버퍼)에 플래그 하나를 더해 감싼다:

```
typedef struct {
    proven_buf_t internal; /* the bytes + length + capacity; always keeps room for a NUL */
    bool        borrowed; /* false = allocator-owned (default); true = wraps caller memory */
} proven_u8str_t;
```

- internal.len이 바이트 길이다(proven_buf_t는 ptr / len / cap이며 — size 멤버는 없다). 문자열이 유효한 동안 internal.len 위치의 바이트는 항상 NUL 종료자이므로, proven_u8str_as_cstr()은 O(1)이다.
- borrowed는 0으로 초기화된 핸들에서 false이므로, allocator가 소유하는 문자열이 안전한 기본값이다. 이 값은 여러분이 소유한 [buf, buf+cap)을 감싸는 proven_u8str_borrow에 의해서만 true가 된다: 그때는 성장 연산이 재할당을 거부하고(PROVEN_ERR_OUT_OF_BOUNDS), proven_u8str_destroy는 no-op이 된다.

- 문자열을 바꾸려고 이 필드들을 직접 읽거나 쓰지 **말라**. 아래의 함수를 사용하라. 길이를 얻으려고 `internal.len`을 읽는 것은 괜찮지만, `proven_u8str_as_view()`를 선호하라.

반례 — borrowed 문자열을 owned 문자열처럼 다루기:

```
proven_byte_t stack[8];
proven_u8str_t s = proven_u8str_borrow(stack, sizeof stack); /* borrowed */

/* This would have to reallocate caller memory, so it refuses: the call returns
   PROVEN_ERR_OUT_OF_BOUNDS and `stack` is left exactly as it was. A borrowed
   string never silently escapes to the heap. */
proven_err_t e = proven_u8str_append_grow(&alloc, &s, PROVEN_LIT("far too long for eight bytes"));
(void)e; /* == PROVEN_ERR_OUT_OF_BOUNDS */

/* destroy is a no-op here: `stack` is yours, and the library will not free it.
   Writing it anyway is correct, and it keeps teardown code uniform. */
proven_u8str_destroy(&alloc, &s);
```

매크로

매크로	의도	비고
PROVEN_LIT(s)	문자열 리터럴로부터 <code>proven_u8str_view_t</code> 를 만든다.	문자열 리터럴에만 사용하라.
PROVEN_LIT_INIT(s)	리터럴 view의 초기화자 형태.	집합체 초기화에 사용하라.
PROVEN_INDEX_NOT_FOUND	find 함수가 반환하는 센티널.	(<code>proven_size_t</code>)-1과 같다.

U8 함수

API	의도	반환
<code>proven_u8str_create(alloc, limit)</code>	limit 바이트에 NUL을 더한 용량을 가진 빈 owned 문자열을 만든다.	<code>proven_result_u8str_t</code> .
<code>proven_u8str_create_from_view(alloc, view)</code>	view를 새 owned 문자열로 복사한다.	<code>proven_result_u8str_t</code> .
<code>proven_u8str_borrow(buf, cap)</code>	호출자가 소유한 <code>[buf, buf+cap)</code> 을 고정 용량 문자열로 감싼다(할당 없음). 고정 용량 연산과 <code>append_fmt</code> 는 동작하고, 성장 연산은 호출자 메모리 재할당을 거부하며, <code>destroy</code> 는 no-op이다. <code>cap</code> 은 NUL을 포함한다.	<code>proven_u8str_t</code> .
<code>proven_u8str_reset(str)</code>	버퍼/용량은 재사용을 위해 유지한 채 빈 상태로 자른다(owned든 borrowed든).	<code>proven_err_t</code> .
<code>proven_u8str_is_valid(str)</code>	공개 문자열 불변식을 검증한다.	<code>bool</code> .
<code>proven_u8str_reserve(alloc, str, new_cap)</code>	내부 용량이 최소 <code>new_cap</code> 바이트가 되도록 보장한다. (borrowed 문자열은 성장할 수 없다.)	<code>proven_err_t</code> .
<code>proven_u8str_append(str, data)</code>	원자적 고정 용량 append.	PROVEN_OK 또는 PROVEN_ERR_OUT_OF_BOUNDS.
<code>proven_u8str_append_partial(str, data)</code>	잘라내는 append.	<code>proven_result_size_t</code> ; value는 쓰인 바이트 수.

proven_u8str_append_grow(alloc, str, data)	원자적 성장 가능 append.	proven_err_t.
proven_u8str_append_byte(alloc, str, b)	바이트 하나를 append하며, 필요하면 성장한다.	proven_err_t.
proven_u8str_replace_at(str, index, old_len, data)	정확한 바이트 범위를 고정 용량 의미로 치환한다.	proven_err_t.
proven_u8str_replace_at_grow(alloc, str, index, old_len, data)	replace_at과 같지만, 편집이 들어가지 않을 때 실패하는 대신 버퍼를 성장시킨다(2배씩).	proven_err_t; 할당 실패 시 문자열은 변경되지 않는다.
proven_u8str_insert(str, index, data)	고정 용량 의미로 index에 바이트를 삽입한다.	proven_err_t.
proven_u8str_insert_grow(alloc, str, index, data)	insert와 같지만 필요할 때 버퍼를 성장시킨다. 수동 reserve가 필요 없다.	proven_err_t; 할당 실패 시 문자열은 변경되지 않는다.
proven_u8str_remove(str, index, len)	바이트 범위를 제거한다.	proven_err_t.
proven_u8str_replace_first(str, start_offset, target, replacement)	start 이후 처음 일치하는 target을 치환한다.	찾지 못해도 PROVEN_OK.
proven_u8str_view_find(haystack, start_offset, needle)	바이트 부분 문자열의 첫 등장을 찾는다(어떤 바이트 값이든 올바르게 처리하며, NUL 종료를 요구하지 않는다).	인덱스 또는 PROVEN_INDEX_NOT_FOUND.
proven_u8str_view_starts_with(str, prefix)	접두사 검사.	int 진리값.
proven_u8str_view_ends_with(str, suffix)	접미사 검사.	int 진리값.
proven_u8str_view_slice(str, index, len)	잘라 맞춘(clamped) 부분 view를 반환한다.	proven_u8str_view_t.
proven_u8str_as_cstr(str)	내부의 NUL 종료 포인터를 반환한다.	const char *; 성장/파괴에 의해 무효화된다.
proven_mem_view_from_u8(view)	U8 view를 바이트 view로 변환한다.	proven_mem_view_t.
proven_u8str_view_to_cstr(view, alloc)	임의의 view로부터 NUL 종료 C 문자열을 할당한다.	proven_result_cstr_t; 호출자가 allocator로 해제한다.
proven_cstr_len(s)	NUL까지의 바이트를 센다.	proven_size_t.
proven_u8str_view_from_cstr(s)	신뢰할 수 있는 NUL 종료 문자열로부터 view를 만든다.	널 포인터에는 빈 view.
proven_u8str_view_eq(a, b)	바이트 동등성 검사.	int 진리값.
proven_u8str_destroy(alloc, str)	일치하는 allocator로 owned 저장소를 해제한다.	void.
proven_u8str_as_view(str)	현재 내용을 borrow한다.	proven_u8str_view_t.

기본 U8 예제

```

proven_result_u8str_t r = proven_u8str_create_from_view(alloc, PROVEN_LIT("log"));
if (!proven_is_ok(r.err)) {
    return;
}
proven_u8str_t s = r.value;

```

```

proven_err_t e = proven_u8str_append_grow(alloc, &s, PROVEN_LIT(": ready"));
if (!proven_is_ok(e)) {
    proven_u8str_destroy(alloc, &s);
    return;
}

/* Valid until the next growing call: as_cstr points into the string's storage. */
const char *cstr = proven_u8str_as_cstr(&s);
(void)cstr;

proven_u8str_destroy(alloc, &s);

```

2. U16 문자열과 view

두 번째 문자열 타입이 아예 존재하는 이유

UTF-8이 옳은 기본값이고 이 라이브러리는 거기에 전념한다. proven_u16str_t는 단 하나의 이유로 존재한다: **Windows API가 UTF-16이기 때문이다.** 모든 "와이드" 진입점 — CreateFileW, GetEnvironmentVariableW, W 계열 전체 — 은 wchar_t *를 받고, Windows에서 그것은 16비트다. 그 API들과 대화하는 라이브러리에는 그들의 코드 유닛을 바이트인 척하지 않고 답을 타입이 필요하다.

그러니 이것은 경계 타입이다. UTF-16 API에 닿는 곳에서 이것을 쓰고, 그 밖의 모든 곳에서는 proven_u8str_t를 쓴다. 이 타입은 의도적으로 작다 — create, destroy, append, 길이 — 프로그램이 이 타입으로 사고하도록 만든 것이 아니기 때문이다.

불들어 둘 것이 둘 있다. 모든 UTF-16 버그의 근원이기 때문이다:

- **코드 유닛은 문자가 아니다.** UTF-16은 Basic Multilingual Plane 밖의 것을 *서로게이트 쌍*으로 인코딩한다 — 한 문자를 뜻하는 두 개의 proven_u16 값이다. 이모지는 코드 유닛 두 개다. 그 사이를 잘라내면 짝 없는 서로게이트가 나오고, 그것은 유효한 UTF-16이 아니다.
- **여기서 size는 바이트가 아니라 코드 유닛을 센다.** proven_u16str_view_t.size는 proven_u16 값의 개수다. 밑에 깔린 proven_buf_t는 바이트를 추적하며, 그래서 proven_u16str_len이 나눗셈을 한다. 이 두 단위를 섞는 것이 이 타입에서 가장 흔한 실수다.

이 라이브러리에는 UTF-8과 UTF-16 사이의 변환이 없다. 이는 진짜 빈틈이며, 잊어버린 것이 아니라 의도적이다: 올바른 변환이란 유효하지 않은 입력, 짝 없는 서로게이트, 과장 인코딩을 어떻게 할지 결정하는 일이고, 그것은 Unicode 계층의 몫이다. 오늘날 proven_u16str_t는 u"..." 리터럴이나 이미 가지고 있는 코드 유닛으로부터 만든다.

PROVEN_NO_U16STR이 정의되면 U16 API는 제외되며, 이는 프리스탠딩(freestanding) 빌드의 기본값이다 — 베틀어메탈 타깃에는 대화할 Windows API가 없다.

잘못된 예 — 코드 유닛 하나가 문자 하나라고 가정하기:

```

proven_u16str_view_t v = ...; /* text containing an emoji */
proven_size_t chars = proven_u16str_len(&s); /* wrong: that is code units */

```

구조체

```

typedef struct {
    const proven_u16 *ptr;
    proven_size_t size;
} proven_u16str_view_t;

```

```

typedef struct {
    proven_buf_t internal;
} proven_u16str_t;

```

```

typedef struct {
    proven_err_t err;
}

```

```
    proven_u16str_t value;
} proven_result_u16str_t;
```

매크로

매크로	의도
PROVEN_U16_LIT(s)	UTF-16 리터럴 표현식 u"... "로부터 proven_u16str_view_t를 만든다.

U16 함수

API	의도	반환
proven_u16str_create(alloc, unit_limit)	빈 owned U16 문자열을 만든다.	proven_result_u16str_t.
proven_u16str_create_from_view(alloc, view)	U16 view를 owned 문자열로 복사한다.	proven_result_u16str_t.
proven_u16str_destroy(alloc, str)	owned U16 저장소를 해제한다.	void.
proven_u16str_append(str, data)	원자적 고정 용량 append.	proven_err_t.
proven_u16str_append_partial(str, data)	잘라내는 append.	proven_result_size_t.
proven_u16str_append_grow(alloc, str, data)	원자적 성장 가능 append.	proven_err_t.
proven_u16str_as_ptr(str)	내부 proven_u16 포인터를 반환한다.	const proven_u16 *.
proven_u16str_len(str)	코드 유닛 단위 길이를 반환한다.	proven_size_t.

예제:

```
#ifndef PROVEN_NO_U16STR
proven_result_u16str_t r =
    proven_u16str_create_from_view(alloc, PROVEN_U16_LIT("hello"));
if (!proven_is_ok(r.err)) {
    return;
}
proven_u16str_t s = r.value;

(void)proven_u16str_append_grow(alloc, &s, PROVEN_U16_LIT(" world"));

/* Length is in code units, not characters: "hello world" is 11 units. */
proven_size_t units = proven_u16str_len(&s);
(void)units;

proven_u16str_destroy(alloc, &s);
#endif
```

참고: proven_u16은 코드 유닛이지, 반드시 하나의 완전한 Unicode 문자인 것은 아니다. UTF-16 서로게이트 쌍은 코드 유닛 두 개를 쓴다.

3. 포매팅

문제: printf는 타입을 두 번 듣는다

printf("%d", x)는 x의 타입을 두 번 말한다 — 한 번은 포맷 문자열에서, 한 번은 x를 전달하면서 — 그리고 그 둘이 일치하는지는 아무것도 검사하지 않는다. 가변 인자는 타입을 지우므로, 함수는 호출 규약이 남긴 바이트가 무엇이든 포맷이 요구한 모양으로 읽는다:

```
printf("%d\n", 3.0);    /* wrong: reads a double's bytes as an int */
printf("%s\n", 42);     /* wrong: dereferences 42 as a pointer */
printf("%d %d\n", 1);   /* wrong: reads an argument that was never passed */
```

셋 다 컴파일된다. 현대 컴파일러는 포맷이 리터럴일 때 경고해 주는데, 그건 포맷이 변수가 되기 전까지의 이야기다 — 그리고 그때 여러분에게는 요청만 하면 임의의 스택 메모리를 읽어 주는 함수가 생기며, 이는 자기 이름까지 붙은 취약점 부류다.

두 번째 문제는 출력이 어디로 가느냐다. `sprintf`는 자기가 크기를 모르는 버퍼에 쓴다. `snprintf`는 크기를 받고 나서 **잘라내며**, 썼을 뻔한 길이를 반환한다 — 그러니 비교하는 것을 잊은 호출자는 조용히 짧아진 경로나 명령이나 식별자를 얻는다.

이 라이브러리가 대신 하는 것

자리표시자 안에는 타입이 없다. `{}`는 "여기에 값이 온다"고 말하고, 타입은 인자에서 오며 컴파일 시간에 결정된다:

```
proven_printf("{ scored %f}", PROVEN_ARG(name), PROVEN_ARG(score));
```

`PROVEN_ARG`는 `_Generic` 디스패치다 — 컴파일러가 인자의 정적 타입에 맞는 생성자를 고른다. 포맷은 결코 타입을 말하지 않으므로 포맷과 인자가 어긋나는 일은 애초에 불가능하다. : 뒤의 스펙이 제어하는 것은 표현 — 폭, 채움, 정렬, 정밀도, 진법 — 이지 결코 해석이 아니다.

목적지는 크기를 아는 객체이며, 고정 용량 형태는 잘라내는 대신 거부한다: `proven_u8str_append_fmt`은 `PROVEN_ERR_OUT_OF_BOUNDS`로 실패하고 아무것도 쓰지 않으며, `proven_u8str_append_fmt_grow`는 allocator를 받아 성장한다. 어느 쪽을 호출했는지는 호출 그 자체에 드러난다.

여러분의 타입도 여기에 참여할 수 있다. `PROVEN_ARG_OF(&obj, render_fn)`은 여러분이 정의한 타입이 다른 모든 것처럼 `{}`로 출력되게 해 준다 — 확장 지점은 이름 레지스트리가 아니라 컴파일 시간의 타입 있는 지점이다.

8장 § 5.1이 방법을 보여준다.

비용을 그대로 말하자면: 인자마다 `PROVEN_ARG`를 두르는 것은 `%d`보다 타이핑이 많고, 이 포맷 언어는 손에 익은 그 언어가 아니다. 그 대가로 사는 것은 이 절 첫머리의 버그 부류를 아예 쓸 수 없게 된다는 것이다.

포매터는 `proven_u8str_t`나 PAL 기반 스트림에 쓴다. `{}` 자리표시자, `{1}` 같은 명시적 인덱스, 이스케이프된 중괄호 `{{}`와 `}}`, 그리고 `{:0>5}`, `{:*^10}`, `{:.<10}` 같은 폭/정렬 스펙을 갖춘 작은 구조적 포맷 언어를 사용한다.

잘못된 예 — 스펙이 타입을 고른다고 가정하기:

```
proven_printf("{:d}", PROVEN_ARG(3.5)); /* wrong: the spec formats, it does not convert */
```

구조체와 열거형

```
typedef struct {
    proven_err_t err;
    proven_size_t written;
    proven_size_t required;
} proven_fmt_result_t;
```

필드:

- `err`: 결과 코드.
- `written`: 실제로 쓰인 바이트.
- `required`: 전체 출력에 필요한 바이트.

`proven_arg_type_t` 변종:

- `PROVEN_ARG_NONE`
- `PROVEN_ARG_I32`
- `PROVEN_ARG_U32`
- `PROVEN_ARG_I64`
- `PROVEN_ARG_U64`
- `PROVEN_ARG_F64` — `PROVEN_FMT_NO_FLOAT`가 정의되지 않은 경우
- `PROVEN_ARG_CSTR`
- `PROVEN_ARG_STR_VIEW`
- `PROVEN_ARG_DATETIME`

- PROVEN_ARG_PTR
- PROVEN_ARG_FN

```
typedef struct {
    proven_arg_type_t type;
    union {
        proven_i32 i32;
        proven_u32 u32;
        proven_i64 i64;
        proven_u64 u64;
        double f64;
        const char *cstr;
        proven_u8str_view_t str_view;
        proven_datetime_t datetime;
        const void *ptr;
        void (*fn)(void);
    } value;
} proven_arg_t;
```

포맷 인자 생성자

API	의도
proven_arg_none()	내부 센티널 값.
proven_arg_i32(v), proven_arg_u32(v)	정수 인자.
proven_arg_i64(v), proven_arg_u64(v)	넓은 정수 인자.
proven_arg_f64(v)	부동소수점 인자 — float 포매팅이 비활성화되지 않은 경우.
proven_arg_cstr(v)	신뢰할 수 있고 살아 있는 NUL 종료 C 문자열.
proven_arg_cstr_n(v, max_len)	경계가 있는 C 문자열 인자; max_len까지만 NUL을 탐색한다.
proven_arg_str_view(v)	borrowed 문자열 view 인자.
proven_arg_datetime(v)	datetime 인자.
proven_arg_ptr(v)	객체 포인터 인자.
proven_arg_fn(v)	함수 포인터 인자.
proven_arg_ucstr(v)	unsigned char C 문자열 헬퍼.
proven_arg_identity(v)	기존 proven_arg_t를 그대로 통과시킨다.

포맷 매크로

매크로	의도
PROVEN_ARG(x)	지원되는 인자 타입에 대한 _Generic 선택자.
PROVEN_ARG_FN(f)	함수 포인터 포매팅 헬퍼.
PROVEN_ARG_CSTR_N(v, max_len)	경계가 있는 C 문자열 헬퍼.
proven_u8str_append_fmt(str, fmt, ...)	원자적 고정 용량 포매팅.
proven_u8str_append_fmt_trunc(str, fmt, ...)	최선을 다하는 잘라내기 포매팅.
proven_u8str_append_fmt_grow(alloc, str, fmt, ...)	원자적 성장 가능 포매팅.
proven_u8str_append_fmt_with_scratch(alloc, str, fmt, scratch, ...)	임시 패치 할당에 임시 작업용(scratch) allocator를 쓰는 성장 가능 포매팅.
PROVEN_FMT_IS_OK(res)	proven_fmt_result_t를 검사한다.

포매팅 엔진

```
proven_fmt_result_t proven_u8str_fmt_internal(
    proven_allocator_t alloc,
```

```

    proven_u8str_t *str,
    bool trunc,
    const char *fmt,
    proven_allocator_t scratch,
    const proven_arg_t *args,
    proven_size_t args_count
);

```

일반 사용자 코드는 이 내부 엔진 대신 매크로를 호출해야 한다. 엔진은 인덱스 0에 선두 `proven_arg_none()` 센티널이 있을 것을 기대한다.

사용되지 않는 여분의 포맷 인자는 `PROVEN_ERR_INVALID_ARG`를 반환한다.

예제:

```

proven_result_u8str_t r = proven_u8str_create(alloc, 8);
if (!proven_is_ok(r.err)) {
    return;
}
proven_u8str_t s = r.value;

/* The target is only 8 bytes; the _grow form reallocates rather than truncate. */
proven_fmt_result_t fr = proven_u8str_append_fmt_grow(
    alloc,
    &s,
    "name={ } score={:0>4}",
    PROVEN_ARG(PROVEN_LIT("ada")),
    PROVEN_ARG(42)
);
if (!PROVEN_FMT_IS_OK(fr)) {
    proven_u8str_destroy(alloc, &s);
    return;
}
/* s == "name=ada score=0042" */

proven_u8str_destroy(alloc, &s);

```

4. 스캐닝

문제: `scanf`는 어디서 멈췄는지 말해 주지 않는다

입력을 파싱하는 것은 포매팅의 거울이고, `libc`의 답은 더 나쁘다. `sscanf`는 *몇 개의 필드를 채웠는지*만 반환하고 그 외에는 아무것도 주지 않는다 — 어느 것이 실패했는지도, 어디까지 갔는지도, 왜인지도:

```

int n = sscanf(line, "%d %d %d", &a, &b, &c);
if (n != 3) { /* wrong: which field? at what offset? was it malformed or missing? */ }

```

세 번째 필드가 잘못된 형식이라면, 여러분이 아는 것은 돌을 얻었다는 것뿐이다. 사용자에게 위치를 보고할 수 없고, 잘못된 레코드를 건너뛰고 재개할 수 없으며, "입력이 떨어졌다"와 "숫자가 아닌 무언가를 찾았다"를 구별할 수 없다. 그리고 `char *`로의 `%s`는 `strcpy`와 똑같은 무경계 쓰기 문제를 가지며, 이번에는 입력이 프로그램 바깥에서 온다.

그리고 `strtol`이 있는데, 그 계약은 먼저 `errno`를 지우고, 나중에 검사하고, "숫자가 하나도 없음"을 탐지하려고 `endptr`를 입력과 비교할 것을 요구한다 — 변환 하나에 제대로 해내야 할 것이 셋이다.

이 라이브러리가 대신 하는 것

커서는 여러분의 것이다. `proven_scan_t`는 파싱 중인 view와 그 안의 오프셋을 담는다. 각 스캔은 커서에서 읽고 성공하면 커서를 앞으로 옮긴다. 커서는 여러분이 읽을 수 있는 필드이므로, 파싱이 정확히 어디서 멈췄는지 항상 알 수 있다 — 그것이 사용자에게 보여줄 위치다.

각 스캔은 `result`를 반환한다. `proven_scan_i64`는 `{err, val}`을 돌려주므로, "숫자가 아님", "범위를 벗어남", "입력의 끝"이 하나의 빠진 필드가 아니라 서로 다른 에러가 된다.

실패는 커서를 되돌린다 — 원시 스캐너들에 한해서. 실패한 `proven_scan_i64`는 커서를 있던 자리에 남기므로, 같은 위치에서 다른 것을 시도할 수 있다. 그것이 짐작 대신 복구를 가능하게 하는 것이다.

이것에 기대기 전에 알아 둘 것 하나: 구조적 스캔(`proven_scan_fmt`, {} 형태)은 필드 전체에 걸쳐 트랜잭션적이지 **않다**. 세 번째 자리표시자가 실패하면 앞의 두 목적지에는 이미 쓰여 있다. [8장 §11.1](#)이 에러 코드와 복구 패턴을 온전히 다룬다.

스캐너는 borrow된 `proven_u8str_view_t`로부터 파싱한다. 커서가 진행 상황을 추적한다.

잘못된 예 — 부분적인 구조적 스캔을 아무 일도 없었던 것처럼 다루기:

```
proven_err_t e = proven_scan_fmt(&sc, "{} {} {}", ...);
if (!proven_is_ok(e)) {
    /* wrong: destinations for the fields that DID parse have already been written */
}
```

Result 구조체

```
typedef struct { proven_err_t err; proven_i64 val; } proven_result_i64_t;
typedef struct { proven_err_t err; proven_u64 val; } proven_result_u64_t;
typedef struct { proven_err_t err; double val; } proven_result_f64_t;
typedef struct { proven_err_t err; proven_u8str_view_t val; } proven_result_u8str_view_t;
```

```
proven_scan_t
typedef struct {
    proven_u8str_view_t view;
    proven_size_t cursor;
} proven_scan_t;
```

목적: 입력과 현재 파싱 위치를 담는다. `proven_scan_init()`은 유효하지 않은 비어 있지 않은 널 view를 빈 view로 정규화한다.

스캐너 함수

API	의도	반환
<code>proven_scan_init(view)</code>	view로부터 스캐너를 만든다.	<code>proven_scan_t</code> .
<code>proven_scan_skip_whitespace(scan)</code>	공백을 지나 진행한다.	<code>void</code> .
<code>proven_scan_i64(scan)</code>	부호 있는 64비트 정수를 파싱한다.	<code>proven_result_i64_t</code> .
<code>proven_scan_u64(scan)</code>	부호 없는 64비트 정수를 파싱한다.	<code>proven_result_u64_t</code> .
<code>proven_scan_f64(scan)</code>	부동소수점 값을 파싱한다.	<code>proven_result_f64_t</code> .
<code>proven_parse_double_ascii(view)</code>	공백을 건너뛰지 않고 로케일 독립적인 ASCII float 토큰 하나를 파싱한다.	<code>proven_parse_double_result_t</code> .
<code>proven_parse_f64_ascii(view)</code>	같은 로케일 독립 binary64 파서에 대한 호환 별칭.	<code>proven_parse_f64_result_t</code> .
<code>proven_strtod(npstr, endptr)</code>	공백 건너뛰기와 <code>endptr</code> 보고를 갖춘 <code>strtod</code> 스타일 토큰 하나를 파싱한다.	<code>double</code> .
<code>proven_scan_str(scan)</code>	공백으로 구분된 토큰을 입력에 대한 view로 파싱한다.	<code>proven_result_u8str_view_t</code> .
<code>proven_scan_skip_until(scan, target)</code>	<code>target</code> 을 찾으면 커서를 그쪽으로 옮긴다.	<code>proven_err_t</code> .
<code>proven_scan_skip_until_number(scan)</code>	다음으로 숫자처럼 보이는 위치로 커서를 옮긴다.	<code>void</code> .

스캔 인자 타입

`proven_scan_arg_type_t`는 `proven_scan_arg_t`에 저장된 목적지 종류를 식별한다. `PROVEN_SCAN_ARG(&x)`는 다음에 대한 포인터를 지원한다:

- short, unsigned short
- int, unsigned int
- long, unsigned long
- long long, unsigned long long
- double
- proven_u8str_view_t

네이티브 목적지 생성자:

- proven_scan_arg_short, proven_scan_arg_ushort
- proven_scan_arg_int, proven_scan_arg_uint
- proven_scan_arg_long, proven_scan_arg_ulong
- proven_scan_arg_llong, proven_scan_arg_ullong

명시적 고정 폭 및 유틸리티 헬퍼:

- proven_scan_arg_i32, proven_scan_arg_u32
- proven_scan_arg_i64, proven_scan_arg_u64
- proven_scan_arg_f64
- proven_scan_arg_str_view
- proven_scan_arg_none
- proven_scan_arg_identity

Long 별칭:

```
#define PROVEN_SCAN_ARG_LONG(ptr) proven_scan_arg_long(ptr)
#define PROVEN_SCAN_ARG_ULONG(ptr) proven_scan_arg_ulong(ptr)
```

포맷 스캐닝 매크로

매크로	의도
PROVEN_SCAN_ARG(x)	_Generic 목적지 선택자.
proven_scan_fmt_cursor(scan_ptr, fmt, ...)	기존 커서에서 스캔한다.
proven_scan_fmt(view, fmt, ...)	임시 커서로 view에서 스캔한다.

스캔 엔진

```
proven_err_t proven_scan_fmt_internal(
    proven_scan_t *scan,
    const char *fmt,
    const proven_scan_arg_t *args,
    proven_size_t args_count
);
```

API	의도	반환
proven_scan_fmt_internal(scan, fmt, args, args_count)	기존 커서 위에서 동작하는 구조적 스캐너 엔진.	proven_err_t.
proven_scan_fmt_internal_view(view, fmt, args, count)	view 위에 임시 스캐너를 만드는 편의 래퍼.	proven_err_t.

일반 사용자 코드는 매크로를 호출해야 한다. 엔진은 선두 센티널 인자를 기대한다.

중요한 동작: proven_scan_fmt_internal()은 뒤쪽에서 리터럴 불일치가 일어나면 에러를 반환하기 전에 커서를 진행하고 앞선 목적지에 써 넣었을 수 있다. 트랜잭션 같은 파싱이 필요하다면 커서와 목적지 값을 먼저 저장하라.

예제:

```

proven_scan_t scan = proven_scan_init(PROVEN_LIT("ID: 402 SCORE: 99.5 ada"));
int id = 0;
double score = 0.0;
proven_u8str_view_t user = {0};

proven_err_t e = proven_scan_fmt_cursor(
    &scan,
    "ID: {} SCORE: {} {}",
    PROVEN_SCAN_ARG(&id),
    PROVEN_SCAN_ARG(&score),
    PROVEN_SCAN_ARG(&user)
);
if (!proven_is_ok(e)) {
    return;
}
/* id == 402, score == 99.5, user borrows "ada" out of the input - it is not a
 * copy, so it is only valid while the scanned bytes are. */

```

Float 파싱 관련 사항

- proven_scan_f64()와 proven_parse_double_ascii()는 공유된 decimal-to-binary64 백엔드를 거친다.
- 유한한 십진 입력은 round-to-nearest, ties-to-even 동작으로 IEEE-754 binary64로 반올림된다.
- 현재 변환 스택은 Clinger fast path -> staged Eisel-Lemire layer -> exact bigint fallback이며, 대표적인 입력이 어느 경로를 탔는지 테스트가 확인할 수 있도록 내부 카운터를 둔다.
- staged Eisel-Lemire 계층은 현재 생성된 5^q 양수 지수 경우와, 십진 유효숫자가 요구되는 5^q를 깔끔하게 상쇄하는 정확한 음수 지수 경우를 받아들인다.
- __uint128_t가 있는 컴파일러에서는 같은 계층이 정상 범위 경우에 대해 보수적으로 반올림된 음수 지수 비율 부분집합도 받아들이며, 원래의 좁은 프로토타입이 허용하던 것보다 더 넓은 좌시프트 정규화를 포함한다.
- 넓혀진 캐시된 거듭제곱 곱 경로는 이제 5e-324를 포함한 일부 비정규(subnormal) 경우도 단계적으로 처리하지만, true-min에 대한 절반 임계값 아래의 값들은 여전히 정확한 bigint fallback으로 넘긴다.
- 체크인된 캐시 5^q 상수들은 scripts/generate_float_decimal_tables.py로 로컬에서 생성된다.
- proven_parse_double_ascii()는 선행 공백을 건너뛰지 않는다.
- proven_strtod()는 선행 ASCII 공백을 건너뛰고, endptr을 갱신하며, 오버플로 시 부호 있는 무한대를 반환하고, 언더플로 시 부호 있는 0을 보존한다.

5. 예제와 오용 사례

view는 C 문자열이 아니다

잘못된 예:

```

proven_u8str_view_t view = get_view();
printf("%s\n", (const char *)view.ptr); /* wrong: view may not be NUL-terminated */

```

올바른 예:

```

proven_u8str_view_t view = proven_u8str_view_slice(PROVEN_LIT("/etc/hosts"), 5, 5);

/* A view is a pointer and a length into somebody else's bytes. To hand it to a
 * C API that wants a NUL, allocate a real C string from it. */
proven_result_cstr_t c = proven_u8str_view_to_cstr(view, alloc);
if (!proven_is_ok(c.err)) {
    return;
}
/* c.value is "hosts", NUL-terminated. It is yours, so free it. */
alloc.free_fn(alloc.ctx, (void *)c.value);

```

PROVEN_LIT은 리터럴을 위한 것이다

올바른 예:

```
proven_u8str_view_t a = PROVEN_LIT("abc");
(void)a;
```

잘못된 예:

```
const char *runtime = getenv("NAME");
proven_u8str_view_t a = PROVEN_LIT(runtime); /* wrong: macro requires literal syntax */
```

이렇게 하라:

```
const char *runtime = "NAME=value"; /* any trusted NUL-terminated string */
proven_u8str_view_t a = proven_u8str_view_from_cstr(runtime);
(void)a;
```

찾지 못한 것과 치환된 것을 구별하라

proven_u8str_replace_first()는 target을 찾지 못했을 때 PROVEN_OK를 반환한다. 그 차이가 중요하다면 먼저 검색하라.

```
proven_result_u8str_t r = proven_u8str_create_from_view(alloc, PROVEN_LIT("the old way"));
if (!proven_is_ok(r.err)) {
    return;
}
proven_u8str_t s = r.value;

proven_size_t at = proven_u8str_view_find(proven_u8str_as_view(&s), 0, PROVEN_LIT("old"));
if (at != PROVEN_INDEX_NOT_FOUND) {
    (void)proven_u8str_replace_first(&s, 0, PROVEN_LIT("old"), PROVEN_LIT("new"));
}

proven_u8str_destroy(alloc, &s);
```

경계가 있는 포맷 입력

잘못된 예:

```
char *untrusted = get_untrusted_pointer();
proven_println("{", PROVEN_ARG(untrusted));
/* wrong: C-string formatting scans until NUL */
```

올바른 예:

```
char untrusted[16] = { 'n', 'o', ' ', 'n', 'u', 'l', ' ', 'h', 'e', 'r', 'e', '!', '!', '!', '!', '!' };
proven_size_t max_len = sizeof untrusted;

/* The bounded form stops looking for a NUL after max_len bytes. */
proven_println("{", PROVEN_ARG_CSTR_N(untrusted, max_len));
```

Borrow된 고정 용량 문자열

proven_u8str_borrow를 사용하면 아무 할당 없이 스택이나 정적 버퍼에 포매팅할 수 있다 — allocator가 없는 코드와 뜨거운 경로에서 유용하다. 고정 용량 연산(그리고 proven_u8str_append_fmt)을 사용하고, proven_u8str_reset으로 재사용하며, 성장 연산이나 proven_u8str_destroy는 호출하지 말라(메모리는 호출자가 소유한다).

```
int cur = 3;
int total = 10;

proven_byte_t line[64];
proven_u8str_t s = proven_u8str_borrow(line, sizeof line); /* cap includes NUL */

proven_fmt_result_t fr = proven_u8str_append_fmt(&s, "L{}/{}", PROVEN_ARG(cur), PROVEN_ARG(total));
if (PROVEN_FMT_IS_OK(fr)) {
    proven_println("{", PROVEN_ARG(proven_u8str_as_view(&s)));
}
```

```
(void)proven_u8str_reset(&s); /* reuse next frame, no allocation */
```

오용: borrow된 용량을 넘길 성장 호출은 호출자 메모리를 재할당하는 대신 PROVEN_ERR_OUT_OF_BOUNDS를 반환한다.

```
proven_byte_t small[4];
proven_u8str_t t = proven_u8str_borrow(small, sizeof small);
proven_err_t e = proven_u8str_append_grow(&t, PROVEN_LIT("toolong"));
(void)e; /* e == PROVEN_ERR_OUT_OF_BOUNDS; small[] is untouched */
```

자기 참조 포매팅

잘못된 예:

```
const char *inside = proven_u8str_as_cstr(&s);
proven_u8str_append_fmt_grow(&s, "{}", PROVEN_ARG(inside));
/* wrong: self-aliasing C-string arguments are rejected */
```

올바른 예:

```
proven_result_u8str_t r = proven_u8str_create_from_view(&v, PROVEN_LIT("ab"));
if (!proven_is_ok(r.err)) {
    return;
}
proven_u8str_t s = r.value;

/* A view carries a length, so the formatter knows exactly which bytes to snapshot. */
proven_u8str_view_t before = proven_u8str_as_view(&s);
proven_fmt_result_t fr = proven_u8str_append_fmt_grow(&s, "{}", PROVEN_ARG(before));
(void)fr; /* s == "abab" */

proven_u8str_destroy(&s);
```

트랜잭션적 스캐닝

잘못된 가정:

```
proven_err_t e = proven_scan_fmt_cursor(&scan, "{} suffix", PROVEN_SCAN_ARG(&x));
/* if suffix mismatches, x and scan.cursor may already have changed */
```

올바른 패턴:

```
proven_scan_t scan = proven_scan_init(PROVEN_LIT("42 prefix"));
int x = -1;

proven_size_t old_cursor = scan.cursor;
int old_x = x;

proven_err_t e = proven_scan_fmt_cursor(&scan, "{} suffix", PROVEN_SCAN_ARG(&x));
if (!proven_is_ok(e)) {
    /* The literal "suffix" did not match, but 42 was already written into x and
     * the cursor already moved. Put both back yourself. */
    scan.cursor = old_cursor;
    x = old_x;
}
```

실전 예제: owned 문자열과 borrow된 문자열

테스트 스위트에서 컴파일되고 실행된다. 중요한 구분은 이것이다: owned 문자열은 재할당될 수 있고 반드시 파괴해야 한다; borrow된 문자열은 호출자 메모리를 감싸고, 결코 재할당하지 않으며, 여러분이 준 버퍼를 넘어 성장하는 대신 그것을 조용히 옮기지 않고 거부한다.

```
/*
 * 여기 문자열 손잡이가 둘 있고, 둘을 가르는 것은 크기가 아니라 소유다.
 */
```

```

* proven_u8str_t      - 고칠 수 있는 바이트 문자열. 할당을 소유하거나(create)
*                     여러분의 것을 빌린다(borrow).
* proven_u8str_view_t - 남의 바이트를 가리키는 포인터와 길이. 아무것도 소유하지
*                     않고, NUL 로 끝나지 않으며, 그 바이트가 살아 있는 동안에만
*                     쓸 수 있다.
*
* 읽기만 하는 함수에 건네는 것이 뷰이고, 손에 쥐고 있는 것이 u8str 다.
*/

```

```

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* --- *소유하는* 문자열: 할당자의 기억, 여러분이 지울 것 ----- */
    /* 용량 인자는 내용 바이트 수다. NUL 은 그 밖에 따로 있어서, as_cstr 은 언제나
     * 0(1) 이고 언제나 안전하다. */
    proven_result_u8str_t r = proven_u8str_create(alloc, 16);
    EXAMPLE_REQUIRE(proven_is_ok(r.err), "creating a 16-byte string must succeed");
    if (!proven_is_ok(r.err)) {
        return 1;
    }
    proven_u8str_t path = r.value;

    /* append 는 용량이 고정이다. 들어가거나 실패하고, 실패했다면 문자열에 손도 대지
     * 않았다. 재할당하지 않으므로 할당자도 필요 없다. */
    proven_err_t err = proven_u8str_append(&path, PROVEN_LIT("/etc/hosts"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "10 bytes fit in a 16-byte string");

    /* append_grow 는 늘어나는 쌍둥이다. 문자열을 만들 때 쓴 할당자를 주면 필요할 때
     * 재할당한다. 여전히 실패 원자적이다 - 할당이 실패하면 문자열은 그대로다. */
    err = proven_u8str_append_grow(alloc, &path, PROVEN_LIT(".backup.original"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "append_grow must reallocate rather than fail");

    /* 가운데를 고치기. insert 는 꼬리를 오른쪽으로, remove 는 왼쪽으로 민다. */
    err = proven_u8str_insert_grow(alloc, &path, 0, PROVEN_LIT("/srv"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "inserting a prefix must succeed");

    err = proven_u8str_remove(&path, proven_u8str_as_view(&path).size - 9, 9); /* ".original" 을 떼어 낸다 */
    EXAMPLE_REQUIRE(proven_is_ok(err), "removing the trailing suffix must succeed");

    /* 찾는 것이 없으면 replace_first 는 PROVEN_OK 를 돌려준다 - "할 일이 없다" 는
     * 오류가 아니다. 그 차이가 중요하면 먼저 찾아볼 것. */
    err = proven_u8str_replace_first(&path, 0, PROVEN_LIT("hosts"), PROVEN_LIT("fstab"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "replacing an existing substring must succeed");

    /* --- 읽기: 복사하지 말고 뷰를 빌릴 것 ----- */
    /* as_view 는 공짜다. 그 뷰는 다음 편집 전까지만 쓸 수 있다. 늘리는 호출은 재할당할
     * 수 있고, 그러면 뷰(그리고 cstr)는 매달린 포인터가 된다. */
    proven_u8str_view_t v = proven_u8str_as_view(&path);

    EXAMPLE_REQUIRE(proven_u8str_view_eq(v, PROVEN_LIT("/srv/etc/fstab.backup")),
        "the edits above should have produced /srv/etc/fstab.backup");
    EXAMPLE_REQUIRE(proven_u8str_view_starts_with(v, PROVEN_LIT("/srv")),
        "the inserted prefix is at the front");

    proven_size_t dot = proven_u8str_view_find(v, 0, PROVEN_LIT(".backup"));
    EXAMPLE_REQUIRE(dot != PROVEN_INDEX_NOT_FOUND, "the suffix must be found");

    /* 슬라이스는 *같은* 바이트를 가리키는 뷰다 - 할당도 복사도 없다. */
    proven_u8str_view_t stem = proven_u8str_view_slice(v, 0, dot);
    EXAMPLE_REQUIRE(proven_u8str_view_eq(stem, PROVEN_LIT("/srv/etc/fstab")),

```

```

        "slicing at the suffix leaves the stem");

/* as_cstr 은 C API 로 나가는 비상구이고, 소유한 문자열이 길이 뒤에 NUL 을 지키기
 * 때문에만 옳다. 부르는 이렇게 하지 *말 것*. `stem.ptr` 은 NUL 로 끝나지 않는다 -
 * 그냥 `path` 안을 가리킬 뿐이다. */
printf("owned: %s\n", proven_u8str_as_cstr(&path));

/* --- *빌린* 문자열: 여러분의 기억, 할당은 전혀 없다 ----- */
/* 타입도 연산도 같다 - 다만 바이트가 이 스택 버퍼다. `cap` 은 NUL 을 포함하므로
 * 내용은 31 바이트까지 담긴다. */
proven_byte_t line[32];
proven_u8str_t status = proven_u8str_borrow(line, sizeof line);

err = proven_u8str_append(&status, PROVEN_LIT("mounted "));
EXAMPLE_REQUIRE(proven_is_ok(err), "appending into a borrowed buffer needs no allocator");
err = proven_u8str_append(&status, stem);
EXAMPLE_REQUIRE(proven_is_ok(err), "a view can be appended just like a literal");

/* 빌린 문자열에도 늘리는 호출은 있지만, 자기 것이 아닌 기억을 재할당하지는
 * 않는다. 자료가 넘치면 OUT_OF_BOUNDS 이고 `line` 은 손대지 않은 채로 남는다.
 * 빌린 문자열이 등 뒤에서 조용히 힙으로 달아나는 일은 없다. */
err = proven_u8str_append_grow(alloc, &status,
                               PROVEN_LIT(" ...and a great deal more text than fits"));
EXAMPLE_REQUIRE(err == PROVEN_ERR_OUT_OF_BOUNDS,
                "a borrowed string reports overflow instead of reallocating caller memory");
EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&status), PROVEN_LIT("mounted /srv/etc/fstab")),
                "the failed append must have left the string unchanged");

printf("borrowed: %s\n", proven_u8str_as_cstr(&status));

/* reset 은 비어 있는 상태로 자르고 버퍼는 그대로 둔다. 그래서 다음 프레임이 같은
 * 32 바이트를 할당 없이 다시 쓴다. */
err = proven_u8str_reset(&status);
EXAMPLE_REQUIRE(proven_is_ok(err), "reset must succeed on a borrowed string");
EXAMPLE_REQUIRE(proven_u8str_as_view(&status).size == 0, "reset empties the string");

/* --- destroy: 소유 규칙을 그대로 적어 보면 ----- */
/* 빌린 문자열에 대한 destroy 는 아무 일도 하지 않는다 - `line` 은 라이브러리가
 * 해제할 것이 아니다. 그래도 부르는 것이 옳고 값도 들지 않으며, 덕분에 뒷정리 코드는
 * 자기가 뭘 문자열이 어느 쪽인지 몰라도 된다. */
proven_u8str_destroy(alloc, &status);

/* 소유한 문자열에 대한 destroy 는 할당을 해제한다. 그리고 반드시 그 문자열을 만들
 * 때 쓴 할당자를 주어야 한다. */
proven_u8str_destroy(alloc, &path);
return EXAMPLE_OK();
}

```

실전 예제: 버퍼는 고정인데 데이터는 그렇지 않을 때

앞의 예제는 자리가 모자라면 문자열을 늘린다. 실제 코드의 상당 부분은 그럴 수 없다. 레코드에는 고정 폭 필드가 있고, 로그 한 줄에는 상한이 있으며, 아레나(arena) 위에 놓인 문자열은 재할당할 때마다 옛 사본을 다음 reset까지 남긴다. 그런 호출자는 데이터가 들어가지 않을 때 호출이 주는 세 가지 답 중 어느 것인지를 알아야 한다.

호출의 종류	데이터가 들어가지 않을 때 하는 일	해당 호출

원자적(atomic), 고정 용량	PROVEN_ERR_OUT_OF_BOUNDS로 거절하고 아무것도 바꾸지 않는다.	proven_u8str_append, proven_u8str_insert, proven_u8str_replace_at
최선 노력(best-effort, 잘라 쓰기)	들어가는 만큼 쓰고, PROVEN_ERR_OUT_OF_BOUNDS와 함께 실제로 쓴 바이트 수를 돌려준다.	proven_u8str_append_partial
원자적, 확장 가능	allocator를 통해 재할당한다. 할당이 실패하면 아무것도 바꾸지 않는다.	proven_u8str_append_grow, proven_u8str_replace_at_grow, proven_u8str_append_byte

보조 호출 둘도 여기 나온다. proven_u8str_reserve()는 용량을 미리 한 번에 올려 두어 이후의 증가가 재할당을 부르지 않게 한다. 힙(heap)에서는 복사를 아끼고, arena에서는 재할당이 남기는 죽은 저장 공간을 아낀다. proven_u8str_is_valid()는 문자열 핸들 자체의 필드가 서로 모순되지 않는지 본다. 문자열이 다른 코드에서 넘어오는 경계에서 한 번 확인할 값어치가 있고, 편집할 때마다 되풀이할 검사는 아니다.

아래 프로그램은 상한이 있는 로그 한 줄을 쓰고, 넘치는 append를 거절당하고, 최선 노력 호출로 일부러 잘라 내고, proven_u8str_replace_at()으로 경로 가운데를 고친다 — 먼저 짧아지는 방향(언제나 들어간다), 다음에 길어지는 방향(들어가지 않아 거절된다), 그리고 같은 편집을 proven_u8str_replace_at_grow()로 다시 한다. 마지막은 proven_u8str_view_ends_with()로 파일 확장자를 확인하는 것이다.

```
/*
 * 앞의 예제는 자리가 모자라면 문자열을 늘렸다. 이번 것은 늘리는 것이 허용되지 않는
 * 경우 - 크기가 고정된 레코드, 길이 상한이 단단한 로그 줄, 재할당하면 안 되는 아레나
 * 속 버퍼 - 그리고 자료가 들어가지 않을 때 호출이 줄 수 있는 두 가지 정직한 답에 대한
 * 것이다.
 *
 * "안 됩니다, 그리고 아무것도 바꾸지 않았습니다" - 원자적 호출들: append, insert,
 * replace_at. 먼저 용량을 검사하므로, 거부는
 * 문자열을 그대로 남긴다.
 * "일부만 했고, 얼마나 했는지 알려 드립니다" - 최선 노력 호출: append_partial.
 * 답을 수 있는 만큼 담고 몇 바이트를 썼는지
 * 알려 준다.
 *
 * 둘 다 쓸모가 있다. 잘못 고르면 레코드를 조용히 자르거나 조용히 버린다. `_grow` 짝은
 * 세 번째 답 - "네, 자리를 더 찾았습니다" - 이고, 견주어 보라고 끝에 실었다.
 */
```

```
#define FIELD_CAP 32u
```

```
int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    proven_result_u8str_t r = proven_u8str_create(alloc, FIELD_CAP);
    EXAMPLE_REQUIRE(proven_is_ok(r.err), "creating the field buffer must succeed");
    if (!proven_is_ok(r.err)) {
        return 1;
    }
    proven_u8str_t field = r.value;

    /* is_valid 는 손잡이 자신의 구조를 검사한다 - 포인터와, 서로 어긋나지 않는 용량과
     * 길이. 문자열이 다른 데서 넘어올 때 여러분 코드의 경계에서 한 번 단언해 둘 값어치가
     * 있다. 편집할 때마다 할 검사는 아니다. 모든 편집이 그 성질을 지키기 때문이다. */
    EXAMPLE_REQUIRE(proven_u8str_is_valid(&field), "a freshly created string must be structurally valid");

    /* --- 미리 자리를 잡아 두기 ----- */

    /* reserve 는 용량을 지금 올려 두어 나중의 성장이 재할당하지 않게 한다. 힙에서는
```

```

* 복사를 아끼고, 아레나에서는 더 나쁜 것을 아낀다. 거기서는 재할당마다 옛 블록이
* 다음 reset 까지 새기 때문이다. 필요할 만큼을, 한 번에 청할 것. */
proven_err_t err = proven_u8str_reserve(alloc, &field, 64);
EXAMPLE_REQUIRE(proven_is_ok(err), "reserving 64 bytes must succeed");
EXAMPLE_REQUIRE(field.internal.cap >= 64, "the capacity must actually be at least what was asked for");

/* --- 원자적 호출: 들어가거나, 아무것도 바꾸지 않거나 ----- */

err = proven_u8str_append(&field, PROVEN_LIT("2026-01-01 level=info "));
EXAMPLE_REQUIRE(proven_is_ok(err), "the prefix fits in the reserved capacity");

/* append_byte 는 한 바이트를 더한다. 구분자, 종결자, 이스케이프 문자가 그런
* 것들이다. 늘리는 호출이라 할당자를 받는다 - 한 바이트야말로 생각지 못한 경계에서
* 용량 검사가 걸리는 바로 그 경우다. */
err = proven_u8str_append_byte(alloc, &field, (proven_u8)'\n');
EXAMPLE_REQUIRE(proven_is_ok(err), "appending a single separator byte must succeed");

err = proven_u8str_append(&field, PROVEN_LIT("disk full"));
EXAMPLE_REQUIRE(proven_is_ok(err), "the message fits");

err = proven_u8str_append_byte(alloc, &field, (proven_u8)']');
EXAMPLE_REQUIRE(proven_is_ok(err), "closing the bracket must succeed");

/* 이제 용량이 담을 수 있는 것보다 많이 청해 본다. 원자적 append 는 거부하고 -
* 이것이 믿고 기댈 값어치가 있는 성질이다 - 문자열은 호출 전에 담고 있던 것을
* 그대로 담고 있다. */
proven_size_t before = proven_u8str_as_view(&field).size;
err = proven_u8str_append(&field, PROVEN_LIT(" and a very long trailing explanation that certainly does
not fit"));
EXAMPLE_REQUIRE(err == PROVEN_ERR_OUT_OF_BOUNDS, "an oversized atomic append must be refused");
EXAMPLE_REQUIRE(proven_u8str_as_view(&field).size == before, "and must leave the string untouched");

/* --- 최선 노력 호출: 들어가는 만큼, 그리고 그 개수 ----- */

/* 보고서의 폭 고정 열이 이것을 쓰는 경우다. 들어가는 만큼 쓰고, 얼마나 썼는지 알아
* 두어 부르는 쪽이 그 값을 온전한 척하는 대신 잘렸다고 표시할 수 있게 한다. */
proven_result_size_t part = proven_u8str_append_partial(&field, PROVEN_LIT(" ...more text than there is
room for"));
EXAMPLE_REQUIRE(part.err == PROVEN_ERR_OUT_OF_BOUNDS, "a partial append that truncates still reports the
truncation");
EXAMPLE_REQUIRE(part.value > 0, "but it wrote what it could");
EXAMPLE_REQUIRE(proven_u8str_as_view(&field).size == before + part.value,
    "and the string grew by exactly the number of bytes it reports");
printf("partial append wrote %zu byte(s) before the buffer was full\n", (size_t)part.value);

/* --- 늘리지 않고 가운데를 고치기 ----- */

proven_result_u8str_t r2 = proven_u8str_create(alloc, FIELD_CAP);
EXAMPLE_REQUIRE(proven_is_ok(r2.err), "creating the second buffer must succeed");
proven_u8str_t path = r2.value;
err = proven_u8str_append(&path, PROVEN_LIT("var/log/service.log"));
EXAMPLE_REQUIRE(proven_is_ok(err), "the path fits");

/* insert 는 꼬리를 오른쪽으로 민다. 용량 고정이다 - 들어가거나 거부한다. */
err = proven_u8str_insert(&path, 0, PROVEN_LIT("/"));
EXAMPLE_REQUIRE(proven_is_ok(err), "inserting a leading slash must succeed");
EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&path), PROVEN_LIT("/var/log/service.log")),
    "the insert lands at index 0");

/* replace_at 은 어떤 자리의 old_len 바이트를 길이에 상관없는 자료로 바꾼다.

```

```

* 결과가 여전히 들어가기만 하면 된다. "service"(7)를 "daemon"(6)으로 바꾸면 문자열이
* 짧아지므로 용량 때문에 실패할 수 없다. */
proven_size_t at = proven_u8str_view_find(proven_u8str_as_view(&path), 0, PROVEN_LIT("service"));
EXAMPLE_REQUIRE(at != PROVEN_SIZE_MAX, "the substring must be found before it can be replaced");
err = proven_u8str_replace_at(&path, at, 7, PROVEN_LIT("daemon"));
EXAMPLE_REQUIRE(proven_is_ok(err), "a shortening replacement must succeed");
EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&path), PROVEN_LIT("/var/log/daemon.log")),
    "and produce the expected path");

/* 같은 편집을 거꾸로 하면 32바이트 버퍼를 넘치고, 용량 고정 호출은 경로를 자르는
* 대신 거부한다 - 자르는 쪽이 바로 엉뚱한 파일에 조용히 쓰게 되는 그 실패다. */
before = proven_u8str_as_view(&path).size;
err = proven_u8str_replace_at(&path, at, 6, PROVEN_LIT("a-replacement-name-far-too-long-for-this-
buffer"));
EXAMPLE_REQUIRE(err == PROVEN_ERR_OUT_OF_BOUNDS, "a replacement that does not fit must be refused");
EXAMPLE_REQUIRE(proven_u8str_as_view(&path).size == before, "and must leave the path unchanged");

/* replace_at_grow 는 재할당을 허락받은 같은 편집이다. 버퍼가 힙에 있고 길이가 정말
* 한정되지 않았을 때 쓸 것. 한계가 형식의 일부라면 용량 고정 호출을 고를 것. */
err = proven_u8str_replace_at_grow(alloc, &path, at, 6, PROVEN_LIT("a-replacement-name-far-too-long-for-
this-buffer"));
EXAMPLE_REQUIRE(proven_is_ok(err), "the growing variant makes room instead of refusing");
EXAMPLE_REQUIRE(proven_u8str_view_ends_with(proven_u8str_as_view(&path), PROVEN_LIT(".log")),
    "the extension is still at the end after the edit");

/* ends_with 는 확장자 검사가 실제로 던지는 물음에 답한다. 손으로 셈한 인덱스로 하면
* off-by-one 이 사는 자리가 되고, 포인터에 strcmp 로 하면 뷰에는 없는 NUL 이
* 필요해진다. */
EXAMPLE_REQUIRE(!proven_u8str_view_ends_with(proven_u8str_as_view(&path), PROVEN_LIT(".txt")),
    "and it is not a .txt file");

/* 빈 접미사는 무엇이 접미사이기도 하다. 접미사 목록을 도는 반복문이 특별한 경우를
* 두지 않아도 되게 해 주는 답이다. */
EXAMPLE_REQUIRE(proven_u8str_view_ends_with(proven_u8str_as_view(&path), PROVEN_LIT("")),
    "every string ends with the empty suffix");

printf("log line: %s\n", proven_u8str_as_cstr(&field));
printf("path:      %s\n", proven_u8str_as_cstr(&path));

proven_u8str_destroy(alloc, &path);
proven_u8str_destroy(alloc, &field);
return EXAMPLE_OK();
}

```

반례 — 최선 노력 호출을 원자적 호출처럼 다루는 경우:

```
(void)proven_u8str_append_partial(&line, field); /* wrong: ignores the count */
```

잘렸다는 사실이 보고되는 곳은 반환값뿐이다. 그것을 버리면 "레코드가 잘렸다"가 "레코드가 멀쩡해 보인다"로 바뀐다.

반례 — 바이트 수를 글자 수로 읽는 경우:

```
if (part.value == field.size) { /* all of it was written */ }
```

이 비교는 옳고, **바이트** 단위로 옳다. UTF-8 한 글자는 최대 4바이트이므로 최선 노력 append는 글자 중간에서 멈출 수 있다. 뒤끝이 유효한 UTF-8이어야 한다면 자를 지점을 용량이 아니라 여러분이 정한다.

실전 예제: 시스템 호출에 넘길 UTF-16 문자열 조립하기

proven_u16str_t가 제 몫을 하는 곳은 운영체제 호출이 UTF-16을 요구하는 경계뿐이다. 보통은 윈도의 와이드(wide) API가 그 이유다. 방식은 언제나 같다. 코드 단위(code unit)를 모은 다음 proven_u16str_as_ptr()를 그 호출에 넘긴다.

이 코드가 옳은지는 세 가지가 정한다.

- 단위는 코드 단위이지 바이트도 글자도 아니다. 용량 32는 코드 단위 32개, 곧 64바이트다. 기본 다국어 평면(Basic Multilingual Plane, BMP) 밖의 글자 — 이모지, 드문 한중일 한자 다수 — 는 두 개를 차지한다.
- proven_u16str_as_ptr()는 복사하지 않는다. 그 포인터는 문자열을 늘리는 다음 append 전까지만 유효하다.
- 결과는 NUL로 끝난다. 일부러 잘라 낸 뒤에도 그렇기 때문에, 종결자를 기대하는 시스템 호출에 그대로 넘겨도 안전하다.

```
/*
 * UTF-16 이 이 라이브러리에 있는 이유는 하나다. 어떤 운영체제 호출은 그것만 받는다.
 * 윈도의 "와이드" API 가 늘 그 경우다 - CreateFileW 에 건네는 파일 이름은 바이트가
 * 아니라 NUL 로 끝나는 16비트 코드 단위의 줄이다.
 *
 * 그래서 이 타입이 하는 일은 좁다. 코드 단위를 모으고, 개수를 맞게 지키고, 시스템
 * 호출이 원하는 포인터를 내주는 것. 프로그램의 나머지는 UTF-8 로 두어야 한다.
 *
 * 하나만 헛갈리지 말 것 - 단위다. 여기서 용량 32 는 코드 단위 32개, 곧 64 바이트이고,
 * 기본 다국어 평면 밖의 문자는 - 이모지, 드문 한자 대부분 - 두 개를 쓴다. 코드 단위의
 * 개수는 문자의 개수가 아니고, 한 번도 그랬던 적이 없다.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* 인자는 바이트 한계가 아니라 코드 단위 한계다. */
    proven_result_u16str_t r = proven_u16str_create(alloc, 32);
    EXAMPLE_REQUIRE(proven_is_ok(r.err), "creating a 32-code-unit string must succeed");
    if (!proven_is_ok(r.err)) {
        return 1;
    }
    proven_u16str_t name = r.value;
    EXAMPLE_REQUIRE(proven_u16str_len(&name) == 0, "a new string is empty");

    /* PROVEN_U16_LIT 은 u"... " 리터럴에서 뷰를 만들고 단위 개수를 리터럴 자체에서
     * 계산한다. 그래서 개수가 글과 어긋날 수 없다. */
    proven_err_t err = proven_u16str_append(&name, PROVEN_U16_LIT("C:\\logs\\"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "the directory prefix fits in 32 code units");

    err = proven_u16str_append(&name, PROVEN_U16_LIT("service.log"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "the file name fits too");
    EXAMPLE_REQUIRE(proven_u16str_len(&name) == 8 + 11, "the length is a count of code units");

    /* 바이트 문자열 쌍둥이처럼 원자적이다. 자료가 넘치면 거부되고 문자열은 그대로
     * 남는다. 그래서 경로가 반쯤 적히는 일이 없다. */
    proven_size_t before = proven_u16str_len(&name);
    err = proven_u16str_append(&name, PROVEN_U16_LIT(".a-suffix-long-enough-to-overflow-the-capacity"));
    EXAMPLE_REQUIRE(err == PROVEN_ERR_OUT_OF_BOUNDS, "an oversized append must be refused");
    EXAMPLE_REQUIRE(proven_u16str_len(&name) == before, "and must not truncate the path");

    /* --- 시스템 호출이 원하는 포인터 ----- */

    /* as_ptr 은 내부의 코드 단위를 NUL 로 끝난 채로, 복사 없이 돌려준다. 호출 직전의
     * 마지막 걸음이고, 그 포인터는 다음 append 전까지만 쓸 수 있다. 늘리는 append 는
```

```

    * 저장소를 옮길 수 있다. */
    const proven_u16 *wide = proven_u16str_as_ptr(&name);
    EXAMPLE_REQUIRE(wide != NULL, "an assembled string must yield a pointer");
    EXAMPLE_REQUIRE(wide[0] == (proven_u16)'C', "the first code unit is the drive letter");
    EXAMPLE_REQUIRE(wide[proven_u16str_len(&name)] == 0, "the sequence is NUL-terminated for the system
call");
    /* 윈도우에서는 이것이 이 타입의 존재 이유 전부다.
    *     HANDLE h = CreateFileW((LPCWSTR)wide, ...);
    * 여기서는 부르지 않는다. 이 예제는 다른 모든 곳에서도 돌아야 하기 때문이다. */

    /* --- 자르는 것이 옳은 답일 때 ----- */

    /* 어떤 시스템 구조체에는 폭이 고정된 필드가 있다 - 이를테면 16 단위짜리 이름표 -
    * 거기서는 들어가지 않는 이름을 거부하는 것이 아니라 자르는 것이 뜻이다. 부분
    * append 가 그것을 위한 것이다. 담을 수 있는 만큼 담고 몇 단위를 썼는지 알려 주어,
    * 부르는 쪽이 그 값을 잘렸다고 표시할 수 있게 한다. */
    proven_result_u16str_t r2 = proven_u16str_create(alloc, 16);
    EXAMPLE_REQUIRE(proven_is_ok(r2.err), "creating the fixed-width label must succeed");
    proven_u16str_t label = r2.value;

    proven_result_size_t wrote = proven_u16str_append_partial(&label, PROVEN_U16_LIT("a-label-that-is-longer-
than-the-field"));
    EXAMPLE_REQUIRE(wrote.err == PROVEN_ERR_OUT_OF_BOUNDS, "the truncation is reported, not hidden");
    EXAMPLE_REQUIRE(wrote.value == 16, "it filled the field exactly");
    EXAMPLE_REQUIRE(proven_u16str_len(&label) == wrote.value, "and the length matches what it says it wrote");
    EXAMPLE_REQUIRE(proven_u16str_as_ptr(&label)[wrote.value] == 0,
        "a truncated string is still NUL-terminated, so it is still safe to pass on");

    printf("assembled %zu code unit(s); label truncated to %zu\n",
        (size_t)proven_u16str_len(&name), (size_t)wrote.value);

    proven_u16str_destroy(alloc, &label);
    proven_u16str_destroy(alloc, &name);
    return EXAMPLE_OK();
}

```

반례 — UTF-16 버퍼 크기를 바이트로 잡는 경우:

```
proven_result_u16str_t r = proven_u16str_create(alloc, sizeof(buf)); /* wrong */
```

인자는 코드 단위의 개수다. 바이트 수를 넘기면 의도한 것의 두 배를 요구하게 되고, 그 바이트 수가 UTF-8 문자열에서 왔다면 변환 결과를 담지도 못하는 버퍼를 요구하게 된다.

반례 — append를 사이에 두고 포인터를 붙들고 있는 경우:

```

const proven_u16 *w = proven_u16str_as_ptr(&name);
proven_err_t e = proven_u16str_append_grow(alloc, &name, more); /* may reallocate */
use_wide(w); /* wrong: w may dangle */

```

포인터는 그것을 소비하는 호출 바로 앞에서 얻는다.

4장: 컨테이너와 알고리즘

3부 — 자료구조. 선행 조건: 2부 (1, 2, 3). 이 장을 마치면 작업에 맞는 컨테이너를 고르고, 보장된 한계 안에서 정렬하고 검색하며, 올바른 이유로 해싱하고, 바이트를 텍스트로 또 그 반대로 바꿀 수 있다.

이 장은 array.h, list.h, ring.h, map.h, algorithm.h, hash.h, encode.h를 다룬다. 여기 있는 모든 컨테이너는 할당자(allocator)를 받는다. 2장이 먼저 오는 이유가 그것이다.

목차

1. 동적 배열
2. 침습적(intrusive) 리스트
3. 링 버퍼
4. 해시 맵
5. 알고리즘
6. 용도별 해싱
7. 바이트를 텍스트로: hex와 Base64
8. 예제와 응용 사례

1. 동적 배열

문제: 매번 손으로 다시 쓰는 그 배열

확장 가능한 배열은 모든 C 프로그램이 결국 한 번은 작성하는 자료구조이며, 매번 조금씩 다르게 작성된다:

```
int *items = malloc(cap * sizeof(int));
/* ... later ... */
if (n == cap) {
    cap *= 2;
    items = realloc(items, cap * sizeof(int)); /* wrong on two counts */
    items[n++] = x;
}
```

한 줄에 버그가 둘 있고, 둘 다 고전이다. `cap * sizeof(int)`는 큰 `cap`에 대해 **랩어라운드**할 수 있고, 그러면 거대한 개수에 대해 작은 할당이 만들어진다 — 1장 § 4를 보라. 그리고 `realloc`의 결과를 곧바로 `items`에 대입하면 **NULL을 반환할 때 옛 블록이 누수된다**. 원래 포인터가 사라졌는데 그것이 가리키던 메모리는 여전히 할당된 채로 남기 때문이다.

버그를 빼고 보더라도, 손으로 쓴 버전은 원소 타입마다 다시 써야 하거나, `void *`로 제네릭하게 만들어 타입 검사를 잃어야 한다.

이 라이브러리는 대신 무엇을 하는가

`proven_array_t`는 생성될 때의 원소 크기와 정렬을 유지하는 확장 벡터다. 그래서 템플릿 없이도, 호출 지점에서 `void *`를 쓰지 않고도 어떤 타입에나 동작한다 — `PROVEN_ARRAY_*` 매크로가 타입을 받아, 컴파일러가 아직 검사할 수 있는 자리에서 캐스팅을 수행한다.

- 확장은 검사된 산술을 사용하므로, 위의 오버플로는 작은 할당이 아니라 `PROVEN_ERR_OVERFLOW`가 된다.
- 확장은 **실패-원자적(failure-atomic)**이다: 확장할 수 없으면 기존 원소는 손대지 않은 채로 여전히 유효하다. 아무것도 누수되지 않고 아무것도 잃지 않는다.
- 문자열 타입과 달리 **allocator를 내부에 저장한다** — `PROVEN_ARRAY_DESTROY`가 배열만 받는 이유가 그것이다.

`proven_array_t`는 연속된 원소 저장소를 소유하므로, `proven_array_sort`와 § 5의 검색들에 넘기는 것도 바로 이것이다.

잘못된 예 — `push`를 건너 포인터를 붙잡고 있기:

```
int *first = PROVEN_ARRAY_GET_MUT(&arr, int, 0);
(void)PROVEN_ARRAY_PUSH(&arr, int, 42); /* may reallocate */
*first = 7; /* wrong: `first` may point at freed memory */
```

이것은 타입이 제거해 줄 수 없는 유일한 위험이다. 확장은 저장소를 이동시키므로, **배열을 가리키는 모든 포인터나 뷰(view)는 확장을 유발할 수 있는 모든 연산에 의해 무효화된다.** 가리키는 대신 인덱스를 쓰거나, push 이후에 포인터를 다시 가져오라.

구조체

```
typedef struct {
    proven_allocator_t alloc;
    proven_byte_t *data;
    proven_size_t len;
    proven_size_t cap;
    proven_size_t elem_size;
    proven_size_t align;
} proven_array_t;
```

```
typedef struct {
    proven_err_t err;
    proven_array_t value;
} proven_result_array_t;
```

필드:

- alloc: 성장과 파괴에 쓰이는 allocator.
- data: 원소를 위한 바이트 저장소.
- len: 현재 원소 개수.
- cap: 원소 단위 용량.
- elem_size: 각 원소의 크기.
- align: 각 원소의 정렬.

함수

API	의도	반환
proven_array_create(alloc, init_cap, elem_size, align)	제네릭 배열을 만든다.	proven_result_array_t.
proven_array_is_valid(arr)	공개 배열 불변식을 검증한다.	bool.
proven_array_reserve(arr, new_cap)	최소 new_cap개의 원소를 보장한다.	proven_err_t.
proven_array_push(arr, element)	포인터로부터 복사하여 원소 하나를 추가한다.	proven_err_t.
proven_array_pop(arr, out_element)	마지막 원소를 꺼낸다; out_element가 null이면 버린다.	proven_err_t.
proven_array_get_mut(arr, index)	가변 원소 포인터를 얻는다.	포인터 또는 null.
proven_array_get(arr, index)	const 원소 포인터를 얻는다.	포인터 또는 null.
proven_array_destroy(arr)	저장소를 해제하고 상태를 비운다.	void.

매크로

매크로	의도
PROVEN_ARRAY_INIT(alloc, type, init_cap)	sizeof(type)와 alignof(type)를 사용하는 타입 안전 생성.
PROVEN_ARRAY_PUSH(arr_ptr, type, value)	임시 복합 리터럴로 rvalue나 lvalue를 push한다.
PROVEN_ARRAY_POP(arr_ptr, type, out_ptr)	출력 포인터로 pop하거나 null로 버린다.
PROVEN_ARRAY_GET(arr_ptr, type, index)	타입이 지정된 const 원소 포인터.
PROVEN_ARRAY_GET_MUT(arr_ptr, type, index)	타입이 지정된 가변 원소 포인터.
PROVEN_ARRAY_DESTROY(arr_ptr)	배열을 파괴한다.

예제:

```
proven_result_array_t r = PROVEN_ARRAY_INIT(alloc, int, 4);
if (!proven_is_ok(r.err)) {
    return;
}
proven_array_t nums = r.value;

(void)PROVEN_ARRAY_PUSH(&nums, int, 10);
(void)PROVEN_ARRAY_PUSH(&nums, int, 20);

/* GET points into the array's own storage, and returns NULL out of range. */
const int *first = PROVEN_ARRAY_GET(&nums, int, 0);
if (first) {
    proven_println("first={}", PROVEN_ARG(*first));
}

/* The array stores its allocator, so destroy needs nothing but the array. */
PROVEN_ARRAY_DESTROY(&nums);
```

2. 침습적(intrusive) 리스트

문제: 배운 대로의 리스트는 원소마다 노드를 할당한다

```
struct node { struct node *next; void *data; };
```

이것이 교과서의 연결 리스트이며, 교과서가 언급하지 않는 두 가지 비용이 있다. 삽입할 때마다 **할당한다** — 천 개의 항목은 곧 포인터를 담기 위해서만 존재하는 천 번의 할당이고, 각각은 실패할 수 있으며 각각은 짝이 되는 해제를 필요로 한다. 그리고 data는 void *이므로 리스트는 자신이 무엇을 담고 있는지 전혀 모른다: 다시 읽어낼 때마다 컴파일러가 검사할 수 없는 캐스트가 된다.

침습적 리스트는 대신 무엇을 하는가

관계를 뒤집는다: 메모리는 당신이 소유하고, 링크가 그 안에 산다.

```
typedef struct {
    int id;
    proven_list_node_t link; /* the list's hook, inside your struct */
} task_t;
```

이제 삽입은 포인터 두 개를 쓰는 일이다. 아무것도 할당하지 않으므로 **실패할 수 없다** —

proven_list_push_back이 void를 반환한다는 점에 주목하라. 이 라이브러리에서는 이례적인 일이고, 바로 그것이 요점이다. 제거도 마찬가지다. 객체는 스택에도, 배열 안에도, 아레나 안에도, 어디에나 살 수 있다. 리스트는 이미 그 안에 들어 있는 포인터들을 재배치할 뿐이다.

링크에서 다시 당신의 객체로 돌아가는 것은 PROVEN_LIST_ENTRY(node, task_t, link)이며, 노드의 주소에서 멤버의 오프셋을 뺀다. 이것은 맞기를 바라는 캐스트가 아니라, 컴파일러가 타입을 검사하는 산술이다.

포기하는 것: 객체는 자신이 가진 링크 멤버 수만큼의 리스트에만 속할 수 있고, 그 수는 구조체를 선언할 때 당신이 정한다. 어떤 항목이 큐와 인덱스에 동시에 있어야 한다면, 링크가 둘 필요하다.

proven_list_t는 침습적 이중 연결 원형 리스트다 — 헤드가 sentinel 노드이므로, 삽입과 제거가 양 끝을 특별히 취급하는 일이 없다. 노드를 할당하지 않는다.

잘못된 예 — 평범한 반복자로 순회하면서 제거하기:

```
PROVEN_LIST_FOR_EACH(it, &queue) {
    if (should_drop(it)) proven_list_remove(it); /* wrong: it->next is read after unlink */
}
```

proven_list_remove는 노드 자신의 포인터를 통해 쓰기를 하므로, 루프는 방금 떼어낸 노드에서 next를 읽게 된다. PROVEN_LIST_FOR_EACH_SAFE는 두 번째 변수를 받아 본문이 실행되기 전에 다음 포인터를 읽는다. 그것이 바로 이 매크로가 존재하는 이유다.

완성 예제: 링크가 호출자의 구조체 안에 사는 큐

테스트 스위트가 컴파일하고 실행한다. 스택에 할당된 task들의 큐를 만들고, 순회하고, 안전한 순회 안에서 하나를 제거하고, 중간에 삽입한다 — 프로그램 어디에도 allocator가 없다.

```
/*
 * *침입형*(intrusive) 리스트는 여러분의 자료를 담을 노드를 따로 할당하는 대신, 연결
 * 고리를 여러분의 구조체 *안에* 둔다.
 *
 * 배울 때 본 리스트는 이렇게 생겼다.
 *
 * struct node { struct node *next; void *data; };
 *
 * 넣을 때마다 노드를 할당하므로 항목 천 개짜리 리스트는 포인터를 담자고 존재하는 할당
 * 천 번이 되고, 하나하나가 실패할 기회이자 해제할 거리다. 더 나쁜 것은 `data` 가
 * void* 라는 점이다 - 리스트는 자기가 무엇을 담았는지 모르므로 읽을 때마다 컴파일러가
 * 검사할 수 없는 형변환을 한다.
 *
 * 침입형 리스트는 그것을 뒤집는다. 기억은 *여러분*이 소유하고 고리는 그 안에 산다.
 * 넣는 데 할당이 없으니 실패할 수 없다. 빼는 데도 할당이 없으니 실패할 수 없다. 그리고
 * 고리가 알려진 타입의 멤버이므로, 고리에서 물건으로 돌아오는 것은 형변환이 아니라
 * 컴파일러가 대신 해 주는 산술이다.
 *
 * 대가는 물건 하나가 자기가 가진 고리 멤버 수만큼의 리스트에만 들어갈 수 있다는 것이고,
 * 그것은 구조체를 선언할 때 여러분이 내리는 결정이다.
 */

/* 고리가 멤버다. 이 작업은 한 번에 정확히 한 리스트에만 들어갈 수 있다. */
typedef struct {
    int id;
    proven_list_node_t link;
} task_t;

int main(void) {
    /* 이 프로그램에는 할당자가 아예 없다. 작업들은 스택에 있고, 리스트는 그 안에 사는
     * 포인터들을 다시 이어 줄 뿐이다. */
    task_t a = { .id = 1 };
    task_t b = { .id = 2 };
    task_t c = { .id = 3 };

    proven_list_t queue;
    proven_list_init(&queue);
    EXAMPLE_REQUIRE(proven_list_is_empty(&queue), "a freshly initialised list is empty");

    /* --- 넣기는 실패할 수 없다. 할당하는 것이 없으므로 ----- */
    proven_list_push_back(&queue, &a.link);
    proven_list_push_back(&queue, &b.link);
    proven_list_push_back(&queue, &c.link);
    EXAMPLE_REQUIRE(!proven_list_is_empty(&queue), "three tasks are queued");

    /* --- 출기: PROVEN_LIST_ENTRY 가 고리에서 물건을 되찾아 준다 ----- */
    proven_list_node_t *it = NULL;
    int seen[3] = {0}, n = 0;
    PROVEN_LIST_FOR_EACH(it, &queue) {
        task_t *t = PROVEN_LIST_ENTRY(it, task_t, link);
        if (n < 3) seen[n++] = t->id;
    }
    EXAMPLE_REQUIRE(n == 3 && seen[0] == 1 && seen[1] == 2 && seen[2] == 3,
        "the walk visits every task, in insertion order");

    /* --- 훑으면서 빼려면 *안전한* 꼴이 필요하다 ----- */
}
```

```

/* proven_list_remove 는 노드 자신의 next/prev 포인터를 통해 쓴다. 그래서 그냥
 * FOR_EACH 를 쓰면 방금 떼어 낸 노드에서 `it->next` 를 읽게 된다. _SAFE 끝은 본문이
 * 돌기 *전에* 다음 포인터를 읽어 둔다. */
proven_list_node_t *safe = NULL;
PROVEN_LIST_FOR_EACH_SAFE(it, safe, &queue) {
    task_t *t = PROVEN_LIST_ENTRY(it, task_t, link);
    if (t->id == 2) proven_list_remove(it);
}

n = 0;
PROVEN_LIST_FOR_EACH(it, &queue) {
    task_t *t = PROVEN_LIST_ENTRY(it, task_t, link);
    if (n < 3) seen[n++] = t->id;
}
EXAMPLE_REQUIRE(n == 2 && seen[0] == 1 && seen[1] == 3, "task 2 was unlinked");

/* --- 가운데에 끼워 넣기는 포인터 바꿔 끼우기다 ----- */
task_t d = { .id = 4 };
proven_list_insert_after(&a.link, &d.link);

n = 0;
PROVEN_LIST_FOR_EACH(it, &queue) {
    task_t *t = PROVEN_LIST_ENTRY(it, task_t, link);
    if (n < 3) seen[n++] = t->id;
}
EXAMPLE_REQUIRE(n == 3 && seen[0] == 1 && seen[1] == 4 && seen[2] == 3,
    "task 4 sits directly after task 1");

/* 지울 것이 없다. 리스트는 아무것도 소유한 적이 없다. `a`, `c`, `d` 는 여전히
 * 멀쩡한 지역 변수이고, 그 수명은 리스트가 없었을 때와 똑같이 이 함수의 것이다. */
return EXAMPLE_OK();
}

```

구조체

```

typedef struct proven_list_node_t {
    struct proven_list_node_t *next;
    struct proven_list_node_t *prev;
} proven_list_node_t;

```

```

typedef struct {
    proven_list_node_t head;
} proven_list_t;

```

함수와 매크로

API	의도	반환
proven_list_init(list)	원형 센티넬을 초기화한다.	void.
proven_list_insert_after(target, node)	target 뒤에 node를 삽입한다.	void.
proven_list_push_back(list, node)	꼬리에 삽입한다.	void.
proven_list_remove(node)	노드를 떼어내고 링크를 null로 오염 (poison)시킨다.	void.
proven_list_is_empty(list)	빈 리스트 또는 null 리스트인지 검사한다.	int 진리값.
PROVEN_CONTAINER_OF(ptr, type, member)	멤버 포인터를 부모 객체 포인터로 변환한다.	포인터.
PROVEN_LIST_FOR_EACH(iter, list)	노드를 순회한다.	루프 매크로.
PROVEN_LIST_FOR_EACH_SAFE(iter, safe_next, list)	현재 노드를 제거하면서 안전하게 순회한다.	루프 매크로.

PROVEN_LIST_ENTRY(ptr, type, member)	리스트 노드를 담고 있는 객체로 변환한다.	포인터.
--------------------------------------	-------------------------	------

예제:

```
typedef struct Item {
    int value;
    proven_list_node_t link; /* the list lives inside the object; it allocates nothing */
} Item;

proven_list_t list;
proven_list_init(&list);

Item a = { .value = 1 };
Item b = { .value = 2 };
proven_list_push_back(&list, &a.link);
proven_list_push_back(&list, &b.link);

int total = 0;
proven_list_node_t *it = NULL;
PROVEN_LIST_FOR_EACH(it, &list) {
    /* The iterator walks nodes; ENTRY converts a node back to its owner. */
    Item *item = PROVEN_LIST_ENTRY(it, Item, link);
    total += item->value;
}
proven_printf("total=%d", PROVEN_ARG(total)); /* 3 */
```

3. 링 버퍼

문제: 커져서는 안 되는 큐

빠른 생산자와 느린 소비자 사이의 큐는 한 가지 질문에 답해야 한다: 소비자가 뒤처지면 무슨 일이 일어나는가? 확장 가능한 큐는 "더 할당한다"고 답하는데, 그러면 일시적인 지연이 무한한 메모리 증가로, 그리고 결국에는 작업을 버리는 것보다 더 나쁜 무언가로 바뀐다.

손으로 쓰는 대안은 배열 하나에 head 인덱스, tail 인덱스, 그리고 모듈로 산술을 더한 것이다 — 그리고 여기에는 유명한 버그가 하나 있다. head == tail일 때 버퍼는 비어 있는가, 가득 찼는가? 별도의 개수를 유지하거나 슬롯 하나를 일부러 낭비하지 않는 한 두 상태는 똑같아 보이고, 프로그래머 세대마다 이것을 다시 발견한다.

이 라이브러리는 대신 무엇을 하는가

proven_ring_t는 개수를 유지하는 고정 용량 FIFO다. 그래서 비어 있음과 가득 참이 구별되며, 커지는 대신 거부한다:

- 가득 찬 링에 대한 proven_ring_push는 PROVEN_ERR_OUT_OF_BOUNDS를 반환한다. 가장 오래된 항목을 덮어쓰지 않고, 재할당도 하지 않는다. 호출자가 결정한다 — 기다리거나, 새 항목을 버리거나, 배압(backpressure)을 보고하거나 — 무엇이 옳은지는 호출자만 알기 때문이다.
- 비어 있는 링에 대한 proven_ring_pop은 낡은 슬롯을 되돌려주는 대신 실패한다.

용량은 생성 시점에 한 번 정해지며, 그 숫자가 곧 정책이다: 생산자가 얼마나 앞서 나가도 되는지를 말한다. 이벤트 큐, 최근 항목의 로그, 오디오나 센서 버퍼 — 한계가 제약이 아니라 설계의 일부인 곳이라면 링을 꺼내라.

이 라이브러리의 대부분의 타입과 달리, proven_ring_t는 **allocator를 내부에 저장한다**. PROVEN_RING_DESTROY가 링만 받는 이유가 그것이다.

잘못된 예 — 가득 찬 링을 즉시 재시도할 오류로 취급하기:

```
while (PROVEN_RING_PUSH(&ring, event_t, e) != PROVEN_OK) { } /* wrong: spins forever */
```

그 루프 안의 무엇도 소비하지 않으므로 링은 계속 가득 차 있다. 한계가 있는 큐의 거부는 *소비자*에 대한 정보이며, 그것을 무시하는 루프는 곧 멈춤(hang)이다.

완성 예제: 가득 찰 때까지 push하기, 그리고 거부가 어떤 모습인지

테스트 스위트가 컴파일하고 실행한다.

```
/*
 * 고리 버퍼는 내용을 옮기지도, 늘어나지도 않는 고정 크기 큐다. 용량을 한 번 정해 주면
 * push 는 꼬리에 넣고 pop 은 머리에서 빼며, 가득 차면 push 는 *거부한다*.
 *
 * 이것이 없으면 여러분이 쓰게 될 C 는 배열 하나에 인덱스 둘, 그리고 그것을 감아 주는
 * 나머지 연산이고, 버그는 언제나 같다 - "지금 가득 찬 것인가 빈 것인가?" 개수를 따로
 * 세거나 칸 하나를 버리지 않는 한 두 상태 모두 head == tail 이다. 이 고리는 개수를
 * 세므로 그 물음이 생기지 않는다.
 *
 * 만드는 쪽과 쓰는 쪽의 속도가 다르고 만드는 쪽이 얼마나 앞서 나갈지에 단단한 한계를
 * 두고 싶을 때 쓴다 - 이벤트 큐, 로그 고리, 오디오 버퍼. 가득 찔 때의 거부가 바로
 * 그 기능이다 - 역압(backpressure)이다. 늘어나는 큐는 몰려드는 입력에 기억을 먹어
 * 치우는 것으로 답하다가 더 나쁜 일을 맞는다.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* 이벤트 4개짜리 용량. 5가 되는 일은 없다. */
    proven_result_ring_t r = PROVEN_RING_INIT(alloc, int, 4);
    EXAMPLE_REQUIRE(proven_is_ok(r.err), "creating a 4-slot ring should succeed");
    proven_ring_t ring = r.value;

    /* --- 가득 찰 때까지 push ----- */
    for (int i = 1; i <= 4; ++i) {
        proven_err_t err = PROVEN_RING_PUSH(&ring, int, i);
        EXAMPLE_REQUIRE(proven_is_ok(err), "the first four pushes fit");
    }

    /* 다섯 번째 push 는 거부된다. 가장 오래된 항목을 덮어쓰지도, 늘어나지도 않는다.
     * 가득 찬 고리는 가득 찬 고리이고, 어떻게 할지는 부르는 쪽이 정한다 - 기다리든,
     * 새 이벤트를 버리든, 역압을 알리든. */
    proven_err_t full = PROVEN_RING_PUSH(&ring, int, 5);
    EXAMPLE_REQUIRE(full == PROVEN_ERR_OUT_OF_BOUNDS,
        "pushing into a full ring must be refused, not silently absorbed");

    /* --- 넣은 차례대로 pop ----- */
    int out = 0;
    proven_err_t err = PROVEN_RING_POP(&ring, int, &out);
    EXAMPLE_REQUIRE(proven_is_ok(err) && out == 1, "pop returns the oldest entry first");

    /* 이제 자리가 다시 생겼고, 고리는 아무것도 옮기지 않은 채 저장소를 감아 돈다. */
    err = PROVEN_RING_PUSH(&ring, int, 5);
    EXAMPLE_REQUIRE(proven_is_ok(err), "after a pop there is room for one more");

    int expected[] = { 2, 3, 4, 5 };
    for (int i = 0; i < 4; ++i) {
        err = PROVEN_RING_POP(&ring, int, &out);
        EXAMPLE_REQUIRE(proven_is_ok(err) && out == expected[i],
            "entries come out in the order they went in");
    }

    /* --- 빈 상태도 가득 찬 상태처럼 군다: 거부할 뿐 자료를 지어내지 않는다 --- */
    err = PROVEN_RING_POP(&ring, int, &out);
    EXAMPLE_REQUIRE(!proven_is_ok(err), "popping an empty ring must fail rather than return junk");

    PROVEN_RING_DESTROY(&ring);
}
```

```

    return EXAMPLE_OK();
}

```

구조체

```

typedef struct {
    proven_allocator_t alloc;
    proven_mem_mut_t internal;
    proven_size_t head;
    proven_size_t tail;
    proven_size_t len;
    proven_size_t cap;
    proven_size_t elem_size;
    proven_size_t align;
} proven_ring_t;

```

```

typedef struct {
    proven_err_t err;
    proven_ring_t value;
} proven_result_ring_t;

```

함수와 매크로

API	의도	반환
proven_ring_create(alloc, cap, elem_size, align)	고정 용량 링을 만든다.	proven_result_ring_t.
proven_ring_is_valid(ring)	링 불변식을 검증한다.	bool.
proven_ring_push(ring, element)	원소 하나를 push한다; 가득 차면 실패한다.	proven_err_t.
proven_ring_pop(ring, out_element)	원소 하나를 pop한다; null 출력이면 버린다.	proven_err_t.
proven_ring_destroy(ring)	저장소를 해제한다.	void.
PROVEN_RING_INIT(alloc, type, cap)	타입 안전 생성.	proven_result_ring_t.
PROVEN_RING_PUSH(ring_ptr, type, value)	타입 안전 push.	proven_err_t.
PROVEN_RING_POP(ring_ptr, type, out_ptr)	타입 안전 pop.	proven_err_t.
PROVEN_RING_DESTROY(ring_ptr)	링을 파괴한다.	void.

예제:

```

proven_result_ring_t r = PROVEN_RING_INIT(alloc, int, 8);
if (!proven_is_ok(r.err)) {
    return;
}
proven_ring_t q = r.value;

proven_err_t e = PROVEN_RING_PUSH(&q, int, 7);
if (!proven_is_ok(e)) {
    /* The ring is fixed-capacity: a full ring is PROVEN_ERR_OUT_OF_BOUNDS,
     * never a silent grow. */
    PROVEN_RING_DESTROY(&q);
    return;
}

int out = 0;
e = PROVEN_RING_POP(&q, int, &out); /* FIFO: out == 7. Empty ring is OUT_OF_BOUNDS. */
(void)e;

PROVEN_RING_DESTROY(&q);

```

4. 해시 맵

proven_map_t는 tombstone을 갖는 open-addressing 해시 맵이다. 정수 키와 빌려 쓰는(borrowed) 또는 소유(owned) U8 문자열 키를 지원한다. allocator를 내부에 저장한다.

구조체

```
typedef enum {
    PROVEN_KEY_TYPE_INT,          /* keys are proven_size_t integers */
    PROVEN_KEY_TYPE_U8_BORROWED, /* keys are u8 views; caller keeps the bytes alive */
    PROVEN_KEY_TYPE_U8_OWNED     /* keys are u8 views; the map copies and frees the bytes */
} proven_key_type_t;

typedef union {
    proven_size_t id;             /* used when key_type == PROVEN_KEY_TYPE_INT */
    proven_u8str_view_t str;      /* used for the two U8 key modes */
} proven_map_key_t;

typedef struct {
    proven_allocator_t alloc;      /* allocator for the bucket array and owned keys */
    proven_mem_mut_t internal;     /* the single contiguous bucket array (ptr + size) */
    proven_size_t len;             /* live entries */
    proven_size_t used;            /* live entries + tombstones (drives the load factor) */
    proven_size_t cap;            /* number of buckets, always a power of two */
    proven_size_t elem_size;       /* size of one stored value */
    proven_size_t align;          /* alignment requested for the value */
    proven_size_t bucket_stride;   /* bytes per bucket: align_up(header + elem_size) */
    proven_size_t payload_offset;  /* bytes from a bucket start to its value payload */
    proven_key_type_t key_type;    /* key mode chosen at create time */
} proven_map_t;

typedef struct {
    proven_err_t err;
    proven_map_t value;
} proven_result_map_t;
```

내부 동작 원리

맵은 cap개의 버킷으로 이루어진 단일 평면 배열이다 — 값에 대해 항목별 할당이 없다(그리고 INT/BORROWED 모드에서는 키에 대해서도 없다). 그래서 조희가 캐시 친화적으로 유지된다. 각 버킷은 다음과 같이 배치된다:

```
[ header: state + key ][ padding to the value's alignment ][ value payload ]
^ bucket start                               ^ payload_offset
|<----- bucket_stride ----->|
```

헤더의 state는 **EMPTY**(한 번도 사용 안 함), **OCCUPIED**(살아 있는 키+값을 보유), **TOMBSTONE**(제거된 항목) 중 하나다. payload_offset과 bucket_stride는 생성 시점에 elem_size/align으로부터 한 번 계산되므로, 버킷 i의 값 주소는 그냥 internal.ptr + i*bucket_stride + payload_offset이다.

- **해싱, 그리고 기본값이 왜 안전한 쪽인가.** 정수 키는 SplitMix/Murmur 스타일의 비트 혼합 finaliser를 거친다(그래서 순차 id가 버킷 전체에 퍼진다). 문자열 키는 OS CSPRNG에서 한 번 뽑은 프로세스별 비밀 아래에서 keyed SipHash-2-4로 해싱된다 — 왜냐하면 신뢰할 수 없는 키를 예측 가능한 함수로 해싱하는 맵은 터지기를 기다리는 서비스 거부(DoS)이기 때문이다: 키를 통제하는 공격자는 오프라인에서 충돌을 계산해 그 전부를 한 버킷에 몰아넣고, 모든 조희를 선형 스캔으로 바꾼다. 그들이 볼 수 없는 비밀로 해시를 키잉하는 것이 그것을 막으며, 이는 Python, Rust, Linux 커널이 자기네 테이블에 대해 내린 것과 같은 선택이다. 키가 전부 여러분 자신의 프로그램에서 온다면, proven_map_create_trusted가 빠른 unkeyed FNV-1a를 대신 선택한다. proven_map_hash는 주어진 맵이 실제로 어느 함수를 쓰는지 알려준다. (CSPRNG도 공격자 모델도 없는 프리스탠딩(freestanding) 타깃에서는 문자열 키가 FNV로 후퇴한다.)
- **탐사(Probing).** 선형 open addressing: 시작 버킷은 hash & (cap - 1)이며(cap이 2의 거듭제곱이라 저렴하다), 그다음 검색은 한 번에 한 버킷씩 앞으로 걸어가며 감싸고, 키(같은 키를 가진 OCCUPIED)나

EMPTY 버킷(키가 없음을 증명함)을 찾을 때까지 계속한다. 선형 탐색은 걸어가는 버킷들을 메모리 상에서 연속되게 유지한다.

- **제거와 tombstone.** proven_map_remove는 버킷을 그냥 비울 수 없다 — 그러면 탐색 체인이 끊겨 나중 키들이 숨겨진다. 대신 버킷을 TOMBSTONE으로 표시한다. Tombstone은 조회에서 건너뛰어지지만 여전히 슬롯을 차지하므로, len이 아니라 used(살아 있는 것 + tombstone)가 성장을 견인한다.
- **부하율(load factor)과 리사이즈.** used $\geq$ cap * 3/4이 되면, 맵은 다음 2의 거듭제곱 크기의 새 버킷 배열을 할당하고 모든 OCCUPIED 항목을 그리로 **재해싱**하며, 그 과정에서 모든 tombstone을 버린다. 용량은 커지지만 하고, 결코 줄어들지 않는다. proven_map_reserve로 미리 예약하라(특히 아레나 allocator에서는, 뒤에 죽은 배열을 남기지 않기 위해).

키 모드 — 하나 고르기

- PROVEN_KEY_TYPE_INT: 키는 proven_size_t다. 키 저장소 없음.
- PROVEN_KEY_TYPE_U8_BORROWED: 버킷이 view(포인터 + 길이)를 저장한다. **맵은 바이트를 절대 복사하지 않으므로**, 호출자는 항목이 존재하는 동안 내내 그 정확한 바이트를 살아 있고 옮겨지지 않은 채로 유지해야 한다. 이미 안정된 어딘가에 사는 키(문자열 리터럴, interned 문자열)에 가장 저렴하다.
- PROVEN_KEY_TYPE_U8_OWNED: 삽입 시(proven_map_set_u8_owned / PROVEN_MAP_SET_U8_OWNED) 맵이 키 바이트를 자기 저장소로 **복제**하고 remove/destroy 시 해제하므로, 호출 직후 원본 버퍼를 해제하거나 재사용해도 된다.

set_with_scratch / 별칭(alias) 사례

proven_map_set에 넘기는 element가 *맵 자신의 버킷 배열 안*을 가리킨다면(예를 들어 proven_map_get이 반환한 포인터를 통해 한 키에서 다른 키로 값을 복사하는 경우), 바로 그 삽입이 유발하는 재해싱이 여러분이 읽고 있는 바이트를 옮기거나 해제할 수 있다. proven_map_set_with_scratch(그리고 *_WITH_SCRATCH_* 매크로)는 원본 바이트를 먼저 임시 작업용(scratch) allocator의 임시 버퍼로 담아두므로 삽입이 별칭 안전(alias-safe)해진다. 영구 저장소는 여전히 map->alloc을 쓴다. scratch allocator는 그 일시적 복사에만 쓰인다.

함수

API	의도	반환
proven_map_create(alloc, init_cap, key_type, elem_size, align)	맵을 만든다.	proven_result_map_t.
proven_map_is_valid(map)	맵 불변식을 검증한다.	bool.
proven_map_reserve(map, new_cap)	용량을 보장한다.	proven_err_t.
proven_map_set_with_scratch(map, key, element, scratch)	임시 별칭 안전 복사에 scratch를 써서 삽입/갱신한다.	proven_err_t.
proven_map_set(map, key, element)	삽입/갱신한다.	proven_err_t.
proven_map_set_u8_owned(map, key, element)	맵이 소유하는 U8 키 저장소로 삽입/갱신한다.	proven_err_t.
proven_map_get_mut(map, key)	가변 값을 조회한다.	포인터 또는 null.
proven_map_get(map, key)	const 값을 조회한다.	포인터 또는 null.
proven_map_remove(map, key)	키가 있으면 제거한다.	proven_err_t.
proven_map_destroy(map)	맵 저장소를 해제한다.	void.

매크로

매크로	의도
proven_map_create_with_capacity(...)	용량 할당을 강조하는 별칭.
PROVEN_MAP_INIT_INT(alloc, type, init_cap)	정수 키 타입 맵을 만든다.
PROVEN_MAP_INIT_U8_BORROWED(alloc, type, init_cap)	borrowed 문자열 키 타입 맵을 만든다.
PROVEN_MAP_INIT_U8_OWNED(alloc, type, init_cap)	owned 문자열 키 타입 맵을 만든다.
PROVEN_MAP_SET_INT(map_ptr, int_key, type, value)	정수 키를 설정한다.

PROVEN_MAP_SET_WITH_SCRATCH_INT(map_ptr, int_key, type, value, scratch)	scratch allocator를 써서 정수 키를 설정한다.
PROVEN_MAP_SET_U8_BORROWED(map_ptr, u8_view, type, value)	borrowed U8 키를 설정한다.
PROVEN_MAP_SET_U8_OWNED(map_ptr, u8_view, type, value)	owned U8 키를 설정한다.
PROVEN_MAP_SET_WITH_SCRATCH_U8_BORROWED(map_ptr, u8_view, type, value, scratch)	scratch allocator를 써서 borrowed U8 키를 설정한다.
PROVEN_MAP_GET_INT(map_ptr, type, int_key)	정수 키로 const 값을 얻는다.
PROVEN_MAP_GET_U8_BORROWED(map_ptr, type, u8_view)	borrowed U8 키로 const 값을 얻는다.
PROVEN_MAP_GET_U8_OWNED(map_ptr, type, u8_view)	owned U8 키로 const 값을 얻는다.
PROVEN_MAP_GET_MUT_INT(map_ptr, type, int_key)	정수 키로 가변 값을 얻는다.
PROVEN_MAP_GET_MUT_U8_BORROWED(map_ptr, type, u8_view)	borrowed U8 키로 가변 값을 얻는다.
PROVEN_MAP_GET_MUT_U8_OWNED(map_ptr, type, u8_view)	owned U8 키로 가변 값을 얻는다.
PROVEN_MAP_REMOVE_INT(map_ptr, int_key)	정수 키를 제거한다.
PROVEN_MAP_REMOVE_U8_BORROWED(map_ptr, u8_view)	U8 키를 제거한다.
PROVEN_MAP_REMOVE_U8_OWNED(map_ptr, u8_view)	owned U8 키를 제거한다.
PROVEN_MAP_DESTROY(map_ptr)	맵을 파괴한다.

owned 키 경로:

PROVEN_KEY_TYPE_U8_OWNED는 키 바이트의 복제본을 맵 내부에 저장한다. 원본 버퍼가 삽입 후 옮겨지거나 해제될 수 있을 때 사용하라. owned 바이트는 remove와 destroy 시 해제되며, 재해싱 동안 항목과 함께 이동한다.

PROVEN_HARDENED와 디버그 검증은 맵 자신의 내부 저장소 안에 들어가는 일부 borrowed 키 포인터를 거부할 수 있다. 그 검사는 방어적일 뿐이며, borrowed 키를 자기 소유(self-owning)로 만들지는 않는다.

예제:

```
typedef struct UserInfo {
    int level;
    double budget;
} UserInfo;

proven_result_map_t r = PROVEN_MAP_INIT_INT(alloc, UserInfo, 8);
if (!proven_is_ok(r.err)) {
    return;
}
proven_map_t users = r.value;

UserInfo u = { .level = 3, .budget = 99.0 };
(void)PROVEN_MAP_SET_INT(&users, 404, UserInfo, u);

/* GET returns a pointer into the bucket array, or NULL when absent. Any insert
 * that rehashes invalidates it: look it up, use it, drop it. */
const UserInfo *found = PROVEN_MAP_GET_INT(&users, UserInfo, 404);
if (found) {
    proven_println("level={}", PROVEN_ARG(found->level));
}

PROVEN_MAP_DESTROY(&users);
```

5. 알고리즘

배열 알고리즘은 `proven_array_t`와 비교 콜백에 대해 동작한다.

비교 함수

```
typedef int (*proven_compare_fn_t)(const void *a, const void *b);
```

$a < b$ 이면 음수, 같으면 0, $a > b$ 이면 양수를 반환한다.

함수

API	의도	반환
<code>proven_array_sort(arr, cmp)</code>	배열을 제자리(in place) 정렬한다.	void.
<code>proven_array_binary_search(arr, key, cmp)</code>	정렬된 배열을 검색한다.	원소 포인터 또는 null.
<code>proven_array_linear_search(arr, key, cmp)</code>	임의의 배열을 스캔하며 검색한다.	원소 포인터 또는 null.

`proven_array_sort`는 introsort다: Bentley-McIlroy 3방향 분할, 작은 범위에 대한 insertion-sort 컷오프, 그리고 재귀가 $2 \cdot \log_2(n)$ 레벨을 넘으면 쓰이는 heapsort fallback으로 이루어진다.

명시할 만한 성질이 둘 있다. 실제로 발목을 잡는 것들이기 때문이다:

- **$O(n \log n)$ 은 전형적인 경우가 아니라 보장이다.** heapsort fallback이 그것을 보장으로 만든다. 적대적 순서로 최악의 경우에 도달할 수 있는 정렬은, 자기가 작성하지 않은 데이터를 정렬하는 어떤 프로그램에서도 서비스 거부다.
- **중복 키는 느린 경우가 아니라 빠른 경우다.** 피벗과 같은 원소들은 최종적이며 결코 재귀되지 않는 런(run)으로 모이므로, 전부-같은 입력은 한 번의 패스로 끝난다. 이것이 중요한 이유는 저기수성(low-cardinality) 키 — 상태 열, enum, 버킷 id — 가 실제로 호출자들이 정렬 기준으로 삼는 것이고, 그것들이 바로 순진한 2방향 분할이 퇴화하는 대상이기 때문이다.

정렬은 안정(stable)하지 않다: 같은 원소들은 재배열될 수 있다.

비교자는 평범한 파일 스코프 함수이므로, 형태는 이렇다:

```
static int cmp_int(const void *a, const void *b) {
    int x = *(const int *)a;
    int y = *(const int *)b;
    return (x > y) - (x < y);    /* not (x - y): that overflows */
}
```

```
proven_array_sort(&nums, cmp_int);
int key = 20;
int *hit = proven_array_binary_search(&nums, &key, cmp_int);
```

이 장 끝의 완성 예제(`manual/examples/ex_04_array.c`)가 바로 이것을 컴파일하고 실행한 버전이다.

6. 용도별 해싱

단 하나의 "해시"란 없다. 어느 함수가 올바른지는 그 결과로 무엇을 하려는지에 전적으로 달려 있으며, 잘못된 것을 집으면 쓸데없이 느린 프로그램이나 조용히 안전하지 않은 프로그램을 얻는다. `proven/hash.h`는 작업당 정확히 하나의 primitive를 제공하므로, 작업 이름을 대는 순간 선택이 끝난다:

여러분이...	사용	그리고 결정적으로
여러분 자신의 테이블에 키를 해싱, 신뢰된 입력	<code>proven_hash_bytes</code> (FNV-1a)	빠르다; 여기서 crypto 해시는 이득 없이 ~50배 느리다
신뢰할 수 없는 입력의 키를 테이블에 해싱	<code>proven_hash_keyed</code> (SipHash-2-4)	FNV는 공격자가 모든 키를 한 버킷에 충돌시켜 $O(1)$ 테이블을 $O(n^2)$ 로 바꾸게 한다
데이터가 전송/디스크에서 손상되지 않았는지 확인	<code>proven_crc32</code>	체크섬; gzip/zlib/PNG와 상호운용된다

콘텐츠 지문 채취: 중복 제거, 콘텐츠 주소 지정, "같은 파일?"	proven_sha256	고의로 위조된 일치에 대해 안전한 유일한 것
---------------------------------------	---------------	--------------------------

기억할 한 줄: **CRC-32와 FNV는 사고를 탐지하지 공격을 탐지하지 않는다.** 누군가 여러분을 속여서 이득을 볼 수 있을 때 두 가지가 "같은지"를 결정하는 데 이것들을 쓰지 말라 — 그것이 proven_sha256이 있는 이유다. 그리고 keyed 해시는 키가 진짜 무작위성에서 한 번 고른 진짜 비밀일 때에만 안전하다; 고정 키는 키가 아닌 것이나 마찬가지다.

모든 함수는 바이트 단위로 정확하며 엔디안에 독립적이다: 같은 입력은 어떤 타겟에서든 같은 출력을 준다. 왜냐하면 기계에 따라 바뀌는 지문은 콘텐츠의 지문이 아니라 기계의 지문일 것이기 때문이다. 넷 모두 로열티 없는 알고리즘이며(FNV, CRC-32: public domain; SipHash: CC0; SHA-256: FIPS 180-4, 특허 없음), 각 명세로부터 구현되었고 각각의 공식 알려진 답(known-answer) 벡터에 대해 검증되었다.

참조

API	의도	반환
proven_hash_bytes(view)	FNV-1a 64. 여러분이 직접 고른 키를 위한 해시 테이블 해시.	proven_u64.
proven_hash_keyed(view, key[16])	16바이트 비밀 아래의 SipHash-2-4. 같은 작업이되, 공격자가 공급하는 키를 위한 것.	proven_u64.
proven_crc32(view)	한 번에 하는 CRC-32(IEEE, reflected) — gzip/zlib/PNG가 지니는 그것.	proven_u32.
proven_crc32_update(crc, view)	체크 스트림에 대한 같은 CRC. 0에서 시작하라; 호출 사이에 여러분이 쥐고 있는 값이 진짜 CRC이므로, 저장하고, 기록하고, 재개할 수 있다.	proven_u32.
proven_sha256(view, out[32])	한 번에 하는 SHA-256.	void; PROVEN_SHA256_SIZE 바이트를 쓴다.
proven_sha256_init/_update/_final	한 번에 메모리에 담을 수 없는 콘텐츠에 대한 같은 다이제스트. 다이제스트는 오직 바이트에만 의존하며, 어떻게 체크되었는지에는 결코 의존하지 않는다.	void.
proven_sha256_to_hex(digest, out[65])	sha256sum과 git이 출력하는 64자 소문자 표기. NUL 종료.	void.

여러분이 쥐는 유일한 구조체:

```
typedef struct {
    /* Opaque. A running SHA-256: the 8-word chain value, a 64-byte block being
       filled, and the message length. Declare one, init it, update it, finalise it. */
} proven_sha256_t;
```

```
#define PROVEN_SHA256_SIZE 32 /* the digest; size your output buffer with this */
```

proven_sha256_t는 아무것도 할당하지 않는다 — **호출자 소유 상태**이므로 파괴할 것이 없다.

주의사항, 그리고 무엇이 잘못되는가

고정 키를 가진 keyed 해시는 keyed 해시가 아니다. proven_hash_keyed의 보안은 전적으로 키가 진짜 무작위성에서 한 번 뽑은 비밀이라는 데 달려 있다. 이것이 proven_map_create가 그것을 대신 해주는 이유이자, 테이블을 위해 여러분이 직접 proven_hash_keyed를 호출하는 일이 거의 없어야 하는 이유다.

잘못된 예:

```
proven_byte_t key[16] = { 0 };          /* wrong: a "secret" everyone knows */
proven_u64 h = proven_hash_keyed(user_input, key);

proven_byte_t key[16];
memcpy(key, &timestamp, sizeof timestamp); /* wrong: guessable, and mostly zero */
```

올바른 예 — 시작 시점에 OS에서 한 번 뽑아라:

```
proven_byte_t key[16];
if (proven_random_bytes(key, sizeof key)) {
    proven_u64 h = proven_hash_keyed(
        proven_mem_view_from_u8(PROVEN_LIT("session-token")), key);
    (void)h;
}
```

두 가지가 "같다"고 결정하는 데 CRC-32나 FNV를 쓰지 말라 — 누군가 여러분을 속여서 이득을 볼 수 있을 때는. 둘 다 사소하게 위조할 수 있다: 같은 CRC-32를 갖는 두 번째 입력을 만드는 것은 교과서 수준의 산수다.

잘못된 예 — 공격자가 오염시킬 수 있는 콘텐츠 주소 지정 저장소:

```
if (proven_crc32(incoming) == stored_crc) {
    accept_as_identical(incoming); /* wrong: a forged collision costs seconds */
}
```

올바른 예 — 속일 수 없어야 하는 지문은 proven_sha256이며, 32바이트 전체에 걸쳐 비교한다.

PROVEN_SHA256_SIZE가 아닌 다이제스트 버퍼는 버퍼 오버플로다. proven_sha256은 정확히 32바이트를 쓰며, 여러분의 배열이 얼마나 큰지 알지 못한다.

```
proven_byte_t digest[16];          /* wrong: SHA-256 is 32 bytes */
proven_sha256(data, digest);       /* writes 32 into a 16-byte buffer */
```

테스트 스위트가 컴파일하고 실행한다:

```
/*
 * 쓰임새로 나눈 해시. 이 모듈은 할 일마다 함수를 정확히 하나씩 준다. 그러니 고를 것은
 * "내가 할 일이 무엇인가" 하나뿐이고, *그것*을 틀리는 것이 위험의 전부다.
 */

int main(void) {
    proven_mem_view_t data = proven_mem_view_from_u8(PROVEN_LIT("the quick brown fox"));

    /* 할 일 1: 믿을 수 있는 입력을 내 표의 자리로. 빠르고, 암호용이 아니다. */
    proven_u64 table_hash = proven_hash_bytes(data);
    EXAMPLE_REQUIRE(table_hash != 0, "FNV-1a produces a spread-out 64-bit value");

    /* 할 일 2: *믿을 수 없는* 입력을 표의 자리로. 목적은 같지만, 입력을 고르는 공격자도
     * 모듈을 충돌시킬 수 없다. 키를 모르기 때문이다. 키는 시작할 때 진짜 난수로 한 번
     * 고른다. 고정된 키를 쓰면 이 방법의 뜻이 사라진다. */
    proven_byte_t key[16] = { 0 }; /* 실제 코드에서는 난수원에서 한 번 채운다 */
    proven_u64 safe_hash = proven_hash_keyed(data, key);
    EXAMPLE_REQUIRE(safe_hash != table_hash, "a keyed hash is a different function");

    /* 할 일 3: 이 바이트들이 상했는가? 해시가 아니라 검사합이다. gzip/zlib/PNG 와
     * 그대로 맞물린다 - 셋 다 정확히 이 CRC-32 를 쓴다. */
    proven_u32 checksum = proven_crc32(data);
```

```

/* 널리 쓰이는 CRC-32 확인값이다. 진짜 그 함수라는 것을 눈으로 볼 수 있다. */
EXAMPLE_REQUIRE(proven_crc32(proven_mem_view_from_u8(PROVEN_LIT("123456789"))) == 0xcbf43926u,
    "CRC-32 of \"123456789\" is the shared check value");
(void)checksum;

/* 할 일 4: 내용의 지문 - 중복 제거, 내용 주소화, "이 둘이 같은 파일인가". 일치할
 * 위조하려는 상대 앞에서도 안전하게 답한다. 답이 속아 넘어가면 안 되는 자리에서
 * 잡는 것이 이것이다. */
proven_byte_t digest[PROVEN_SHA256_SIZE];
proven_sha256(data, digest);

char hex[65];
proven_sha256_to_hex(digest, hex);
/* sha256sum 과 git 이 찍는 것과 같은 표기라, 그대로 맞물린다. */
EXAMPLE_REQUIRE(hex[64] == '\0' && proven_cstr_len(hex) == 64,
    "a SHA-256 fingerprint is 64 lowercase hex characters");

/* SHA-256 은 흘려 넣을 수도 있다. 한 번에 기억에 담을 수 없는 내용을 위해서다 -
 * 다이제스트는 바이트에만 달렸지, 몇 토막으로 나눠 넣었는지에 달리지 않는다. */
proven_sha256_t ctx;
proven_sha256_init(&ctx);
proven_sha256_update(&ctx, proven_mem_view_from_u8(PROVEN_LIT("the quick ")));
proven_sha256_update(&ctx, proven_mem_view_from_u8(PROVEN_LIT("brown fox")));
proven_byte_t streamed[PROVEN_SHA256_SIZE];
proven_sha256_final(&ctx, streamed);

bool same = true;
for (proven_size_t i = 0; i < PROVEN_SHA256_SIZE; ++i) {
    if (streamed[i] != digest[i]) same = false;
}
EXAMPLE_REQUIRE(same, "two updates of the halves equal one hash of the whole");

return EXAMPLE_OK();
}

```

7. 바이트를 텍스트로: hex와 Base64

일단 어떤 것을 해싱할 수 있고(위) 무작위 토큰을 뽑을 수 있으면(random.h), 그 바이트를 텍스트만 담는 곳 — URL, HTTP 헤더, 로그 줄, JSON 문자열 — 에 써야 한다. 그것이 encode.h다. 암호도, 압축도 없다; 이미 모두가 합의한 두 인코딩을, 숨은 할당 없이, 그리고 보통 잘못되는 두 가지 방식 없이 해낸다.

원하는 것	사용	알파벳
사람이 읽는 다이제스트나 몇 바이트	proven_hex_encode	소문자 hex, sha256sum과 git이 출력하는 것
URL, 쿠키, 파일명 안의 바이트	proven_base64url_encode	- _, 패딩 없음 — 이스케이프할 것도, 망가뜨릴 =도 없음
HTTP 헤더, MIME, JSON 안의 바이트	proven_base64_encode	표준 + /, = 패딩

핵심은 두 가지 거부다:

- 디코더는 한 바이트를 쓰기 전에 입력 전체를 검증한다. 프로그램 바깥에서 온 텍스트는 유효하다고 보장되지 않는다; 엉뚱한 문자, 잘못된 길이, 잘못된 패딩, 끼어든 공백은 아무것도 커밋하지 않은 채 PROVEN_ERR_INVALID_ENCODING이다 — 끝을 넘어선 읽기도 아니고, 호출자가 한 바이트 들어가서 손상을 발견하게 되는 조용히 짧은 결과도 아니다. proven_base64_decode는 알파벳 둘 다와 패딩 있음/없음을 받아들인다. 자기가 방출하는 것만 받는 디코더는 세상의 Base64의 절반을 거부하기 때문이다.

- **출력 크기는 추측이 아니라 호출이다** — `proven_hex_encoded_size`, `proven_base64_encoded_size`, 그리고 그들의 디코드 짝. 한 바이트 모자란 버퍼는 아무것도 쓰지 않은 채 `PROVEN_ERR_OUT_OF_BOUNDS`이며, 결코 잘린 접두사가 아니다.

이것은 순수 계산이다 — 할당도, OS도 없다 — 그리고 freestanding에서 이용 가능하다.

참조

모든 호출은 입력, 호출자 소유 출력 버퍼와 그 용량, 그리고 선택적 `written_out`을 받는다. 그 어느 것도 할당하지 않는다.

API	의도	반환
<code>proven_hex_encoded_size(n)</code>	n 바이트에 대해 <code>proven_hex_encode</code> 가 쓸 문자 수: $n * 2$, NUL 없음.	<code>proven_size_t</code> . $n * 2$ 가 들어가지 않으면 <code>PROVEN_SIZE_MAX</code> .
<code>proven_hex_decoded_size(n)</code>	n 문자에 대해 <code>proven_hex_decode</code> 가 쓸 바이트 수: $n / 2$.	<code>proven_size_t</code> .
<code>proven_hex_encode(data, out, cap, &w)</code>	소문자 hex.	<code>PROVEN_OK</code> ; <code>cap</code> 이 모자라면 <code>OUT_OF_BOUNDS</code> (아무것도 쓰지 않음); NULL <code>out</code> 이나 <code>{NULL, >0}</code> view에는 <code>INVALID_ARG</code> .
<code>proven_hex_decode(text, out, cap, &w)</code>	hex를 디코드; 대문자와 소문자 둘 다 허용.	홀수 길이나 hex가 아닌 바이트에는 <code>INVALID_ENCODING</code> (아무것도 커밋하지 않음).
<code>proven_base64_encoded_size(n)</code>	두 Base64 형식 모두에 대한 상한: $4 * \text{ceil}(n/3)$.	<code>proven_size_t</code> . 그 값이 들어가지 않으면 <code>PROVEN_SIZE_MAX</code> .
<code>proven_base64_decoded_size(n)</code>	패딩 있는 것과 없는 텍스트에 대한 상한: $3 * \text{ceil}(n/4)$.	<code>proven_size_t</code> .
<code>proven_base64_encode(data, out, cap, &w)</code>	표준 알파벳(+ /), = 패딩.	위와 같음.
<code>proven_base64url_encode(data, out, cap, &w)</code>	URL-안전 알파벳(- _), 패딩 없음.	위와 같음.
<code>proven_base64_decode(text, out, cap, &w)</code>	디코드. 알파벳 둘 다, 패딩 있음/없음을 받아들인다.	영뚱한 바이트, 잘못된 길이, 잘못된 패딩에는 <code>INVALID_ENCODING</code> .

`written_out`은 NULL이어도 된다. 진입 시와 모든 오류 경로에서 0으로 설정되므로, 결코 낡은 값이 아니다.

주의사항, 그리고 무엇이 잘못되는가

출력 크기는 눈대중이 아니라 크기 함수로 잡아라. 인코더는 잘라내는 대신 모자란 버퍼를 거부한다 — 즉 손으로 계산한 크기에서 얻는 실패는 조용한 손상이 아니라 여러분이 처리해야 하는 오류다. 그것이 좋은 결과다; 요점은 여러분이 여전히 그것을 처리해야 한다는 것이다.

잘못된 예 — 고전적인 하나 차이(off-by-one) 실수이며, 이제 오버플로 대신 *실패*한다:

```
proven_byte_t out[16];                                /* wrong: 12 bytes of hex needs 24 chars */
proven_hex_encode(twelve_bytes, out, sizeof out, &w); /* OUT_OF_BOUNDS, nothing written */
```

올바른 예:

```
proven_mem_view_t data = proven_mem_view_from_u8(PROVEN_LIT("twelve bytes"));
proven_byte_t out[64];
proven_size_t w = 0;
if (proven_hex_encoded_size(data.size) <= sizeof out &&
    proven_is_ok(proven_hex_encode(data, out, sizeof out, &w))) {
    /* `w` characters of lowercase hex in `out` - not NUL-terminated */
}
```

Base64URL은 패딩을 방출하지 않으며, 디코더는 그것을 받아들인다. proven_base64_decoded_size는 *올림*하므로 패딩 없는 텍스트에 대해서도 올바른 상한이다. 그것을 $3 * (n / 4)$ 로 "개선"하지 말라 — 그것은 패딩 없는 꼬리가 실어 나르는 1~2바이트를 버림하며, 여러분은 이 라이브러리 자신의 URL-안전 출력을 디코드하는데 실패할 것이다. (실제로 한 번 그런 일이 있었고, 감사(audit)가 그것을 잡아냈다.)

```
proven_size_t cap = 3 * (text_len / 4); /* wrong: 0 for "QQ", which decodes to 1 byte */
```

디코더를 직접 손으로 만들지 말라. 이것들이 존재하는 이유는 바로, 디코드가 여러분의 프로그램 바깥에서 텍스트를 읽고, 모두가 작성하는 두 줄짜리 루프가 그것을 신뢰하기 때문이다.

잘못된 예 — 홀수 입력에서 끝을 넘어 읽고, 쓰레기를 데이터로 받아들인다:

```
for (size_t i = 0; i < len; i += 2) /* wrong: no length check */
    out[i/2] = (hexval(text[i]) << 4) | hexval(text[i+1]); /* wrong: no validation */
```

proven_hex_decode는 한 바이트를 쓰기 전에 입력 전체를 검증하므로, 끝 근처의 엉뚱한 문자 하나가 여러분이 완전하다고 믿는 절반만 디코드된 점두사를 쥐게 만들 수 없다.

공백은 일부러 건너뛰지 않는다. 붙여넣은, 줄바꿈된 Base64 덩어리는 INVALID_ENCODING이지 조용히 다른 결과가 아니다. 줄바꿈된 입력을 받아들이고 싶다면, 공백을 직접 제거하라 — 의도적으로, 여러분이 볼 수 있는 곳에서.

목적지 크기는 손으로 짐작하지 말고 크기 함수로 구한다. 인코딩마다 짝이 하나씩 있고 —

proven_hex_encoded_size() / proven_hex_decoded_size(), 그리고 proven_base64_encoded_size() / proven_base64_decoded_size() — 둘이 약속하는 것이 다르다.

방향	그 수의 의미	쓰는 법
인코딩	hex와 표준 Base64에서는 정확한 값 . Base64URL은 패딩을 빼므로 결과가 더 짧을 수 있으나, 이 값은 여전히 안전한 상한이다.	이만큼 할당한다. 호출이 실제로 쓴 양을 알려 준다.
디코딩	상한 이다. 패딩과 쓰인 알파벳은 텍스트를 읽기 전에는 알 수 없다.	상한만큼 할당한 뒤, 길이로는 상한이 아니라 보고된 개수를 쓴다.

아래 예제는 그것으로 끝맺는다. proven_base64_encoded_size()로 크기를 잡아 목적지를 할당하고, 거기에 인코딩하고, proven_base64_decoded_size()로 역방향 버퍼 크기를 잡아 디코딩하고, 같은 바이트를 proven_hex_decode()로도 왕복시킨다 — 실제 도구들이 내놓는 대문자 입력과, 온전한 바이트 수가 될 수 없는 홀수 길이 입력까지 포함해서다.

테스트 스위트가 컴파일하고 실행한다:

```
/*
 * 바이트를 글자로, 쓰임새별로. 규칙은 해시와 같다. 할 일마다 함수 하나, 그리고 위험은
 * 할 일을 잘못 고르는 것. 사람이 읽을 것에는 hex, URL 에 들어갈 것에는 Base64URL,
 * 선로로 나갈 것에는 표준 Base64.
 */

int main(void) {
    proven_mem_view_t data = proven_mem_view_from_u8(PROVEN_LIT("the quick brown fox"));

    /* 할 일 1: 사람이 읽거나 붙여 넣을 다이제스트 - hex, sha256sum 과 git 이 쓰는 표기. */
    proven_byte_t hex[64]; /* proven_hex_encoded_size(19) = 38 */
    proven_size_t hn = 0;
    EXAMPLE_REQUIRE(proven_is_ok(proven_hex_encode(data, hex, sizeof hex, &hn)),
        "hex encode into a buffer sized by proven_hex_encoded_size");
    EXAMPLE_REQUIRE(hn == proven_hex_encoded_size(data.size), "two hex chars per byte");

    /* 할 일 2: URL 에 들어갈 토큰 - Base64URL. 퍼센트 이스케이프가 필요 없고, 파서가
     * 걸려 넘어질 '=' 채움도 없다. */
    proven_byte_t token_bytes[16] = { 0 }; /* 실제 코드에서는: proven_random_bytes(token_bytes, 16) */
```

```

proven_byte_t url[32];
proven_size_t un = 0;
EXAMPLE_REQUIRE(proven_is_ok(proven_base64url_encode(
    (proven_mem_view_t){ token_bytes, sizeof token_bytes }, url, sizeof url, &un)),
    "base64url encode a token");
/* URL 안전 토큰에는 '=' 가 없다. */
bool has_pad = false;
for (proven_size_t i = 0; i < un; ++i) if (url[i] == '=') has_pad = true;
EXAMPLE_REQUIRE(!has_pad, "the URL form emits no padding");

/* 할 일 3: 선행로 나가는 바이트 - 표준 Base64, HTTP 와 MIME 이 기대하는 +/- 문자표. */
proven_byte_t b64[64];
proven_size_t bn = 0;
EXAMPLE_REQUIRE(proven_is_ok(proven_base64_encode(data, b64, sizeof b64, &bn)),
    "standard base64 encode");

/* 그리고 왕복한다. 디코드는 바이트를 그대로 되돌려 준다. 두 문자표와 채움이 있든
 * 없든 받아 주는 디코더는 일부러 그렇게 만든 것이다 - 실제 입력은 온갖 모양으로 온다. */
proven_byte_t back[32];
proven_size_t dn = 0;
EXAMPLE_REQUIRE(proven_is_ok(proven_base64_decode(
    (proven_mem_view_t){ b64, bn }, back, sizeof back, &dn)),
    "decode the base64 back");
EXAMPLE_REQUIRE(dn == data.size && proven_memcmp(back, data.ptr, dn) == 0,
    "what comes back is exactly what went in");

/* 검증하는 디코더의 요점: 쓰레기는 추측하지 않고 거부한다. 이것을 두 줄짜리 반복문에
 * 먹었다면 끝을 넘어 읽거나 조용히 짧은 결과를 얻었을 것이다. */
proven_err_t bad = proven_base64_decode(
    proven_mem_view_from_u8(PROVEN_LIT("not valid base64!!")), back, sizeof back, &dn);
EXAMPLE_REQUIRE(bad == PROVEN_ERR_INVALID_ENCODING,
    "a stray character is INVALID_ENCODING, with nothing committed");

/* 그리고 한 바이트 모자란 버퍼는 거부된다. 잘리는 일은 없다. */
proven_byte_t tiny[4];
EXAMPLE_REQUIRE(proven_hex_encode(data, tiny, sizeof tiny, &hn) == PROVEN_ERR_OUT_OF_BOUNDS,
    "a too-small output buffer is OUT_OF_BOUNDS, not a truncated prefix");

/* --- 짐작하지 말고 목적지 버퍼의 크기를 셈하기 ----- */

/* 위의 인코드와 디코드는 모두 넉넉한 것이 뻔한 고정 배열에 썼다. 실제 코드는
 * 목적지를 할당하고, 그러면 크기는 기억해 내는 것이 아니라 셈해야 한다. 크기 함수
 * 넷이 그것을 위한 것이다 - 인코딩마다, 방향마다 하나씩. */
proven_allocator_t alloc = proven_heap_allocator();

proven_size_t need = proven_base64_encoded_size(data.size);
EXAMPLE_REQUIRE(need > 0, "an encoded size must be computed, not guessed");

proven_result_mem_mut_t out = alloc.alloc_fn(alloc.ctx, need, 1);
EXAMPLE_REQUIRE(proven_is_ok(out.err), "allocating exactly the encoded size must succeed");

proven_size_t wrote = 0;
EXAMPLE_REQUIRE(proven_is_ok(proven_base64_encode(data, out.value.ptr, out.value.size, &wrote)),
    "encoding into a buffer sized by the size function must fit exactly");
EXAMPLE_REQUIRE(wrote == need, "and use all of it: this size is exact for standard base64");

/* 디코드 방향은 정확한 개수가 아니라 *상한*이다. base64 글에는 채움이 있을 수
 * 있으므로, 디코더가 실제로 몇 바이트를 썼는지 알려 준다. 버퍼는 상한으로 잡고,
 * 그다음에는 알려 준 개수를 쓸 것. */
proven_size_t bound = proven_base64_decoded_size(wrote);

```

```

EXAMPLE_REQUIRE(bound >= data.size, "the decoded bound must cover the real output");

proven_result_mem_mut_t plain = alloc.alloc_fn(alloc.ctx, bound, 1);
EXAMPLE_REQUIRE(proven_is_ok(plain.err), "allocating the decode bound must succeed");

proven_size_t got = 0;
EXAMPLE_REQUIRE(proven_is_ok(proven_base64_decode(
    (proven_mem_view_t){ out.value.ptr, wrote }, plain.value.ptr, plain.value.size,
    &got)),
    "decoding back must succeed");
EXAMPLE_REQUIRE(got == data.size && proven_memcmp(plain.value.ptr, data.ptr, got) == 0,
    "and reproduce the original bytes exactly");

/* hex 도 같은 짝을 갖는다. 입력 길이가 짝수일 때 디코드 상한은 정확하고, 올바른
 * hex 는 언제나 짝수다 - 홀수 길이는 잘못된 입력이고, 디코더는 마지막 자리를
 * 버리는 대신 그렇다고 말한다. */
proven_size_t hex_len = 0;
EXAMPLE_REQUIRE(proven_is_ok(proven_hex_encode(data, hex, sizeof hex, &hex_len)),
    "re-encode to hex, since the refused call above left its count unusable");

proven_size_t hex_bound = proven_hex_decoded_size(hex_len);
EXAMPLE_REQUIRE(hex_bound == data.size, "two hex characters decode to one byte");

proven_byte_t from_hex[32];
proven_size_t hex_got = 0;
EXAMPLE_REQUIRE(proven_is_ok(proven_hex_decode(
    (proven_mem_view_t){ hex, hex_len }, from_hex, sizeof from_hex, &hex_got)),
    "hex decodes back to the original bytes");
EXAMPLE_REQUIRE(hex_got == data.size && proven_memcmp(from_hex, data.ptr, hex_got) == 0,
    "and the round trip is exact");

/* hex 는 들어올 때 대소문자를 가리지 않는다. 도구마다 같은 다이제스트를 다른
 * 대소문자로 찍기 때문에 중요하다. */
EXAMPLE_REQUIRE(proven_is_ok(proven_hex_decode(
    proven_mem_view_from_u8(PROVEN_LIT("DEADBEEF")), from_hex, sizeof from_hex,
    &hex_got)),
    "uppercase hex decodes too");
EXAMPLE_REQUIRE(hex_got == 4, "and yields one byte per two characters");

/* 글자 수가 홀수면 온전한 바이트 수가 될 수 없다. */
EXAMPLE_REQUIRE(proven_hex_decode(proven_mem_view_from_u8(PROVEN_LIT("abc")),
    from_hex, sizeof from_hex, &hex_got) == PROVEN_ERR_INVALID_ENCODING,
    "an odd-length hex string is malformed, not silently truncated");

alloc.free_fn(alloc.ctx, plain.value.ptr);
alloc.free_fn(alloc.ctx, out.value.ptr);

(void)url; (void)un;
return EXAMPLE_OK();
}

```

8. 예제와 오용 사례

배열 안을 가리키는 포인터는 낚을 수 있다

잘못된 예:

```

int *p = PROVEN_ARRAY_GET_MUT(&nums, int, 0);
PROVEN_ARRAY_PUSH(&nums, int, 30);
*p = 99; /* wrong: push may have reallocated the array */

```

올바른 예:

```
proven_result_array_t r = PROVEN_ARRAY_INIT(alloc, int, 2);
if (!proven_is_ok(r.err)) {
    return;
}
proven_array_t nums = r.value;
(void)PROVEN_ARRAY_PUSH(&nums, int, 10);

/* Push first, then take the pointer. A pointer fetched before a push may point
 * at a block the array has already reallocated away. */
(void)PROVEN_ARRAY_PUSH(&nums, int, 30);
int *p = PROVEN_ARRAY_GET_MUT(&nums, int, 0);
if (p) {
    *p = 99;
}

PROVEN_ARRAY_DESTROY(&nums);
```

하나의 침습적 노드는 한 번에 하나의 리스트에 속한다
잘못된 예:

```
proven_list_push_back(&list_a, &item.link);
proven_list_push_back(&list_b, &item.link); /* wrong */
```

멤버십마다 내장 노드를 하나씩 써라.

안전 매크로로 순회 중 제거하기

올바른 예:

```
typedef struct Item {
    int value;
    proven_list_node_t link;
} Item;

proven_list_t list;
proven_list_init(&list);
Item a = { .value = 1 };
Item b = { .value = 2 };
proven_list_push_back(&list, &a.link);
proven_list_push_back(&list, &b.link);

/* The SAFE form reads `it->next` into `next` before the body runs, so removing
 * `it` (which nulls its links) cannot strand the loop. */
proven_list_node_t *it = NULL;
proven_list_node_t *next = NULL;
PROVEN_LIST_FOR_EACH_SAFE(it, next, &list) {
    Item *item = PROVEN_LIST_ENTRY(it, Item, link);
    if (item->value % 2 == 0) {
        proven_list_remove(it);
    }
}
```

링 push는 커지지 않는다

잘못된 예:

```
PROVEN_RING_PUSH(&q, int, value); /* wrong if you ignore full-ring errors */
```

올바른 예:

```
proven_result_ring_t r = PROVEN_RING_INIT(alloc, int, 2);
if (!proven_is_ok(r.err)) {
    return;
}
```

```

}
proven_ring_t q = r.value;

int value = 7;
for (int i = 0; i < 3; ++i) {
    proven_err_t e = PROVEN_RING_PUSH(&q, int, value);
    if (e == PROVEN_ERR_OUT_OF_BOUNDS) {
        /* The ring is full. Drop the item, or drain one first - but decide. */
        int drop = 0;
        (void)PROVEN_RING_POP(&q, int, &drop);
        e = PROVEN_RING_PUSH(&q, int, value);
    }
    (void)e;
}

PROVEN_RING_DESTROY(&q);

```

borrowed 맵 키는 항목보다 오래 살아야 한다

잘못된 예:

```

proven_u8str_t key = make_key(alloc);
PROVEN_MAP_SET_U8_BORROWED(&m, proven_u8str_as_view(&key), int, 1);
proven_u8str_destroy(alloc, &key);
/* wrong: map still points at freed key bytes */

```

올바른 선택지:

- borrowed 키에는 문자열 리터럴이나 다른 안정된 저장소를 사용하라.
- owned 키 객체를 다른 곳에 저장하고, 맵 항목을 제거한 뒤 그것들을 파괴하라.
- 맵이 키 저장소를 소유해야 한다면 PROVEN_KEY_TYPE_U8_OWNED와 proven_map_set_u8_owned()를 사용하라.

같은 맵에서 borrow한 값을 삽입할 때는 scratch를 써라

element가 맵 저장소 안을 가리키고 삽입이 재해싱할 수 있다면, proven_map_set_with_scratch()나 scratch 매크로를 사용하라.

```

proven_result_map_t r = PROVEN_MAP_INIT_INT(alloc, int, 4);
if (!proven_is_ok(r.err)) {
    return;
}
proven_map_t m = r.value;
(void)PROVEN_MAP_SET_INT(&m, 1, int, 42);

/* `src` points into the map's own bucket array. Passing it straight to
 * proven_map_set would be an alias: a rehash triggered by that same insert can
 * free the bytes it is still reading from. The scratch form captures the source
 * bytes into a temporary from `scratch` first, so the insert is alias-safe. */
const int *src = PROVEN_MAP_GET_INT(&m, int, 1);
if (src) {
    (void)proven_map_set_with_scratch(&m, (proven_map_key_t){ .id = 2 }, src, scratch);
}

PROVEN_MAP_DESTROY(&m);

```

이진 검색은 정렬된 입력을 요구한다

잘못된 예:

```

int *hit = proven_array_binary_search(&nums, &key, cmp_int);
/* wrong if nums was not sorted by cmp_int */

```

완성 예제: 배열, 정렬, 이진 검색

테스트 스위트가 컴파일하고 실행한다. 중복 키에 대한 정렬의 동작에 주목하라: 그것들은 빠른 경우이지, 이차(quadratic)인 경우가 아니다.

```
/*
 * proven_array_t 는 원소 타입 하나짜리 늘어나는 벡터다. 만들 때 쓴 할당자를 스스로
 * 담아 두므로 push 는 늘릴 수 있고 destroy 는 할당자를 다시 받지 않고도 해제할 수
 * 있다 - 뒤집어 말하면 배열은 *자기 자신만으로* 지워져야 한다는 뜻이다.
 *
 * PROVEN_ARRAY_* 매크로는 void* 로 된 속을 타입 안전하게 감싼 얼굴이다. 타입에서
 * sizeof/alignof 를 뺀고 원소를 포인터로 건네므로, 레코드 배열에 int 를 밀어 넣는
 * 코드는 컴파일되지 않는다.
 */

/* 우선순위를 키로 하는 작업 큐. 실제 자료는 값의 가짓수가 적다 - 우선순위 셋에 작업은
 * 여럿. 그 사실이 정렬에서 중요하다. 아래를 볼 것. */
typedef struct {
    int priority;
    int id;
} task_t;

/* 비교자가 곧 배열의 순서다. 정렬과 이진 탐색에는 *반드시* 같은 것을 써야 한다 -
 * 다른 순서로 찾는 것은 컴파일러가 볼 수 없는 버그다.
 *
 * (x > y) - (x < y) 꼴은 일부러 쓴 것이다. 두 int 를 빼면 큰 값에서 넘쳐 엉뚱한
 * 부호를 돌려준다. */
static int cmp_priority(const void *a, const void *b) {
    int x = ((const task_t *)a)->priority;
    int y = ((const task_t *)b)->priority;
    return (x > y) - (x < y);
}

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* --- push / get / pop ----- */
    /* init_cap 은 한계가 아니라 힌트다. push 는 그것을 넘어 늘어난다. 잘 잡아 두면
     * 재할당을 피할 뿐이다. */
    proven_result_array_t r = PROVEN_ARRAY_INIT(alloc, task_t, 4);
    EXAMPLE_REQUIRE(proven_is_ok(r.err), "creating an array of task_t must succeed");
    if (!proven_is_ok(r.err)) {
        return 1;
    }
    proven_array_t tasks = r.value;

    /* 일부러 중복이 많게 했다. 실제 키가 그렇게 생겼기 때문이다 - 상태 열, enum,
     * 버킷 번호. */
    static const task_t seed[] = {
        { .priority = 2, .id = 10 }, { .priority = 0, .id = 11 },
        { .priority = 2, .id = 12 }, { .priority = 1, .id = 13 },
        { .priority = 2, .id = 14 }, { .priority = 0, .id = 15 },
        { .priority = 1, .id = 16 }, { .priority = 2, .id = 17 },
    };

    for (proven_size_t i = 0; i < sizeof seed / sizeof seed[0]; ++i) {
        proven_err_t err = PROVEN_ARRAY_PUSH(&tasks, task_t, seed[i]);
        EXAMPLE_REQUIRE(proven_is_ok(err), "pushing into a growable array must succeed");
        if (!proven_is_ok(err)) {
            PROVEN_ARRAY_DESTROY(&tasks);
            return 1;
        }
    }
}
```

```

    }
}
EXAMPLE_REQUIRE(tasks.len == 8, "eight pushes give eight elements");

/* get 은 배열 저장소 *안*을 가리키는 포인터를 돌려준다 - 사본이 아니고, 다음
 * push 가 재할당하면 매달린 포인터가 된다. push 를 다 한 뒤에 얻어 쓰고, 담아
 * 두지는 말 것. */
const task_t *front = PROVEN_ARRAY_GET(&tasks, task_t, 0);
EXAMPLE_REQUIRE(front && front->id == 10, "element 0 is the first one pushed");

/* 범위를 벗어나면 널 포인터다. 죽는 것도, 미정의 동작도 아니다. */
EXAMPLE_REQUIRE(PROVEN_ARRAY_GET(&tasks, task_t, 99) == NULL, "an out-of-range index yields NULL");

/* pop 은 마지막 원소를 복사해 내보낸다(그냥 버리려면 NULL 을 건넨다). */
task_t last = {0};
proven_err_t err = PROVEN_ARRAY_POP(&tasks, task_t, &last);
EXAMPLE_REQUIRE(proven_is_ok(err), "popping a non-empty array must succeed");
EXAMPLE_REQUIRE(last.id == 17 && tasks.len == 7, "pop returns the last element and shrinks the array");

/* 도로 넣는다. 이 예제의 나머지가 여덟 개를 모두 필요로 한다. */
err = PROVEN_ARRAY_PUSH(&tasks, task_t, last);
EXAMPLE_REQUIRE(proven_is_ok(err), "pushing the popped element back must succeed");

/* --- 정렬 ----- */
/* 인트로소트다.  $O(n \log n)$  은 평균이 아니라 보장이자 - 힙소트로 물러나는 길이
 * 있어, 상대가 유도할 수 있는 제곱 시간 경우를 배제한다. 그리고 여기서는 같은 키가
 * *빠른* 쪽이다. 피벗과 같은 것은 모두 한 구간으로 갈라 두고 다시 들어가지 않으므로,
 * 위의 중복된 우선순위는 서로 다른 값보다 값이 덜 든다.
 *
 * 안정 정렬은 아니다. 작업 10 과 12 는 어느 쪽이 먼저 나올지 정해져 있지 않다. */
proven_array_sort(&tasks, cmp_priority);

for (proven_size_t i = 1; i < tasks.len; ++i) {
    const task_t *prev = PROVEN_ARRAY_GET(&tasks, task_t, i - 1);
    const task_t *cur = PROVEN_ARRAY_GET(&tasks, task_t, i);
    EXAMPLE_REQUIRE(prev && cur && prev->priority <= cur->priority,
        "after sorting, priorities must be non-decreasing");
}

/* --- 이진 탐색 ----- */
/* 바로 이 비교자로 정렬해 두었기 때문에만 옳다. 키는 원소 하나 통째지만, 비교자가
 * 읽는 필드만 뜻이 있다. */
task_t key = { .priority = 1, .id = 0 };
const task_t *hit = (const task_t *)proven_array_binary_search(&tasks, &key, cmp_priority);
EXAMPLE_REQUIRE(hit != NULL, "priority 1 is present, so the search must find it");
EXAMPLE_REQUIRE(hit && hit->priority == 1, "the hit must be an element with the searched key");
/* 키가 중복되면 그 키를 가진 *어떤* 원소를 찾지, 첫 번째를 찾지는 않는다. 첫
 * 번째가 필요하면 찾은 자리에서 뒤로 훑을 것. */

task_t absent = { .priority = 9, .id = 0 };
EXAMPLE_REQUIRE(proven_array_binary_search(&tasks, &absent, cmp_priority) == NULL,
    "a key that is not there must return NULL");

printf("tasks: %zu sorted, found priority %d (id %d)\n",
    (size_t)tasks.len, hit ? hit->priority : -1, hit ? hit->id : -1);

/* 배열은 원소 저장소를 소유한다. destroy 는 배열을 만들 때 쓴 할당자로 그것을
 * 돌려준다. 여기 원소는 그냥 자료다 - 원소가 무언가를 소유했다면 하나씩 먼저
 * 지워야 했을 것이다. */
PROVEN_ARRAY_DESTROY(&tasks);

```

```

    return EXAMPLE_OK();
}

```

완성 예제: 정수 키 맵과 owned 문자열 키 맵

테스트 스위트가 컴파일하고 실행한다. owned 키 절반이 잘못 이해하기 쉬운 요점을 증명한다: 맵이 키를 복사하므로, 그것을 만든 버퍼는 즉시 재사용해도 되는 여러분의 것이다.

```

/*
 * proven_map_t 는 납작한 개방 주소법 해시 맵이다. 값 타입은 만들 때 정해지고 버킷
 * 배열 안에 그대로 담긴다 - 값을 위한 항목별 할당이 없고, get 은 그 배열 속을 곧장
 * 가리키는 포인터를 돌려준다.
 *
 * 재미있는 결정은 *키* 쪽이다.
 *
 *   PROVEN_KEY_TYPE_INT           - 키가 proven_size_t 다. 소유할 것이 없다.
 *   PROVEN_KEY_TYPE_U8_BORROWED - 버킷이 여러분의 포인터와 길이를 담는다. 맵은
 *                                   바이트를 복사하지 않으므로, 항목이 살아 있는
 *                                   동안 *여러분*이 그 바이트를 살아 있고 움직이지
 *                                   않게 지켜야 한다. 문자열 리터럴에 알맞다.
 *   PROVEN_KEY_TYPE_U8_OWNED     - 맵이 키 바이트를 제 저장소로 복사하고 나중에
 *                                   해제한다. 실행 중에 만든 키에 알맞고, 대개의
 *                                   키가 그렇다.
 *
 * 이 예제의 뒷부분이 OWNED 가 존재하는 이유다.
 */

typedef struct {
    int level;
    long score;
} player_t;

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* --- 정수 키 ----- */
    proven_result_map_t r = PROVEN_MAP_INIT_INT(alloc, player_t, 8);
    EXAMPLE_REQUIRE(proven_is_ok(r.err), "creating an int-keyed map must succeed");
    if (!proven_is_ok(r.err)) {
        return 1;
    }
    proven_map_t by_id = r.value;

    proven_err_t err = PROVEN_MAP_SET_INT(&by_id, 404, player_t, ((player_t){ .level = 3, .score = 990 }));
    EXAMPLE_REQUIRE(proven_is_ok(err), "inserting into the map must succeed");
    err = PROVEN_MAP_SET_INT(&by_id, 7, player_t, ((player_t){ .level = 1, .score = 10 }));
    EXAMPLE_REQUIRE(proven_is_ok(err), "inserting a second key must succeed");

    /* 이미 있는 키에 set 하면 값을 바꾼다. 항목을 더하지 않는다. */
    err = PROVEN_MAP_SET_INT(&by_id, 7, player_t, ((player_t){ .level = 2, .score = 40 }));
    EXAMPLE_REQUIRE(proven_is_ok(err), "re-setting an existing key must succeed");
    EXAMPLE_REQUIRE(by_id.len == 2, "re-setting a key replaces its value rather than adding an entry");

    /* get 은 버킷 배열 속을 가리키는 포인터를, 없으면 NULL 을 돌려준다. 다시 해싱하는
     * 삽입이 일어나면 무효가 된다 - 찾고, 쓰고, 버릴 것. */
    const player_t *p = PROVEN_MAP_GET_INT(&by_id, player_t, 7);
    EXAMPLE_REQUIRE(p && p->level == 2 && p->score == 40, "get must see the replaced value");
    EXAMPLE_REQUIRE(PROVEN_MAP_GET_INT(&by_id, player_t, 999) == NULL, "a missing key yields NULL");

    err = PROVEN_MAP_REMOVE_INT(&by_id, 7);
    EXAMPLE_REQUIRE(proven_is_ok(err), "removing a present key must succeed");

```

```

EXAMPLE_REQUIRE(PROVEN_MAP_GET_INT(&by_id, player_t, 7) == NULL, "a removed key is gone");
EXAMPLE_REQUIRE(by_id.len == 1, "removal decrements the live entry count");

PROVEN_MAP_DESTROY(&by_id);

/* --- 소유하는 문자열 키 ----- */
/* 같은 맵인데, 실행 중에 지어낸 이름을 키로 쓴다 - 빌린 키였다면 매달린 포인터가
 * 되기를 기다리는 자리다. */
proven_result_map_t rm = PROVEN_MAP_INIT_U8_OWNED(alloc, player_t, 8);
EXAMPLE_REQUIRE(proven_is_ok(rm.err), "creating an owned-string-keyed map must succeed");
if (!proven_is_ok(rm.err)) {
    return 1;
}
proven_map_t by_name = rm.value;

/* 키마다 다시 쓸 걱정인 임시 버퍼. BORROWED 맵에서 그 걱정은 치명적이다. 모든
 * 항목이 바로 이 같은 바이트를 가리키게 된다. */
proven_byte_t scratch[32];
proven_u8str_t name = proven_u8str_borrow(scratch, sizeof scratch);

err = proven_u8str_append(&name, PROVEN_LIT("ada"));
EXAMPLE_REQUIRE(proven_is_ok(err), "building the first key must succeed");

/* set_u8_owned 는 키 바이트를 맵 저장소로 *복사한다*. 돌아온 뒤로 맵의 키는
 * `scratch` 와 아무 상관이 없다. */
err = PROVEN_MAP_SET_U8_OWNED(&by_name, proven_u8str_as_view(&name), player_t,
    ((player_t){ .level = 9, .score = 5000 }));
EXAMPLE_REQUIRE(proven_is_ok(err), "inserting with an owned key must succeed");

/* 그래서 버퍼는 곧바로 다음 키에 다시 쓸 수 있고... */
err = proven_u8str_reset(&name);
EXAMPLE_REQUIRE(proven_is_ok(err), "the key buffer is ours again the moment set returns");
err = proven_u8str_append(&name, PROVEN_LIT("grace"));
EXAMPLE_REQUIRE(proven_is_ok(err), "overwriting the buffer with the next key must succeed");

err = PROVEN_MAP_SET_U8_OWNED(&by_name, proven_u8str_as_view(&name), player_t,
    ((player_t){ .level = 4, .score = 700 }));
EXAMPLE_REQUIRE(proven_is_ok(err), "inserting the second owned key must succeed");

/* ...첫 항목은 그 덮어쓰기에 조금도 다치지 않는다. 이것이 요점 전부다. 맵은
 * 지금 "grace" 라고 적힌 버퍼의 뷰가 아니라 "ada" 의 제 사본을 쥐고 있다. BORROWED
 * 맵이었다면 둘 다 "grace" 로 키가 잡힌 항목 둘을 보고했을 것이다 - 아니면 더 나쁘게,
 * 해제된 기억이 키인 항목 하나를. */
const player_t *ada = PROVEN_MAP_GET_U8_OWNED(&by_name, player_t, PROVEN_LIT("ada"));
EXAMPLE_REQUIRE(ada && ada->score == 5000, "the copied key survives the caller reusing its buffer");

const player_t *grace = PROVEN_MAP_GET_U8_OWNED(&by_name, player_t, PROVEN_LIT("grace"));
EXAMPLE_REQUIRE(grace && grace->score == 700, "the second key is a separate entry");
EXAMPLE_REQUIRE(by_name.len == 2, "two distinct keys means two entries");

/* remove 는 맵이 만든 키 사본을 해제한다 - 여러분이 직접 해제하는 일은 없다. */
err = PROVEN_MAP_REMOVE_U8_OWNED(&by_name, PROVEN_LIT("ada"));
EXAMPLE_REQUIRE(proven_is_ok(err), "removing an owned key must succeed");
EXAMPLE_REQUIRE(PROVEN_MAP_GET_U8_OWNED(&by_name, player_t, PROVEN_LIT("ada")) == NULL,
    "the removed entry is gone");

printf("map: %zu name(s) left, grace at level %d\n",
    (size_t)by_name.len, grace ? grace->level : -1);

/* destroy 는 버킷 배열과 그 안에 남은 키 사본을 모두 해제한다(여기서는 "grace").

```

```

    * `scratch` 는 우리 것이고 맵보다 오래 살며, 빌린 `name` 손잡이는 해제할 것이 없다. */
    PROVEN_MAP_DESTROY(&by_name);
    proven_u8str_destroy(alloc, &name);
    return EXAMPLE_OK();
}

```

실전 예제: 컨테이너 크기 잡기, 정렬되지 않은 데이터 찾기, 흘러가는 데이터 검사함 구하기

앞의 실전 예제들은 컨테이너를 하나씩 다룬다. 이 예제는 그 조각들이 들어가 있는 프로그램 — 작은 이벤트 수집기 — 이고, 컨테이너가 여럿 얹히고 나서야 생기는 질문들에 답한다.

- 채우는 도중에 컨테이너가 재할당하지 않게 한다. `proven_array_reserve()`와 `proven_map_reserve()`는 용량을 한 번에 정한다. 힙에서는 복사를 아끼고, 아레나(arena) 뒤에서는 재할당이 다음 `reset`까지 남기는 죽은 저장 공간을 아낀다. `map`에서는 포인터의 정확성 문제이기도 하다. 리해시(rehash, 버킷 배열을 다시 만드는 일)는 앞서 `get_mut`이 돌려준 모든 포인터를 무효로 만든다.
- 내가 만들지 않은 핸들을 확인한다. `proven_array_is_valid()`, `proven_ring_is_valid()`, `proven_map_is_valid()`는 핸들 자체의 필드가 서로 맞는지 묻는다. 0으로만 초기화된, 혹은 아예 만든 적 없는 컨테이너를 첫 `push`가 아니라 내 코드의 경계에서 잡아내는 검사다.
- 정렬되지 않은 데이터를 찾는다. 이벤트 로그는 도착 순서로 둔다. 그 순서 자체가 기록하려는 대상이기 때문이다. 그래서 이진 탐색(binary search)은 쓸 수 없다 — 정렬되지 않은 입력에서 그것은 "없음"을 돌려주지 않고 틀린 답을 돌려준다. 올바른 호출은 `proven_array_linear_search()`이고, 순서상 첫 번째 일치를 돌려준다.
- `map`에 이미 있는 값을 한 번의 조회로 고친다. `proven_map_get_mut()`은 `map` 자신의 저장 공간을 가리키는 포인터를 돌려주므로, 카운터를 읽고 고쳐 다시 쓰는 대신 있는 자리에서 증가한다.
- 키에 어떤 해시가 어울리는지 고른다. `proven_map_create()`는 문자열 키를 키가 있는 SipHash로 해싱하므로, 키를 고르는 공격자가 모든 키를 한 버킷에 몰아넣을 수 없다. `proven_map_create_trusted()`는 빠른 FNV-1a를 쓰며, 모든 키를 내 코드가 고를 때만 옳다. `proven_map_hash()`는 `map`이 실제로 키를 놓는 데 쓴 값을 돌려주므로, 둘의 차이는 주장이 아니라 관찰할 수 있는 사실이 된다.
- 조각으로 도착하는 데이터의 검사함을 구한다. `proven_crc32_update()`는 각 조각을 진행 중인 값에 접어 넣는다. 조각 경계를 어떻게 나누었든 결과는 전체에 대한 `proven_crc32()`와 같다.

```

/*
 * 컨테이너 장들은 구조를 하나씩 따로 보인다. 실제 프로그램은 여럿을 한꺼번에 쓰고, 그때
 * 떠오르는 물음들은 어느 한 컨테이너에 대한 것이 아니다.
 *
 * - 채우는 동안 컨테이너가 재할당하지 않게 하려면? reserve.
 * - 남이 건넌 컨테이너가 쓸 만한지 확인하려면? is_valid.
 * - 자료가 정렬돼 있지 *않을* 때 찾으려면? 선형 탐색 - 그리고 여기서 이진 탐색이 왜
 *   틀린 답을 주는지 아는 것.
 * - 맵에 이미 있는 값을 두 번 찾지 않고 고치려면? get_mut.
 * - 공격에 견디는 해시가 필요한가, 빠른 쪽인가? 키를 누가 고르느냐에 달렸고,
 *   map_hash 가 그 차이를 눈으로 보게 해 준다.
 * - 토막토막 도착하는 자료의 검사함을 구하려면? crc32_update.
 *
 * 이 프로그램은 작은 이벤트 수집기다. 이벤트가 도착하고, 클라이언트별로 세어지고, 최근
 * 몇 개는 진단 덤프용으로 남고, 지나가는 동안 묶음의 검사함이 구해진다.
 */

```

```

typedef struct {
    proven_u32 client;
    proven_u32 code;      /* 이벤트 코드. 0 은 "연결이 닫혔다" 는 뜻이다 */
} event_t;

```

```

/* 클라이언트 번호로 찾기. 이것은 이벤트를 *찾기* 위한 비교이지 정렬하기 위한 것이
 * 아니다. 아래 배열은 *도착한 차례*로 있고 그대로 둔다. */

```

```

static int by_client(const void *a, const void *b) {
    const event_t *x = (const event_t *)a;
    const event_t *y = (const event_t *)b;

```

```

    return (x->client > y->client) - (x->client < y->client);
}

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* --- 1. reserve: 용량을 한 번에 정한다 ----- */

    proven_result_array_t ar = PROVEN_ARRAY_INIT(alloc, event_t, 4);
    EXAMPLE_REQUIRE(proven_is_ok(ar.err), "creating the event log must succeed");
    if (!proven_is_ok(ar.err)) {
        return 1;
    }
    proven_array_t events = ar.value;

    /* 시작하기 전에 묶음 크기를 알고 있으니 자리를 한 번에 청한다. 이것이 없으면 배열은
     * 채워지는 동안 두 배씩 늘어 그때마다 내용을 복사하고, 아레나 할당자 뒤에서라면 그
     * 복사마다 옛 블록이 아레나가 reset 될 때까지 남는다. */
    proven_err_t err = proven_array_reserve(&events, 16);
    EXAMPLE_REQUIRE(proven_is_ok(err), "reserving room for the whole batch must succeed");
    EXAMPLE_REQUIRE(events.cap >= 16, "the capacity is now at least what was asked for");

    /* is_valid 는 손잡이 자체가 구조적으로 온전한지 묻는다 - 서로 맞아떨어지는 포인터와
     * 길이와 용량. 컨테이너가 다른 코드에서 넘어오는 자리나, 0 으로 초기화된 손잡이가
     * 빠져나갔을 수 있는 자리에서 단언할 것. push 마다 되풀이할 것은 아니다. */
    EXAMPLE_REQUIRE(proven_array_is_valid(&events), "a created array must be structurally valid");

    proven_array_t never_created = {0};
    EXAMPLE_REQUIRE(!proven_array_is_valid(&never_created),
        "a zero-initialised handle is not a usable array");

    static const event_t batch[] = {
        { .client = 7, .code = 200 }, { .client = 3, .code = 200 },
        { .client = 7, .code = 404 }, { .client = 9, .code = 200 },
        { .client = 3, .code = 500 }, { .client = 7, .code = 0 },
    };
    proven_size_t batch_len = sizeof batch / sizeof batch[0];

    proven_size_t cap_before = events.cap;
    for (proven_size_t i = 0; i < batch_len; ++i) {
        err = PROVEN_ARRAY_PUSH(&events, event_t, batch[i]);
        EXAMPLE_REQUIRE(proven_is_ok(err), "pushing an event must succeed");
    }
    EXAMPLE_REQUIRE(events.cap == cap_before, "the reserve was enough: nothing reallocated while filling");

    /* --- 2. 정렬되지 않은 자료에서 찾기 ----- */

    /* 로그는 도착한 차례로 있고, 그 차례를 지키고 싶다. 그것이 기록하려는 대상이기
     * 때문이다. 여기서 이진 탐색은 더 빠르고 *틀리다*. 정렬된 구간에서만 써야 하는데,
     * 정렬되지 않은 자료에서는 "없다" 를 돌려주는 것이 아니라 헛소리를 돌려준다. 옳은
     * 도구는 선형 탐색이고, 이 크기의 묶음에서 O(n) 은 아무것도 아니다. */
    event_t key = { .client = 9, .code = 0 };
    const event_t *found = (const event_t *)proven_array_linear_search(&events, &key, by_client);
    EXAMPLE_REQUIRE(found != NULL, "client 9 appears in the batch");
    EXAMPLE_REQUIRE(found->code == 200, "and linear search returns the FIRST match in order");

    event_t absent = { .client = 42, .code = 0 };
    EXAMPLE_REQUIRE(proven_array_linear_search(&events, &absent, by_client) == NULL,
        "a client that never appeared is reported as not found");

```

```

/* --- 3. 가장 최근 이벤트를 담는 고리 ----- */

proven_result_ring_t rr = PROVEN_RING_INIT(alloc, event_t, 4);
EXAMPLE_REQUIRE(proven_is_ok(rr.err), "creating the recent-events ring must succeed");
proven_ring_t recent = rr.value;
EXAMPLE_REQUIRE(proven_ring_is_valid(&recent), "a created ring must be structurally valid");

proven_ring_t unset = {0};
EXAMPLE_REQUIRE(!proven_ring_is_valid(&unset), "a zero-initialised ring handle is not usable");

/* 이 고리는 가득 차면 덮어쓰는 대신 거부한다. 그래서 "최근 넷을 남긴다" 는 것은
 * 넣기 전에 가장 오래된 것을 우리가 직접 버린다는 뜻이다. */
for (proven_size_t i = 0; i < batch_len; ++i) {
    if (recent.len == recent.cap) {
        event_t dropped;
        err = PROVEN_RING_POP(&recent, event_t, &dropped);
        EXAMPLE_REQUIRE(proven_is_ok(err), "a full ring must yield its oldest element");
    }
    err = PROVEN_RING_PUSH(&recent, event_t, batch[i]);
    EXAMPLE_REQUIRE(proven_is_ok(err), "pushing after making room must succeed");
}
EXAMPLE_REQUIRE(recent.len == 4, "the ring holds the four most recent events");

/* --- 4. 클라이언트별로 세기, 고칠 때마다 찾기는 한 번 ----- */

/* create_with_capacity 는 proven_map_create 인데, 용량 인자가 왜 거기 있는지 말해
 * 주는 이름을 달았다. 지금 크기를 잡아 두면 나중의 재해싱을 피한다. 재해싱은 모든
 * 버킷을 복사하고, 전에 get_mut 이 돌려준 모든 포인터를 무효로 만든다. */
proven_result_map_t mr = proven_map_create_with_capacity(alloc, 8, PROVEN_KEY_TYPE_INT,
                                                         sizeof(proven_u32), alignof(proven_u32));
EXAMPLE_REQUIRE(proven_is_ok(mr.err), "creating the counter map must succeed");
proven_map_t counts = mr.value;
EXAMPLE_REQUIRE(proven_map_is_valid(&counts), "a created map must be structurally valid");

err = proven_map_reserve(&counts, 32);
EXAMPLE_REQUIRE(proven_is_ok(err), "reserving map capacity must succeed");

for (proven_size_t i = 0; i < batch_len; ++i) {
    proven_map_key_t k = { .id = batch[i].client };

    /* get_mut 은 맵 저장소 *안*을 가리키는 포인터를 돌려준다. 그래서 계수기는 그것이
     * 사는 자리에서 올라간다 - 찾기 한 번, 되쓰는 복사 없음. 그 포인터는 다음 삽입
     * 전까지만 쓸 수 있고, 위에서 용량을 미리 잡아 둔 또 하나의 이유가 그것이다. */
    proven_u32 *seen = (proven_u32 *)proven_map_get_mut(&counts, k);
    if (seen) {
        *seen += 1;
    } else {
        proven_u32 one = 1;
        err = proven_map_set(&counts, k, &one);
        EXAMPLE_REQUIRE(proven_is_ok(err), "inserting a new client must succeed");
    }
}

const proven_u32 *seven = PROVEN_MAP_GET_INT(&counts, proven_u32, 7);
EXAMPLE_REQUIRE(seven != NULL && *seven == 3, "client 7 sent three events");

/* 닫힘 이벤트(코드 0)는 그 클라이언트가 갔다는 뜻이다. 계수기를 버린다. */
for (proven_size_t i = 0; i < batch_len; ++i) {
    if (batch[i].code == 0) {
        err = proven_map_remove(&counts, (proven_map_key_t){ .id = batch[i].client });
    }
}

```

```

        EXAMPLE_REQUIRE(proven_is_ok(err), "removing a present key must succeed");
    }
}
EXAMPLE_REQUIRE(PROVEN_MAP_GET_INT(&counts, proven_u32, 7) == NULL, "the closed client is gone");

/* --- 5. 어느 해시인가, 그리고 그 차이를 보는 법 ----- */

/* 같은 키를 담은 문자열 키 맵 둘. 기본 쪽은 키가 있는 SipHash 로 해싱하므로, 키를
 * 고르는 공격자도 그것들을 한 버킷으로 몰아넣을 수 없다. 믿는 쪽은 빠른 FNV-1a 를
 * 쓰고, 모든 키를 여러분 코드가 고를 때*만* 옳은 선택이다. */
proven_result_map_t untrusted = PROVEN_MAP_INIT_U8_BORROWED(alloc, proven_u32, 8);
EXAMPLE_REQUIRE(proven_is_ok(untrusted.err), "creating the default string-key map must succeed");
proven_map_t from_network = untrusted.value;

proven_result_map_t tr = proven_map_create_trusted(alloc, 8, PROVEN_KEY_TYPE_U8_BORROWED,
                                                    sizeof(proven_u32), alignof(proven_u32));
EXAMPLE_REQUIRE(proven_is_ok(tr.err), "creating the trusted-key map must succeed");
proven_map_t internal = tr.value;

EXAMPLE_REQUIRE(from_network.trusted_keys == false, "the default map defends against chosen keys");
EXAMPLE_REQUIRE(internal.trusted_keys == true, "the trusted map opts out of that defence");

/* map_hash 는 맵이 실제로 키를 넣을 때 쓰는 값을 드러낸다. 그래서 이 선택이 믿고
 * 받아들이는 것이 아니라 눈으로 볼 수 있는 것이 된다. */
proven_map_key_t name = { .str = PROVEN_LIT("user-agent") };
proven_u64 keyed = proven_map_hash(&from_network, name);
proven_u64 fast = proven_map_hash(&internal, name);
EXAMPLE_REQUIRE(keyed != fast, "the same key hashes differently under the two functions");
EXAMPLE_REQUIRE(proven_map_hash(&internal, name) == fast, "and each function is deterministic");

/* 맵이 소유하지 않은 기억에 사는 키는 *빌린* 것이다. 그 바이트가 맵보다 오래 살아야
 * 한다. 여기서는 문자열 리터럴이니 그렇다. 곧 다시 쓸 버퍼에서 키가 온다면 대신
 * 소유하는 키 맵(PROVEN_MAP_INIT_U8_OWNED / proven_map_set_u8_owned)을 쓸 것.
 * 그쪽은 복사한다. */
proven_u32 hits = 1;
err = proven_map_set(&from_network, name, &hits);
EXAMPLE_REQUIRE(proven_is_ok(err), "inserting a borrowed string key must succeed");
EXAMPLE_REQUIRE(PROVEN_MAP_GET_U8_BORROWED(&from_network, proven_u32, PROVEN_LIT("user-agent")) != NULL,
                "and it can be looked up by an equal view, not the same pointer");

/* --- 6. 흘러가는 자료를 토막으로 검사함 ----- */

/* 묶음은 지나가는 동안 검사합이 구해진다. 파일이나 소켓을 읽는 프로그램이 해야 하는
 * 일이 그것이다 - 전체를 한꺼번에 쥐지 않는다. 흐르는 값을 0 에서 시작해 토막을 하나씩
 * 먹이면, 마지막 값은 이어 붙인 것을 한 번에 부른 결과와 같다. */
proven_u32 running = 0;
for (proven_size_t i = 0; i < batch_len; ++i) {
    proven_mem_view_t chunk = { .ptr = (const proven_byte_t *)&batch[i], .size = sizeof batch[i] };
    running = proven_crc32_update(running, chunk);
}
proven_mem_view_t whole = { .ptr = (const proven_byte_t *)batch, .size = sizeof batch };
EXAMPLE_REQUIRE(running == proven_crc32(whole),
                "chunked and whole-buffer CRC-32 must agree, whatever the chunking");

printf("events=%zu recent=%zu crc32=%08x\n",
        (size_t)events.len, (size_t)recent.len, (unsigned)running);

proven_map_destroy(&internal);
proven_map_destroy(&from_network);
proven_map_destroy(&counts);

```

```

    PROVEN_RING_DESTROY(&recent);
    PROVEN_ARRAY_DESTROY(&events);
    return EXAMPLE_OK();
}

```

반례 — 도착 순서 로그에 이진 탐색을 쓰는 경우:

```
const event_t *e = proven_array_binary_search(&events, &key, by_client); /* wrong */
```

proven_array_binary_search()는 같은 비교 함수로 정렬된 구간에서만 쓸 수 있다. 정렬되지 않은 입력에서 이 함수는 실패하지 않는다. 반씩 좁혀 가다 그 자리에 있던 원소를 답이라고 내놓거나, 찾는 원소가 두 칸 옆에 있는데도 없다고 말한다.

반례 — 삽입을 사이에 두고 get_mut 포인터를 붙들고 있는 경우:

```

proven_u32 *seen = proven_map_get_mut(&counts, k);
proven_err_t e = proven_map_set(&counts, other_key, &one); /* may rehash */
*seen += 1; /* wrong: may dangle */

```

삽입할 수 있는 무언가를 사이에 두고 붙들 것은 포인터가 아니라 키다.

반례 — 프로그램 밖에서 온 키에 빠른 해시를 쓰는 경우:

```

proven_result_map_t m = proven_map_create_trusted(alloc, 64, PROVEN_KEY_TYPE_U8_BORROWED,
                                                    sizeof(session_t), alignof(session_t));
/* keys are HTTP header names taken from the request */ /* wrong */

```

그것이 바로 키가 있는 해시가 막으려는 경우다. 키를 나 아닌 누군가가 고른다면 proven_map_create()를 쓴다.

Chapter 8: Formatting and Scanning (v0.1.1)

이 장은 `fmt.h`와 `scan.h`에 대한 상세 레퍼런스다. Chapter 3은 더 짧은 개요와 일상적인 예제를 제공한다. 이 장은 정확한 문법, 파라미터 형태, 반환값, 그리고 호출자가 흔히 실수하는 지점에 초점을 맞춘다.

목차

1. Design model
2. Formatter data model
3. Formatter constructors and selectors
4. Format string grammar
5. Formatting APIs
- 5.1. Formatting a type of your own
 1. Console print helpers
 2. Scanner data model
 3. Scanner primitive APIs
 4. Scan argument model
 5. Structural scan grammar
 6. Scan formatting APIs
- 11.1. Scan error code guide and recovery
 1. Examples and misuse cases
 2. Freestanding and build-mode notes

1. 설계 모델

타입을 두 번 쓰지 않는 이유

이 장의 두 절반은 모두 한 가지를 없애기 위해 존재한다. 프로그래머가 타입을 말하는데 컴파일러가 그것을 값과 대조해 볼 수 없는 자리다.

`printf("%d", x)`는 타입을 두 번 말한다 — 한 번은 `%d`로, 한 번은 `x`를 넘기면서 — 그리고 `varargs`가 두 번째를 지워버리므로 아무것도 둘을 비교할 수 없다. `scanf("%d", &x)`는 더 나쁘다. 포맷이 어떻게 파싱할지 그리고 포인터를 통해 무엇을 쓸지를 둘 다 결정하므로, 불일치는 헛소리를 출력하는 대신 메모리를 망가뜨린다. 둘은 반대 방향에서 온 같은 결함이고, 둘 다 조용히 컴파일된다.

여기서 플레이스홀더는 **타입을 전혀 담지 않는다**. `{}`는 위치를 표시할 뿐이고, 타입은 포매팅 쪽에서는 `PROVEN_ARG(x)`, 스캐닝 쪽에서는 `PROVEN_SCAN_ARG(&x)`에서 오며, 둘 다 인자의 정적 타입에 대해 컴파일 시점에 `_Generic`으로 결정된다. : 뒤의 `spec`은 표현만 제어한다 — `width`, `fill`, 정렬, 정밀도, 진법 — 결코 해석을 제어하지 않는다. 문자열에 타입을 쓴 적이 없으므로 `%d`나 `double`이냐를 틀릴 여지가 없다.

두 번째 설계 결정은 **어느 쪽도 버퍼를 소유하지 않는다**는 것이다. 포매팅은 여러분이 제공한 목적지에 덧붙이고 들어가지 않으면 거부한다. 스캐닝은 여러분이 제공한 뷰(view)에서 읽고 여러분이 읽을 수 있는 커서를 옮긴다. 할당자(allocator)를 건네지 않는 한 여기서는 아무것도 할당하지 않으며, 바로 그 점이 이 장 전체가 프리스탠딩(freestanding) 빌드에서 동작하게 만든다 (§ 13).

포매팅 측면과 스캐닝 측면은 정반대의 문제를 해결한다.

- 포매팅은 타입이 있는 값을 받아 텍스트로 렌더링한다.
- 스캐닝은 텍스트를 받아 타입이 있는 값을 기록한다.

이 프로젝트는 양쪽 모두 의도적으로 작게 유지한다.

- 포매팅은 간결한 플레이스홀더 언어, positional 재사용, 단순한 정렬(alignment), `width`, 그리고 숫자 값에 대한 hex 렌더링을 지원한다.
- 스캐닝은 타입이 있는 목적지 포인터, 엄격한 플레이스홀더 개수 검사, 그리고 공백 축약(whitespace collapsing)을 포함한 리터럴 매칭을 지원한다.

- 어느 쪽도 완전한 printf나 scanf 복제를 목표로 하지 않는다.

실용적인 결과로, 이 API들은 대규모 범용 포맷 엔진보다 추론하기 쉬우면서도, 문법은 일반적인 시스템 코드 작업에 충분히 표현력이 있다.

2. 포매터 데이터 모델

```
proven_fmt_result_t
typedef struct {
    proven_err_t err;
    proven_size_t written;
    proven_size_t required;
} proven_fmt_result_t;
```

의미:

- err: 연산의 상태 코드.
- written: 목적지에 실제로 기록된 바이트 수.
- required: 전체 포맷 출력에 필요한 총 바이트 수.

먼저 err를 사용하라. 나머지 필드는 잘림(truncating)이나 부분적으로 성공한 연산에서 가장 유용하다.

성공한 result는 다음과 같은 모습이다.

```
proven_result_u8str_t rs = proven_u8str_create(alloc, 64);
if (proven_is_ok(rs.err)) {
    proven_u8str_t s = rs.value;

    proven_fmt_result_t r = proven_u8str_append_fmt_trunc(
        &s,
        "hello {}",
        PROVEN_ARG("world")
    );
    if (proven_is_ok(r.err)) {
        proven_println("{} ", PROVEN_ARG(proven_u8str_as_view(&s)));
    }

    proven_u8str_destroy(alloc, &s);
}
```

잘림이 발생한 result도 얼마나 더 많은 공간이 필요했는지는 여전히 알려줄 수 있다. 여기서는 목적지를 일 부러 너무 작게 만들었으므로 written이 required에 못 미치게 되며, 그럼에도 문자열은 여전히 유효하다.

```
proven_result_u8str_t rs = proven_u8str_create(alloc, 8); /* on purpose: too small */
if (proven_is_ok(rs.err)) {
    proven_u8str_t s = rs.value;

    proven_fmt_result_t r = proven_u8str_append_fmt_trunc(
        &s,
        "name={} score={}",
        PROVEN_ARG("ada"),
        PROVEN_ARG(42)
    );
    /* r.written is what fit; r.required is what the whole output needed. */
    proven_println("wrote {} of {} bytes",
        PROVEN_ARG(r.written), PROVEN_ARG(r.required));

    proven_u8str_destroy(alloc, &s);
}
```

```
proven_arg_type_t
typedef enum {
    PROVEN_ARG_NONE,
```

```

PROVEN_ARG_I32,
PROVEN_ARG_U32,
PROVEN_ARG_I64,
PROVEN_ARG_U64,
#ifdef PROVEN_FMT_NO_FLOAT
PROVEN_ARG_F64,
#endif
PROVEN_ARG_CSTR,
PROVEN_ARG_STR_VIEW,
PROVEN_ARG_DATETIME,
PROVEN_ARG_PTR,
PROVEN_ARG_FN,
} proven_arg_type_t;

```

포매터는 현재 다음 값 클래스들을 인식한다.

- signed 32비트 정수
- unsigned 32비트 정수
- signed 64비트 정수
- unsigned 64비트 정수
- 부동소수점 값 (PROVEN_FMT_NO_FLOAT가 정의되지 않은 경우)
- 신뢰할 수 있는 C 문자열
- 빌려 쓰는(borrowed) U8 문자열 뷰
- datetime
- 객체 포인터
- 함수 포인터

```

proven_arg_t
typedef struct {
    proven_arg_type_t type;
    union {
        proven_i32 i32;
        proven_u32 u32;
        proven_i64 i64;
        proven_u64 u64;
        double f64;
        const char *cstr;
        proven_u8str_view_t str_view;
        proven_datetime_t datetime;
        const void *ptr;
        void (*fn)(void);
    } value;
} proven_arg_t;

```

union 필드는 선택된 type과 일치해야 한다. union 필드를 잘못 채워 놓고 포매터가 알아서 추측해 주기를 바라며 proven_arg_t를 억지로 만들지 마라.

잘못된 예:

```

proven_arg_t arg = {0};
arg.type = PROVEN_ARG_I64;
arg.value.u64 = 123; /* wrong: type and union field do not match */

```

올바른 예:

```

proven_arg_t arg = proven_arg_i64(123);
(void)arg; /* pass it to a formatting macro; PROVEN_ARG(arg) accepts it as-is */

```

3. 포매터 생성자와 선택자

생성자 요약

API	파라미터	반환	의도
proven_arg_none(void)	없음	proven_arg_t	내부 sentinel 값.
proven_arg_i32(int v)	signed 정수	proven_arg_t	32비트 signed 정수로 렌더링.
proven_arg_u32(unsigned int v)	unsigned 정수	proven_arg_t	32비트 unsigned 정수로 렌더링.
proven_arg_i64(long long v)	넓은 signed 정수	proven_arg_t	64비트 signed 정수로 렌더링.
proven_arg_u64(unsigned long long v)	넓은 unsigned 정수	proven_arg_t	64비트 unsigned 정수로 렌더링.
proven_arg_f64(double v)	부동소수점 값	proven_arg_t	float 포매팅이 비활성화되지 않은 한, 부동소수점 텍스트로 렌더링.
proven_arg_cstr(const char *v)	신뢰할 수 있는 live C 문자열	proven_arg_t	NUL로 종료되는 C 문자열을 렌더링.
proven_arg_cstr_n(const char *v, proven_size_t max_len)	경계가 있을 수 있는 C 문자열	proven_arg_t	NUL을 찾되 max_len까지만 렌더링.
proven_arg_str_view(proven_u8str v)	비교된 U8 뷰	proven_arg_t	NUL 종료를 가정하지 않고 빌려온 뷰를 렌더링.
proven_arg_datetime(proven_datetime v)	datetime 값	proven_arg_t	포매터의 datetime 규칙으로 datetime을 렌더링.
proven_arg_ptr(const void *v)	객체 포인터	proven_arg_t	포인터 값을 렌더링.
proven_arg_fn(void (*v)(void))	함수 포인터	proven_arg_t	원시 함수 포인터 표현을 렌더링.
proven_arg_ucstr(const unsigned char *v)	unsigned-char 문자열	proven_arg_t	proven_arg_cstr에 대한 편의 래퍼.
proven_arg_identity(proven_arg_t v)	기존 argument 객체	proven_arg_t	통과(pass-through) 헬퍼.
proven_arg_bool(bool v)	boolean	proven_arg_t	1 / 0이 아니라 true / false를 단어로 렌더링.
proven_arg_char(char v)	문자 하나	proven_arg_t	문자 를 렌더링. 이것이 바로 char 변수는 문자로 렌더링되는데 리터럴 PROVEN_ARG('Z')는 여전히 90으로 렌더링되는 이유다. C에서 'Z'는 int 타입이며, 어떤 _Generic으로도 이것을 숫자 90과 구별할 수 없다.
proven_arg_custom(const void *v, proven_fmt_custom_fn fn)	어떤 타입이든	proven_arg_t	라이브러리가 전혀 알지 못하는 타입을, 당신이 제공하는 함수를 통해 렌더링. Formatting a user-defined type 참조.

PROVEN_ARG(x)

PROVEN_ARG(x)는 통상적인 진입점이다. _Generic을 사용하여 컴파일러가 x의 타입으로부터 생성자를 고른다.

현재 매핑은 다음과 같다.

- _Bool, char, signed char, short, int -> proven_arg_i32
- unsigned char, unsigned short, unsigned int -> proven_arg_u32
- long, long long -> proven_arg_i64
- unsigned long, unsigned long long -> proven_arg_u64
- double, float -> proven_arg_f64 (PROVEN_FMT_NO_FLOAT가 정의되지 않은 경우)
- const char *, char * -> proven_arg_cstr
- unsigned char *, const unsigned char * -> proven_arg_ucstr
- void *, const void * -> proven_arg_ptr
- proven_u8str_view_t -> proven_arg_str_view
- proven_datetime_t -> proven_arg_datetime
- proven_arg_t -> proven_arg_identity

중요한 결과:

- PROVEN_ARG는 함수 포인터를 선택하지 않는다.
- 함수 포인터에는 PROVEN_ARG_FN(f)를 사용하라.

잘못된 예:

```
void helper(void) {}
proven_u8str_append_fmt_grow(alloc, &s, "{}", PROVEN_ARG(helper)); /* wrong */
```

올바른 예:

```
void helper(void); /* whatever function you want to print the address of */

proven_result_u8str_t rs = proven_u8str_create(alloc, 64);
if (proven_is_ok(rs.err)) {
    proven_fmt_result_t r = proven_u8str_append_fmt_grow(alloc, &rs.value, "{}",
                                                         PROVEN_ARG_FN(helper));
    (void)r;
    proven_u8str_destroy(alloc, &rs.value);
}
```

PROVEN_ARG_FN(f)

이 매크로는 호출자가 함수 포인터를 void *로 캐스팅하지 않고도 넘길 수 있도록 존재한다. proven_arg_fn을 감싼 작은 안전 래퍼다.

예:

```
void helper(void);

proven_result_u8str_t rs = proven_u8str_create(alloc, 64);
if (proven_is_ok(rs.err)) {
    proven_u8str_t s = rs.value;

    proven_fmt_result_t r = proven_u8str_append_fmt_grow(
        alloc,
        &s,
        "callback = {}",
        PROVEN_ARG_FN(helper)
    );
    if (!PROVEN_FMT_IS_OK(r)) {
        proven_eprintln("formatting the callback failed");
    }
}
```

```
    proven_u8str_destroy(alloc, &s);
}
```

PROVEN_ARG_CSTR_N(v, max_len)

이 매크로는 경계가 있는 문자열 헬퍼다. 소스가 완전히 신뢰할 수 있는 C 문자열이 아닐 수 있지만 여전히 C 문자열과 유사한 입력 처리를 원할 때 사용하라.

이 매크로는 max_len까지만 NUL을 탐색한 뒤 경계 지어진 접두부를 뷰로서 포매팅한다.

좋은 사용 사례 - buf가 통신선(wire)에서 들어왔으므로 NUL 종료점이 전혀 보장되지 않는 경우:

```
char buf[128]; /* filled from an untrusted source */
(void)proven_mem_copy(buf, sizeof buf, proven_mem_view_from_u8(PROVEN_LIT("payload")));

proven_result_u8str_t rs = proven_u8str_create(alloc, 64);
if (proven_is_ok(rs.err)) {
    proven_fmt_result_t r = proven_u8str_append_fmt_grow(
        alloc,
        &rs.value,
        "payload={}",
        PROVEN_ARG_CSTR_N(buf, sizeof buf) /* looks for NUL only within 128 bytes */
    );
    (void)r;
    proven_u8str_destroy(alloc, &rs.value);
}
```

나쁜 사용 사례:

```
const char *buf = get_network_buffer();
proven_u8str_append_fmt_grow(alloc, &s, "{}", PROVEN_ARG(buf)); /* wrong if buf is not trusted */
```

부동소수점 형식화 참고

PROVEN_FMT_NO_FLOAT가 정의되면 float 지원은 제네릭 셀렉터에서 제거되고 float 생성자는 사용할 수 없다. 이는 런타임 토글이 아니라 컴파일 시점의 구성 선택이다.

double의 기본 {} 렌더링은 유한 값에 대해 소수점 이하 6자리 고정 형식을 만들며, 크기가 간결한 형식에 비해 너무 크거나 너무 작으면 과학적 표기(scientific)로 전환된다. 이전 버전과 달리, 이 출력은 이제 **정확하고 정수만 사용하는** 엔진으로 계산된다(double/long double 근사 없음). 자릿수는 정확히 반올림되므로(round-half-to-even), {}는 같은 값에 대해 printf("%.6f") / %.6e가 출력하는 것과 일치한다. 최단(shortest) 라운드 트립 형식이나 6이 아닌 다른 precision이 필요하면 아래에 설명하는 proven_float_format_* 정책 API를 사용하라.

정확도와 한계

- 기본 {} float 출력은 소수점 이하 6자리를 사용하며, 가장 가까운 값으로 정확히 반올림하되 동점(tie)은 짝수로 처리한다(glibc %.6f/%.6e와 일치). 어떤 크기에서도 정확하며, 구성 가능한 큰 정수(big-integer) 용량을 넘어서는 precision/크기 한계는 없다.
- 라운드트립 직렬화에는 **최단(shortest)** 정책 (proven_float_format_options_shortest())을 사용하라. 이는 다시 파싱하면 정확히 같은 값이 되는 가장 짧은 십진 문자열을 방출한다.
- 십진수-double 스캐닝은 IEEE-754 binary64로, 가장 가까운 값으로 반올림하되 동점은 짝수로 처리하도록 정확히 반올림되며, 호스트의 strtod와 비트 단위로 일치한다. 가장 작은 subnormal에 대한 중간 임계값 아래의 값들은 입력 부호를 보존한 채 부호 있는 0으로 반올림된다.
- 파서는 Clinger fast path -> Eisel-Lemire -> 정확한 big-integer 폴백으로 계층화되어 있다. 모든 계층이 정확하고 폴백이 최종 판정자이므로, 결과는 모든 입력에 대해 정확히 반올림된다. 캐시된 거듭제곱 테이블은 생성된 소스다 (scripts/generate_float_decimal_tables.py).
- 정확 폴백 big-integer 용량은 임베디드 타겟을 위해 PROVEN_FLOAT_BIGINT_LIMBS(include/proven/float_config.h 참조)로 조정할 수 있다. fast path는 big integer를 절대 건드리지 않는다.

- 검증: 포맷터와 파서는 40.8억 개의 모든 유한 binary32 값과 25.6억 개의 무작위 binary64 값에 대해 호스트 C 라이브러리와 남김없이 대조 검사되었으며, 불일치가 하나도 없었다. 알고리즘, 방법론, glibc와의 벤치마크는 docs/float-correctness-and-performance.md를 참조하라.

엔진 내부(개념적으로)

API를 사용하는 데 이 내용이 전혀 필요하지 않다. 이는 출력이 왜 신뢰할 수 있는지 알고 싶은 독자를 위한 것이다. 전체 내용은 docs/float-correctness-and-performance.md에 있다.

파싱(십진수 → binary64), 세 계층, 빠른 것 우선. 결과는 항상 정확히 반올림된다. 계층은 순전히 속도의 계단일 뿐이며, 각 계층은 정확한 답을 보장할 수 있을 때만 선택된다.

1. Clinger fast path. 값의 유효 자릿수가 적고 지수가 작을 때, 유효숫자(significand)와 10^{exp} 모두 double로 정확히 표현 가능하므로, 단 한 번의 반올림된 곱셈/나눗셈이 증명 가능하게 올바르다. 일상적인 대부분의 숫자를 커버한다.
2. Eisel-Lemire. 캐시된 10의 거듭제곱과의 64×128 비트 고정소수점 곱셈이며, 결과가 반올림 경계로부터 충분히 멀어 확실한지를 검사한다. 검사가 불확실하면(값이 중간의 동점 지점에 놓이면) 다음 계층으로 넘어간다.
3. 정확한 big-integer 폴백. 값을 큰 정수들의 비(significand와 $5^q/2^q$)로 구성하고, 후보 double 및 그 이웃과 정확히 비교한다. 이것이 동점과 subnormal을 올바르게 만드는 최종 판정자다. 시드된 ± 16 -ULP 윈도우가 탐색을 몇 번의 비교로 유지한다. big-integer 용량은 PROVEN_FLOAT_BIGINT_LIMBS로 제한된다. 이 계층만이 limb를 (스택에) 할당하며, fast path는 여기에 절대 도달하지 않는다.

포맷팅(binary64/32 → 십진수). 두 엔진, 그 어디에도 long double 없음:

- 최단(shortest) (proven_float_format_options_shortest()): **Grisu3** fast path가 거의 모든 입력에 대해 최소한의 라운드트립 자릿수를 ~ 90 ns에 만든다. Grisu3가 최소성을 증명할 수 없는 드문 경우에는 정확한 **Dragon4**(Burger-Dybvig, round-to-even) 코어로 폴백한다. 결과는 같은 비트로 다시 파싱되는 유일한 최단 십진수다.
- 고정 %f / 과학적 %e (기본 {}와 고정 옵션): 정확한 정수 엔진이 값을 big-integer mul_pow5/시프트로 $10^{\text{precision}}$ 만큼 스케일링하고, 정수 divmod를 수행한 뒤, round-half-to-even으로 반올림한다. 따라서 어떤 precision과 크기에서도 glibc와 일치하며, $2^{64}/\text{precision}$ 한계가 없다. 극단적인 지수는 실제 임의정밀도(arbitrary-precision) 연산을 하며 그만큼 느리다(실무에서는 드물다).

공개 부동소수점 파싱 API

세 진입점이 하나의 정확히 반올림되는 백엔드를 공유한다.

- proven_scan_f64(scan) — proven_scan_t 커서로부터 파싱하며, 실패 시 커서를 복원한다. 네이티브하고 길이로 경계 지어진 경로(NUL 종료 불필요).
- proven_parse_double_ascii(view) — proven_u8str_view_t로부터 로케일 독립적인 ASCII 토큰 하나를 파싱하고 소비한 길이를 보고한다.
- proven_strtod(npstr, endptr) — C 문자열 위의 strtod 스타일 편의 래퍼로, 선행 ASCII 공백을 건너뛰고 endptr을 보고한다.

실전 예제: 파싱

```
#include "proven/scan.h"
#include "proven/float_parse.h"

/* (1) Native, view-based parsing through a scanner cursor. */
proven_scan_t sc = proven_scan_init(proven_u8str_view_from_cstr("3.14159e2 rest"));
proven_result_f64_t r = proven_scan_f64(&sc);
if (r.err == PROVEN_OK) {
    /* r.val == 314.159; the cursor now sits at " rest". */
    proven_println("parsed {}", PROVEN_ARG(r.val));
}

/* (2) strtod-style wrapper for C strings. endptr reports where parsing stopped. */
char *end = NULL;
```

```
double v = proven_strtod(" -0.5\t", &end); /* v == -0.5, *end == '\t' */
(void)v;

/* A trailing exponent marker with no digits stops like strtod: "1e" parses 1,
   leaving endptr at 'e'. Inputs with hundreds of significant digits and extreme
   exponents are still rounded correctly via the exact fallback. */
```

실전 예제: 정책 API로 형식화하기

proven_float_format_f64_policy / _f32_policy는 호출자 버퍼에 직접 기록하고 기록한 바이트 수를 보고한다. 이들은 절대 할당하지 않는다.

```
#include "proven/float_format.h"

char buf[64];
proven_size_t n = 0;

/* Shortest round-trippable form: 0.1 -> "0.1" (not "0.10000000000000001"). */
(void)proven_float_format_f64_policy(buf, sizeof buf, 0.1,
    PROVEN_FLOAT_FORMAT_POLICY_RYU,
    proven_float_format_options_shortest(), &n);
/* buf == "0.1", n == 3 */

/* Fixed precision (correctly rounded, round-half-to-even). */
proven_float_format_options_t opt = proven_float_format_options_fixed_default();
opt.precision = 2;
(void)proven_float_format_f64_policy(buf, sizeof buf, 3.14159,
    PROVEN_FLOAT_FORMAT_POLICY_DEFAULT, opt, &n);
/* buf == "3.14" */

/* Always scientific - this is what {:e} selects. Six fractional digits by default,
   a signed two-digit-minimum exponent: exactly what printf's %e prints. */
proven_float_format_options_t sci = proven_float_format_options_scientific();
sci.precision = 2;
(void)proven_float_format_f64_policy(buf, sizeof buf, 42.0,
    PROVEN_FLOAT_FORMAT_POLICY_DEFAULT, sci, &n);
/* buf == "4.20e+01" - where fixed would give "42.00" and shortest "42" */
```

- PROVEN_FLOAT_FORMAT_POLICY_RYU는 최단 출력을 선택한다. DEFAULT/SIMPLE은 정확한 고정 precision 경로(%f, 아주 크거나 작은 크기에서는 %e로 전환)를 선택한다.
- 버퍼가 너무 작으면 PROVEN_ERR_OUT_OF_BOUNDS를 반환하며(값은 절대 소리 없이 잘리지 않는다), 지원되지 않는 정책에는 PROVEN_ERR_INVALID_ARG를 반환한다.
- 제네릭 {} 포매터(proven_u8str_append_fmt*)는 내부적으로 DEFAULT 정책을 사용하므로, 일상적인 로깅에서는 이 API를 직접 호출할 필요가 없다.

4. 형식 문자열 문법

포매터는 의도적으로 작은 문법을 받아들인다.

치환 필드

지원되는 형식:

- {}: 다음 positional argument
- {0}: 첫 번째 사용자 argument
- {1}: 두 번째 사용자 argument
- {2}: 세 번째 사용자 argument
- 이하 계속

번호 매김은 사용자 대상이며 0부터 시작한다. 구현은 인덱스 0에 숨겨진 sentinel을 저장하고, 사용자 인덱스 0을 내부 argument 슬롯 1로 매핑한다.

중괄호 이스케이프

- {{는 리터럴 {가 된다
- }}는 리터럴 }가 된다

레이아웃 명세

{:[fill]align[sign][#][0][width][.precision][type]}

모든 부분은 선택적이며, 순서는 나머지 세계가 사용하는 것과 같다. 따라서 Python이나 Rust에서 복사한 스펙은 여기서도 그곳과 같은 의미를 가진다.

부분	값	하는 일
align	< > ^	왼쪽, 오른쪽, 가운데. 기본값 >.
fill	align 앞에 오는 임의의 문자	패딩 문자. 기본값은 공백.
sign	+ 또는 공백	음수가 아닌 수에 부호를 강제하거나, 부호 자리를 하나 예약한다.
#		대체 형식(alternate form): 0x, 0X, 0o, 0b 접두어. 정수 전용.
0		0 채움. 42에 대한 {08}은 00000042.
width	숫자, 최대 10000	최소 필드 width.
.precision	.N, 최대 60	소수 자릿수. float 전용.
type	x X o b d(int), f g e(float)	기수(base)와 대소문자. f 고정, g 최단 라운드로우, e 과학적 표기(sprintf %e).

과거에 거졌이었던 만큼 알아둘 가치가 있는 두 가지가 있다.

- 선행 0은 0 채움이지 width의 첫 자리가 아니다. {08}은 v26.07.12f 전까지 " 42"를 만들었다(공백 패딩, 오류 없음). 명시적 fill은 여전히 우선한다: {:>08}은 *로 채운다.
- 0 채움은 부호와 자릿수 사이에 들어간다. 42에 대한 {:+08}은 +0000042이며, 0000+42가 아니다. 패딩은 숫자의 일부이고, 숫자의 부호가 먼저 온다.

부동소수점

{f}는 정확히 반올림된 소수점 이하 6자리를 준다. {:.3}은 3자리, {:.0}은 0자리를 주고, {e}는 과학적 표기를 강제한다 - 가수(mantissa), precision 소수 자릿수(기본 6자리), 부호 있는 최소 두 자리 지수, 정확히 반올림, 정확히 printf의 %e와 같다 - 이는 {:.f}와 {:.g}가 도달하지 못하는 형식이다: {:.f}는 지수를 절대 보여주지 않고, {:.g}는 더 짧을 때만 지수를 사용한다. {:.f}는 고정 형식을 강제하고, {:.g}는 라운드로우되는 가장 짧은 표현을 준다.

v26.07.12i 전까지 **이 중 어느 것도 존재하지 않았다**: 모든 float은 영원히 정확히 6자리 소수로만 나왔다. 정확 엔진은 언제나 이 모든 것을 할 수 있었으나, {} 문법이 그것에 도달하지 못했을 뿐이다. 눈에 보이는 대가는 float 컬럼을 정렬할 수 없다는 것이었다. 12.5는 9자 폭으로, 100.0은 10자 폭으로 렌더링되었기 때문이다.

```
proven_byte_t buf[64];
proven_u8str_t line = proven_u8str_borrow(buf, sizeof buf);
(void)proven_u8str_append_fmt(&line, "{:>9.2}", PROVEN_ARG(12.5)); /* " 12.50" */
```

인자가 지킬 수 없는 명세는 예러다

double에 대한 {:.x}, 정수에 대한 {:.2}, 정수에 대한 {:.f}, 문자열에 대한 {:#}: 모두

PROVEN_ERR_INVALID_FORMAT.

이들은 과거에 *무시*되었다 - double에 대한 {:.x}는 3.500000을 출력하고 성공을 보고했다. 호출자는 무언가를 요청하고, 다른 것을 받았으며, 그것이 제대로 되었다는 말을 들었다. 그것은 있을 수 있는 최악의 결과이며, 거부하는 것보다 나쁘다.

char and bool

PROVEN_ARG('Z')는 Z로 렌더링되고, bool은 true 또는 false로 렌더링된다. 둘 다 과거에는 정수 경로를 거쳤으므로, 문자가 90으로 출력되었고 문자 하나를 방출할 방법이 아예 없었다 - hex 덤프의 ASCII 컬럼은 별도 버퍼에서 손으로 만들어 문자열로 넘겨야 했다. 이제 대문자 hex 덤프는 한 개의 루프다.

```
proven_byte_t hexbuf[64];
proven_u8str_t hexline = proven_u8str_borrow(hexbuf, sizeof hexbuf);
unsigned char byte = 0xde;
(void)proven_u8str_append_fmt(&hexline, "{:02X} ", PROVEN_ARG((unsigned)byte));
(void)proven_u8str_append_fmt(&hexline, "{}", PROVEN_ARG((char)'.'));
```

5. 형식화 API

```
proven_u8str_fmt_internal(...)
proven_fmt_result_t proven_u8str_fmt_internal(
    proven_allocator_t alloc,
    proven_u8str_t *str,
    bool trunc,
    const char *fmt,
    proven_allocator_t scratch,
    const proven_arg_t *args,
    proven_size_t args_count
);
```

이것은 내부 포매팅 엔진이다. 사용자 코드는 보통 대신 공개 매크로를 호출해야 한다.

파라미터:

- alloc: 문자열이 커져야 할 때 사용되는 할당자
- str: 목적지 U8 문자열
- trunc: true이면 best-effort 잘림을 허용하고, false이면 atomic(원자적) 동작을 유지
- fmt: 포맷 텍스트
- scratch: 필요할 때 임시 alias 패칭에 사용되는 할당자
- args: 인덱스 0의 숨겨진 sentinel을 포함한 포맷 argument 배열
- args_count: sentinel을 포함한 args의 총 길이

반환값:

- proven_fmt_result_t

중요한 규칙:

- args_count는 플레이스홀더 개수에 숨겨진 sentinel을 더한 값과 일치해야 한다
- 사용되지 않는 여분의 argument는 오류다
- 누락된 argument는 오류다
- 엔진이 목적지 문자열과 빌려온 뷰 argument 사이의 aliasing을 감지하면, 실패 atomicity를 보존하기 위해 임시 작업용(scratch) 할당자를 사용할 수 있다

```
proven_u8str_append_fmt(str, fmt, ...)
```

고정 용량 문자열로의 atomic(원자적) 포매팅. 결과가 들어맞지 않으면, 함수는 실패를 보고하고 목적지를 변경하지 않는다.

전부 아니면 전무(all-or-nothing) 동작을 원할 때 사용하라.

```
proven_u8str_append_fmt_trunc(str, fmt, ...)
```

Best-effort 포매팅. 들어맞는 만큼 기록하고, 얼마나 기록되었는지와 얼마나 필요했는지를 보고한다.

부분 출력이 허용될 때 사용하라.

`proven_u8str_append_fmt_grow(alloc, str, fmt, ...)`

성장 가능한(growable) 포매팅. 제공된 할당자를 통해 목적지 문자열을 재할당할 수 있다. 할당 실패 시, 기존 문자열은 유효하게 유지된다.

수동 용량 계획 없이 출력을 들어맞게 하고 싶을 때 사용하라.

`proven_u8str_append_fmt_with_scratch(alloc, str, fmt, scratch, ...)`

별도의 scratch 할당자를 사용하는 성장 가능한 포매팅. argument 목록에 목적지 버퍼와 alias할 수 있는 문자열 뷰가 있어 임시 패칭이 필요할 때 유용하다.

`alloc`과 `scratch` 모두에 실제 할당자를 사용하라. 호출에 충분할 만큼 수명이 길지 않은 한, 죽은 아레나(arena)나 일회용 임시 버퍼를 넘기지 마라.

부동소수점 형식화 정책 이음매

공개 float 정책 헤더는 float 포매팅을 위한 명시적 정책 레이어를 제공한다. 의도적으로 작으며, 정확한 고정 precision 포매터를 기본 경로로 유지한다.

주요 진입점은 다음과 같다.

- `proven_float_format_f64_policy(...)`
- `proven_float_format_f32_policy(...)`
- `proven_float_format_options_fixed_default()`
- `proven_float_format_options_shortest()`

정책 참고 사항:

- `PROVEN_FLOAT_FORMAT_POLICY_DEFAULT`와 `PROVEN_FLOAT_FORMAT_POLICY_SIMPLE`은 정확한 고정 precision 출력(정확히 반올림, round-half-to-even)을 선택한다.
- `PROVEN_FLOAT_FORMAT_POLICY_RYU`는 최단 출력 정책 분기다.
- 정책 API는 지원되지 않는 enum 값에 대해 `PROVEN_ERR_INVALID_ARG`를 반환한다.
- 정책 API는 호출자가 제공한 버퍼가 너무 작을 때 `PROVEN_ERR_OUT_OF_BOUNDS`를 반환한다.

예:

```
char buf[128];
proven_size_t written = 0;
proven_err_t err = proven_float_format_f64_policy(
    buf,
    sizeof buf,
    0.1,
    PROVEN_FLOAT_FORMAT_POLICY_RYU,
    proven_float_format_options_shortest(),
    &written
);
if (proven_is_ok(err)) {
    /* buf holds the shortest form that parses back to exactly 0.1 */
    proven_println("{", PROVEN_ARG_CSTR_N(buf, written));
}
```

`PROVEN_FMT_IS_OK(res)`

`proven_fmt_result_t`를 검사하는 작은 헬퍼 매크로. 의도를 간결하게 유지하고 싶을 때 사용하라.

예:

```
proven_result_u8str_t rs = proven_u8str_create(alloc, 32);
if (proven_is_ok(rs.err)) {
    proven_u8str_t s = rs.value;

    proven_fmt_result_t r = proven_u8str_append_fmt_grow(
        alloc,
        &s,
        "name={ } score={:0>4}",
```

```

    PROVEN_ARG("ada"),
    PROVEN_ARG(42)
);
if (!PROVEN_FMT_IS_OK(r)) {
    /* the string is untouched: grow-mode formatting is failure-atomic */
    proven_eprintln("formatting failed");
}

proven_u8str_destroy(alloc, &s);
}

```

콘솔식 헬퍼

sysio 레이어는 같은 포매터 기계를 사용하는 print 헬퍼들을 제공한다.

- proven_print(fmt, ...)
- proven_println(fmt, ...)
- proven_eprint(fmt, ...)
- proven_eprintln(fmt, ...)

이들은 stdout이나 stderr로 직접 포맷 출력을 원할 때 편리하다. 여전히 proven_err_t를 반환하므로, 출력이 중요할 때는 결과를 검사하라.

예:

```

if (!proven_is_ok(proven_println("hello {} ", PROVEN_ARG("world")))) {
    /* the write to stdout failed - a closed pipe, a full disk */
    proven_eprintln("stdout is not writable");
}

```

5.1. 내가 만든 타입 형식화하기

PROVEN_ARG는 _Generic 위에 세워졌고, _Generic은 컴파일 시점에 알려준 타입에 대해서만 디스패치할 수 있다. 당신의 타입은 알려줄 수 없다. 그래서 포매터의 argument 집합 — 정수, float, 문자열, 포인터, datetime — 은 닫힌 집합이었다: rect_t, uuid_t, vec3_t는 {}에 전혀 넘길 수 없었다.

이를 우회하는 두 방법은 모두 나빴다. 값을 scratch 문자열로 미리 포매팅해서 *그것*을 넘기는 방법: 값마다 할당과 복사가 로깅 경로에서 발생하는데, 로깅 경로야말로 할당이 바로 실패한 상황에서도 계속 동작해야 하는 유일한 경로다. 아니면 필드를 하나씩 출력하고 컬럼 정렬은 포기하는 방법.

PROVEN_ARG_OF(&obj, render)가 그 문이다.

```
proven_err_t render(proven_fmt_sink_t out, const void *obj);
```

그 시그니처의 형태로부터 세 가지가 따라 나온다.

- 렌더러는 버퍼가 아니라 sink를 받는다. 렌더러는 공간이 얼마나 있는지 알 필요가 없고, 무엇도 오버플로할 수 없다. proven_fmt_put으로 방출한다.
- 합성(compose)된다. 렌더러는 포매터를 다시 호출할 수 있으며 — 스택 버퍼로, 할당자 없이 — 그 결과를 sink에 넘길 수 있다. 필드 자체가 사용자 타입인 타입도 자연스럽게 중첩된다.
- Width, fill, alignment가 동작한다. 포매터는 렌더러를 두 번 실행한다: 먼저 counting sink에 대해 출력 폭을 알아내고, 그다음 실제로, 그 주위에 패딩을 적용하여 실행한다. 이것이 {:>10}이 당신의 타입 컬럼을 int 컬럼을 정렬하는 것과 똑같이 정렬하는 이유이며, 렌더러가 결정적(deterministic)이어야 하고 obj를 변경해서는 안 되는 이유다. 두 패스가 불일치하면, 포매터는 정렬된 컬럼에 잘못된 폭의 필드를 방출하고 나중에 알아차리게 하는 대신 PROVEN_ERR_INVALID_ARG를 반환한다.

포매터가 하지 않을 일은 추측이다. 사각형에 대한 {:x}, UUID에 대한 {:.2}, 행렬에 대한 {:+} — 라이브러리는 그것들이 당신의 타입에 무엇을 의미하는지 전혀 모르므로, PROVEN_ERR_INVALID_FORMAT으로 거부한다. 그럴 듯한 답을 지어내고 성공을 보고하는 것이야말로 포매터가 거짓말을 시작하는 방식이다. 당신이 요청하지 않은 타입 글자는 오류보다 낫지 않다.

테스트 스위트가 컴파일하고 실행함:

```

/*
 * 라이브러리가 들어 본 적 없는 타입을 형식화하기.
 *
 * `PROVEN_ARG` 는 `_Generic` 위에 서 있고, 그것은 미리 알려 준 타입에만 갈래를 태울 수
 * 있다 - 그리고 여러분의 타입을 알려 줄 방법이 없다. 그래서 `PROVEN_ARG_OF` 가 생기기
 * 전에는 `rect_t` 를 찍을 방법이 아예 없었다. 임시 문자열에 미리 형식화해서 그것을
 * 건네거나(값마다 할당 한 번과 복사 한 번, 그것도 할당하면 안 되는 그 로그 경로에서),
 * 필드를 하나씩 찍고 열 맞추기는 포기하거나 둘 중 하나였다.
 *
 * 렌더러는 버퍼가 아니라 *그릇*을 받는다. 그것이 이 방식을 조립 가능하게 만든다. 렌더러는
 * 형식화기를 다시 부를 수 있고, 그 출력은 그저 어딘가로 가는 바이트다. 그리고 형식화기가
 * 내보내기 전에 렌더러의 출력을 재기 때문에 - 세는 그릇에 대고 한 번 돌려 본다 - 폭과
 * 채움과 정렬이 사용자 타입에서도 int 에서와 똑같이 먹는다.
 */

typedef struct { int w, h; } rect_t;

static proven_err_t render_rect(proven_fmt_sink_t out, const void *obj) {
    const rect_t *r = (const rect_t *)obj;

    /* 조립: 형식화기를 스택 버퍼로. 할당자는 어디에도 없다. */
    proven_byte_t tmp[64];
    proven_u8str_t s = proven_u8str_borrow(tmp, sizeof tmp);
    proven_fmt_result_t f = proven_u8str_append_fmt(&s, "{}x{}",
                                                    PROVEN_ARG(r->w), PROVEN_ARG(r->h));
    if (!PROVEN_FMT_IS_OK(f)) return f.err;

    return proven_fmt_put(out, proven_u8str_as_view(&s));
}

int main(void) {
    rect_t a = { .w = 1920, .h = 1080 };
    rect_t b = { .w = 640, .h = 480 };

    proven_byte_t buf[128];
    proven_u8str_t line = proven_u8str_borrow(buf, sizeof buf);

    /* 다른 인자와 똑같다. */
    proven_fmt_result_t r = proven_u8str_append_fmt(&line, "mode={}", PROVEN_ARG_OF(&a, render_rect));
    EXAMPLE_REQUIRE(PROVEN_FMT_IS_OK(r), "a user type should format");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&line), PROVEN_LIT("mode=1920x1080")),
                    "the renderer's bytes should be what came out");

    /* 그리고 정렬된다. 형식화기가 먼저 재는 이유가 바로 이것이다 - 사용자 정의 값의 열이
     * 다른 무엇의 열과 똑같이 줄을 맞춘다. */
    (void)proven_u8str_reset(&line);
    r = proven_u8str_append_fmt(&line, "[{:>10}]\n[{:>10}]",
                                PROVEN_ARG_OF(&a, render_rect),
                                PROVEN_ARG_OF(&b, render_rect));
    EXAMPLE_REQUIRE(PROVEN_FMT_IS_OK(r), "two user types should format");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&line),
                                        PROVEN_LIT("[ 1920x1080]\n[ 640x480]")),
                    "both rows should be right-aligned to the same width");

    /* 여러분의 타입에 대해 라이브러리가 해석할 수 없는 지정자는 추측하지 않고 거부된다.
     * 사각형에 `{:x}` 는 뜻이 없고, 성공했다고 알리면서 그럴듯한 무언가로 답하는 것이
     * 형식화기가 여러분에게 거짓말을 시작하는 방식이다. */
    (void)proven_u8str_reset(&line);
    r = proven_u8str_append_fmt(&line, "{:x}", PROVEN_ARG_OF(&a, render_rect));
    EXAMPLE_REQUIRE(r.err == PROVEN_ERR_INVALID_FORMAT,

```

```

        "a type letter on a user type should be an error");

    return EXAMPLE_OK();
}

```

6. 콘솔 출력 헬퍼

`proven_println`은 무엇이고, 비용은 얼마인가

`proven_println("{", PROVEN_ARG(x))`는 프로그램에서 텍스트를 내보내는 가장 짧은 방법이며, 진단 출력이나 일회성 도구, 출력이 몇 줄뿐인 프로그램에는 알맞은 도구다.

루프에는 잘못된 도구인데, 그 이유는 호출 지점에서 보이지 않으므로 명시할 가치가 있다. **호출 하나가 각각 하나의 `write` 시스템 콜이다.** `proven_println` 만 번 호출은 시스템 콜 만 번이고, 이는 포매팅 자체보다 대략 두 자릿수만큼 비싸다. `printf`는 `libc`가 대신 flush해 주는 버퍼 뒤에 이것을 숨긴다. 이 라이브러리는 여러분 모르게 버퍼링하지 않는데, 요청하지 않은 버퍼는 종료 시점에, 크래시 때, 또는 두 기록자가 뒤섞일 때 여러분을 놀라게 하는 버퍼이기 때문이다.

할당에 관한 주의 하나. 줄은 512바이트 스택 버퍼에 포매팅되므로 일반적인 줄은 할당 비용이 0이다. 들어가지 않는 줄은 거부되는 대신 그 호출에 한해 전역 힙(heap)으로 몰려난다 — 길다는 이유로 출력을 거부하는 편이 더 나쁘기 때문이다. 이는 allocator 매개변수가 없는 호출이 그래도 할당할 수 있는 라이브러리 내 유일한 지점이며, 과도하게 긴 경우로 한정된다. 아무것도 할당하지 않는다는 보장이 필요하다면 직접 버퍼에 포매팅한 뒤 그것을 기록할 것.

출력량이 문제가 될 때는 `proven_sysio_stdout_buffered`에서 버퍼드 쓰기 스트림(writer)를 받아 거기에 포매팅한다 — argument 규칙은 동일하고, 줄마다 한 번이 아니라 flush마다 한 번의 시스템 콜이 된다. [Chapter 5](#)가 스트림 계층을 다룬다.

잘못된 예 — 출력하면서 스캐닝용 argument 생성자를 집어들이:

```
proven_println("{", PROVEN_SCAN_ARG(&x)); /* wrong: that builds a scan destination */
```

이 섹션은 상세 I/O API가 Chapter 5에 있기 때문에 의도적으로 짧다. 포매팅 사용자에게 중요한 점은 콘솔 헬퍼들이 문자열 append API와 같은 argument 규칙을 공유한다는 것이다.

흔한 실수:

- `proven_println`에 `PROVEN_SCAN_ARG`를 사용하기
- 모든 포맷 문자열에 `PROVEN_LIT`이 필요하다고 가정하기
- 출력 함수도 여전히 실패할 수 있다는 것을 잊기

7. 스캐너 데이터 모델

스캐너는 당신이 이미 가지고 있는 바이트 위의 커서다. 텍스트를 소유하지 않고, 복사하지 않으며, 어디에서도 읽지 않는다 - `proven_scan_t`는 뷰에 오프셋을 더한 것이며, 그것이 전부다.

```

typedef struct {
    proven_u8str_view_t view; /* the bytes being read; not owned */
    proven_size_t cursor; /* how far in we are */
} proven_scan_t;

```

이것이 `scanf`와 다른 이유이므로 언급할 가치가 있는 두 가지 결과:

- 스캐너는 **절대 할당하지 않고 당신의 입력에 절대 기록하지 않는다.** 스캔된 단어는 원본 바이트 *안*을 가리키는 `proven_u8str_view_t`로 반환된다. 그것은 정확히 그 바이트가 유효한 동안만 유효하며, 그 이상은 아니다. 그것들보다 오래 살아야 한다면, `proven_u8str_create_from_view()`로 복사하라.
- 커서는 **당신의 것이다.** 그것은 평범한 필드다. 저장하거나, 복원하거나, 손으로 진행시킬 수 있다 (§ 12는 `proven_scan_skip_until` 이후 정확히 그렇게 한다). 스캐너의 어떤 것도 당신에게 숨겨져 있지 않으므로, 당신을 위해 되돌려야 할 것도 없다.

`proven_scan_init()`은 잘못된 형식의 뷰(size > 0인데 null 포인터)를 신뢰하는 대신 빈 뷰로 정규화한다. 따라서 쓰레기로부터 만들어진 스캐너는 역참조 대신 입력 끝(EOF)으로 읽힌다.

각 primitive는 값을 그것을 지키는 오류와 짝지은 result 구조체를 반환한다: proven_result_i64_t, proven_result_u64_t, proven_result_f64_t, proven_result_u8str_view_t. 오류가 PROVEN_OK가 아닌 한 값은 무의미하다 - 스캐너는 실패를 뜻하는 sentinel 값을 사용하지 않는데, 모든 sentinel은 동시에 정당한 입력이기도 하기 때문이다.

8. 스캐너 기본 API

```
void proven_scan_skip_whitespace(proven_scan_t *scan);
proven_result_i64_t proven_scan_i64(proven_scan_t *scan);
proven_result_u64_t proven_scan_u64(proven_scan_t *scan);
proven_result_f64_t proven_scan_f64(proven_scan_t *scan);
proven_result_u8str_view_t proven_scan_str(proven_scan_t *scan);
proven_err_t proven_scan_skip_until(proven_scan_t *scan, proven_u8str_view_t target);
void proven_scan_skip_until_number(proven_scan_t *scan);
```

값을 반환하는 것들은 [[nodiscard]]다: 결과를 버리는 스캔은 애초에 할 필요가 없던 스캔이다.

공통 동작

- **선행 공백은 건너뛰다** — 모든 값 스캐너가 그렇게 한다. 먼저 proven_scan_skip_whitespace()를 호출할 필요가 없다. 그 함수는 커서를 직접 위치시키고 싶을 때를 위해 존재한다.
- **스캐닝은 값에 속할 수 없는 첫 바이트에서 멈춘다.** "12abc"는 12를 내고 커서를 a에 남긴다. 그것은 오류가 아니다 - 스캐너는 당신이 묻은 질문에 답했고 나머지는 다음에 묻는 이를 위해 남겼다.
- **실패 시 커서는 복원된다.** 따라서 실패한 스캔은 비사건(non-event)이다: 돌아서서 같은 위치를 다른 것으로 파싱할 수 있다. § 12가 이렇게 한다.

정수 스캐너

입력	proven_scan_i64	이유
"42", "+42", "-42"	OK - 42, 42, -42	부호는 숫자의 일부다
"9223372036854775808"	PROVEN_ERR_OVERFLOW	INT64_MAX보다 하나 큼. 랩(wrap)하지 않는다
"abc", ""	PROVEN_ERR_INVALID_ARG	여기에 숫자가 없다
"0x10"	OK - 0, 커서는 1에	십진수만: 0, 그 뒤에 텍스트

마지막 행이 사람들을 놀라게 하는 것이다. proven_scan_i64와 proven_scan_u64는 십진수를 읽는다. hex도, 8진수도, 기수 접두어도 없다. 0x10은 정수 0이고, x10은 여전히 입력에 남아 있다.

proven_scan_u64는 unsigned를 뜻한다: "-1"은 18446744073709551615로 랩하는 것이 아니라 PROVEN_ERR_INVALID_ARG다. 음수를 조용히 거대한 양수로 바꾸는 스캐너는 경계 검사가 무력화되는 방식이다.

부동소수점 스캐너

proven_scan_f64는 라이브러리의 나머지와 같은 정확히 반올림되는 십진 엔진을 거친다: 가장 가까운 값으로 반올림, 동점은 짝수로, 그 어디에도 long double 없음. nan과 inf를 받아들인다.

두 경계 동작은 **의도적으로 비대칭**이며, 그 비대칭이 요점이다.

- "1e309"는 PROVEN_ERR_OVERFLOW를 준다. 올바른 유한 답이 없으므로, 요청하지 않은 무한대를 건네는 대신 거부한다.
- "1e-400"은 PROVEN_OK와 0.0을 주며, 부호는 보존된다. 0으로의 언더플로는 정확히 반올림된 답 *그 자체*다. 그것을 오류로 보고하는 것은 올바른 산술을 실패로 보고하는 것이 될 것이다.

날말과 커서 이동

proven_scan_str은 다음 공백으로 구분된 구간을 입력 안을 가리키는 뷰로 반환한다. 공백 외에 아무것도 남지 않은 경우는 PROVEN_ERR_INVALID_ARG다.

완전한 뷰에서는 "입력이 떨어졌다"와 "입력이 틀렸다"가 같은 사실이다 - 더 이상 입력이 없으므로, 끝에서 잘린 숫자는 정말로 잘못된 형식이다. 스트림 위에서는 이 둘이 반대되는 사실이며, 그 차이가 기다릴지 오류를 보고할지를 결정한다. 파싱이 가진 것의 끝을 넘어 달렸을 때 스캐너는 proven_scan_t::needs_more를 설정하

며, 버퍼드 스캐너는 정확히 그것을 사용해 다시 채우고 재시도한다: -를 전달한 뒤 잠시 후 12를 전달하는 파이프는 -12로 스캔된다. 이전에는 잘못된 형식의 숫자였다. 실제로 존재하는 잘못된 바이트 - 숫자가 와야 할 자리의 글자 - 는 여전히 오류이며, 이후 어떤 입력도 그것을 바꾸지 못한다. 스캐너는 그것을 기다리지 않는다.

`proven_scan_skip_until(scan, target)`은 커서를 `target`을 지나서가 아니라 `target`으로 옮긴다 - 그것을 얼마나 소비할지는 당신이 결정한다. `target`이 거기 없으면 결과는 `PROVEN_ERR_NOT_FOUND`이고 커서는 움직이지 않는다: 스캐너는 탐색에 실패한 입력을 소비하지 않는다.

`proven_scan_skip_until_number`는 첫 숫자에서, 또는 바로 뒤에 숫자가 따라오는 부호에서 멈춘다. 숫자가 없으면 커서를 입력의 끝까지 몰아간다 - 따라서 읽을 것이 있다고 가정하기 전에 커서(cursor), 곧 스캐너가 지금까지 읽어 온 위치를 `scan.cursor < scan.view.size`로 검사하라.

9. 스캔 인자 모델

스캔 argument는 컴파일러가 선택하는 당신의 목적지를 가리키는 타입 있는 포인터다.

```
PROVEN_SCAN_ARG(&x)    /* _Generic on the pointer type */
```

이것이 스캐너가 `scanf`와 가장 첨예하게 다른 지점이다. 잘못 쓸 포맷 글자가 없는데, 포맷 글자가 아예 없기 때문이다. `long`에 대한 `%d`나, 너무 작은 버퍼에 대한 `%s`는 여기서 가능한 실수가 아니다: 목적지의 타입 *자체*가 명세이며, 불일치는 손상된 스택이 아니라 컴파일 오류다.

지원되는 목적지: `short`, `unsigned short`, `int`, `unsigned int`, `long`, `unsigned long`, `long long`, `unsigned long long`, `double`, 그리고 `proven_u8str_view_t`.

`PROVEN_SCAN_ARG_LONG(&x)`와 `PROVEN_SCAN_ARG_ULONG(&x)`는 호출 지점에서 명시적이길 원하는 호출자를 위해 존재한다. `PROVEN_SCAN_ARG`는 이미 `long*`와 `unsigned long*`를 처리한다.

좁은 목적지는 범위 검사된다. `"70000"`을 `short`로 스캔하는 것은 잘린 4464가 아니라 `PROVEN_ERR_OVERFLOW`다. 값은 64비트로 파싱되어, 무언가를 저장하기 전에 목적지의 범위와 대조 검사된다.

`proven_scan_arg_*` 생성자들은 argument 배열을 손으로 만들어야 할 때를 위해 공개되어 있지만, 호출자가 사용하는 것은 매크로다.

10. 구조적 스캔 문법

포맷 문자열로 파싱하는 것이 출력하는 것보다 위험한 이유

잘못된 포맷 문자열로 포매팅하면 잘못된 출력이 나온다. **그것으로 파싱하면 메모리가 망가진다.** 포맷이 여러 분이 넘긴 포인터를 통해 무엇을 쓸지 결정하기 때문이다. `c`가 `char`인데 `sscanf("%d", &c)`를 하면 1바이트 객체에 4바이트를 쓰는데, 호출문 어디에도 그런 말이 없다.

이 비대칭이 이 섹션의 모양을 결정한다. 구조적 스캔은 포매터와 같은 {} 문법을 쓰지만, 목적지 타입은 `PROVEN_SCAN_ARG(&x)`에서 온다 — 같은 `_Generic` 디스패치이므로 기록되는 폭은 객체의 폭이고, 거기 담기에 너무 큰 값은 이웃한 세 바이트가 아니라 `PROVEN_ERR_OVERFLOW`가 된다.

쓰기 전에 반드시 체크해야 할 성질 하나. **구조적 스캔은 플레이스홀더 전체에 걸쳐 트랜잭션이 아니다.** 세 번째 {}가 실패하면 앞의 두 목적지는 이미 기록된 뒤다. 이는 의도한 절충이다 — 줄 전체가 파싱될 때까지 모든 목적지를 버퍼링하려면 할당이 필요한데, 이 스캐너는 아무것도 할당하지 않는다 — 그러나 그 결과로 실패한 `proven_scan_fmt`는 변수들을 일부만 갱신된 상태로 남긴다. 실패 시 그것들을 쓰레기로 취급하거나, 지역 변수에 스캔한 뒤 성공했을 때만 복사해 낼 것. § 11.1이 두 패턴을 모두 보여준다.

패턴 속 리터럴은 문법의 나머지 절반이며 사람들이 잘 활용하지 않는 부분이다. 플레이스홀더가 아닌 모든 것은 입력과 정확히 일치해야 하므로, `"12-34"`에 대한 `"{:}:{:}"`는 한 필드를 조용히 돌려주는 대신 리터럴에서 실패한다.

스캔 포맷 문자열은 포매터의 것을 거꾸로 읽은 것이다.

- 플레이스홀더는 argument 하나를 순서대로 소비한다.
- 그 외의 모든 것은 입력과 정확히 일치해야 하는 리터럴이다.

스캐닝 측에서는 플레이스홀더 안에 스펙이 없다. Width, fill, alignment는 포매팅의 관심사이며, 스캐너는 거기 있는 것을 읽는다.

포맷 안의 공백은 특별하지 않다. 값 스캐너들이 스스로 선행 공백을 건너뛰므로, 두 플레이스홀더 사이에 공백이 있는 포맷과 없는 포맷은 "7 8"을 동일하게 파싱한다 - 포맷의 공백은 입력의 공백과 일치하고, 그것이 없었더라도 두 번째 스캐너가 어차피 건너뛰었을 것이다.

플레이스홀더의 수는 argument의 수와 같아야 한다. 입력의 값이 너무 적은 것은 오류다. **너무 많은 것은 오류가 아니다**(§ 11.1).

11. 스캔 형식화 API

```
proven_scan_fmt(view, fmt, ...) /* scan a view from the beginning */
proven_scan_fmt_cursor(&scan, fmt, ...) /* continue from an existing cursor */
proven_err_t proven_scan_fmt_internal(...) /* what the macros expand to */
```

자기완결적인 한 줄에는 proven_scan_fmt를 사용하라. 스캔이 같은 입력 위를 걷는 더 긴 여정의 한 단계일 때는 proven_scan_fmt_cursor를 사용하라: 이것은 당신이 소유한 커서를 진행시키므로, § 8의 primitive들과 자유롭게 섞인다.

11.1. 스캔 에러 코드 안내와 복구

코드	실제로 일어난 일	할 일
PROVEN_OK	모든 플레이스홀더가 채워졌고 모든 리터럴이 일치했다.	값들을 발행(publish)하라.
PROVEN_ERR_INVALID_ARG	입력이 당신이 요청한 형태가 아니다 - 플레이스홀더가 읽을 값이 없었거나, 입력이 떨어졌다.	그 줄은 일치하지 않는다. 보고하되, 같은 형태로 재시도하지 마라.
PROVEN_ERR_NEED_MORE	버퍼드 스캐너 전용. 토큰이 읽기 경계에서 반으로 잘렸다: 그 나머지가 아직 도착하지 않았다. 보통은 이것을 보지 않는다 - proven_sysio_scanner_scan이 당신을 위해 다시 채우고 재시도한다 - 이는 스캐너가 스스로에게 하는 말이다.	아무것도. 이미 처리되었다.
PROVEN_ERR_NOT_FOUND	포맷 안의 리터럴 이 일치하지 않았다.	그 줄은 예상과 다른 형태다. 저장된 커서에서 다른 포맷을 시도하라.
PROVEN_ERR_OVERFLOW	숫자는 제대로 된 형식이었지만 목적지에 들어맞지 않는다.	입력이 유효한데 목적지가 너무 좁은 것일 수도, 입력이 적대적인 것일 수도 있다. 그 둘은 매우 다른 상황이다 - 타입을 넓히기 전에 구별하라.
PROVEN_ERR_INVALID_FORMAT	포맷 문자열 자체가 잘못된 형식이다.	입력이 아니라 당신 코드의 버그다.

구조적 스캐너는 트랜잭션이 아니며, 이것이 물어뜯는 그 지점이다.

리터럴이 일치에 실패했을 때, 불일치 *이전*의 플레이스홀더들은 이미 기록되어 있다. 호출은 PROVEN_ERR_NOT_FOUND를 반환하는데 당신의 목적지는 그래도 값을 담고 있다 - 아래의 id는 실패한 호출에서 온 7이다.

```
int id = -1;
double ratio = -1.0;
proven_err_t err = proven_scan_fmt(line, "id={ } XXX={ }",
                                   PROVEN_SCAN_ARG(&id), PROVEN_SCAN_ARG(&ratio));
/* err == PROVEN_ERR_NOT_FOUND, and id == 7: it was written before the failure. */
```

그러므로: 실패 시 모든 목적지를 오염된(clobbered) 것으로 취급하라. 전부 아니면 전무가 필요하다면, 지역 변수로 스캔하고 호출이 성공한 뒤에만 발행하라 - § 12의 예제가 그 형태를 보여준다. 대안으로, 호출 전에 scan.cursor를 저장하고 이후에 복원하라. 커서는 평범한 필드이며, 그것은 의도적이다.

후행 입력은 오류가 아니다. "7 8"에 대해 플레이스홀더 하나를 스캔하면 값 7로 성공하고 8은 소비되지 않은 채 남는다. 스캐너는 당신이 요청한 것을 일치시키고 멈췄다. 당신이 묻지 않은 것을 단속하지 않는다. 줄 전체가 소비되어야 한다면, 커서를 직접 검사하라.

```
if (scan.cursor != scan.view.size) { /* there is unparsed input left */ }
```

12. 예제와 오용 사례

실전 예제: 한 줄을 형식화하고 다시 읽어 들이기

테스트 스위트가 컴파일하고 실행함. 스택에서 빌린 문자열(오버플로 시 atomic)과 할당자 기반 문자열(성장함)로 포매팅하고, 신뢰할 수 없는 바이트에 경계 argument를 사용한 뒤, 그 줄을 다시 파싱한다 - float은 정확히 라운드트립된다.

```
/*
 * 형식화와 파싱은 같은 생각의 두 반쪽이다. `{}` 는 타입 있는 값을 글로 그려 내고,
 * 스캐너는 글을 타입 있는 목적지로 되읽는다. 둘 다 호출 자리에서 타입이 검사되므로
 * (_Generic 이 생성자를 고른다), 실행 중에 어긋날 서식 문자열/인자 짝이 없다.
 *
 * 중요한 선택은 *바이트가 어디로 가는가* 다.
 *
 * append_fmt      - 용량 고정, 원자적. 너무 길면? 아무것도 쓰이지 않고
 *                   PROVEN_ERR_OUT_OF_BOUNDS 를 받는다. 할당자가 끼지 않으므로
 *                   스택 버퍼에서도 돈다.
 * append_fmt_grow - 할당자를 등에 업는다. 들어가도록 늘리고, 할당이 실패하면
 *                   문자열은 있던 그대로 남는다.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* --- 용량 고정: 할당자도, 할당도 없다 ----- */
    /* borrow 는 부르는 쪽의 기억을 감싼다. 그래서 이 문자열은 통째로 스택에 산다. `cap`
     * 은 NUL 을 포함하므로 32 바이트는 내용 31 바이트를 담는다. 지울 것도 없다. */
    proven_byte_t stack_buf[32];
    proven_u8str_t fixed = proven_u8str_borrow(stack_buf, sizeof stack_buf);

    proven_fmt_result_t r = proven_u8str_append_fmt(&fixed, "port={}", PROVEN_ARG(8080));
    EXAMPLE_REQUIRE(PROVEN_FMT_IS_OK(r), "a short line should fit in 32 bytes");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&fixed), PROVEN_LIT("port=8080")),
        "the fixed-capacity append should have rendered the port");

    /* 원자적이라는 말은 원자적이라는 뜻이다. 들어가지 않는 append 는 아무것도 바꾸지
     * 않는다. 문자열은 여전히 온전하고 전에 담고 있던 것을 그대로 담고 있다 - 치울 잘린
     * 꼬리가 없다. (잘린 꼬리가 원하는 것이라면 append_fmt_trunc 를 쓸 것.) */
    proven_fmt_result_t too_long = proven_u8str_append_fmt(
        &fixed, " and a great deal more text than will ever fit here {}", PROVEN_ARG(1));
    EXAMPLE_REQUIRE(too_long.err == PROVEN_ERR_OUT_OF_BOUNDS, "the overlong append must fail");
    EXAMPLE_REQUIRE(too_long.required > too_long.written, "it reports what it would have needed");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&fixed), PROVEN_LIT("port=8080")),
        "a failed atomic append must leave the string untouched");

    /* --- 지정자: 채움, 정렬, 폭, 16진 ----- */
    proven_result_u8str_t created = proven_u8str_create(alloc, 8); /* 일부러 작게 */
    EXAMPLE_REQUIRE(proven_is_ok(created.err), "creating the output string should succeed");
    if (!proven_is_ok(created.err)) return 1;
    proven_u8str_t out = created.value;
```

```

/* grow 는 필요한 만큼 재할당하므로, 처음 용량은 한계가 아니라 힌트다.
 * `{:0>4}` = 채움 '0', 오른쪽 정렬, 폭 4. `{:x}` = 소문자 16진, 0x 없음. */
r = proven_u8str_append_fmt_grow(alloc, &out, "id={:0>4} tag={:^9} addr=0x{:x}",
    PROVEN_ARG(7),
    PROVEN_ARG(PROVEN_LIT("ok")),
    PROVEN_ARG(48879));
EXAMPLE_REQUIRE(PROVEN_FMT_IS_OK(r), "the growing append should succeed");
EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&out),
    PROVEN_LIT("id=0007 tag=***ok*** addr=0xbeef")),
    "fill/align/width/hex should render exactly this");
printf("%s\n", proven_u8str_as_cstr(&out));

/* --- 믿을 수 없는 글은 경계가 있고, NUL 로 끝난다고 믿지 않는다 ----- */
/* char* 에 PROVEN_ARG 를 쓰면 "NUL 이 나올 때까지 걸러라" 는 뜻이다 - 리터럴에는
 * 관찰지만 소켓에서 온 것에는 버퍼 넘어 읽기다. 이 버퍼에는 NUL 이 아예 없다.
 * PROVEN_ARG_CSTR_N 은 대신 길이에서 멈추므로 실제로 있는 것만 읽는다. 여러분이 직접
 * 만들지 않은 것에는 이것을 쓸 것. */
const char untrusted[4] = {'a', 'b', 'c', 'd'}; /* 일부러 종결자를 두지 않았다 */
EXAMPLE_REQUIRE(proven_is_ok(proven_u8str_reset(&out)), "reset should keep the buffer");
r = proven_u8str_append_fmt_grow(alloc, &out, "payload={}",
    PROVEN_ARG_CSTR_N(untrusted, sizeof untrusted));
EXAMPLE_REQUIRE(PROVEN_FMT_IS_OK(r), "the bounded append should succeed");
EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&out), PROVEN_LIT("payload=abcd")),
    "the bounded argument should render its whole 4 bytes and stop");

/* --- 레코드를 형식화하고, 도로 파싱하기 ----- */
proven_i64 sensor_id = 42;
double reading = 3.14159;

EXAMPLE_REQUIRE(proven_is_ok(proven_u8str_reset(&out)), "reset should keep the buffer");
r = proven_u8str_append_fmt_grow(alloc, &out, "{ } { } { }",
    PROVEN_ARG(sensor_id),
    PROVEN_ARG(PROVEN_LIT("boiler")),
    PROVEN_ARG(reading));
EXAMPLE_REQUIRE(PROVEN_FMT_IS_OK(r), "formatting the record should succeed");
printf("record: %s\n", proven_u8str_as_cstr(&out));

/* 뷰 하나 위의 스캐너 하나. 호출마다 커서를 자기가 삼킨 만큼 앞으로 옮기므로 호출이
 * 왼쪽에서 오른쪽으로 이어진다 - 그리고 하나하나가 따로 실패할 수 있는데, 그것이
 * 파서와 짐작을 가르는 차이이다. */
proven_scan_t sc = proven_scan_init(proven_u8str_as_view(&out));

proven_result_i64_t id = proven_scan_i64(&sc);
EXAMPLE_REQUIRE(proven_is_ok(id.err), "the first field should parse as an integer");
EXAMPLE_REQUIRE(id.val == sensor_id, "the integer should round-trip");

/* scan_str 은 *파싱 중인 문자열 안*을 가리키는 뷰를 돌려준다 - 복사도 소유도 하지
 * 않으므로 `out` 이 살아 있는 동안에만 쓸 수 있다. */
proven_result_u8str_view_t name = proven_scan_str(&sc);
EXAMPLE_REQUIRE(proven_is_ok(name.err), "the second field should parse as a word");
EXAMPLE_REQUIRE(proven_u8str_view_eq(name.val, PROVEN_LIT("boiler")), "the name should round-trip");

proven_result_f64_t temp = proven_scan_f64(&sc);
EXAMPLE_REQUIRE(proven_is_ok(temp.err), "the third field should parse as a float");

/* 근사하게가 아니라 정확히 같다. 스캐너는 올바르게 반올림하므로 글자에 가장 가까운
 * double 을 돌려주고 - 형식화기가 내놓은 글자(소수 여섯 자리)는 이 값을 모호함 없이
 * 지목한다. 비트 하나까지 우리가 시작한 그 double 이다. 소수 여섯 자리보다 더 필요한
 * 값이라면 가장 짧은 정책(proven_float_format_options_shortest)으로 형식화하면 같은

```

```

    * 왕복이 성립한다. */
    EXAMPLE_REQUIRE(temp.val == reading, "the float must round-trip exactly, not approximately");

    /* 입력을 남김없이 삼켰다. 조용히 남겨 둔 것이 없다. */
    proven_result_i64_t extra = proven_scan_i64(&sc);
    EXAMPLE_REQUIRE(!proven_is_ok(extra.err), "there should be nothing left to scan");

    proven_u8str_destroy(alloc, &out);
    return EXAMPLE_OK();
}

```

실전 예제: 스캐너의 에러 코드와 그로부터의 복구

테스트 스위트가 컴파일하고 실행함. 위 표의 모든 코드를 여기서 일부러 유발한다. 비-트랜잭션 실패까지 포함하는데 - 읽기만 한 계약은 배우지 못한 계약이기 때문이다.

```

/*
 * 스캐너의 오류 코드들, 그리고 거기서 되살아나는 법.
 *
 * 이 스캐너는 scanf 가 아니다. 받지 않은 포인터를 통해 쓰는 일이 없고, 폭을 짐작하지
 * 않으며, 여러 가지 중 무엇이 잘못됐는지를 말해 준다. 마지막 것은 그 코드들이 무슨
 * 뜻인지 알아야 도움이 되므로, 이 프로그램은 하나하나를 일부러 일으켜 본다.
 */

static proven_u8str_view_t v(const char *s) {
    return proven_u8str_view_from_cstr(s);
}

int main(void) {
    /* --- 기본 호출들은 실패하면 커서를 되돌린다 ----- */
    /* 실패한 파싱은 없던 일이다. 커서는 있던 자리에 있으므로, 같은 자리를 다른 것으로
     * 읽어 볼 수 있다. */
    {
        proven_scan_t sc = proven_scan_init(v("abc"));
        proven_result_i64_t n = proven_scan_i64(&sc);
        EXAMPLE_REQUIRE(n.err == PROVEN_ERR_INVALID_ARG, "'abc' is not an integer");
        EXAMPLE_REQUIRE(sc.cursor == 0, "a failed integer scan leaves the cursor alone");

        /* 그래서 같은 자리를 낱말로 읽을 수 있다. */
        proven_result_u8str_view_t w = proven_scan_str(&sc);
        EXAMPLE_REQUIRE(proven_is_ok(w.err) && proven_u8str_view_eq(w.val, PROVEN_LIT("abc")),
            "the same bytes parse fine as a word");
    }

    /* --- 들어가지 않는 수는 감긴 값이 아니라 OVERFLOW 다 ----- */
    {
        proven_scan_t sc = proven_scan_init(v("9223372036854775808")); /* INT64_MAX + 1 */
        proven_result_i64_t n = proven_scan_i64(&sc);
        EXAMPLE_REQUIRE(n.err == PROVEN_ERR_OVERFLOW, "one past INT64_MAX must not wrap");
        EXAMPLE_REQUIRE(sc.cursor == 0, "the cursor is restored on overflow too");
    }

    /* --- 그러나 아래로 넘치는 실수는 오류가 *아니다* ----- */
    /* 너무 크면 OVERFLOW 이고, 너무 작으면 부호를 지킨 0 이다. 그 비대칭은 일부러다.
     * 0 으로 내려앉는 것은 올바르게 반올림한 답이지만, 위로 넘치는 데는 올바른 유한한
     * 답이 아예 없다. */
    {
        proven_scan_t big = proven_scan_init(v("1e309"));
        proven_result_f64_t b = proven_scan_f64(&big);
        EXAMPLE_REQUIRE(b.err == PROVEN_ERR_OVERFLOW, "1e309 does not fit a double");
    }
}

```

```

    proven_scan_t tiny = proven_scan_init(v("-1e-400"));
    proven_result_f64_t t = proven_scan_f64(&tiny);
    EXAMPLE_REQUIRE(proven_is_ok(t.err), "1e-400 underflows, which is not an error");
    EXAMPLE_REQUIRE(t.val == 0.0, "it rounds to zero");
}

/* --- 정수 스캐너는 십진만 읽는다 ----- */
/* "0x10" 은 열여섯이 아니다. 0 하나이고, 그 뒤는 스캐너가 보라고 하지 않은 글이다.
 * 사람들이 놀라는 자리라 알아 둘 값어치가 있다. */
{
    proven_scan_t sc = proven_scan_init(v("0x10"));
    proven_result_i64_t n = proven_scan_i64(&sc);
    EXAMPLE_REQUIRE(proven_is_ok(n.err) && n.val == 0, "0x10 scans as the integer 0");
    EXAMPLE_REQUIRE(sc.cursor == 1, "and the cursor stops before the 'x'");
}

/* --- 파싱은 그 값에 속할 수 없는 첫 바이트에서 멈춘다 ----- */
{
    proven_scan_t sc = proven_scan_init(v("12abc"));
    proven_result_i64_t n = proven_scan_i64(&sc);
    EXAMPLE_REQUIRE(proven_is_ok(n.err) && n.val == 12, "12abc yields 12");
    EXAMPLE_REQUIRE(sc.cursor == 2, "and leaves 'abc' for whoever asks next");
}

/* --- 부호 없음은 부호 없음이라는 뜻이다 ----- */
{
    proven_scan_t sc = proven_scan_init(v("-1"));
    proven_result_u64_t n = proven_scan_u64(&sc);
    EXAMPLE_REQUIRE(n.err == PROVEN_ERR_INVALID_ARG,
        "-1 is rejected rather than wrapping to a huge unsigned value");
}

/* --- 값까지 찾아가기: skip_until ----- */
/* skip_until 은 커서를 목표 *위*에 두지, 지나쳐 두지 않는다. 그것을 얼마나 삼킬지는
 * 여러분이 정한다. */
{
    proven_scan_t sc = proven_scan_init(v("port=8080"));
    proven_err_t err = proven_scan_skip_until(&sc, PROVEN_LIT("="));
    EXAMPLE_REQUIRE(proven_is_ok(err), "the '=' is there");
    EXAMPLE_REQUIRE(sc.cursor == 4, "the cursor sits on the '=' itself");

    ++sc.cursor; /* 그것을 넘어선다 */
    proven_result_i64_t port = proven_scan_i64(&sc);
    EXAMPLE_REQUIRE(proven_is_ok(port.err) && port.val == 8080, "the port parses");

    /* 못 찾으면 NOT_FOUND 이고 커서는 움직이지 않는다 - 스캐너는 찾아가지 못한
     * 입력을 삼키지 않는다. */
    proven_scan_t sc2 = proven_scan_init(v("port=8080"));
    proven_err_t missing = proven_scan_skip_until(&sc2, PROVEN_LIT("#"));
    EXAMPLE_REQUIRE(missing == PROVEN_ERR_NOT_FOUND, "there is no '#'");
    EXAMPLE_REQUIRE(sc2.cursor == 0, "and the cursor stayed put");
}

/* --- 구조를 읽는 스캐너 ----- */
{
    int id = 0;
    double ratio = 0.0;
    proven_u8str_view_t name = {0};

    proven_err_t err = proven_scan_fmt(v("id=7 ratio=0.5 name=ada"),

```

```

        "id={} ratio={} name={}",
        PROVEN_SCAN_ARG(&id),
        PROVEN_SCAN_ARG(&ratio),
        PROVEN_SCAN_ARG(&name));
EXAMPLE_REQUIRE(proven_is_ok(err), "the line matches the shape");
EXAMPLE_REQUIRE(id == 7 && ratio == 0.5, "the values land in the right places");
EXAMPLE_REQUIRE(proven_u8str_view_eq(name, PROVEN_LIT("ada")), "including the word");
}

/* --- 구조를 읽는 스캐너는 트랜잭션이 *아니다* ----- */
/*
* 이것이 사람을 무는 자리다. 리터럴이 맞지 않으면 파싱은 오류를 돌려주는데 - 그
* 어긋남 *앞*의 자리표들은 이미 목적지에 쓰여 버렸다. 호출이 실패했는데도 `id` 는
* 7 이다.
*
* 그러니 실패했을 때는 모든 목적지가 더럽혀졌다고 여길 것. 전부 아니면 전무가
* 필요하면 지역 변수로 파싱하고 호출이 성공한 뒤에만 공표할 것. 아래 코드가 하는
* 일이 그것이다.
*/
{
    int id = -1;
    double ratio = -1.0;
    proven_err_t err = proven_scan_fmt(v("id=7 ratio=0.5"),
        "id={} XXX={}", /* 리터럴이 틀렸다 */
        PROVEN_SCAN_ARG(&id),
        PROVEN_SCAN_ARG(&ratio));
    EXAMPLE_REQUIRE(err == PROVEN_ERR_NOT_FOUND, "the literal 'XXX=' is not in the input");
    EXAMPLE_REQUIRE(id == 7, "and yet id was already written: the scan is not atomic");

    /* 안전한 모양: 지역 변수로 파싱하고 성공했을 때 공표한다. */
    int good_id = 0;
    double good_ratio = 0.0;
    int published_id = -1;
    proven_err_t ok = proven_scan_fmt(v("id=7 ratio=0.5"), "id={} ratio={}",
        PROVEN_SCAN_ARG(&good_id), PROVEN_SCAN_ARG(&good_ratio));
    if (proven_is_ok(ok)) published_id = good_id;
    EXAMPLE_REQUIRE(published_id == 7, "publish only what a successful scan produced");
}

/* --- 입력이 모자랄 때, 그리고 남을 때 ----- */
{
    int a = 0, b = 0;
    proven_err_t short_input = proven_scan_fmt(v("5"), "{} {}",
        PROVEN_SCAN_ARG(&a), PROVEN_SCAN_ARG(&b));
    EXAMPLE_REQUIRE(!proven_is_ok(short_input), "two placeholders, one value: that fails");

    /* 뒤에 남은 입력은 오류가 *아니다*. 스캐너는 청한 것을 맞추고 멈췄다. 청하지
    * 않은 것까지 단속하지는 않는다. 줄 전체를 삼켜야 한다면 그것은 여러분이
    * 확인할 일이다. */
    int only = 0;
    proven_scan_t sc = proven_scan_init(v("7 8"));
    proven_err_t err = proven_scan_fmt_cursor(&sc, "{}", PROVEN_SCAN_ARG(&only));
    EXAMPLE_REQUIRE(proven_is_ok(err) && only == 7, "the first value scans");
    EXAMPLE_REQUIRE(sc.cursor < sc.view.size, "and '8' is still sitting there, unconsumed");
}

/* --- 좁은 목적지는 범위가 검사된다 ----- */
{
    short small = 0;
    proven_err_t err = proven_scan_fmt(v("70000"), "{}", PROVEN_SCAN_ARG(&small));
}

```

```

EXAMPLE_REQUIRE(err == PROVEN_ERR_OVERFLOW,
                "70000 does not fit a short, and the scanner says so rather than truncating");
}

return EXAMPLE_OK();
}

```

오용: 0x10이 16이라고 가정하기

그것은 0이다. 정수 스캐너들은 십진수 전용이며, x10은 여전히 입력에 남아 있다. hex가 필요하다면, 그 자릿수 루프는 당신이 직접 작성하는 것이다.

오용: 남은 입력을 예러로 다루기

그것은 오류가 아니다. "7 8"에 대한 플레이스홀더 하나는 성공한다. 신경 쓰인다면 커서를 검사하라.

오용: 실패한 스캔 뒤의 목적지를 믿기

그것들은 오염되었다. § 11.1을 참조하라.

오용: 입력이 사라진 뒤에도 스캔한 낱말을 들고 있기

proven_scan_str은 복사본이 아니라 **입력 안을 가리키는 뷰**를 반환한다. 버퍼가 사라지면, 그 단어도 사라진다. 그것이 온 바이트보다 오래 살아야 한다면 proven_u8str_create_from_view()로 복사하라.

13. 프리스탠딩과 빌드 모드 참고

float만 컴파일에서 빠지는 이유

이 장의 모든 것은 이식 가능한 계산이지만 한 부분만 예외이고, 그 부분은 유난히 비싸다.

double을 올바르게 포매팅하는 것 — 왕복(round-trip)하는 가장 짧은 십진 표현을, subnormal을 포함한 모든 입력에 대해 얻는 것 — 에는 큰 정수 연산과 조회 테이블이 필요하다. 포맷터에서 단일 코드 조각으로는 가장 크며, 대부분의 펌웨어는 double을 아예 출력하지 않는다. 그래서 freestanding 프로파일은 PROVEN_FMT_NO_FLOAT를 설정해 그것을 털어내고, 마이크로컨트롤러용 빌드는 결코 호출하지 않을 수 킬로바이트의 십진 변환 테이블을 지고 다니지 않는다.

이것이 런타임 결정이 아니라 빌드 타임 결정인 것은 의도적이다. 코드가 단지 도달 불가능한 것이 아니라 *부재*하므로, 링커를 설득해 남겨둘 수 없다. 작은 타깃에서도 float를 원하는 빌드를 위한 관련 손잡이 — 큰 정수 용량 — 는 [freestanding 가이드](#) § 8a가 다룬다.

스캐너는 코어다: I/O를 하지 않고, 아무것도 할당하지 않으며, 어떤 플랫폼 레이어도 건드리지 않는다. 따라서 freestanding 빌드에서도 호스티드(hosted) 빌드에서와 정확히 똑같이 사용할 수 있다.

유일한 빌드 모드 의존성은 float다. Freestanding 빌드는 PROVEN_FMT_NO_FLOAT(float *포맷터*를 컴파일에서 제외)와 PROVEN_NO_U16STR를 설정한다. proven_scan_f64는 십진 파싱 엔진(float_parse.c와 float_decimal.c)을 끌어오는데, 이는 정수 전용이며

- long double 없음, libm 없음, 소프트 float 헬퍼 호출 없음 - 코드 크기 면에서

공짜는 아니다. 그것이 중요한 타겟이면서 float을 읽을 필요가 없다면, 그저 호출하지 마라: 스캐너의 다른 어떤 것도 float 경로를 참조하지 않는다.

proven_sysio_scanner_*(Chapter 5)는 **다른** 것이다: 파일 위의 버퍼드 스캐너로, I/O를 하기 때문에 hosted 전용이다. 이 장에서 설명한 스캐너는 당신이 이미 가지고 있는 바이트를 읽는다.

실전 예제: 더 넓은 수, 자유로운 텍스트, 부동소수점

앞의 예제들은 정해진 서식을 proven_i32로 읽어 들인다. 입력이 실제 데이터가 되는 순간 세 가지가 따라오며, 이 예제는 그것들을 한꺼번에 다룬다.

목적지 타입이 곧 계약이다. proven_scan_arg_i64(), proven_scan_arg_u32(), proven_scan_arg_u64()는 값이 들어갈 타입을 이름으로 말하고, 스캐너는 감기는 대신 넘침을 보고한다. 32비트로는 모자란 바이트 수와, 결코 음수일 수 없는 값 — 이 둘이 목적지 타입을 잘못 고른 것이 숫자가 커지기 전까지 아무도 모르는 버그가 되는 경

우다. 일반 매크로 `PROVEN_SCAN_ARG()`는 넘겨받은 포인터로 이들 중 하나를 골라 주며, 이름 있는 형태는 그 매크로가 고르는 것을 손으로 적은 것이다.

모든 입력이 정해진 서식인 것은 아니다. `proven_scan_skip_whitespace()`는 공백·탭·줄바꿈을 지나가고, `proven_scan_skip_until_number()`는 다음 숫자 자리까지, 또는 바로 뒤에 숫자가 붙은 부호 자리까지 커서를 옮긴다. 음수가 부호를 잃지 않는 이유다. 둘 다 실패할 수 없다. "건너뛴 것이 없었다"는 에러가 아니며, 입력의 끝에서 커서는 넘어가지 않고 멈춘다.

부동소수점에는 로케일(locale) 문제가 있고, 이 라이브러리에는 없다. `strtod()`는 "3,5"를 어떤 로케일에서는 3.5로, 다른 로케일에서는 3으로 읽으므로 같은 프로그램이 기계마다 다른 말을 한다.

`proven_parse_double_ascii()`는 구조적으로 로케일에서 자유롭다. 쉼표는 결코 소수점이 아니다. 이 함수는 숫자가 쓴 바이트 수를 알려 주어 호출자가 그 지점부터 이어 갈 수 있게 하고, 해석할 수 없는 텍스트를 진짜 0과 구분되지 않는 0이 아니라 에러로 보고한다. `proven_parse_f64_ascii()`는 같은 함수의 예전 이름이며, 기존 호출 자리가 그대로 읽히도록 남아 있다.

내보내는 쪽에서는 `proven_arg_f64()`가 부동소수점 값이 형식화기로 들어가는 통로다. `float`도 `double`도 이것을 지나므로, 출력이 그 값을 담아 둔 변수의 쪽에 좌우되지 않는다. 정확한 표기가 중요할 때는 `proven_float_format_f32_policy()`와 그 `f64` 짝이 정책과 옵션을 명시적으로 받는다. 최단(shortest) 모드는 다시 읽었을 때 정확히 같은 값이 되는 가장 짧은 자릿수를 요구한다. 직렬화기가 원하는 성질이 이것이다. 필요 없는 수에까지 열일곱 자리를 찍지 않으면서 왕복을 정확하게 만든다.

```
#include <string.h>
```

```
/*
 * 글에서 수를 읽어 내고, 다시 써 넣기.
 *
 * 여기 쓰인 글은 프로그램이 실제로 만나는 종류다. 폭과 부호가 뒤섞인 로그 줄, 그리고
 * 관심 있는 수가 산문 속에 묻혀 있는 측정값 파일. 그래서 앞의 8장 예제들이 다루지 않는
 * 세 가지 일이 나온다.
 *
 * - int 보다 *넓은* 타입, 그리고 부호 없는 타입으로 파싱하기. 여기서는 목적지 타입이
 *   물음의 전부터 - 32비트에 들어가지 않는 바이트 수야말로 파일이 5 GB 가 될 때까지
 *   아무도 알아채지 못하는 고전적인 넘침이다.
 * - 입력이 뽀뽀한 형식이 아닐 때 커서를 손으로 옮기기 - 공백을 건너뛰거나, 다음 수가
 *   시작하는 자리까지 건너뛰기.
 * - 십진 문자열을 double 로, 그리고 다시 글로, C 라이브러리의 로케일에 매인 변환을
 *   거치지 않고 정확하게 옮기기.
 *
 * 부동소수점 쪽은 들리는 것보다 중요하다. strtod 는 어느 로케일에서는 "3,5" 를 3.5 로,
 * 다른 로케일에서는 3 으로 읽고, 그러면 같은 프로그램이 기계마다 서로 다른 소리를 한다.
 * 여기 쓰인 파서는 만들어질 때부터 로케일이 없다. 환경이 무어라 하든 쉼표가 소수점이
 * 되는 일은 없다.
 */
```

```
int main(void) {
    /* --- 1. 값이 필요로 하는 타입으로 파싱하기 ----- */

    /* 로그 줄 하나: 64비트가 필요한 요청 번호, 결코 음수가 아니고 4 GB 를 넘을 수 있는
     * 바이트 수, 작은 상태 코드, 그리고 음수 오프셋. */
    proven_scan_t scan = proven_scan_init(
        PROVEN_LIT("id=9007199254740993 bytes=5368709120 status=404 delta=-17"));

    proven_i64 id = 0;
    proven_u64 bytes = 0;
    proven_u32 status = 0;
    proven_i32 delta = 0;

    /* 인자 생성자마다 목적지 타입을 이름으로 밝히므로, 스캐너는 옳은 쪽으로 쓰고 감기는
     * 대신 넘침을 알린다. 일반 매크로 PROVEN_SCAN_ARG() 는 포인터의 타입에서 이것들을
```

```

* 대신 골라 준다. 이름 붙은 끝은 그것이 고르는 것 자체이고, 그 선택을 눈에 보이게
* 하고 싶을 때 여러분이 적는 것이다. */
proven_err_t err = proven_scan_fmt_cursor(&scan, "id={} bytes={} status={} delta={}",
    proven_scan_arg_i64(&id),
    proven_scan_arg_u64(&bytes),
    proven_scan_arg_u32(&status),
    proven_scan_arg_i32(&delta));
EXAMPLE_REQUIRE(proven_is_ok(err), "scanning the log line must succeed");
EXAMPLE_REQUIRE(id == 9007199254740993LL, "a 64-bit id survives, which a 32-bit destination could not");
EXAMPLE_REQUIRE(bytes == 5368709120ULL, "and so does a byte count larger than 4 GiB");
EXAMPLE_REQUIRE(status == 404u, "the small unsigned value reads normally");
EXAMPLE_REQUIRE(delta == -17, "and a signed destination accepts the minus sign");

/* 목적지 타입은 힌트가 아니라 계약이다. 음수에는 부호 없는 표현이 없고, 스캐너는
* 그것을 어마어마한 값으로 감아 버리는 대신 거부한다. */
proven_scan_t neg = proven_scan_init(PROVEN_LIT("-17"));
proven_u32 nowhere = 12345;
err = proven_scan_fmt_cursor(&neg, "{}", proven_scan_arg_u32(&nowhere));
EXAMPLE_REQUIRE(err != PROVEN_OK, "a negative value cannot be scanned into an unsigned destination");
EXAMPLE_REQUIRE(nowhere == 12345, "and the destination is left as it was");

/* --- 2. 입력이 뽀뽀하지 않을 때 커서 옮기기 ----- */

/* 자유로운 글: 수가 중요하고 그 사이의 낱말은 아니다. */
proven_scan_t notes = proven_scan_init(
    PROVEN_LIT(" sample A measured 42 units; sample B measured -8 units"));

/* skip_whitespace 는 공백과 탭과 줄바꿈을 지나 나아간다. 손으로 물고 가는 형식의 필드
* 사이에서 부르는 것이고, 실패하는 일이 없다. "공백이 없었다" 는 오류가 아니기
* 때문이다. */
proven_scan_skip_whitespace(&notes);
EXAMPLE_REQUIRE(notes.cursor == 3, "the three leading spaces are consumed");

/* skip_until_number 는 커서를 첫 숫자, 또는 곧바로 숫자가 따라오는 부호까지 물고
* 간다. "이 줄에서 수를 찾아라" 를 손으로 쓴 반복문에서 한 문장으로 바꿔 주는
* 호출이다. */
proven_scan_skip_until_number(&notes);
proven_i32 first = 0;
err = proven_scan_fmt_cursor(&notes, "{}", proven_scan_arg_i32(&first));
EXAMPLE_REQUIRE(proven_is_ok(err) && first == 42, "the first number in the line is 42");

proven_scan_skip_until_number(&notes);
proven_i32 second = 0;
err = proven_scan_fmt_cursor(&notes, "{}", proven_scan_arg_i32(&second));
EXAMPLE_REQUIRE(proven_is_ok(err) && second == -8,
    "and the next one keeps its sign, because the sign is part of the number");

/* 끝에 이르면 더 찾을 것이 없고, 커서는 달려 나가는 대신 멈춘다. 파싱이 망가진 것이
* 아니라 끝난 것이다. */
proven_scan_skip_until_number(&notes);
EXAMPLE_REQUIRE(notes.cursor == notes.view.size, "with no number left, the cursor lands at the end");

/* --- 3. 십진 글을 double 로, 로케일 없이 ----- */

proven_parse_double_result_t d = proven_parse_double_ascii(PROVEN_LIT("3.14159 rest"));
EXAMPLE_REQUIRE(proven_is_ok(d.err), "parsing a decimal number must succeed");
EXAMPLE_REQUIRE(d.val > 3.14158 && d.val < 3.14160, "and produce the value the digits spell");

/* consumed 는 수가 어디서 끝났는지 알려 주므로 부르는 쪽이 거기서 이어 갈 수 있다 -
* 목록을 토막으로 복사하지 않고 파싱하는 방법이 그것이다. */

```

```

EXAMPLE_REQUIRE(d.consumed == 7, "consumed reports exactly the bytes the number used");

/* 여기서 쉼표는 소수점이 아니고 앞으로도 아니다. 프로그램이 어느 로케일에서 돌든
 * 마찬가지로. 수는 쉼표에서 끝난다. */
proven_parse_double_result_t comma = proven_parse_double_ascii(PROVEN_LIT("3,5"));
EXAMPLE_REQUIRE(proven_is_ok(comma.err) && comma.consumed == 1 && comma.val == 3.0,
    "a comma ends the number: the parser is locale-free by construction");

/* proven_parse_f64_ascii 는 예전 이름을 단 같은 함수다. 지금 이름이 생기기 전에 쓰인
 * 코드를 위해 남겨 두었다. 새 코드는 proven_parse_double_ascii 를 쓸 것. 옛 호출
 * 자리도 그대로 읽히도록 둘 다 여기 있다. */
proven_parse_f64_result_t same = proven_parse_f64_ascii(PROVEN_LIT("3.14159 rest"));
EXAMPLE_REQUIRE(same.val == d.val && same.consumed == d.consumed,
    "the compatibility name is the same parser");

/* 아예 수가 아닌 글은 조용한 0 이 아니라 오류다 - atof() 였다면 진짜 "0" 과 구별할
 * 방법 없이 그것을 돌려주었을 것이다. */
proven_parse_double_result_t junk = proven_parse_double_ascii(PROVEN_LIT("not a number"));
EXAMPLE_REQUIRE(!proven_is_ok(junk.err), "unparsable text is reported, not turned into 0");
EXAMPLE_REQUIRE(junk.consumed == 0, "and nothing was consumed");

/* --- 4. float 를 다시 글로 ----- */

/* proven_arg_f64 가 부동소수점 값이 형식화기로 들어가는 길이다. float 도 double 도
 * 그것을 지나므로, 프로그램이 찍는 자릿수가 그 값이 어쩌다 담겨 있던 변수의 폭에
 * 좌우되지 않는다. */
proven_allocator_t alloc = proven_heap_allocator();
proven_result_u8str_t line = proven_u8str_create(alloc, 64);
EXAMPLE_REQUIRE(proven_is_ok(line.err), "creating the output string must succeed");

proven_fmt_result_t out = proven_u8str_append_fmt(&line.value, "measured {} units",
    proven_arg_f64(d.val));
EXAMPLE_REQUIRE(proven_is_ok(out.err), "formatting the parsed value must succeed");
EXAMPLE_REQUIRE(proven_u8str_view_starts_with(proven_u8str_as_view(&line.value),
    PROVEN_LIT("measured 3.14159")),
    "and spell the number it was given");

/* 정책을 주는 꼴이 명시적인 쪽이다. 기본값이 아니라 특정한 표기가 필요할 때 쓴다.
 * SHORTEST 는 다시 읽었을 때 정확히 이 값이 되는 가장 적은 자릿수를 청한다 -
 * 직렬화기가 원하는 성질이 그것이다. 필요 없는 수까지 열일곱 자리로 찍지 않고도
 * 왕복을 정확하게 만들어 주기 때문이다. */
char shortest[64];
proven_size_t wrote = 0;
float measured = 0.1f;
proven_err_t ferr = proven_float_format_f32_policy(shortest, sizeof shortest, measured,
    PROVEN_FLOAT_FORMAT_POLICY_RYU,
    proven_float_format_options_shortest(),
    &wrote);
EXAMPLE_REQUIRE(proven_is_ok(ferr), "formatting a float in shortest mode must succeed");
EXAMPLE_REQUIRE(wrote > 0 && shortest[wrote] == '\0', "the result is written and terminated");
EXAMPLE_REQUIRE(strcmp(shortest, "0.1") == 0,
    "0.1f prints as 0.1: the shortest text that reads back as the same float");

/* 그리고 왕복한다. 그 글은 자기가 나온 값으로 되읽힌다. 가장 짧은 방식이 존재하는
 * 이유가 그 보장이다. */
proven_parse_double_result_t roundtrip = proven_parse_double_ascii(
    (proven_u8str_view_t){ .ptr = (const proven_byte_t *)shortest, .size = wrote });
EXAMPLE_REQUIRE(proven_is_ok(roundtrip.err), "the shortest form parses back");
EXAMPLE_REQUIRE((float)roundtrip.val == measured, "as exactly the float it was printed from");

```

```

printf("scanned id=%lld bytes=%llu; formatted %s and %s\n",
      (long long)id, (unsigned long long)bytes, proven_u8str_as_cstr(&line.value), shortest);

proven_u8str_destroy(alloc, &line.value);
return EXAMPLE_OK();
}

```

반례 — 큰 값을 32비트 목적지로 읽는 경우:

```

proven_i32 bytes = 0;
proven_err_t e = proven_scan_fmt_cursor(&scan, "bytes={}", proven_scan_arg_i32(&bytes));
/* input says 5368709120 */ /* wrong type */

```

값이 들어가지 않는다. 필드가 담을 수 있는 범위가 아니라 손에 익은 것으로 목적지 타입을 고르는 일은, 파 일 오프셋을 int로 선언하는 것과 같은 실수다.

반례 — 기계를 건너다니는 데이터에 atof나 strtod를 쓰는 경우:

```

double v = strtod(text, NULL); /* wrong: the decimal point depends on the locale */

```

게다가 atof는 실패를 보고할 방법이 아예 없다. 해석할 수 없는 텍스트가 0이 되고, 그것은 데이터 안의 진짜 0과 구분되지 않는다.

반례 — skip_until_number가 "없음"을 알려 준다고 가정하는 경우:

```

proven_scan_skip_until_number(&scan);
proven_i32 n = 0;
proven_err_t e = proven_scan_fmt_cursor(&scan, "{}", proven_scan_arg_i32(&n)); /* may fail */

```

이 함수는 커서를 옮길 뿐 아무것도 돌려주지 않는다. 남은 숫자가 없으면 커서는 입력의 끝에 놓이고, 그 사실을 알려 주는 것은 뒤따르는 스캔이다. 숫자를 찾았다고 가정하는 대신 그 에러를 확인한다.

실전 예제: 인자 타입을 손으로 이름 붙이기

PROVEN_ARG(x)와 PROVEN_SCAN_ARG(&x)는 넘겨받은 것의 타입에서 인자 생성자를 골라 주며, 대부분의 코드는 무엇이 골라졌는지 알 필요가 없다. 그 선택을 도로 가져와야 하는 상황이 둘 있다.

1. **매크로가 판단할 타입이 내 의도를 담고 있지 않을 때.** proven_u8str_view_t, 분해된 날짜, 진단용으로 찍는 주소 — proven_arg_str_view(), proven_arg_datetime(), proven_arg_ptr()가 있는 이유는 이들이 얼떨결에 빠지는 기본값이 아니라 의도적인 선택이기 때문이다.
2. **인자 목록을 실행 중에 만들 때.** 매개변수가 이미 proven_arg_t인 로그 헬퍼는 그것을 다시 감쌀 수 없다. 값을 인자로 바꾸는 매크로는 이미 인자인 것과는 상관이 없다. proven_arg_identity()와 proven_scan_arg_identity()가 바로 그 자리에서, 매크로로 굴러가는 같은 코드 경로가 미리 만들어진 인자를 받아들이게 해 준다.

예제는 양쪽을 이름으로 적는다. 형식화 쪽은 문자 하나, 불(bool) 값, 부호 없는 32·64비트 값, 텍스트를 넘기는 세 가지 방법(proven_arg_str_view(), proven_arg_cstr(), proven_arg_ucstr()), 날짜와 포인터. 스캔 쪽은 C의 모든 기본 정수 폭(short, int, long, long long과 그 부호 없는 형태), double, 그리고 복사 없이 붙잡는 문자열 view.

텍스트 생성자 셋은 호출자를 얼마나 믿는가에서 갈리며, 그 순서를 기억해 둘 값어치가 있다.

생성자	무엇을 받는가	무엇이 잘못될 수 있는가
proven_arg_str_view	포인터 와 길이	종결자를 찾아 훑는 일이 없으니 빠질 종결자도 없다 — 기본으로 이것을 쓴다
proven_arg_cstr	NUL로 끝나는 C 문자열	종결자가 있어야 하고 메모리가 살아 있어야 한다. 아니면 형식화기가 끝을 넘어 읽는다
proven_arg_ucstr	같은 것을 unsigned char *로	바이트 버퍼에 진짜 경고를 지우는 캐스트를 붙이지 않아도 되도록 존재한다

```

/*
 * 형식화기에 건네는 모든 값은 proven_arg_t 로 도착하고, 스캐너가 써 넣는 모든 목적지는
 * proven_scan_arg_t 로 도착한다. 대개는 둘 다 볼 일이 없다. PROVEN_ARG(x) 와
 * PROVEN_SCAN_ARG(&x) 가 건넨 것의 타입에서 알맞은 생성자를 골라 주고, 그것이 그
 * 매크로들의 존재 이유다.
 *
 * 두 상황에서는 그 선택을 매크로에게서 도로 가져오게 되는데, 둘 다 평범한 일이다.
 *
 * 1. 매크로가 알아볼 타입이 없을 때. `proven_u8str_view_t`, 쪼개 놓은 낱짜, 진단용으로
 * 찍는 낱주소 - 이런 것들은 생성자를 이름으로 불러야 한다. 여러분이 뜻하는 바를
 * 뜻하는, 매크로가 갈래를 태울 수 있는 평범한 C 타입이 없기 때문이다.
 *
 * 2. 인자 목록을 *실행 중에* 짓고 있을 때. "부르는 쪽이 이미 모아 둔 무엇이든" 을 받는
 * 로그 도우미는 int 나 문자열이 아니라 proven_arg_t 값을 받는다 - 그리고 값을
 * 인자로 바꾸는 매크로에 이미 인자인 것을 건넬 수는 없다. 항등 생성자가 그것을 위한
 * 것이다. 매크로로 굴러가는 같은 코드 길이, 앞서 만들어진 인자도 받아들이게 해 준다.
 *
 * 그래서 이 예제는 양쪽을 다 이름으로 적는다. 목적지의 폭과 부호 유무는 장식이 아니다 -
 * 70000 이 70000 으로 도착할지 4464 로 도착할지를 정하는 것이 그것이다.
 */

/* 부르는 쪽이 이미 만들어 둔 인자를 받는 로그 도우미. 매개변수가 proven_arg_t 이므로 그
 * 안에서 PROVEN_ARG 는 틀린 도구다 - 그 값은 이미 인자이고, 그렇다고 말해 주는 것이
 * proven_arg_identity 다. */
static proven_fmt_result_t log_pair(proven_u8str_t *out, const char *fmt,
                                    proven_arg_t a, proven_arg_t b) {
    return proven_u8str_append_fmt(out, fmt, proven_arg_identity(a), proven_arg_identity(b));
}

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    proven_result_u8str_t line = proven_u8str_create(alloc, 256);
    EXAMPLE_REQUIRE(proven_is_ok(line.err), "creating the output string must succeed");
    if (!proven_is_ok(line.err)) {
        return 1;
    }
    proven_u8str_t out = line.value;

    /* --- 형식화 인자를 직접 이름으로 부르기 ----- */

    /* 문자 하나와 플래그 하나. bool 이 1/0 이 아니라 true/false 로 찍힌다는 것이 눈에
     * 보이도록 여기서는 이름으로 적었다. */
    proven_fmt_result_t r = proven_u8str_append_fmt(&out, "flag={} mark={}",
                                                    proven_arg_bool(true),
                                                    proven_arg_char('!'));
    EXAMPLE_REQUIRE(proven_is_ok(r.err), "formatting a bool and a char must succeed");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&out), PROVEN_LIT("flag=true mark=!")),
                    "a bool renders as a word, not as a digit");

    EXAMPLE_REQUIRE(proven_is_ok(proven_u8str_reset(&out)), "clearing the line must succeed");

    /* 부호 없는 폭들. 생성자가 그 값이 무엇인지에 대한 선언이다. 부호 없는 32비트 개수와
     * 부호 없는 64비트 바이트 합계는 서로 다른 사실이고, 그것을 적어 두면 자기가 가진
     * 것이 어느 쪽인지 말하게 된다. */
    r = proven_u8str_append_fmt(&out, "files={} bytes={}",
                                proven_arg_u32(1200u),
                                proven_arg_u64(5368709120ULL));
    EXAMPLE_REQUIRE(proven_is_ok(r.err), "formatting unsigned values must succeed");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&out), PROVEN_LIT("files=1200

```

```

bytes=5368709120")),
    "a 64-bit total is printed in full, not truncated to 32 bits");

EXAMPLE_REQUIRE(proven_is_ok(proven_u8str_reset(&out)), "clearing the line must succeed");

/* 형식화기에 글을 주는 세 가지 방법. 부르는 쪽을 얼마나 믿는지 순서대로.
 *
 * proven_arg_str_view - 포인터 *와* 길이. 종결자를 찾아 훑는 일이 없으므로,
 *                      없어서 문제가 될 종결자도 없다. 이것을 고를 것.
 * proven_arg_cstr     - NUL 로 끝나는 C 문자열. 형식화기가 종결자를 찾아 걸으므로
 *                      그것이 반드시 있어야 하고, 그 기억도 살아 있어야 한다.
 * proven_arg_ucstr    - 같은 것인데 `unsigned char *` 용이다. 바이트 버퍼는 보통
 *                      그 타입으로 적힌다. 부르는 쪽이 진짜 경고를 입막음하는
 *                      형변환을 쓰지 않아도 되도록 있는 것이다.
 */
const char *name = "report.txt";
const unsigned char *tag = (const unsigned char *)"draft";
proven_u8str_view_t note = PROVEN_LIT("first pass");

r = proven_u8str_append_fmt(&out, "{} [{}] {}",
    proven_arg_cstr(name),
    proven_arg_ucstr(tag),
    proven_arg_str_view(note));
EXAMPLE_REQUIRE(proven_is_ok(r.err), "formatting the three text forms must succeed");
EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&out),
    PROVEN_LIT("report.txt [draft] first pass")),
    "all three produce the same kind of output from different kinds of pointer");

EXAMPLE_REQUIRE(proven_is_ok(proven_u8str_reset(&out)), "clearing the line must succeed");

/* 날짜 하나와 주소 하나. 둘 다 자기 뜻을 뜻하는 평범한 C 타입이 없다. 날짜는
 * 구조체이고, 진단용으로 찍는 주소는 우연히 빠지는 것이 아니라 일부러 하는 일이다. */
proven_datetime_t when = proven_time_breakdown(0); /* 기점: 고정된, 확인할 수 있는 값 */
int local = 0;

r = proven_u8str_append_fmt(&out, "at {} object {}",
    proven_arg_datetime(when),
    proven_arg_ptr(&local));
EXAMPLE_REQUIRE(proven_is_ok(r.err), "formatting a date and a pointer must succeed");
EXAMPLE_REQUIRE(proven_u8str_view_starts_with(proven_u8str_as_view(&out), PROVEN_LIT("at 1970-01-01")),
    "the epoch breaks down to the first of January 1970");
EXAMPLE_REQUIRE(proven_u8str_view_find(proven_u8str_as_view(&out), 0, PROVEN_LIT("0x")) !=
PROVEN_SIZE_MAX,
    "and an address is rendered in hexadecimal");

EXAMPLE_REQUIRE(proven_is_ok(proven_u8str_reset(&out)), "clearing the line must succeed");

/* 여기서 지은 인자를, 넘겨, 저기서 형식화한다. */
r = log_pair(&out, "status={} retries={}", proven_arg_cstr("ok"), proven_arg_u32(3u));
EXAMPLE_REQUIRE(proven_is_ok(r.err), "formatting pre-built arguments must succeed");
EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&out), PROVEN_LIT("status=ok retries=3")),
    "an argument built by the caller formats exactly as one built in place");

/* --- 파싱 목적지를 직접 이름으로 부르기 ----- */

/* 들어오는 쪽에서는 생성자가 목적지를 이름으로 밝히고, 목적지가 "너무 크다" 의 뜻을
 * 정한다. 이것들은 평범한 C 타입들 - short, int, long, long long 과 그 부호 없는 꼴 -
 * 이고, 이미 가진 변수가 고정 폭 타입이 아니라 그중 하나인 아주 흔한 경우를 위한
 * 것이다. */
proven_scan_t scan = proven_scan_init(

```

```

    PROVEN_LIT("h=-32000 uh=65000 i=-2000000 ui=4000000000 l=-9000000 ul=9000000 "
               "ll=-9007199254740993 ull=18446744073709551615 f=2.5 word=alpha"));

short h = 0;
unsigned short uh = 0;
int i = 0;
unsigned int ui = 0;
long l = 0;
unsigned long ul = 0;
long long ll = 0;
unsigned long long ull = 0;
double f = 0.0;
proven_u8str_view_t word = {0};

proven_err_t err = proven_scan_fmt_cursor(
    &scan, "h={ } uh={ } i={ } ui={ } l={ } ul={ } ll={ } ull={ } f={ } word={ }",
    proven_scan_arg_short(&h),
    proven_scan_arg_ushort(&uh),
    proven_scan_arg_int(&i),
    proven_scan_arg_uint(&ui),
    proven_scan_arg_long(&l),
    proven_scan_arg_ulong(&ul),
    proven_scan_arg_llong(&ll),
    proven_scan_arg_ullong(&ull),
    proven_scan_arg_f64(&f),
    proven_scan_arg_str_view(&word));
EXAMPLE_REQUIRE(proven_is_ok(err), "scanning every plain integer width must succeed");
EXAMPLE_REQUIRE(h == -32000 && uh == 65000u, "the short forms hold their values");
EXAMPLE_REQUIRE(i == -2000000 && ui == 4000000000u, "and so do the int forms");
EXAMPLE_REQUIRE(l == -9000000L && ul == 9000000UL, "and the long forms");
EXAMPLE_REQUIRE(ll == -9007199254740993LL, "a value needing 64 bits arrives whole");
EXAMPLE_REQUIRE(ull == 18446744073709551615ULL, "including the largest unsigned 64-bit value");
EXAMPLE_REQUIRE(f > 2.4999 && f < 2.5001, "the floating-point destination reads a decimal");

/* 파싱된 문자열 뷰는 파싱 중인 글 *안*을 가리킨다. 복사한 것도 할당한 것도 없으므로,
 * 그 글이 사는 동안 정확히 그만큼만 쓸 수 있다 - 파싱보다 오래 살아야 한다면
 * proven_u8str_t 로 복사할 것. */
EXAMPLE_REQUIRE(proven_u8str_view_eq(word, PROVEN_LIT("alpha")), "the word is captured as a view");

/* 목적지의 폭은 스캐너가 지키는 약속이다. 70000 은 short 에 들어가지 않으므로 4464 로
 * 감기는 대신 거부된다 - 형변환이었다면 아무도 다시 보지 않을 파일 안에서 조용히
 * 그렇게 만들어 냈을 값이다. */
proven_scan_t narrow = proven_scan_init(PROVEN_LIT("70000"));
short too_small = 7;
err = proven_scan_fmt_cursor(&narrow, "{ }", proven_scan_arg_short(&too_small));
EXAMPLE_REQUIRE(err != PROVEN_OK, "a value that does not fit the destination is refused");
EXAMPLE_REQUIRE(too_small == 7, "and the destination keeps the value it had");

/* 파싱 쪽 항등 생성자. 이유는 형식화 쪽과 같다. 이미 만들어진 파싱 인자를 받는
 * 도우미는 그것을 다시 감쌀 수 없다. */
proven_scan_t again = proven_scan_init(PROVEN_LIT("41"));
proven_i32 answer = 0;
proven_scan_arg_t prebuilt = proven_scan_arg_i32(&answer);
err = proven_scan_fmt_cursor(&again, "{ }", proven_scan_arg_identity(prebuilt));
EXAMPLE_REQUIRE(proven_is_ok(err) && answer == 41, "a pre-built scan argument works unchanged");

printf("arguments: %s\n", proven_u8str_as_cstr(&out));

proven_u8str_destroy(alloc, &out);

```

```

    return EXAMPLE_OK();
}

```

반례 — 이미 인자인 것을 다시 감싸는 경우:

```

static proven_fmt_result_t log_one(proven_u8str_t *out, proven_arg_t a) {
    return proven_u8str_append_fmt(out, "{}", PROVEN_ARG(a));    /* wrong */
}

```

proven_arg_identity(a)를 쓴다. PROVEN_ARG는 값을 위한 것이고, a는 이미 인자다.

반례 — 텍스트가 사라진 뒤에도 스캔한 view를 들고 있는 경우:

```

proven_u8str_view_t word = {0};
{
    proven_u8str_t line = read_a_line(alloc);
    proven_scan_t s = proven_scan_init(proven_u8str_as_view(&line));
    proven_err_t e = proven_scan_fmt_cursor(&s, "word={}", proven_scan_arg_str_view(&word));
    proven_u8str_destroy(alloc, &line);    /* wrong: `word` pointed into `line` */
}
use(word);

```

스캔한 view는 스캔 대상 텍스트 안쪽을 가리킨다. 그 텍스트보다 오래 살아야 한다면 proven_u8str_t로 복사한다.

Chapter 5: Hosted Services

5부 — 운영체제와 대화하기. 선행 조건: 2부 (1, 2, 3). 이 장을 마치면 충돌 시에도 데이터를 잃지 않고 파일을 읽고 쓰며, 입력을 한 줄씩 읽고, 작업에 맞는 무작위성을 생성하고, 벽시계 시간과 경과 시간을 구별할 수 있다.

이 장에서는 `fs.h`, `sysio.h`, `mmap.h`, `time.h`, `stream.h`, `random.h`를 다룬다. 이 안의 모든 것은 운영체제를 필요로 한다: 이 장은 [프리스탠딩\(freestanding\)](#) 빌드에 적용되지 않는 매뉴얼의 유일한 부분이다.

이 API들은 호스티드 플랫폼 지원이 필요하며, 현재의 freestanding 하위 집합에서는 제외된다.

목차

1. 파일시스템 API
2. 시스템 I/O와 환경
3. 메모리 매핑
4. Time API
5. 예제와 오용 사례
6. 트리 순회
7. 스트림: writer와 reader
8. 표준 스트림
9. 용도별 무작위성

1. 파일시스템 API

파일시스템 계층은 플랫폼 파일 핸들, 경로, 디렉터리 목록, 메타데이터, 권한, 링크, 잠금을 감싼다.

원시 파일시스템 헬퍼는 신뢰할 수 없는 경로를 정제하지 않고, 루트 감금(root confinement)을 강제하지 않으며, symlink-race TOCTOU를 방어하지 않는다. 신뢰할 수 없는 경로를 받아들이는 호출자는 API를 사용하기 전에 그것을 검증해야 한다.

구조체와 열거형

```
typedef struct {
    union {
        void *ptr;
        int fd;
    } internal;
} proven_fs_handle_t;
```

```
typedef proven_fs_handle_t proven_file_t;
```

의도: POSIX나 Win32 세부사항을 상위 계층에 노출하지 않으면서 플랫폼 파일 핸들을 표현한다.

```
typedef enum {
    PROVEN_FS_READ    = 1 << 0,
    PROVEN_FS_WRITE   = 1 << 1,
    PROVEN_FS_APPEND  = 1 << 2,
    PROVEN_FS_CREATE  = 1 << 3,
    PROVEN_FS_TRUNC   = 1 << 4,
    PROVEN_FS_CREATE_NEW = 1 << 5
} proven_fs_mode_t;
```

파일 모드 플래그는 조합할 수 있다. 예:

- PROVEN_FS_READ
- PROVEN_FS_WRITE | PROVEN_FS_CREATE | PROVEN_FS_TRUNC
- PROVEN_FS_WRITE | PROVEN_FS_APPEND | PROVEN_FS_CREATE

```
typedef enum {
    PROVEN_FS_TYPE_FILE,
    PROVEN_FS_TYPE_DIR,
```

```

    PROVEN_FS_TYPE_OTHER
} proven_fs_type_t;

```

```

typedef struct {
    proven_u8str_t name;
    proven_fs_type_t type;
    proven_size_t size;
} proven_fs_entry_t;

```

디렉터리 엔트리는 name을 소유(owned)로 소유한다. proven_fs_list()가 반환한 배열에는 proven_fs_list_destroy()를 사용하라.

```

typedef struct {
    proven_err_t err;
    proven_file_t value;
} proven_result_file_t;

```

```

typedef enum {
    PROVEN_FS_PERM_OWNER_R = 1 << 8,
    PROVEN_FS_PERM_OWNER_W = 1 << 7,
    PROVEN_FS_PERM_OWNER_X = 1 << 6,
    PROVEN_FS_PERM_GROUP_R = 1 << 5,
    PROVEN_FS_PERM_GROUP_W = 1 << 4,
    PROVEN_FS_PERM_GROUP_X = 1 << 3,
    PROVEN_FS_PERM_OTHER_R = 1 << 2,
    PROVEN_FS_PERM_OTHER_W = 1 << 1,
    PROVEN_FS_PERM_OTHER_X = 1 << 0,
    PROVEN_FS_PERM_DEFAULT = PROVEN_FS_PERM_OWNER_R | PROVEN_FS_PERM_OWNER_W |
                             PROVEN_FS_PERM_GROUP_R | PROVEN_FS_PERM_OTHER_R
} proven_fs_perms_t;

```

```

typedef enum {
    PROVEN_FS_LOCK_SHARED,
    PROVEN_FS_LOCK_EXCLUSIVE,
    PROVEN_FS_LOCK_UNLOCK
} proven_fs_lock_type_t;

```

```

typedef struct {
    proven_size_t size;           /* size in bytes (0 for directories on some hosts) */
    proven_fs_type_t type;        /* PROVEN_FS_TYPE_FILE / _DIR - there is no symlink type */
    proven_fs_perms_t perms;      /* the nine permission bits only; the file type is in `type` */
    proven_i64 created_at;        /* always 0: no birth time is queried - see below */
    proven_i64 modified_at;       /* last-modification time, seconds since the Unix epoch */
    unsigned long long dev;       /* device id (POSIX st_dev; 0 where unavailable) */
    unsigned long long ino;       /* inode number (POSIX st_ino; 0 where unavailable) */
    unsigned long long uid;       /* owner id (POSIX st_uid; 0 on Windows - no uid/gid) */
    unsigned long long gid;       /* group id (POSIX st_gid; 0 on Windows) */
} proven_fs_stat_t;

```

uid/gid(v26.06.22a에서 추가됨)는 POSIX 소유자 및 그룹 id를 담는다. 둘 다 Windows에서는 0인데, Windows에는 uid/gid 개념이 없기 때문이다. ls -l 스타일의 소유자/그룹 열을 표시할 때는 호스트의 getpwuid/getgrgid로 이들을 이름으로 해석하라.

한 필드는 보이는 것보다 좁고, 다른 한 필드에는 날카로운 모서리가 있다:

- created_at은 **항상 0**이다. 평범한 stat()에는 이식 가능한 birth time이 없으며, PAL은 그것을 요청하지 않는다. modified_at만이 실제 타임스탬프를 담는다.
- type은 일반 파일이면 FILE, 디렉터리면 DIR, 그 밖의 모든 것—FIFO, 소켓, 디바이스, 끊어진(dangling) symlink—에 대해서는 **PROVEN_FS_TYPE_OTHER**이다. (v26.07.13a 이전까지는 이 모든 것이 FILE로 stat되어, 호출자에게 그것들을 열어 바이트를 읽을 수 있다고 알려주었다. 끊어진 symlink는 애초에 열 수 없다.)

type은 여기서도, 그리고 디렉터리 순회에서 **symlink를 따라간다**. 그것이 두 값을 일치시키는 원리다: 일반 파일을 가리키는 symlink는 FILE이고, 디렉터를 가리키는 symlink는 DIR이다. 이어지는 모서리는 실재

한다—재귀 순회기는 루프에 빠질 수 있다. 조상을 가리키는 symlink는 사이클이고 그 type이 DIR이라고 말하기 때문이다. 깊이 제한을 두거나, (dev, ino) 쌍을 기억해 두고 이미 본 것으로는 내려가기를 거부하라.

```
proven_fs_stat_t st;
if (proven_is_ok(proven_fs_stat(scratch, PROVEN_LIT("/etc/hosts"), &st))) {
    proven_println("size={} uid={} gid={}",
        PROVEN_ARG(st.size), PROVEN_ARG(st.uid), PROVEN_ARG(st.gid));
}
```

매크로

매크로	의도
PROVEN_FS_PATH_SEP	라이브러리 수준에서 선호되는 구분 문자, 현재는 '/'.

파일 함수

API	의도	반환
proven_fs_open(scratch, path, mode)	파일 경로를 연다. scratch는 경로 변환에 사용된다.	proven_result_file_t.
proven_fs_close(file)	파일 핸들을 닫는다.	에러를 반환한다. 쓰기를 한 파일에서는 close *자체*가 쓰기의 일부다: NFS, CIFS 및 쿼터를 강제하는 파일시스템은 write-back 실패를 오직 이곳에서만 보고한다. 읽기만 한 파일에서는 (void)로 무시해도 괜찮다.
proven_fs_read(file, dest)	가변 슬라이스로의 단일 읽기 시도.	proven_result_size_t.
proven_fs_write(file, src)	바이트 view로부터의 단일 쓰기 시도.	proven_result_size_t.
proven_fs_write_all(file, src)	모든 바이트가 쓰이거나 에러가 발생할 때까지 재시도.	proven_err_t.
proven_fs_size(file)	열린 파일의 크기를 조회.	proven_result_size_t.
proven_fs_rename(scratch, src, dest)	경로 이름 변경 또는 이동.	proven_err_t.
proven_fs_remove(scratch, path)	파일 제거.	proven_err_t.
proven_fs_copy(temp_alloc, src, dest)	임시 버퍼 할당을 사용해 파일 복사.	proven_err_t.
proven_fs_mkdir(scratch, path)	디렉터리 생성.	proven_err_t.
proven_fs_rmdir(scratch, path)	빈 디렉터리 제거.	proven_err_t.
proven_fs_list(alloc, path)	디렉터리를 proven_fs_entry_t의 proven_array_t로 나열.	proven_result_array_t.
proven_fs_list_destroy(alloc, list)	디렉터리 목록과 엔트리 이름을 파괴.	void.
proven_fs_chmod(scratch, path, perms)	권한 설정.	proven_err_t.
proven_fs_lock(file, type, wait)	파일 잠금 획득/해제.	proven_err_t.
proven_fs_stat(scratch, path, out_stat)	메타데이터 채우기.	proven_err_t.

proven_fs_stat()은 perms에 아홉 개의 권한 비트만 보고하므로, stat의 perms를 그대로 proven_fs_chmod()에 다시 넘길 수 있다. 예전에는 원시 POSIX st_mode를 담았는데, chmod가 거부하는 파일 타입 비트가 포함되어 있어서 —이 필드의 자명한 용도인 그 왕복(round-trip)이 실제 파일마다 PROVEN_ERR_INVALID_ARG로 실패했다. 파일 타입은 type에서 읽어라.

위치, 그리고 atomic과 durable의 차이

seek할 수 없는 핸들은 그렇다고 말한다. 파이프, FIFO, 터미널은 proven_fs_seek에서 PROVEN_ERR_IO가 아니라 PROVEN_ERR_UNSUPPORTED를 반환한다. seek 불가능은 호출의 실패가 아니라 그 대상의 속성이며, 그것에 적응하는 코드—스캐너가 그렇다—는 둘을 구별할 수 있어야 한다.

pread와 pwrite는 위치를 이동하지 않는다. 그것이 바로 그들의 목적이다: 하나의 핸들을 공유하는 두 읽기 스트림(reader)는 어느 쪽도 이동시키지 않는 커서를 두고 경합할 수 없다.

atomic과 durable은 서로 다른 약속이며, 이 둘을 혼동하는 것이 데이터가 손실되는 방식이다.

- proven_fs_write_file_atomic은 *reader*가 절반만 쓰인 파일을 결코 보지 않도록 보장한다. 전원 차단에 대해서는 아무것도 말하지 않는다: 커널이 여전히 당신의 바이트를 붙들고 있을 수 있고, rename이 그것이 가리키는 데이터보다 먼저 디스크에 도달할 수 있다.
- proven_fs_write_file_durable은 그 창(window)을 닫는다. 오직 작동하는 하나의 순서로: 임시 파일을 fsync하고, **그 다음** rename하고, **그 다음** 디렉터리를 fsync한다. 파일은 sync하되 디렉터리는 하지 않으면, 바이트는 안전하지만 그것을 가리키는 이름은 안전하지 않은 크래시 창이 남는다—그것이 바로 atomic 쓰기가 막고자 존재하는 그 손상이다.

durable 형태는 저장 장치를 두 번 기다린다. 쓰기를 잃는 것이 기다림보다 나쁠 때는 이것을, 그렇지 않을 때는 atomic 형태를 사용하라.

예전에는 여기에 proven_sysio_flush가 있었는데, 그것은 이 어느 것도 아니었다: 존재하지 않는 버퍼를 flush한다고 주장했다. 그것은 **삭제되었다**. 버퍼드 쓰기 스트림(writer)의 바이트를 OS로 밀어 넣는 것은 proven_writer_flush이고, OS의 바이트를 디스크로 밀어 넣는 것은 proven_fs_sync이다. 이 둘은 서로 다른 연산이며, 한 단어가 정직하게 둘 모두를 뜻할 수는 없었다.

중요한 동작:

- proven_fs_read()와 proven_fs_write()는 단일 연산 API이며 요청된 것보다 적은 바이트를 처리할 수 있다.
- **파일 끝에서의 읽기는 0바이트 성공이 아니라 PROVEN_ERR_EOF를 반환한다.** 자명한 방식으로 작성된 루프—`if (r.value == 0) break;`—는 결코 그 분기를 타지 않으며, 파일의 끝을 I/O 실패로 취급한다. PROVEN_ERR_EOF를 명시적으로 검사하라. 이 장 끝의 완성 예제가 그 형태를 보여준다.
- 모든 바이트를 반드시 써야 할 때는 proven_fs_write_all()을 사용하라.
- 크기 0의 읽기/쓰기 요청은 non-null 버퍼를 요구하지 않고 0바이트를 처리한 채로 성공해야 한다.
- proven_fs_is_absolute()는 POSIX 절대 경로, Windows 드라이브-루트 경로, UNC 경로, 확장된 Windows 경로 형식을 인식한다.
- proven_fs_read_all()은 EOF까지 읽는다. 미리 측정된 크기까지 읽는 것이 아니다. 파일이 보고한 크기는 초기 용량의 씨앗(seed)으로만 쓰이므로, 일반 파일은 여전히 한 번의 할당과 한 번의 패스로 읽힌다. 이것이 중요한 이유는 proven_fs_size()가 일반 파일이 아닌 모든 것에 대해 0을 보고하기 때문이다: FIFO, 캐릭터 디바이스, /proc 엔트리에는 미리 알 수 있는 크기가 없으며, EOF까지 읽는 것만이 그 내용을 얻는 유일한 방법이다. 또한 이는 읽히는 도중 커지는 파일이 조용히 절단되지 않음을 뜻한다. value.size는 언제나 실제 바이트 수이며, 빈 소스는 PROVEN_OK와 함께 { .ptr = NULL, .size = 0 }을 넣는다.
- proven_fs_read_all()과 proven_fs_read_all_u8str()은 소스가 보고된 크기를 초과해 커질 경우 realloc_fn을 가진 할당자(allocator)를 필요로 한다. 그렇지 않은(non-growing) allocator는 그 경우 PROVEN_ERR_UNSUPPORTED를 반환한다.
- proven_fs_read_all_u8str()은 대부분의 호출자가 원하는 파일 전체 읽기다: 결과가 NUL로 종료되므로 proven_u8str_as_view()와 proven_u8str_as_cstr()가 두 번째 복사 없이 그 위에서 동작한다. 종료 슬롯은 미리 예약되므로 추가 할당 비용이 들지 않는다. 내용은 UTF-8로 검증되지 않는다. proven_u8str_destroy()로 해제하라.

- `proven_fs_write_file()`은 `atomic`이 아니다: `reader`가 부분적으로 쓰인 파일을 관찰할 수 있고, 쓰기 도중의 실패는 파일을 절단된 채로 남긴다. `proven_fs_write_file_atomic()`은 형제 임시 파일을 쓰고 그것을 대상 위로 `rename`하므로, 동시 `reader`는 전체 이전 파일 또는 전체 새 파일 둘 중 하나를 본다. 이는 `reader`에 대해 `atomic`이지만 전원 손실에 대해 `durable`하지는 않다: `rename`이 데이터보다 먼저 디스크에 도달할 수 있다. `durability`가 필요할 때는 위에서 설명한 `proven_fs_write_file_durable`(또는 `proven_fs_sync`)로 명시적으로 요청하라.

읽기 전용 대상은 넷 다 거절한다. `proven_fs_write_file`, `proven_fs_write_file_atomic`, `proven_fs_write_file_durable`, `proven_fs_copy`는 대상의 소유자 쓰기 비트가 꺼져 있으면 `PROVEN_ERR_PERMISSION`을 돌려주고 파일을 그대로 둔다. 그 비트가 두 플랫폼이 "이 파일에 쓰지 말라"를 기록하는 유일한 자리다 — POSIX에서는 모드 `0200`, Windows에서는 `READONLY` 속성이 같은 비트로 보고된다.

하나의 규칙인 이유는, 한 플랫폼 안에서 같은 질문에 세 가지 답이 나왔기 때문이다. `write_file`은 거절했다. 대상을 쓰기로 열기 때문이다. `write_file_atomic`은 성공했다. `rename`은 파일이 아니라 *디렉터리*에 권한을 묻기 때문에 파일의 모드를 아예 보지 않는다. `copy`는 성공했고 **그 뒤 파일을 쓰기 가능한 상태로 남겼다**. 원본의 모드를 옮겨 오기 때문이다. 호출자가 걸어 둔 보호가 사라졌는데 아무것도 그 사실을 말하지 않았다. 어느 함수를 골랐는지는 권한에 대한 결정이 아니다.

보호된 파일을 정말로 바꾸려는 호출자는 표시를 먼저 본다 — 한 줄이고, 눈에 보인다. 거절은 되돌릴 수 있는 쪽이고, 실수로 덮어쓴 것은 되돌릴 수 없다. 보안 경계가 아니라 사고 방지 장치다: 모드는 작업 전에 읽고 작업 후에 반영되며, `chmod`를 할 수 있는 사람은 표시를 풀 수 있다.

`proven_fs_rename`도 그 목록에 있고, 있어야 한다. 대상을 대체하는 함수이고, `atomic` 쓰기가 바로 그 위에 지어져 있다 — 이것이 빠지면 한 함수에서 거절당한 호출자가 다른 함수로 같은 결과를 얻는다.

`proven_fs_remove`는 일부터 **빠져 있다**: 이름을 지우는 것은 디렉터리 연산이고, POSIX는 파일 자신의 모드에 발언권을 준 적이 없다. 이 지점에서 두 플랫폼은 실제로 다르다 — Windows는 읽기 전용 파일을 지우지 않는다 — 그래서 `proven_fs_remove`는 그 차이를 입출력 오류 뒤에 숨기지 않고 `PROVEN_ERR_PERMISSION`으로 보고한다.

거절은 이제 어떤 거절인지 말한다. `proven_fs_open`, `proven_fs_rename`, `proven_fs_remove`는 예전에 전부 `PROVEN_ERR_IO`로 답하던 자리에서 `PROVEN_ERR_NOT_FOUND`, `PROVEN_ERR_PERMISSION`, `PROVEN_ERR_BUSY`를 답한다. 사용자에게 묻기, 다시 시도하기, 포기하기는 서로 다른 세 가지 답이고, 오류 코드 하나로는 그중 어느 것도 할 수 없다.

누가 읽고 있는 파일도 바꿔 넣는다. 다른 프로세스가 대상을 열어 두고 있어도 그 프로세스가 삭제 공유(delete sharing)를 허용했다면 — `proven_fs_open`이 그렇게 연다 — `atomic` 쓰기는 성공하고, 읽던 쪽은 열어 둔 핸들로 옛 내용을 끝까지 읽는다. POSIX가 원래 그렇고, Windows 10 1809 이후에서는 POSIX 방식 `rename`으로 같은 동작을 한다. 그 `rename`을 모르는 곳 — 1809 이전 Windows, FAT32/exFAT, 네트워크 드라이브 — 에서는 `MoveFileExW`로 물려서며, 거기서는 누가 열어 두기만 해도 교체가 거절되고 `PROVEN_ERR_BUSY`가 온다. 삭제 공유를 허용하지 않고 연 파일은 어느 Windows에서든 `PROVEN_ERR_BUSY`다. 보호가 아니라 사용 중이므로, 다시 시도하면 될 수 있다.

예:

```
proven_result_file_t of = proven_fs_open(
    alloc,
    PROVEN_LIT("out.txt"),
    PROVEN_FS_WRITE | PROVEN_FS_CREATE | PROVEN_FS_TRUNC
);
if (proven_is_ok(of.err)) {
    proven_err_t e = proven_fs_write_all(
        of.value,
        proven_mem_view_from_u8(PROVEN_LIT("hello\n"))
    );
    /* The close is part of the write. Close on the failure path too - the handle is
     * ours either way - but do not throw the answer away: on a network filesystem or
     * over quota, close() is the only place the failure appears. */
}
```

```

    proven_err_t ce = proven_fs_close(of.value);
    if (proven_is_ok(e)) e = ce;
    if (!proven_is_ok(e)) {
        proven_eprintln("writing out.txt failed");
    }
}

```

2. 시스템 입출력과 환경변수

표준 스트림

```

proven_file_t proven_sysio_stdin(void);
proven_file_t proven_sysio_stdout(void);
proven_file_t proven_sysio_stderr(void);

```

목적: 표준 스트림을 proven_file_t 핸들로 노출한다. 이들은 또한 writer와 reader이기도 하다— 아래 [표준 스트림](#)을 보라. 그것이 stdin을 한 줄씩 읽고, stdout을 버퍼링하며, 둘 중 어느 쪽으로든 직접 포매팅할 수 있게 해준다.

```

proven_sysio_scanner_t
typedef struct {
    proven_file_t file;
    proven_allocator_t alloc;
    proven_u8 *buffer;
    proven_size_t capacity;
    proven_size_t cursor;
    proven_size_t length;
    bool eof;
} proven_sysio_scanner_t;

```

목적: 경계가 있는 스트림 입력을 위한 버퍼드 스캐너로, seek 가능한 파일뿐 아니라 파이프와 stdin에도 안전하다. 토큰이 EOF 전에 현재 로드된 조각의 끝에 도달하면, 스캐너는 버퍼를 다시 채우고 재시도한다. 다시 채운 후에도 버퍼 전체에 들어갈 수 없는 토큰만이 절단되어 받아들여지는 대신 PROVEN_ERR_OUT_OF_BOUNDS로 거부된다.

Sysio 함수와 매크로

API	의도	반환
proven_sysio_scanner_init(scanner, file, alloc, buffer_capacity)	스캐너 버퍼를 할당하고 스트림을 바인딩.	proven_err_t.
proven_sysio_scanner_deinit(scanner)	스캐너 버퍼를 해제.	void.
proven_sysio_scanner_scan_impl(scanner, fmt, args, args_count)	내부 스캐너 엔진.	proven_err_t.
proven_sysio_scanner_scan(scanner, fmt, ...)	타입 안전 버퍼드 스캔 매크로.	proven_err_t.
proven_sysio_print_impl(handle, fmt, args, args_count)	내부 출력 엔진.	proven_err_t.
proven_sysio_scan_chunk_impl(handle, fmt, args, args_count)	한-청크 스캔 엔진.	proven_err_t.
proven_print(fmt, ...)	stdout으로 출력.	proven_err_t.
proven_println(fmt, ...)	개행과 함께 stdout으로 출력.	proven_err_t.
proven_eprint(fmt, ...)	stderr로 출력.	proven_err_t.
proven_eprintln(fmt, ...)	개행과 함께 stderr로 출력.	proven_err_t.
proven_scan_fmt_from_file(file, fmt, ...)	파일의 고정 크기 청크 하나에서 스캔.	proven_err_t.
proven_scan_fmt_from_stdin(fmt, ...)	stdin에서 고정 크기 청크 하나를 스캔.	proven_err_t.

proven_env_get(alloc, key)	환경 변수를 owned U8 문자열로 읽음.	proven_result_u8str_t.
----------------------------	--------------------------	------------------------

proven_sysio_scan_chunk_impl()은 seek 가능한 파일 입력을 위한 것이다. 최대 하나의 고정 크기 청크를 읽는다. 핸들을 되감을 수 없으면, 읽기 전에 PROVEN_ERR_UNSUPPORTED를 반환한다. 완전한 토큰이 준비되기 전에 청크가 가득 차면, PROVEN_ERR_OUT_OF_BOUNDS를 반환하고 파일 커서를 청크의 시작으로 되감는다. 문자열 뷰(view) 목적지도 읽기 전에 PROVEN_ERR_UNSUPPORTED로 거부한다. 그런 view는 이 함수의 지역 청크 버퍼를 빌리므로 반환 즉시 무효가 되기 때문이다. 반복적인 버퍼드 스캔이나 빌린 문자열 결과에는 호출자가 버퍼를 소유하는 proven_sysio_scanner_t를 사용하라. 그 스캐너가 반환한 문자열 view는 다음 scan 호출이나 스캐너 해제 전까지만 유효하다. 어느 쪽이든 공유 버퍼를 다시 채우거나 압축하거나 해제할 수 있기 때문이다.

예:

```
proven_println("answer={}", PROVEN_ARG(42));
proven_eprintln("warning: {}", PROVEN_ARG(PROVEN_LIT("low memory")));
```

환경 변수 예. proven_env_get은 owned 문자열을 돌려주므로 반드시 파괴해야 한다:

```
proven_result_u8str_t env = proven_env_get(alloc, PROVEN_LIT("PATH"));
if (proven_is_ok(env.err)) {
    proven_u8str_view_t path = proven_u8str_as_view(&env.value);
    proven_println("PATH is {} bytes", PROVEN_ARG(path.size));
    proven_u8str_destroy(alloc, &env.value);
}
```

3. 메모리 매핑

문제: 복사하고 싶지 않은 파일 읽기

proven_fs_read_all_u8str는 파일을 당신이 소유하는 메모리로 읽어 들인다. 설정 파일이라면 그것이 정확히 옳다. 하지만 4 GB짜리 데이터베이스, 메모리 매핑된 인덱스, 또는 두 프로세스가 동시에 봐야 하는 파일이라면 세 가지 면에서 틀렸다: 4 GB의 RAM이 필요하고, 읽든 읽지 않든 모든 바이트를 복사하며, 그 복사본은 오직 당신만의 것이다.

메모리 매핑은 대신 운영체제에게 파일의 내용이 어떤 주소에 나타나게 해 달라고 요청한다. 미리 복사되는 것은 없다. 당신이 건드리는 페이지만 필요할 때 디스크에서 읽히고, 결코 건드리지 않는 페이지는 결코 읽히지 않는다. 두 프로세스가 같은 파일을 PROVEN_MMAP_SHARED로 매핑하면 하나의 페이지 집합을 보게 되므로, 한 쪽에서의 쓰기가 다른 쪽에서 보인다.

이것의 비용, 그리고 쓰지 말아야 할 때

실패 양상이 오류처럼 보이지 않기 때문에, 이 절은 매뉴얼에서 독실이 가장 침예한 부분이다:

- **읽기가 fault를 일으킬 수 있다.** 파일이 메모리에 있으면 I/O 오류는 반환값이다. 매핑에서는, 디스크 읽기가 실패하는 페이지를 건드리면 오류 코드가 아니라 **시그널(SIGBUS)**이 전달된다. 이를 우회하도록 작성할 수 있는 if는 없다.
- **잘림(truncation)은 지뢰다.** 매핑한 뒤에 다른 프로세스가 파일을 줄이면, 새 끝을 넘어선 페이지를 건드리는 것 역시 SIGBUS다. 다른 무언가가 잘라낼 수 있는 파일을 매핑하는 것은 어떤 API도 대신 고쳐 줄 수 없는 방식으로 안전하지 않다.
- **공짜가 아니다.** 매핑을 설정하는 것은 syscall이자 페이지 테이블 작업이며, 첫 접촉마다 페이지 fault가 난다. 작은 파일이라면 proven_fs_read_all_u8str가 그냥 더 빠르다.
- **proven_mmap_sync가 지속성의 지점이다.** 공유 매핑에 대한 쓰기는 결국 파일에 도달한다. 지금 디스크에 있어야 한다면, 요청하라.

큰 파일을 드문드문 읽을 때, 프로세스 간에 공유하는 읽기 전용 데이터에, 그리고 큰 파일에 대한 임의 접근에 매핑을 쓰라. 그 밖의 모든 것에는 평범한 읽기를 쓰라.

매핑은 호출자가 소유하는 상태다: proven_mmap_as_view는 **매핑 안을 가리키는 view**를 건네주므로, 그 view는 proven_mmap_destroy가 실행되는 순간 죽는다.

잘못된 예 — 매핑을 파괴한 뒤에 view를 사용하기:

```

proven_u8str_view_t data = proven_mmap_as_view(m);
proven_err_t e = proven_mmap_destroy(&m);
(void)e;
parse(data);          /* wrong: those addresses are no longer mapped - SIGSEGV */

```

잘못된 예 — 매핑을 키울 수 있는 버퍼처럼 취급하기:

```

/* wrong: a mapping is a window onto a file of a fixed size at map time.
   Appending means changing the file and mapping it again. */

```

구조체와 열거형

```

typedef enum {
    PROVEN_MMAP_READ  = 0x01,
    PROVEN_MMAP_WRITE = 0x02,
    PROVEN_MMAP_EXEC  = 0x04
} proven_mmap_prot_t;

```

```

typedef enum {
    PROVEN_MMAP_PRIVATE = 0x01,
    PROVEN_MMAP_SHARED  = 0x02
} proven_mmap_flags_t;

```

```

typedef struct {
    void *ptr;
    proven_size_t size;
    proven_fs_handle_t file;
    void *internal_handle;
} proven_mmap_t;

```

```

typedef struct {
    proven_err_t err;
    proven_mmap_t value;
} proven_result_mmap_t;

```

함수

API	의도	반환
proven_mmap_create(file, offset, size, prot, flags)	파일 영역을 매핑. size == 0은 EOF까지 매핑한다. 오프셋은 플랫폼의 매핑 세분성(granularity)과 일치해야 한다.	proven_result_mmap_t.
proven_mmap_destroy(mmap)	영역을 언매핑하고 상태를 정리.	proven_err_t.
proven_mmap_sync(mmap)	공유된 쓰기 가능 변경분을 저장소로 flush.	proven_err_t.
proven_mmap_as_view(mmap)	매핑된 메모리를 U8 view로 빌림 (borrow).	proven_u8str_view_t.

예:

```

proven_result_file_t f = proven_fs_open(alloc, PROVEN_LIT("data.bin"), PROVEN_FS_READ);
if (proven_is_ok(f.err)) {
    proven_result_mmap_t mr = proven_mmap_create(
        f.value,
        0,
        0,
        PROVEN_MMAP_READ,
        PROVEN_MMAP_PRIVATE
    );
    if (proven_is_ok(mr.err)) {
        /* The view borrows the mapping: it dies when the mapping is destroyed. */
    }
}

```

```

    proven_u8str_view_t bytes = proven_mmap_as_view(mr.value);
    proven_println("mapped {} bytes", PROVEN_ARG(bytes.size));
    (void)proven_mmap_destroy(&mr.value);
}
(void)proven_fs_close(f.value);
}

```

4. 시간 API

문제: 서로 다른 두 질문, 하나의 단어

"시간"은 서로 닮았지만 하는 짓은 전혀 다른 두 가지를 뜻한다.

벽시계(wall clock)는 *지금 몇 시인가?*에 답한다 — 사용자가 읽는 그것이다. 이 시계는 튀어도 된다: NTP가 보정하고, 서버타임이 옮기고, 관리자가 설정한다. 이것으로 경과 시간을 재면 음수가 나올 수 있는데, 이는 윤초가 있는 날에 나타나고 테스트 중인 누구의 노트북에서도 나타나지 않는 버그다.

단조(monotonic) 시계는 *언제부터 얼마나 지났는가?*에 답한다. 앞으로만 움직이며 어떤 달력과도 관계가 없다. 연산의 시간을 썰 때 쓰는 것이 이것이다.

libc는 이 구분을 흐린다. `time()`은 벽시계의 초 단위 정수를 준다 — 무언가를 재기에는 너무 거칠다. `clock()`은 CPU 시간을 재므로, 소켓을 기다리는 프로그램은 시간이 전혀 걸리지 않은 것처럼 보인다. 어느 이름도 자신이 어느 질문에 답하는지 말해 주지 않으며, 둘 다 서로의 용도로 흔히 쓰인다.

이 라이브러리는 대신 무엇을 하는가

`proven_time_now()`는 **Unix epoch 이후의 나노초**를 부호 있는 64비트 값으로 반환한다. 빼면 경과 시간이 되고, 분해하면 달력 날짜가 되는 하나의 숫자다 — 실제 작업의 시간을 썰 만큼 충분히 고운 해상도로.

`proven_time_breakdown()`은 그 숫자를 `proven_datetime_t`로 바꾸며, 그 필드는 사람이 쓰는 것들이다: `month`는 libc의 0-11이 아니라 **1-12**이고, `year`는 1900년 이후의 해수가 아니라 실제 연도다. `struct tm`의 그 두 가지 off-by-N 관례는 의도적으로 갈라설 만큼 충분히 많은 버그를 낳았다.

포매팅은 3장과 같은 {} 자리표시자를 쓴다: 이름이 필드를 고르고 스펙이 자리를 채우므로, {month:0>2}는 너비 2로 0을 채운다. 월과 요일 이름은 `proven_time_locale_t`에서 옴프로, 다른 언어로 렌더링하는 것은 전역을 설정하는 것이 아니라 다른 locale을 넘기는 일이다.

잘못된 예 — 벽시계로 시간을 재고 부호를 믿기:

```

proven_time_t t0 = proven_time_now();
do_work();
proven_time_t t1 = proven_time_now();
proven_u64 ns = (proven_u64)(t1 - t0); /* wrong: an NTP step back makes this enormous */

```

뻘썸은 괜찮다. 캐스트가 문제다. 차이를 부호 있는 값으로 유지하고, 음수 경과 시간은 지속 시간이 아니라 "시계가 움직였다"로 취급하라.

잘못된 예 — `sleep`이 정확하다고 가정하기:

```

proven_time_sleep(15);
/* wrong to assume exactly 15ms have passed: sleep guarantees AT LEAST that long,
   and the scheduler decides when you actually run again. */

```

완성 예제: 경과 시간, 날짜, 그리고 포맷된 타임스탬프

테스트 스위트가 컴파일하고 실행한다. 이 예제가 무엇을 단언하지 *않는지*에 주목하라 — `sleep`의 상한이다. 그것을 단언하면 바쁜 기계에서 실패하는 테스트가 되기 때문이다.

```

/*
 * 시간에는 똑같이 생겼지만 같지 않은 두 갈래가 있고, 잘못 고르는 것이 시간 재기의
 * 고전적 버그다.
 *
 * - *벽시계*는 "지금 몇 시인가"에 답한다. 사용자가 보고 싶어 하는 것이고, 뛰는 것이
 *   허용된다 - NTP가 고치고, 서버타임이 옮기고, 관리자가 맞춘다. 그것으로 기간을
 *   재면 음수 경과 시간이 나올 수 있고, 윤초가 있던 날에 실제로 유명하게 그랬다.

```

```

*
* - *단조* 시계는 "그때로부터 얼마나" 에 답한다. 앞으로만, 고른 속도로 가고, 어떤
*   달력과도 관계가 없다. 어떤 작업의 시간을 잴 때 쓰는 것이 이것이다.
*
* libc 는 이 구분을 흐린다. time() 은 초 단위 벽시계라 재는 데는 쓸모없다. clock() 은
* 경과 시간이 아니라 CPU 시간을 재므로, 잠든 프로그램은 즉시 끝난 것처럼 보인다. 어느
* 이름도 자기가 둘 중 어느 물음에 답하는지 말해 주지 않는다.
*
* proven_time_now() 는 유닉스 기점부터의 나노초다. 날짜로 형식화되기도 하고 기간으로
* 빼지기도 하는 수 하나이며, 실제 작업의 시간을 재기에 충분히 고운 해상도를 갖는다.
*/

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* --- 기간으로 쓰기 ----- */
    proven_time_t start = proven_time_now();
    proven_time_sleep(15); /* 밀리초 */
    proven_time_t end = proven_time_now();

    proven_i64 elapsed_ns = end - start;
    EXAMPLE_REQUIRE(elapsed_ns > 0, "time must move forward across a sleep");
    /* sleep 은 청한 시간 *이상*을 보장하지, 이하를 보장하지 않는다. 언제 다시 돌지는
     * 스케줄러가 정한다. 여기서 상한을 단언하면 바쁜 기계에서 실패하는 시험이 되고,
     * 그래서 이 예제는 그러지 않는다. */
    EXAMPLE_REQUIRE(elapsed_ns >= 10 * 1000 * 1000,
        "sleeping 15ms must take at least ~10ms of wall time");

    /* --- 날짜로 쓰기 ----- */
    proven_datetime_t dt = proven_time_breakdown(start);
    EXAMPLE_REQUIRE(dt.year >= 2020 && dt.year < 3000, "the epoch breakdown gives a plausible year");
    EXAMPLE_REQUIRE(dt.month >= 1 && dt.month <= 12, "month is 1-12, not 0-11 as in libc's tm");
    EXAMPLE_REQUIRE(dt.day >= 1 && dt.day <= 31, "day is 1-31");
    EXAMPLE_REQUIRE(dt.hour <= 23 && dt.min <= 59 && dt.sec <= 60, "sec allows 60 for leap seconds");
    EXAMPLE_REQUIRE(dt.weekday <= 6, "weekday is 0-6 with 0 = Sunday");

    /* proven_time_now_datetime() 은 위의 두 호출을 하나로 합친 것이다. 달력 끝만
     * 필요할 때 쓴다. */
    proven_datetime_t now = proven_time_now_datetime();
    EXAMPLE_REQUIRE(now.year == dt.year, "both routes read the same clock");

    /* --- 시각을 글자로 ----- */
    proven_result_u8str_t s = proven_u8str_create(alloc, 64);
    EXAMPLE_REQUIRE(proven_is_ok(s.err), "a 64-byte string is enough for a timestamp");

    /* 달 이름과 요일 이름은 로케일이 준다. proven_time_locale_en 이 내장된 영어
     * 로케일이다. 다른 언어로 찍으려면 여러분의 것을 건넨다. */
    proven_err_t err = proven_time_u8_fmt(alloc, &s.value, dt, &proven_time_locale_en,
        "{year}-{month:0>2}-{day:0>2} {hour:0>2}:{min:0>2}:{sec:0>2}");
    EXAMPLE_REQUIRE(proven_is_ok(err), "formatting a datetime should succeed");

    proven_u8str_view_t out = proven_u8str_as_view(&s.value);
    EXAMPLE_REQUIRE(out.size == 19, "year-month-day hour:min:sec is exactly 19 characters");
    EXAMPLE_REQUIRE(out.ptr[4] == '-' && out.ptr[7] == '-' && out.ptr[13] == ':',
        "the separators land where the pattern put them");

    proven_println("formatted: {}", PROVEN_ARG(out));

    proven_u8str_destroy(alloc, &s.value);

```

```

    return EXAMPLE_OK();
}

```

타입

```
typedef proven_i64 proven_time_t;
```

UNIX epoch 이후 나노초.

```

typedef struct {
    proven_i32 year;
    proven_u8 month;
    proven_u8 day;
    proven_u8 hour;
    proven_u8 min;
    proven_u8 sec;
    proven_u32 ms;
    proven_u8 weekday;
} proven_datetime_t;

```

필드 범위:

- month: 1 ~ 12.
- day: 1 ~ 31.
- hour: 0 ~ 23.
- min: 0 ~ 59.
- sec: 0 ~ 60.
- ms: 0 ~ 999.
- weekday: 0 ~ 6, 일요일이 0.

```

typedef struct {
    const proven_u8str_view_t *month_names;
    const proven_u8str_view_t *month_short_names;
    const proven_u8str_view_t *weekday_names;
    const proven_u8str_view_t *weekday_short_names;
} proven_time_locale_t;

```

proven_time_locale_en은 기본 영어 로케일이다.

함수

API	의도	반환
proven_time_u8_fmt(alloc, str, dt, locale, fmt)	포맷된 날짜시간을 U8 문자열에 덧붙임.	proven_err_t.
proven_time_u16_fmt(alloc, str, dt, locale, fmt)	U16이 비활성화되지 않은 한 포맷된 날짜시간을 U16 문자열에 덧붙임.	proven_err_t.
proven_time_now()	현재 타임스탬프(나노초).	proven_time_t.
proven_time_breakdown(time_ns)	epoch 나노초를 분해된 UTC 시간으로 변환.	proven_datetime_t.
proven_time_now_datetime()	현재 로컬 분해 시간.	proven_datetime_t.
proven_time_sleep(ms)	밀리초 동안 sleep.	void.

날짜시간 포맷 키:

- {year}, {month}, {day}, {hour}, {min}, {sec}, {ms}, {wday_num}
- 로케일을 사용하는 {Month}, {mon}
- 로케일을 사용하는 {Weekday}, {wday}

예:

```

proven_result_u8str_t r = proven_u8str_create(alloc, 32);
if (proven_is_ok(r.err)) {
    proven_u8str_t s = r.value;

    proven_datetime_t now = proven_time_now_datetime();
    proven_err_t e = proven_time_u8_fmt(
        alloc,
        &s,
        now,
        &proven_time_locale_en,
        "{year}-{month:0>2}-{day:0>2} {hour:0>2}:{min:0>2}:{sec:0>2}"
    );
    if (proven_is_ok(e)) {
        proven_println("{f}", PROVEN_ARG(proven_u8str_as_view(&s)));
    }

    proven_u8str_destroy(alloc, &s);
}

```

5. 예제와 응용 사례

단일 쓰기는 부분적일 수 있다

잘못됨(Wrong):

```

proven_result_size_t w = proven_fs_write(file, data);
/* wrong: one write may be partial, and w.value is never looked at */

```

올바름(Correct):

```

proven_result_file_t f = proven_fs_open(alloc, PROVEN_LIT("out.txt"),
                                         PROVEN_FS_WRITE | PROVEN_FS_CREATE | PROVEN_FS_TRUNC);
if (proven_is_ok(f.err)) {
    proven_mem_view_t data = proven_mem_view_from_u8(PROVEN_LIT("payload"));
    proven_err_t e = proven_fs_write_all(f.value, data); /* loops until done */
    proven_err_t ce = proven_fs_close(f.value);          /* and the close can fail too */
    if (proven_is_ok(e)) e = ce;
    (void)e;
}

```

디렉터리 목록에는 특별한 파괴가 필요하다

잘못됨(Wrong):

```

proven_result_array_t r = proven_fs_list(alloc, PROVEN_LIT("."));
PROVEN_ARRAY_DESTROY(&r.value); /* wrong: entry names leak */

```

올바름(Correct):

```

proven_result_array_t r = proven_fs_list(alloc, PROVEN_LIT("."));
if (proven_is_ok(r.err)) {
    proven_fs_list_destroy(alloc, &r.value);
}

```

destroy 이후에 매핑을 사용하지 말라

잘못됨(Wrong):

```

proven_mmap_destroy(&map);
use_bytes(map.ptr, map.size); /* wrong: mapping has been released */

```

스트림에는 버퍼드 sysio 스캔을 사용하라

stdin이나 파이프에서 반복적으로 읽을 때는 다음을 선호하라:

```

proven_sysio_scanner_t scanner = {0};
proven_err_t e = proven_sysio_scanner_init(&scanner, proven_sysio_stdin(), alloc, 4096);
if (proven_is_ok(e)) {

```

```

    int value = 0;
    e = proven_sysio_scanner_scan(&scanner, "{}", PROVEN_SCAN_ARG(&value));
    proven_sysio_scanner_deinit(&scanner);
}

```

환경 값은 owned 문자열이다

잘못됨(Wrong):

```

proven_result_u8str_t env = proven_env_get(alloc, PROVEN_LIT("PATH"));
use_path(proven_u8str_as_view(&env.value)); /* wrong if env.err was not checked, and it leaks */

```

올바름(Correct):

```

proven_result_u8str_t env = proven_env_get(alloc, PROVEN_LIT("HOME"));
if (proven_is_ok(env.err)) {
    proven_u8str_view_t home = proven_u8str_as_view(&env.value);
    proven_println("HOME={}", PROVEN_ARG(home));
    proven_u8str_destroy(alloc, &env.value);
}

```

완성 예제: 파일 전체를 읽고 쓰기

테스트 스위트에 의해 컴파일되고 실행됨. 이것은 대부분의 호출자가 실제로 원하는 파일 전체 API이며, 대상의 권한을 보존하는 atomic 재작성을 포함한다.

```

/*
 * 파일 통째 API: 한 번 부르면 들어가고, 한 번 부르면 나온다. 이것이 있는 이유는 열고 -
 * 읽는 반복문 - 닫는 그 춤사위가 파일 다루기 버그 대부분이 사는 자리이기 때문이다.
 * 읽은 close, EOF 로 오해한 부분 읽기, 실패한 쓰기가 남긴 잘린 파일. 파일을 통째로 읽거나
 * 쓰는 것이라면 이것이 그 API 다.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* 지금 디렉터리의 상대 경로다. 이 예제가 쓸 수 있는 /tmp 에 기대면 안 되고, 만든
     * 것은 돌아가기 전에 지운다. */
    proven_u8str_view_t path = PROVEN_LIT("proven_example_wholefile.tmp");
    proven_u8str_view_t text = PROVEN_LIT("first line\nsecond line\n");

    /* --- 한 번의 호출로 쓰기 ----- */
    /* 원자적이지 않다. 동시에 읽는 쪽은 이 파일이 반쯤 쓰인 것을 볼 수 있다. 여기서는
     * 괜찮다. 아직 아무도 이 파일을 보고 있지 않기 때문이다. */
    proven_err_t err = proven_fs_write_file(alloc, path, proven_mem_view_from_u8(text));
    EXAMPLE_REQUIRE(proven_is_ok(err), "writing the whole file should succeed");
    if (!proven_is_ok(err)) return 1;

    /* --- 날바이트로 되읽기 ----- */
    /* proven_fs_read_all 은 미리 켜 크기까지가 아니라 EOF 까지 읽는다. 그래서 크기를
     * 미리 알 수 없는 파이프나 /proc 항목에서도 돈다. */
    proven_result_mem_mut_t raw = proven_fs_read_all(alloc, path);
    EXAMPLE_REQUIRE(proven_is_ok(raw.err), "reading the whole file should succeed");
    if (proven_is_ok(raw.err)) {
        EXAMPLE_REQUIRE(raw.value.size == text.size, "read_all should return every byte written");
        /* 그 덩이는 그냥 할당자의 기억이다 - 그것을 내준 할당자에게 돌려줄 것.
         * proven_fs_read_all_destroy 같은 것은 없다. */
        alloc.free_fn(alloc.ctx, raw.value.ptr);
    }

    /* --- 문자열로 되읽기 ----- */
    /* 대개의 부르는 쪽이 원하는 것이 이것이다. 결과가 NUL 로 끝나므로 뷰에도, as_cstr
     * 에도, 스캐너에도 두 번째 복사 없이 건넬 수 있다. 종결자 자리는 미리 잡아 두므로

```

```

    * 할당이 더 들지 않는다. */
    proven_result_u8str_t s = proven_fs_read_all_u8str(alloc, path);
    EXAMPLE_REQUIRE(proven_is_ok(s.err), "reading the whole file as a string should succeed");
    if (!proven_is_ok(s.err)) {
        (void)proven_fs_remove(alloc, path);
        return 1;
    }
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&s.value), text),
        "the file's contents should come back unchanged");
    printf("read back %zu bytes: %s", (size_t)proven_u8str_as_view(&s.value).size,
        proven_u8str_as_cstr(&s.value));
    proven_u8str_destroy(alloc, &s.value);

    /* --- stat, 그리고 권한의 왕복 ----- */
    proven_fs_stat_t st = {0};
    err = proven_fs_stat(alloc, path, &st);
    EXAMPLE_REQUIRE(proven_is_ok(err), "stat on a file we just wrote should succeed");
    EXAMPLE_REQUIRE(st.type == PROVEN_FS_TYPE_FILE, "a regular file should stat as a FILE");
    EXAMPLE_REQUIRE(st.size == text.size, "stat should report the size we wrote");

    /* `perms` 는 권한 비트 아홉 개만 담고 그 밖의 것은 담지 않는다. 그래서 그대로
    * chmod 에 도로 먹일 수 있다. 이 필드의 요점이 그것이다 - 파일의 모드를 읽어 두었다가
    * 나중에 되돌리는 것. (예전에는 날 POSIX st_mode 를 담았는데, 그 파일 종류 비트를
    * chmod 가 거부해서 이 뻘한 왕복이 실패했다.) */
    err = proven_fs_chmod(alloc, path, st.perms);
    EXAMPLE_REQUIRE(proven_is_ok(err), "a stat's perms must be accepted back by chmod");

    /* 이제 파일을 소유자 전용으로 바꾼다. 그래야 다음 검사가 증명할 것이 생긴다. */
    proven_fs_perms_t private_perms = PROVEN_FS_PERM_OWNER_R | PROVEN_FS_PERM_OWNER_W;
    err = proven_fs_chmod(alloc, path, private_perms);
    EXAMPLE_REQUIRE(proven_is_ok(err), "restricting the file to its owner should succeed");

    /* --- 원자적으로 다시 쓰기 ----- */
    /* 형제 임시 파일 하나에 rename 하나. 동시에 읽는 쪽은 옛 파일 전체이거나 새 파일
    * 전체를 보지, 반쯤 섞인 것을 보지 않는다. 읽는 쪽에게 원자적이지, 전원이 나가도
    * 견디는 것은 아니다. 그래야 할 때는 proven_fs_write_file_durable 이 그것을 청한다. */
    proven_u8str_view_t text2 = PROVEN_LIT("replacement\n");
    err = proven_fs_write_file_atomic(alloc, path, proven_mem_view_from_u8(text2));
    EXAMPLE_REQUIRE(proven_is_ok(err), "the atomic rewrite should succeed");

    proven_fs_stat_t st2 = {0};
    err = proven_fs_stat(alloc, path, &st2);
    EXAMPLE_REQUIRE(proven_is_ok(err), "stat after the atomic rewrite should succeed");
    EXAMPLE_REQUIRE(st2.size == text2.size, "the file should now hold the replacement text");
    /* rename 은 옛 이름 위에 *새* 아이노드를 씌우므로, 권한을 옮겨 주지 않으면 읽는다.
    * 옮겨 준다 - 0600 파일을 다시 써도 0644 로 다시 공개되지 않는다. */
    EXAMPLE_REQUIRE(st2.perms == private_perms,
        "the atomic rewrite must preserve the target's permissions");

    /* --- 뒷정리 ----- */
    err = proven_fs_remove(alloc, path);
    EXAMPLE_REQUIRE(proven_is_ok(err), "removing the temp file should succeed");

    return EXAMPLE_OK();
}

```

완성 예제: open, read, write, close

테스트 스위트에 의해 컴파일되고 실행됨. read 루프에 주목하라: 파일 끝에서의 읽기는 0바이트 성공이 아니라 PROVEN_ERR_EOF를 반환하므로, 0바이트만 검사하는 루프는 작성자가 기대한 방식으로 결코 종료되지 않는다.

```
/*
 * 열고-읽고-쓰고-닫는 길. 파일 통째 API(ex_05_fs_wholefile)로는 모자랄 때 쓴다.
 * 흘러 보내고 있거나, 버퍼를 여러분이 소유하고 싶을 때다.
 *
 * 여기서 반드시 맞춰야 할 하나: 읽기나 쓰기 한 번은 청한 바이트 수 *만큼까지*를 옮기지,
 * 정확히 그만큼을 옮기지 않는다. 짧은 읽기 한 번을 파일 끝으로 여기는 것이 파일의 꼬리를
 * 조용히 읽는 고전적인 방법이다.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    proven_u8str_view_t path = PROVEN_LIT("proven_example_stream.tmp");
    proven_u8str_view_t text = PROVEN_LIT("streamed bytes, read back in chunks\n");

    /* --- 쓰기 --- */
    /* CREATE 는 없으면 만들고, TRUNC 는 있으면 비운다. 할당자는 플랫폼 호출을 위해 경로를
     * 변환하는 데만 쓰인다. */
    proven_result_file_t out = proven_fs_open(alloc, path,
                                              PROVEN_FS_WRITE | PROVEN_FS_CREATE | PROVEN_FS_TRUNC);
    EXAMPLE_REQUIRE(proven_is_ok(out.err), "opening the file for writing should succeed");
    if (!proven_is_ok(out.err)) return 1;

    /* write_all 이 우리 대신 반복해 준다. proven_fs_write 는 한 번만 시도하고 더 적게 쓸
     * 수도 있는데, 그것은 부르는 쪽이 뜻한 바가 거의 아니다. */
    proven_err_t err = proven_fs_write_all(out.value, proven_mem_view_from_u8(text));

    /* close 는 쓰기의 일부이고, 네트워크나 할당량이 걸린 파일 시스템에서는 실패가 드러나는
     * *유일한* 자리다. 바이트는 버퍼에 담겼고 write() 는 그렇다고 했으며, 디스크가 마침내
     * 아니라고 말하는 자리가 close() 다. 실패 경로에서도 닫을 것 - 손잡이는 어느 쪽이든
     * 우리 것이다 - 다만 그 답을 버리지는 말 것. */
    proven_err_t cerr = proven_fs_close(out.value);
    if (proven_is_ok(err)) err = cerr;

    EXAMPLE_REQUIRE(proven_is_ok(err), "writing the whole buffer should succeed");
    if (!proven_is_ok(err)) {
        (void)proven_fs_remove(alloc, path);
        return 1;
    }

    /* --- 읽기 --- */
    proven_result_file_t in = proven_fs_open(alloc, path, PROVEN_FS_READ);
    EXAMPLE_REQUIRE(proven_is_ok(in.err), "opening the file for reading should succeed");
    if (!proven_is_ok(in.err)) {
        (void)proven_fs_remove(alloc, path);
        return 1;
    }

    /* size 는 버퍼 크기를 잡는 힌트이지, 읽기 한 번이 몇 바이트를 건넬지에 대한 약속이
     * 아니다 - 그리고 보통 파일이 아닌 것(파이프, 장치, /proc 항목)에서는 0 이다. 아래
     * 반복문은 그것에 기대지 않는다. */
    proven_result_size_t sz = proven_fs_size(in.value);
    EXAMPLE_REQUIRE(proven_is_ok(sz.err), "querying the size of an open file should succeed");
    EXAMPLE_REQUIRE(sz.value == text.size, "the file should be as long as what we wrote");
}
```

```

proven_byte_t buf[128];
proven_size_t total = 0;

/* 부분 읽기 반복문. 한 바퀴마다 버퍼에 남은 만큼을 청하고 실제로 온 만큼 나아간다.
 * 짧은 읽기는 파일의 끝이 아니라 정상이다. 파일의 끝은 따로 있는 상태 -
 * 바이트 0 과 함께 오는 PROVEN_ERR_EOF - 이므로 반복문은 그것으로만 끝난다. 원본이
 * 버퍼보다 커져도 반복문은 멈추는데, 그것을 알아채는 일은 부르는 쪽의 몫이다(여기서는
 * 일어날 수 없지만, 자라나는 파일이라면 그럴 수 있다). */
for (;;) {
    if (total == sizeof buf) break; /* 버퍼가 가득 찼다: 어떻게 할지는 부르는 쪽이 정한다 */

    proven_mem_mut_t dest = { .ptr = buf + total, .size = sizeof buf - total };
    proven_result_size_t r = proven_fs_read(in.value, dest);
    if (r.err == PROVEN_ERR_EOF) break;
    if (!proven_is_ok(r.err)) {
        (void)proven_fs_close(in.value);
        (void)proven_fs_remove(alloc, path);
        EXAMPLE_REQUIRE(false, "reading from the open file should not fail");
        return 1;
    }
    total += r.value;
}

(void)proven_fs_close(in.value);

EXAMPLE_REQUIRE(total == text.size, "the loop should have read every byte in the file");
proven_u8str_view_t got = { .ptr = buf, .size = total };
EXAMPLE_REQUIRE(proven_u8str_view_eq(got, text), "the bytes should come back unchanged");

printf("read %zu bytes in chunks: %.*s", (size_t)total, (int)total, (const char *)buf);

/* --- 뒷정리 ----- */
err = proven_fs_remove(alloc, path);
EXAMPLE_REQUIRE(proven_is_ok(err), "removing the temp file should succeed");

return EXAMPLE_OK();
}

```

디렉터리를 한 번에 한 엔트리씩 읽기

proven_fs_list는 당신이 그 중 어느 것을 보기 전에 디렉터리 전체를 읽으며, 모든 이름마다 문자열을 할당한 다. 50,000개 엔트리로 측정: 189 ms, +4.2 MB 상주(resident), 50,008회 할당, 마지막 엔트리가 읽힐 때 까지 아무것도 보이지 않는다. 이는 설정 디렉터리에는 괜찮고 메일 스펠에는 무용하다.

proven_fs_dir_open / _next / _close는 같은 디렉터리를 한 번에 한 엔트리씩 순회하며 엔트리당 아무것도 할당 하지 않는다.

API	의도	반환
proven_fs_dir_open(scratch, path)	스트리밍 반복자를 연다. scratch는 경로 변환에만 쓰인다.	proven_result_dir_t.
proven_fs_dir_next(&dir, &entry)	다음 엔트리. 더 이상 없으면 PROVEN_ERR_EOF.	proven_err_t.
proven_fs_dir_close(&dir)	반복자를 해제.	void.

```

typedef struct {
    proven_u8str_view_t name; /* BORROWED: points into the iterator's own storage,
                               valid only until the next proven_fs_dir_next */
    proven_fs_type_t type; /* FILE / DIR / OTHER - and it follows symlinks */
} proven_fs_dir_entry_t;

```

이렇게 사용하라:

```
proven_result_dir_t d = proven_fs_dir_open(alloc, PROVEN_LIT("."));
if (proven_is_ok(d.err)) {
    proven_fs_dir_t dir = d.value;
    proven_fs_dir_entry_t entry;
    for (;;) {
        proven_err_t e = proven_fs_dir_next(&dir, &entry);
        if (e == PROVEN_ERR_EOF) break;
        if (!proven_is_ok(e)) break; /* a real error: report it */
        proven_println("{", PROVEN_ARG(entry.name));
    }
    proven_fs_dir_close(&dir);
}
```

이름은 빌려 쓰는(borrowed)이며, 다음 호출에서 소멸한다. 이것이 전체를 무할당으로 만드는 것—그리고 당신이 그것을 붙들어 두는 순간 dangling 포인터로 만드는 것이다.

잘못됨(Wrong):

```
proven_u8str_view_t names[100];
int n = 0;
while (proven_is_ok(proven_fs_dir_next(&dir, &entry)))
    names[n++] = entry.name; /* wrong: every entry aliases the same storage, and the
                             next _next_ overwrites it. All 100 end up equal - to
                             whatever the last entry happened to be. */
```

올바름(Correct): 보관해야 하는 것들에 대해 바이트를 복사하라(proven_u8str_create_from_view)—또는 proven_fs_list를 사용하라. 그것은 모든 엔트리에 대해 정확히 그렇게 하고 그 대가를 청구한다.

PROVEN_ERR_EOF가 끝이며, 그 밖의 것은 실패다. "OK 아님"에서 멈추는 루프는 권한 에러를 완전한 목록으로 취급한다.

```
while (proven_is_ok(proven_fs_dir_next(&dir, &entry))) { ... }
/* wrong: an I/O error ends the loop exactly like the end of the directory does,
   and you cannot tell whether you saw everything. */
```

트리 순회

proven_fs_dir_*는 디렉터리 하나를 순회한다. proven_fs_walk는 트리를 순회한다—그리고 그것이 하기를 거부하는 것을 정확히 말할 가치가 있다. 그 거부들이 곧 기능이기 때문이다:

루프에 빠질 수 없다	결코 symlink를 통과해 내려가지 않는다.
탈출할 수 없다	같은 규칙: 트리 밖으로 나가는 링크는 보고되되 따라가지 않는다.
거짓말할 수 없다	읽을 수 없는 디렉터리는 그 디렉터리를 명명하는 *에러*로 돌아오고, 순회는 계속된다. 읽을 수 없는 하위 트리를 조용히 건너뛰는 트리 순회기는 백업이 파일을 놓치고도 성공을 보고하는 방식이다.
비대해질 수 없다	현재 경로의 *레벨*당 하나의 열린 핸들, 그리고 재사용되는 하나의 경로 버퍼. 메모리는 파일 수가 아니라 깊이의 함수다.

symlink된 디렉터리는 여전히 **보고된다**—그것은 존재하고, type은 DIR이며, is_symlink는 true다—단지 진입되지 않을 뿐이다. 그것을 숨기는 것은 그 자체로 일종의 거짓말이 될 것이다. 그것을 따라가고 싶다면, 당신에게는 경로가 있다: 그것에 대해 두 번째 순회를 열면, 사이클 문제는 당신의 소유가 된다.

당신이 받는 구조체

```
typedef struct {
    proven_u8str_view_t path; /* the whole path, from the root you passed in.
                              BORROWED: it points into the walk's one reused
                              buffer and is valid only until the next call. */
```

```

    proven_u8str_view_t name; /* the last component of `path`. Same lifetime. */
    proven_fs_type_t     type; /* FILE / DIR / OTHER - and it FOLLOWS symlinks,
                                exactly as proven_fs_stat does. */
    proven_size_t        size; /* bytes, for a regular file; 0 otherwise. */
    proven_size_t        depth; /* 0 for an entry directly inside the root. */
    bool                 is_symlink; /* reached through a symlink. `type` describes
                                the TARGET; the walk does not enter it. */
} proven_fs_walk_entry_t;

```

entry.path와 entry.name은 순회의 재사용되는 하나의 버퍼에서 빌린 것이며 **다음 호출 전까지만** 유효하다. 이 단계보다 오래 살아남게 하려면 복사하라. 그것이 백만 엔트리의 순회가 백만 번이 아니라 한 번의 할당으로 드는 것의 대가다.

잘못됨(Wrong)—디렉터리 반복자가 가진 것과 같은 함정, 같은 이유로:

```

proven_u8str_view_t found[100];
int n = 0;
while (proven_is_ok(proven_fs_walk_next(&walk, &entry)))
    found[n++] = entry.path; /* wrong: every entry aliases ONE buffer. When the loop
                                ends, all 100 point at whatever the last path was. */

```

두 개의 제한, 둘 다 조용해지는 대신 그렇다고 말한다:

- max_depth—얼마나 깊이 내려갈지. 제한에 있는 디렉터리는 여전히 보고된다(그것은 하나의 엔트리다). 진입되지 않을 뿐이다.
- PROVEN_FS_WALK_DEPTH_LIMIT(256)—순회 자체의 스택이 결코 넘지 않는 깊이. 그것을 지나친 디렉터리는 그 디렉터리를 명명하며 PROVEN_ERR_OUT_OF_BOUNDS로 돌아온다.

테스트 스위트에 의해 컴파일되고 실행됨:

```

/*
 * 나무 훑기.
 *
 * 재귀 훑기가 틀리는 세 가지, 그리고 이것이 대신 하는 일.
 *
 * 맴돈다.      조상을 가리키는 심링크는 순환이다. 이 훑기는 심링크를 *지나* 내려가지
 *              않는다 - 심링크된 디렉터리도 보고는 되고, 다만 들어가지 않을 뿐이다 -
 *              그래서 순환이 불가능하고, 물어본 나무를 벗어나 나머지 파일 시스템으로
 *              걸어 나가는 일도 없다.
 *
 * 거짓말한다. 읽을 수 없는 디렉터리를 건너뛰고는 성공했다고 보고한다. 백업이 하위
 *              나무 하나를 통째로 놓치는 방식이 그것이다. 여기서는 오류가
 *              proven_fs_walk_next 에서 돌아오고, 그 항목이 어느 디렉터리인지 말해
 *              주며, 훑기는 다음 형제부터 이어 간다. 어떻게 할지는 여러분이 정한다.
 *
 * 부푼다.      하나라도 내주기 전에 디렉터리 전체를 기억에 읽어 들이면, 큰 나무를
 *              훑는 데 큰 할당이 든다. 이것은 지금 경로의 *층*마다 열린 손잡이 하나와
 *              다시 쓰는 경로 버퍼 하나만 된다 - 그래서 기억 사용량은 파일이 몇
 *              개인가가 아니라 깊이의 함수다.
 */

```

```

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* 훑어 볼 작은 나무: 파일 하나, 디렉터리 하나, 그 안의 파일 하나. */
    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_mkdir(alloc, PROVEN_LIT("ex_walk"))), "mkdir");
    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_mkdir(alloc, PROVEN_LIT("ex_walk/inner"))), "mkdir inner");
    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_write_file(alloc, PROVEN_LIT("ex_walk/top.txt"),
        proven_mem_view_from_u8(PROVEN_LIT("top"))), "write top.txt");
    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_write_file(alloc, PROVEN_LIT("ex_walk/inner/deep.txt"),
        proven_mem_view_from_u8(PROVEN_LIT("deep"))), "write deep.txt");

```

```

proven_result_walk_t walk = proven_fs_walk_open(alloc, PROVEN_LIT("ex_walk"),
                                                PROVEN_FS_WALK_UNLIMITED);
EXAMPLE_REQUIRE(proven_is_ok(walk.err), "the walk should open");

proven_size_t files = 0;
proven_size_t dirs = 0;
proven_size_t unreadable = 0;
proven_size_t total_bytes = 0;

for (;;) {
    proven_fs_walk_entry_t entry = {0};
    proven_err_t err = proven_fs_walk_next(&walk.value, &entry);

    if (err == PROVEN_ERR_EOF) break;

    if (!proven_is_ok(err)) {
        /* 읽을 수 없었던 디렉터리이거나, 훑기의 스택보다 깊은 것이다. 건너뛰는 것이
        * 아니라 *보고된다* - `entry.path` 가 어느 것인지 말해 준다 - 그리고 훑기는
        * 계속된다. 나무를 복사하는 도구라면 여기서 실패해야 하고, 나무를 보고하는
        * 도구라면 세어서 알려야 한다. 하지 말아야 할 것은 없던 일로 하는 것이다. */
        unreadable++;
        continue;
    }

    /* `entry.path` 와 `entry.name` 은 빌린 것이다. 훑기가 다시 쓰는 버퍼 하나를
    * 가리키고 다음 호출 전까지만 쓸 수 있다. 더 오래 필요하면 복사할 것. */
    if (entry.type == PROVEN_FS_TYPE_DIR) {
        dirs++;
    } else if (entry.type == PROVEN_FS_TYPE_FILE) {
        files++;
        total_bytes += entry.size;
    }
}

proven_fs_walk_close(&walk.value);

EXAMPLE_REQUIRE(files == 2, "two files: top.txt and inner/deep.txt");
EXAMPLE_REQUIRE(dirs == 1, "one directory: inner");
EXAMPLE_REQUIRE(unreadable == 0, "and nothing in this tree is unreadable");
EXAMPLE_REQUIRE(total_bytes == 7, "three bytes plus four");

/* 깊이 제한: max_depth 0 은 뿌리 바로 안에 있는 것을 보고하고 아무 데도 내려가지
* 않는다. 한계에 걸린 디렉터리도 여전히 항목이므로 여전히 보고된다. */
walk = proven_fs_walk_open(alloc, PROVEN_LIT("ex_walk"), 0);
EXAMPLE_REQUIRE(proven_is_ok(walk.err), "the shallow walk should open");

proven_size_t shallow = 0;
for (;;) {
    proven_fs_walk_entry_t entry = {0};
    proven_err_t err = proven_fs_walk_next(&walk.value, &entry);
    if (err == PROVEN_ERR_EOF) break;
    if (proven_is_ok(err)) shallow++;
}
proven_fs_walk_close(&walk.value);

EXAMPLE_REQUIRE(shallow == 2, "top.txt and inner - but nothing inside inner");

(void)proven_fs_remove(alloc, PROVEN_LIT("ex_walk/inner/deep.txt"));
(void)proven_fs_remove(alloc, PROVEN_LIT("ex_walk/top.txt"));
(void)proven_fs_remove(alloc, PROVEN_LIT("ex_walk/inner"));

```

```

    (void)proven_fs_remove(alloc, PROVEN_LIT("ex_walk"));

    return EXAMPLE_OK();
}

```

스트림: writer와 reader

포매터의 유일한 sink는 예전에 proven_u8str_t였다. 파일은 proven_file_t였다. 인메모리 스캐너는 view를 읽었고, 파일 스캐너는 또 다른 무언가를 읽었다. **네 개의 타입, 네 개의 함수 계열, 공통 인터페이스 없음**—그래서 메모리와 파일 양쪽에서 동작하는 하나의 serialize(sink, value)를 쓸 수 없었고, 파일로 포매팅하는 것 자체가 불가능했으며, 파일을 한 줄씩 읽을 방법이 전혀 없었다.

writer는 바이트 sink다. **reader**는 바이트 source다. 둘 다 값으로 전달되는 작은 vtable로, 정확히 proven_allocator_t처럼, 같은 이유로 그렇다: 호출자가 바이트가 어디로 가는지 결정하며, 아무것도 숨겨지지 않는다.

API	의도
proven_writer_from_file(&file)	열린 파일 위의 언버퍼드 sink.
proven_writer_from_u8str(&state, &str, alloc)	owned 문자열에 덧붙인다.
proven_writer_from_buffer(&state)	고정된 호출자 메모리. 절대로 아무것도 할당하지 않는다.
proven_writer_buffered(&state, inner, buf)	작은 쓰기들을 누적한다. 버퍼는 당신이 공급한다.
proven_fprint(w, fmt, ...) / proven_fprintln	writer로 직접 포매팅. 무할당.
proven_reader_from_file(&file) / _from_view(&state, view)	바이트 source.
proven_reader_buffered(&state, inner, buf)	버퍼드 source; 라인 읽기에 필수.
proven_reader_read_line(&state)	개행 없는 한 줄.
proven_writer_is_valid(w) / proven_reader_is_valid(r)	생성자가 성공했는가? 0으로 채워진 핸들은 invalid이고, 모든 생성자는 잘못된 인자에 대해 그런 것을 반환한다—따라서 NULL 검사가 아니라 이것이 그 검사다.
proven_fwrite_fmt(w, scratch, fmt, ...)	당신이 크기를 정한 임시 작업용(scratch) 버퍼를 쓰는 proven_fprint. proven_fprint는 512바이트 스택 버퍼를 사용하고 더 긴 줄에 대해 OUT_OF_BOUNDS를 반환한다. 이것이 그런 줄을 포매팅하는 방법이다.
proven_fmt_to_writer_impl(w, scratch, fmt, args, n)	위의 두 매크로가 확장되는 함수. 자신만의 가변 인자 래퍼를 만드는 경우에만 직접 호출하라. 매크로는 당신이 인자를 세지 않아도 되도록 존재한다.

명확히 언급할 가치가 있는 네 가지 규칙. 각각은 이것이 잘못 설계될 수 있었던 방식이기 때문이다:

- **버퍼링은 당신이 공급하는 메모리를 사용한다.** proven_writer_buffered는 proven_arena_create가 그러하듯 proven_mem_mut_t를 받는다. 숨겨진 전역 버퍼는 없으며, 이는 곧 당신을 위해 그것을 flush해 줄 소멸자도 없다는 뜻이다—버퍼가 스킵을 벗어나기 전에 당신이 **flush해야 한다**. 그 대가로, 당신의 로깅 경로는 결코 할당하지 않으며, out-of-memory 상태에서 빠져나오며 로그를 남기는 프로그램이 여전히 로그를 남길 수 있다.
- **가득 찬 sink는 거부한다. 절단하지 않는다.** 가득 찬 고정 버퍼는 PROVEN_ERR_OUT_OF_BOUNDS를 반환하고 overflowed를 기록한다. 당신의 데이터 끝을 조용히 버리는 sink는 받을 수 없다고 말하는 sink보다 나쁘다.
- **reader의 버퍼에 비해 너무 긴 줄은 에러다, 절단된 줄이 아니다.** 온전한 것처럼 돌려받은 절단된 줄은 호출자가 탐지할 방법이 없는 손상이다. 버퍼는 당신 것이다; 예상하는 입력에 맞춰 크기를 정하라.

- **부분 쓰기는 얼버무릴 실패가 아니라 하나의 사실이다.** `write_fn`은 `proven_result_size_t`를 반환한다: sink가 얼마나 받았는지, 그리고 무엇이 잘못되었는지. 파이프, 소켓, 또는 차오르는 디스크는 정말로 당신의 6000 바이트 중 4096바이트를 받아들이고 나서 실패한다. 그리고 "전부 소비하거나 실패한다"고 말하는 트레이트는 그런 sink를 올바르게 작성하는 것을 그저 불가능하게 만든다. `proven_writer_write`는 여전히 전부-아니면-전무를 뜻한다(루프를 돈다); `proven_writer_write_partial`은 당신이 얼마나 진행했는지 봐야 할 때 있다. 버퍼드 writer는 sink가 받지 않은 꼬리만 보관한다—첫 버전은 버퍼 전체를 보관하고 다시 보냈으며, 그래서 실패하는 sink가 받아들인 접두부를 두 번 받았다.
- **한 번 실패한 writer는 실패한 채로 남는다.** writer가 바이트를 잃은 뒤에는—sink가 죽은 버퍼드 writer, 오버플로한 고정 버퍼—그것이 만들어내던 스트림에 수신자가 볼 수 없는 구멍이 생기므로, 이후의 모든 쓰기와 `flush`는 에러를 반환한다. 들어갈 수 있는 더 짧은 청크조차 거부된다: 그것을 쓰면 구멍 뒤에 놓여, 결과가 완전한 출력처럼 보일 것이기 때문이다. `clear()`는 없다: 복구 스토리가 있다면 그것은 이 writer가 괜찮은 척하는 것이 아니라 새 sink 위의 새 writer를 수반한다. (이전에는 실패한 쓰기 이후 `flush`가 `PROVEN_OK`를 답했다—버퍼가 비어 있어서 실패할 것이 남지 않았기 때문이다—그리고 "쓰고, 쓰고, 쓰고, `flush`를 검사한다"는, 거의 모두가 버퍼드 writer를 쓰는 방식이 가득 찬 디스크에서 성공을 보고했다.)

reader의 규칙은 그 거울상이다: 실패한 읽기는 에러이지, 결코 파일의 끝이 아니다. `proven_reader_read`는 source가 깨질 때 깨끗한 0바이트 EOF가 아니라 `PROVEN_ERR_IO`를 반환한다—디스크 에러로 잘려나간 파일과 그냥 끝난 파일은, 둘을 구별할 수 없는 호출자에게는 같은 것이며, 그 중 하나만 안전하게 처리할 수 있기 때문이다.

10,000줄을 `stdout`으로 내보내며 측정한 비용:

	write() 시스템 콜	malloc()
<code>proven_println</code>	10,000	0 (이전엔 10,000)
버퍼드 writer, 호출자 메모리 8 KiB	24	0

`malloc()` 열은 512바이트 스택 버퍼에 들어가는 줄, 즉 일반적인 경우에 대한 값이다. 그보다 긴 줄은 거부되는 대신 그 한 번의 호출에 한해 힙(heap)으로 물러난다.

`proven_println`은 의도적으로 여전히 줄당 하나의 시스템 콜이다: 그것을 버퍼링하려면 숨겨진 전역 상태가 필요할 것이다. 24를 원하는 호출자는 버퍼드 writer를 만들고 그렇게 하겠다고 말한다.

완성 예제: 하나의 직렬화기, 세 개의 목적지, 그리고 되읽기

테스트 스위트에 의해 컴파일되고 실행됨. `render_row`가 자신의 바이트가 어디로 가는지 모른다는 점에 주목하라—그것이 바로 요점 전부다.

```
/*
 * 쓰는 쪽과 읽는 쪽. "바이트가 어디로 가는가" 를 위한 인터페이스 하나와 "바이트가
 * 어디서 오는가" 를 위한 인터페이스 하나.
 *
 * 요점은 아래 코드 - render_row - 가 자기가 문자열로 쓰는지, 고정 버퍼로 쓰는지, 파일로
 * 쓰는지 모르고 신경도 쓰지 않는다는 것이다. 예전에는 그럴 수 없었다. 형식화기가 받느
 * 그릇은 proven_u8str_t 하나뿐이었다.
 */

/* 직렬화기 하나. 목적지가 아니라 그릇을 받는다. */
static proven_err_t render_row(proven_writer_t w, int id, const char *name) {
    proven_fmt_result_t r = proven_fprintln(w, "{:>4} | {}", PROVEN_ARG(id), PROVEN_ARG(name));
    return r.err;
}

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* --- 같은 코드로, 늘어나는 문자열에 ----- */
    proven_result_u8str_t s = proven_u8str_create(alloc, 16);
```

```

EXAMPLE_REQUIRE(proven_is_ok(s.err), "string create");

proven_writer_u8str_t s_state;
proven_writer_t to_string = proven_writer_from_u8str(&s_state, &s.value, alloc);
EXAMPLE_REQUIRE(proven_is_ok(render_row(to_string, 7, "ada")), "render into a string");

printf("into a string:\n%s", proven_u8str_as_cstr(&s.value));
proven_u8str_destroy(alloc, &s.value);

/* --- 같은 코드로, 여러분이 소유한 기억에: 할당은 0 ----- */
proven_byte_t fixed[64];
proven_writer_buf_t b_state = { .buf = { .ptr = fixed, .size = sizeof fixed } };
proven_writer_t to_buffer = proven_writer_from_buffer(&b_state);
EXAMPLE_REQUIRE(proven_is_ok(render_row(to_buffer, 8, "grace")), "render into a buffer");
EXAMPLE_REQUIRE(b_state.len > 0, "the buffer received the row");

/* 가득 찬 버퍼는 *거부한다*. 자르지 않는다. 여러분 자료의 끝을 조용히 버리는 그릇은
 * 받을 수 없다고 말하는 그릇보다 나쁘다. */
proven_byte_t tiny[4];
proven_writer_buf_t t_state = { .buf = { .ptr = tiny, .size = sizeof tiny } };
proven_writer_t to_tiny = proven_writer_from_buffer(&t_state);
EXAMPLE_REQUIRE(proven_writer_write_str(to_tiny, PROVEN_LIT("far too long")) == PROVEN_ERR_OUT_OF_BOUNDS,
    "a full buffer refuses rather than truncating");
EXAMPLE_REQUIRE(t_state.overflowed, "and it records that it did");

/* --- 같은 코드로, 파일에, 버퍼를 두고 ----- */
/*
 * 버퍼는 아래나에서와 똑같이 *여러분이* 대는 기억이다. 이 라이브러리에는 숨은 전역
 * 상태가 없으므로 종료 시점에 대신 흘려 줄 수 없다 - 그래서 버퍼가 스킵을 벗어나기
 * 전에 반드시 flush 해야 한다. 그 대가로 로그 경로에서는 할당이 전혀 일어나지 않는다.
 * 여기서 만 줄은 malloc 0 번에 write 시스템 호출 스물 몇 번이지만, proven_println 을
 * 만 번 부르면 시스템 호출 10,000 번이다.
 */
proven_u8str_view_t path = PROVEN_LIT("example_stream_rows.txt");
proven_result_file_t f = proven_fs_open(alloc, path,
    (proven_fs_mode_t)(PROVEN_FS_WRITE | PROVEN_FS_CREATE | PROVEN_FS_TRUNC));
EXAMPLE_REQUIRE(proven_is_ok(f.err), "open the output file");
proven_file_t file = f.value;

proven_byte_t out_buf[4096];
proven_writer_buffered_t w_state;
proven_writer_t to_file = proven_writer_buffered(&w_state,
    proven_writer_from_file(&file),
    (proven_mem_mut_t){ .ptr = out_buf, .size = sizeof out_buf });

for (int i = 0; i < 3; ++i) {
    EXAMPLE_REQUIRE(proven_is_ok(render_row(to_file, i, "row")), "render into the file");
}
EXAMPLE_REQUIRE(proven_is_ok(proven_writer_flush(to_file)),
    "flush: nothing is written until you say so");
/* 그리고 close - 쓰기가 도착하지 못했다고 알려 줄 수 있는 마지막 자리다. */
EXAMPLE_REQUIRE(proven_is_ok(proven_fs_close(file)), "closing the written file");

/* --- 한 줄씩 되읽기 ----- */
/* 파일을 한 줄씩 읽는 것은 예전에는 아예 되지 않았다. 길은 파일 전체를 기억에 올려
 * 손으로 자르는 것뿐이었다. */
proven_result_file_t rf = proven_fs_open(alloc, path, PROVEN_FS_READ);
EXAMPLE_REQUIRE(proven_is_ok(rf.err), "reopen for reading");
proven_file_t rfile = rf.value;

```

```

proven_byte_t in_buf[128];
proven_reader_buffered_t r_state;
(void)proven_reader_buffered(&r_state, proven_reader_from_file(&rfile),
                             (proven_mem_mut_t){ .ptr = in_buf, .size = sizeof in_buf });

int lines = 0;
for (;;) {
    proven_result_u8str_view_t line = proven_reader_read_line(&r_state);
    if (line.err == PROVEN_ERR_EOF) break;
    EXAMPLE_REQUIRE(proven_is_ok(line.err), "read a line");
    /* 그 뷰는 읽는 쪽의 버퍼 *안*을 가리키고, 다음 호출 전까지만 쓸 수 있다. 그보다
     * 오래 살아야 한다면 복사할 것. */
    printf("line %d: %.*s\n", lines, (int)line.val.size, (const char *)line.val.ptr);
    ++lines;
}
EXAMPLE_REQUIRE(lines == 3, "three rows in, three lines out");

(void)proven_fs_close(rfile);
(void)proven_fs_remove(alloc, path);

return EXAMPLE_OK();
}

```

당신이 보유하는 구조체

두 핸들은 값으로 전달되며 저렴하다:

```

typedef struct {
    void *ctx;
    proven_result_size_t (*write_fn)(void *ctx, proven_mem_view_t chunk);
    proven_err_t (*flush_fn)(void *ctx); /* may be NULL: this sink holds nothing back */
} proven_writer_t;

typedef struct {
    void *ctx;
    proven_result_size_t (*read_fn)(void *ctx, proven_mem_mut_t dest);
} proven_reader_t;

```

write_fn은 그 뒤에 실패하더라도 얼마나 나갔는지 보고한다. 그리고 그것은 사소한 배려가 아니다: 파이프나 가득 찬 디스크로의 쓰기는 정말로 일부 바이트를 내보내고 나서 실패한다. 더 깔끔한 "전부 아니면 전무"라는 거짓말 위에 세워진 버퍼드 writer는 실패 시 버퍼 전체를 보관하고 다음 flush에 다시 보냈다—6000바이트 페이로드가 처음 4096바이트가 중복된 채로 10,096바이트로 도착했다. 데이터를 잃는 것은 나쁘다; 조용히 두 배로 만드는 것은 더 나쁘다. 수신자가 알 수 없기 때문이다.

그 밖의 모든 것은 **호출자 소유 상태**다—proven_writer_buf_t, proven_writer_u8str_t, proven_writer_buffered_t, proven_reader_view_t, proven_reader_buffered_t. 이들은 아무것도 할당하지 않고, destroy가 없으며, 핸들이 그 안을 가리키고 있는 동안 복사되거나 이동되어서는 안 된다.

주의사항, 그리고 무엇이 잘못되는가

결코 flush되지 않는 버퍼드 writer는 결코 일어나지 않은 출력이다. 숨겨진 상태가 없으므로, 당신을 위해 그것을 flush해 줄 소멸자도 없다—그리고 여기 있는 어떤 것도 atexit 핸들러를 등록하지 않는다. 당신의 프로세스를 소유하는 라이브러리는 당신이 추론할 수 없는 라이브러리이기 때문이다.

잘못됨(Wrong):

```

proven_writer_buffered_t st;
proven_writer_t w = proven_writer_buffered(&st, inner, buf);
(void)proven_fprintln(w, "the important line");
return; /* wrong: the buffer dies with the frame, and so does the line */

```

올바름(Correct)—버퍼나 내부 sink가 사라지기 전에 flush하라:

```

proven_byte_t buf[256];
proven_sysio_out_t out;
proven_writer_t w = proven_sysio_stdout_buffered(&out,
    (proven_mem_mut_t){ .ptr = buf, .size = sizeof buf });
(void)proven_fprintln(w, "the important line");
(void)proven_writer_flush(w);          /* now it has happened */

```

reader가 당신에게 건네는 줄은 그 버퍼 안을 가리킨다. 그것은 다음 호출 전까지만 유효하다. 그것이 백만 줄을 읽는 데 백만 번의 할당이 아니라 하나의 버퍼가 들게 하는 원리다—그리고 당신이 그것을 붙들어 두는 순간 dangling 포인터다.

잘못됨(Wrong):

```

proven_u8str_view_t lines[100];
for (int i = 0; i < 100; ++i) {
    proven_result_u8str_view_t ln = proven_reader_read_line(&st);
    lines[i] = ln.val;          /* wrong: every entry aliases the SAME buffer, and the
                                next read_line overwrites what the last one returned */
}

```

올바름(Correct): 다시 호출하기 전에, 보관해야 하는 바이트를 복사하라(proven_u8str_t로, 아레나로, 어디로든).

flush는 durability 장벽이 아니다. proven_writer_flush는 버퍼드 writer의 바이트를 그 뒤의 것으로 밀어 넣는다; 파일의 바이트를 디스크에 올리는 것은 proven_fs_sync다. 이 둘은 서로 다른 연산이며, 한 단어가 정직하게 둘 모두를 뜻할 수는 없었다—그것이 바로 둘 다라고 주장하면서 어느 쪽도 아니었던 옛 proven_sysio_flush가 사라진 이유다.

버퍼보다 긴 줄은 절단이 아니라 거부된다. PROVEN_ERR_OUT_OF_BOUNDS—그리고 reader는 그 줄에서 멈춘 채로 남는다: 재동기화(resync)는 없다. 답을 수 없는 줄은 그 바이트를 어떻게 할지 결정하지 않고서는 건너뛸 수 있는 줄이 아니기 때문이다. 예상하는 입력에 맞춰 버퍼의 크기를 정하라. (버퍼를 정확히 채우는 줄은 괜찮다: 개행이 함께 들어가지 않아도 되고, 개행이 전혀 없는 마지막 줄도 마찬가지다.)

표준 스트림

stream.h에는 writer, reader, 버퍼드 writer, 그리고 라인 reader가 있다. sysio.h에는 stdin, stdout, stderr가 있다. 이 둘이 서로 소개되기 전까지는, 두 가지가 그저 불가능했다—그리고 한 호출은 거짓말이었다.

stdin을 한 줄씩 읽는 방법이 없었다. 프로그램이 stdin으로 하는 가장 흔한 일인데, 선택지는 토큰 스캐너이거나, 결코 끝나지 않을 수도 있는 스트림 전체를 읽는 것이었다. 그 다리(bridge)가 그것을 고친다: 표준 핸들이 호출자 소유 저장소에 주차되므로, 라인 reader에게 가리킬 안정적인 무언가가 생긴다.

```

proven_byte_t buf[4096];
proven_sysio_lines_t lines;
if (proven_is_ok(proven_sysio_stdin_lines(&lines, (proven_mem_mut_t){ .ptr = buf, .size = sizeof buf }))) {
    for (;;) {
        proven_result_u8str_view_t line = proven_sysio_read_line(&lines);
        if (line.err == PROVEN_ERR_EOF) break;
        if (!proven_is_ok(line.err)) break; /* OUT_OF_BOUNDS: a line longer than `buf` */
        /* `line.val` points INTO `buf` and is valid only until the next call. */
        proven_println("{ }", PROVEN_ARG(line.val));
    }
}

```

이것은 라인 reader의 속성을 물려받는데, 이는 두 번째 것을 쓰지 않는 것의 요점이다: view는 무할당, "\r\n"이 처리되고, 후행 개행이 없는 마지막 줄도 여전히 반환되며, 당신의 버퍼보다 긴 줄은 PROVEN_ERR_OUT_OF_BOUNDS다—결코 조용히 절단된 줄이 아니다.

포매터를 표준 스트림으로 겨눌 수 없었다. proven_fprintln은 writer를 받는데; stdout은 그것이 아니었다. 이제는 그렇고, 버퍼드 writer일 수도 있다—그래서 천 개의 작은 줄이 천 번이 아니라 한 번의 시스템 콜로 든다.

proven_sysio_stdout_writer(&st)	stdout 위의 언버퍼드 writer. 모든 쓰기가 하나의 write 시스템 콜이다.
proven_sysio_stderr_writer(&st)	stderr에 대해 같은 것—에러에 대해 당신이 원하는 것이다: 다음 코드 줄이 실행되기 전에 그것이 나간다.
proven_sysio_stdin_reader(&st)	stdin 위의 reader.
proven_sysio_stdout_buffered(&out, buf)	당신이 소유하는 버퍼 위의 버퍼드 writer 뒤에 놓인 stdout.
proven_sysio_file_buffered(&out, file, buf)	임의의 열린 파일에 대해 같은 것.

그리고 이제 flush는 무언가를 뜻한다. proven_sysio_flush는 예전에 존재하지 않는 버퍼를 flush한다고 주장했다: POSIX에서는 no-op, Windows에서는 디스크 sync. 그것은 **삭제되었다**. 버퍼드 writer의 바이트를 OS로 밀어 넣는 것은 proven_writer_flush이고, OS의 바이트를 디스크로 밀어 넣는 것은 proven_fs_sync다. 이 둘은 서로 다른 연산이며 이제는 그렇다고 말한다.

당신은 버퍼드 writer를 flush해야 한다. 버퍼가 차거나 당신이 flush하기 전까지는 아무것도 터미널에 도달하지 않으며, 이 라이브러리의 어떤 것도 당신 몰래 그것을 하기 위해 atexit 핸들러를 등록하지 않는다—당신의 프로세스를 소유하는 라이브러리는 당신이 추론할 수 없는 라이브러리이기 때문이다. 결코 flush되지 않는 버퍼드 출력은 결코 일어나지 않은 출력이다. 직접 호출들 (proven_print, proven_println, proven_eprint)은 바로 이 이유로 언버퍼드로 남는다: 그것들이 쓰는 것은 반환되기 전에 이미 나가는 중이다.

당신이 보유하는 구조체

```
typedef struct { proven_file_t file; } proven_sysio_std_t;
/* Storage for a standard handle, so a writer or reader has something stable to
   point at. proven_writer_from_file takes a proven_file_t * and the file must
   outlive the writer - so it cannot be a temporary. This is that storage. */

typedef struct {
    proven_sysio_std_t      std;
    proven_writer_buffered_t buffered;
} proven_sysio_out_t;      /* a buffered writer over a standard stream or a file */

typedef struct {
    proven_sysio_std_t      std;
    proven_reader_buffered_t buffered;
} proven_sysio_lines_t;    /* a line reader over a standard stream or a file */
```

셋 모두 **호출자 소유 상태**다.

주의사항, 그리고 무엇이 잘못되는가

이 상태 구조체들은 자기 자신을 가리키는 포인터를 담는다. 당신이 돌려받는 writer는 당신이 넘긴 구조체 안의 &st->std.file을 주소로 삼는다. 구조체를 복사하거나 값으로 반환하면, writer는 여전히 원본을 가리킨다—그것은 죽은 프레임일 수 있다.

잘못됨(Wrong)—그리고 한 감사에서 정확히 이것이 heap-use-after-free로 재현되었다:

```
proven_sysio_out_t out;
proven_writer_t w = proven_sysio_stdout_buffered(&out, buf);

proven_sysio_out_t copy = out;      /* wrong: `w` still points into `out` */
/* ... `out` goes out of scope ... */
(void)proven_writer_write_str(w, PROVEN_LIT("boom")); /* writes through dead storage */
```

유일한 예외는 proven_sysio_lines_t다: proven_sysio_read_line은 매 호출마다 그것을 다시 바인딩하므로, 라인 reader는 이동될 수 있다. 그것은 의도된 배려인데, 그것이 상태를 포인터로 받기 때문이다—"재배치 가능"을 뜻하는 형태—그리고 라이브러리는 약속의 형태를 한 함정을 놓아서는 안 되기 때문이다.

0으로 초기화된 proven_file_t는 invalid 핸들이 아니다—POSIX에서 그것은 fd 0, 즉 stdin이다. 라이브러리는 당신이 채워 넣기를 잊은 핸들과 정당하게 fd 0을 가리키는 핸들을 구별할 수 없다.

```
proven_sysio_out_t out;
proven_file_t f = {0}; /* wrong: this is stdin */
proven_writer_t w = proven_sysio_file_buffered(&out, f, buf); /* writes to fd 0 */
```

버퍼드 stdout과 언버퍼드 stderr는 당신이 쓴 순서대로 뒤섞이지 않는다. 당신이 버퍼링하는 것은 당신의 버퍼에 앉아 있는 동안 stderr는 곧장 나간다. 당신의 출력 뒤에 나타나야 하는 에러를 출력하기 전에 flush 하라.

난수, 용도별

단일한 "random"은 없다. 동일해 보이지만 그렇지 않은 두 가지 작업이 있다:

당신의 작업	사용	이유
키, 토큰, nonce—공격자가 추측해서는 안 되는 모든 것.	proven_random_bytes, 또는 그것으로 시드된 proven_chacha_rng_t.	오직 암호학적 source만이 추측 불가능하다.
같지만, OS가 없는 타깃에서.	보드 자체의 엔트로피로 시드된 proven_chacha_rng_t.	ChaCha20은 순수 산술이다; OS가 필요 없다. 그것은 오직 그 시드만큼만 추측 불가능하다.
시뮬레이션, 테스트, 게임, 표본추출.	proven_xoshiro256ss_t.	빠르고, 재현 가능하다 : 같은 시드가 같은 실행을 재생하며, 그것이 실패하는 테스트를 디버깅 가능하게 만든다.
범위 내의 수, 셔플, [0,1)의 float.	proven_rng_below, proven_rng_range, proven_rng_f64, proven_rng_shuffle—어떤 source 위에서든.	% n은 편향되어 있고, 모두가 그림에도 그것을 쓴다. 이것들은 그렇지 않다.

두 요구사항은 정면으로 대립한다. 재현 가능이란 예측 가능을 뜻하고, 예측 가능은 정확히 토큰이 그래서 안 되는 것이다: proven_xoshiro256ss_t의 몇 개 출력이 그 전체 상태를, 따라서 그것이 앞으로 낼 모든 수를 드러낸다. 그것은 재생해야 하는 시뮬레이션에는 기능이고 세션 토큰에는 재앙이다—그래서 둘은 혼동될 수 없는 이름을 지니며, 선택은 무언가가 어떻게 시드되었는가에 묻히는 대신 호출 지점에서 가시적이다.

트레이트는 실패하지 않는다(infallible). proven_rng_t는 무작위 바이트의 source이며, valid한 것에서 뽑는 것은 실패할 수 없다. 그것은 단순화가 아니라, 실패가 어디로 갔는지다. 운영체제에 엔트로피를 요청하는 것은 실패할 수 있으므로, 그 실패는 정확히 한 곳—시딩—에 국한되며, 당신은 그것을 시작 시점에 한 번 검사한다. 하류의 모든 draw는 전역적(total)이다.

proven_random_bytes(buf, len)	OS CSPRNG. 실패 시 false를 반환한다; 그러면 buf를 사용하지 말라. len == 0은 성공적인 no-op이다.
proven_random_u64()	OS로부터의 강한 한 워드, 실패 시 0.
proven_chacha_rng_seed_from_entropy(&g)	엔트로피 source로부터 암호학적 생성기를 시드. 이것이 실패할 수 있는 호출이다.
proven_random_set_source(fn, ctx)	엔트로피 source를 설치. 호스티드 타깃에는 OS가 이미 설치되어 있다; 베어메탈 타깃은 보드의 TRNG를 설치한다.
proven_chacha_rng_seed(&g, seed32)	당신이 공급하는 32바이트로부터 시드—보드의 하드웨어 엔트로피 source. 결코 시계로부터는 아니다.

proven_xoshiro256ss_seed(&g, seed)	재현 가능 생성기를 시드. 시드 0도 괜찮다: SplitMix64를 통해 확장된다.
------------------------------------	--

엔트로피는 어디서 오는가

위의 모든 것은 순수 산술이다—생성기들, 헬퍼들, 그리고 proven_random_bytes 자체까지. 플랫폼에 따라 다른 것은 그 뒤의 **엔트로피 source**인데, 그것이 프로그램이 스스로 계산할 수 없는 유일한 것이기 때문이다.

- **호스티드:** OS CSPRNG가 당신을 위해 설치되어 있다—Linux의 getrandom, BSD와 macOS의 getentropy, Windows의 BCryptGenRandom, 그리고 이들 중 어느 것도 없는 곳의 /dev/urandom. 당신은 아무것도 호출하지 않는다.
- **베어메탈:** 당신이 하나를 설치하기 전까지는 source가 없다. 보드는 실제 엔트로피를 가지고 있다—온칩 TRNG, 링 오실레이터, ADC의 노이즈 플로어—그리고 라이브러리는 그것이 어디 있는지 알 수 없다.

```
/* On a board: hand the library its hardware entropy, once, at startup.
 * (A listing, not a fragment: it defines a function, and `hardware_rng_read` is
 * whatever your SoC calls its entropy register.) */
static bool board_trng(void *ctx, void *buf, proven_size_t len) {
    (void)ctx;
    /* read the SoC's entropy register into buf; return false if it is not ready */
    return hardware_rng_read(buf, len);
}

proven_random_set_source(board_trng, NULL);

/* From here everything above works unchanged - including the one call that turns a few
 * hundred bytes of hardware entropy into an endless cryptographic stream. */
proven_chacha_rng_t g;
if (!proven_chacha_rng_seed_from_entropy(&g)) {
    /* the TRNG was not ready. The generator is INERT - it yields zeros and an invalid
     * trait - so ignoring this does not get you plausible-looking bytes. */
}
```

설치된 source가 없으면 proven_random_bytes는 **false**를 반환한다. 그것은 시계로 시드된 PRNG로 폴백하지 않는다. 그것은 성공처럼 보이지만 아무것도 보고하지 않는 보안 구멍이기 때문이다—거부는 호출자가 처리할 수 있는 하나의 사실이다.

의도적으로 내장된 **RDRAND / RNDR 백엔드는 없다**. 호스티드 타깃에서는 OS가 이미 CPU의 명령을 자신의 풀에 섞어 넣으므로, 그것을 직접 호출하는 것은 아무것도 얻지 못하면서 그 섞임을 잃게 하고; 유일한 source로 쓰이는 원시 하드웨어 명령은 사람들이 10년간 논쟁해 온 바로 그 방식이다. 원한다면, 그것은 이 후 뒤로 네 줄이다—그러면 그 선택은 가시적으로 당신 것이다.

당신이 보유하는 구조체

셋 모두 **호출자 소유 상태**: 아무것도 할당하지 않고, 파괴할 것이 없으며, **하나를 복사하면 그 시퀀스를 복제한다**.

```
typedef struct { const proven_rng_vtable_t *vt; void *ctx; } proven_rng_t;
/* The trait: two pointers, held by value. `ctx` points at one of the generators
   below, which must outlive it. Drawing from a VALID one cannot fail - that is
   the whole design: the failure lives in seeding, not in drawing. */

typedef struct { proven_u64 s[4]; } proven_xoshiro256ss_t;
/* 256 bits of state. Reproducible, and NOT secret-grade. */

typedef struct {
    proven_u32    state[16]; /* the ChaCha state: constants, key, counter, nonce */
    proven_byte_t block[64]; /* the keystream block currently being handed out */
    proven_size_t used;      /* how much of it is spent */
    proven_u32    seeded;    /* set only by seeding. A zero-initialised struct is
```

the shape of "never seeded", and must stay inert. */

```
} proven_chacha_rng_t;
```

레퍼런스

API	의도	반환
proven_random_bytes(buf, len)	엔트로피 source로부터 채움(기본은 OS). 실패할 수 있는 유일한 호출.	bool. false이면 buf는 미지정이며 사용해서는 안 된다. len == 0은 성공한다.
proven_random_u64()	같은 source로부터의 강한 한 워드.	proven_u64, 실패 시 0—그것도 valid한 draw이므로, 둘을 구별해야 할 때는 proven_random_bytes를 사용하라.
proven_random_set_source(fn, ctx)	엔트로피 source를 설치. 호스티드 타킷에는 불필요; 이것이 보드가 자신의 TRNG를 넘겨주는 방법이다.	void.
proven_xoshiro256ss_seed(&g, seed)	재현 가능 생성기를 시드. 어떤 시드든 괜찮다—0조차; SplitMix64를 통해 확장된다.	void.
proven_xoshiro256ss_next(&g)	다음 워드. 핫 패스: 트레이트를 통해서가 아니라 직접 호출하라.	proven_u64.
proven_xoshiro256ss_rng(&g)	헬퍼를 위해 proven_rng_t로 본다.	proven_rng_t.
proven_chacha_rng_seed(&g, seed32)	당신이 공급하는 32바이트의 실제 엔트로피 로부터 암호학적 생성기를 시드.	void.
proven_chacha_rng_seed_from_entropy(&g)	설치된 source로부터 시드. 이것을 검사하라.	bool. false이면 생성기는 INERT 상태로 남는다—0을 넣고 invalid한 트레이트를 넣는다.
proven_chacha_rng_next/_fill	draw. 일단 시드되면 실패할 수 없다.	proven_u64 / void.
proven_chacha_rng(&g)	proven_rng_t로 본다.	proven_rng_t—생성기가 성공적으로 시드된 적이 없으면 invalid .
proven_rng_u64(rng) / proven_rng_fill(rng, buf, len)	어느 생성기로부터든 트레이트를 통해 draw.	proven_u64 / void. invalid source에는 0 / no-op.
proven_rng_below(rng, bound)	[0, bound)에서 균일, 무편향 .	proven_u64; bound == 0일 때 0.
proven_rng_range(rng, lo, hi)	[lo, hi)에서 균일, 포함적. 전체 INT64_MIN..INT64_MAX 범위도 오버플로하지 않는다.	proven_i64; hi < lo이면 lo.
proven_rng_f64(rng)	[0, 1)에서 균일. 53비트; 결코 1.0을 반환하지 않는다.	double.
proven_rng_shuffle(rng, base, count, elem_size)	무편향 Fisher-Yates 순열, 제자리(in place).	void.

주의사항, 그리고 무엇이 잘못되는가

proven_xoshiro256ss_t로 결코 비밀을 생성하지 말라. 그것은 예측 가능하기 때문에 빠르다: 그 출력 몇 개가 256비트 상태 전체를, 그리고 그 상태에서부터 그것이 앞으로 낼 모든 수를 드러낸다. 두 생성기는 정확히 이 이유로 혼동될 수 없는 이름을 지닌다.

잘못됨(Wrong)—공격자가 몇 개를 지켜본 뒤 계산할 수 있는 세션 토큰:

```
proven_xoshiro256ss_t g;
proven_xoshiro256ss_seed(&g, 12345);
proven_u64 session_token = proven_xoshiro256ss_next(&g); /* wrong: predictable */
```

암호학적 생성기를 결코 시계, 카운터, 또는 시리얼 번호로 시드하지 말라. ChaCha20은 정확히 그 시드만큼만 추측 불가능하다. 시계에서 유도된 시드는 완벽하게 무작위처럼 보이지만 그렇지 않은 스트림을 낳는다—그것은 명백한 실패보다 나쁘다. 아무것도 그것을 보고하지 않기 때문이다.

```
proven_byte_t seed[32] = { 0 };
memcpy(seed, &now_ns, sizeof now_ns); /* wrong: ~20 bits of real entropy, and guessable */
proven_chacha_rng_seed(&g, seed);
```

시딩을 검사하라. 그것은 여기서 실패할 수 있는 유일한 것이며, 바로 그래서 그것을 무시하는 것이 솔깃하다. 무시하면 생성기는 inert 상태가 되어 당신에게 0을 건넨다—설계상 그럴듯한 값이 아니라 가시적으로 죽은 값이다.

잘못됨(Wrong):

```
proven_chacha_rng_t g;
proven_chacha_rng_seed_from_entropy(&g); /* wrong: the bool was the point */
proven_chacha_rng_fill(&g, key, 32); /* key is now 32 zero bytes */
```

올바름(Correct):

```
proven_chacha_rng_t g;
if (!proven_chacha_rng_seed_from_entropy(&g)) {
    /* No entropy. There is nothing safe to do here except refuse to continue. */
} else {
    proven_byte_t key[32];
    proven_chacha_rng_fill(&g, key, sizeof key); /* cannot fail: it is seeded */
}
```

% n은 편향되어 있고, 모두가 그럼에도 그것을 쓴다. n이 2^{64} 를 나누어떨어지게 하지 않는 한 낮은 값들이 더 자주 나온다—어쩌다 하는 점검에서는 보이지 않지만, 셔플이나 표본추출에서는 실재한다.

```
proven_u64 die = proven_rng_u64(rng) % 6 + 1; /* wrong: 1 and 2 are slightly likelier */
```

올바름(Correct): `proven_rng_below(rng, 6) + 1.`

시드된 생성기를 복사하지 말라—그 스트림을 복제하려는 의도가 아니라면. 하나에서 복사된 두 개의 "독립적인" 생성기는 동일한 출력을 낳는다—그것은 시뮬레이션 재생에는 기능이고 두 개의 토큰을 발급하는 데는 재앙이다.

테스트 스위트에 의해 컴파일되고 실행됨:

```
/*
 * 쓰임새로 나눈 난수. "난수" 하나는 없다. 똑같이 생겼지만 같지 않은 할 일이 둘 있고,
 * 잘못 고르는 것이 위험의 전부다.
 *
 * 키, 토큰, 논스 - 공격자가 알아맞히면 안 되는 것 - 에는 *암호용* 난수원이 필요하다.
 * 시뮬레이션, 시험, 게임에는 *재현 가능한* 것이 필요하다. 다시 돌려 볼 수 없는 실패한
 * 실행은 디버깅할 수 없는 실패한 실행이기 때문이다. 두 요구는 정면으로 부딪친다.
 * 재현 가능하다는 것은 예측 가능하다는 뜻이고, 예측 가능한 것이야말로 토큰이 되어서는
 * 안 되는 것이다. 그래서 라이브러리는 둘에 다른 이름을 주고, 선택은 씨앗을 어떻게
 * 뿌렸는지에 묻히는 대신 여기 호출 자리에서 눈에 보인다.
 */

int main(void) {
    /* ---- 할 일 1: 비밀. OS 의 CSPRNG - 그리고 난수가 실패할 수 있는 유일한 자리. ---- */
    proven_byte_t key[32];
    EXAMPLE_REQUIRE(proven_random_bytes(key, sizeof key),
        "the OS must give us strong bytes on a hosted platform");
```

```

/* ---- 할 일 2: 암호용 바이트를 아주 많이, 또는 OS 가 없는 보드에서 조금이라도.
 * ChaCha20 은 순수한 산술이다. 진짜 엔트로피로 한 번 씨를 뿌리면 그 뒤로는 운영체제가
 * 필요 없다 - 뽑을 때마다 시스템 호출을 하지 않고, 베어메탈에서도 돈다.
 * 씨 뿌리기가 실패할 수 있는 *유일한* 걸음이므로, 확인해야 할 것도 그것 하나다. ---- */
proven_chacha_rng_t crypto;
EXAMPLE_REQUIRE(proven_chacha_rng_seed_from_entropy(&crypto), "seed the CSPRNG from the OS, once");

proven_byte_t token[16];
proven_chacha_rng_fill(&crypto, token, sizeof token); /* 실패할 수 없다: 이미 씨가 뿌려져 있다 */

/* ---- 할 일 3: *재현되는* 실행. xoshiro256** 은 빠르고 씨앗에서 똑같이 다시
 * 재생된다. 그것이 실패한 시뮬레이션을 디버깅할 수 있게 만든다. 비밀급이 *아니다* -
 * 출력 몇 개면 상태 전체가 드러난다. 토큰을 만들라고 건네는 일은 절대 없어야 한다. ---- */
proven_xoshiro256ss_t sim;
proven_xoshiro256ss_seed(&sim, 12345);

proven_xoshiro256ss_t replay;
proven_xoshiro256ss_seed(&replay, 12345);
EXAMPLE_REQUIRE(proven_xoshiro256ss_next(&sim) == proven_xoshiro256ss_next(&replay),
    "the same seed replays the same run - that is the whole point");

/* ---- 도우미들은 proven_rng_t 특성을 통해 *어떤* 난수원 위에서도 돈다. ---- */
proven_rng_t rng = proven_xoshiro256ss_rng(&sim);

/* 범위 안의 수. `rng_u64() % 6` 은 모두가 쓰는 것이고, 상한이 2^64 를 나누어떨어지게
 * 하지 않는 한 *치우쳐* 있다 - 작은 값이 더 자주 나온다. 이것은 그렇지 않다. */
for (int i = 0; i < 100; ++i) {
    proven_u64 die = proven_rng_below(rng, 6) + 1;
    EXAMPLE_REQUIRE(die >= 1 && die <= 6, "a die roll is 1..6, uniformly");
}

proven_i64 temperature = proven_rng_range(rng, -40, 85);
EXAMPLE_REQUIRE(temperature >= -40 && temperature <= 85, "an inclusive range, both ends");

double p = proven_rng_f64(rng);
EXAMPLE_REQUIRE(p >= 0.0 && p < 1.0, "a double in [0, 1) - never 1.0");

/* 치우치지 않은 섞기: 위의 치우치지 않은 인덱스 위에서 도는 피셔-예이츠. 이 반복문의
 * `% n` 판은 어떤 순서를 눈에 띄게 더 좋아한다. */
int deck[10];
for (int i = 0; i < 10; ++i) deck[i] = i;
proven_rng_shuffle(rng, deck, 10, sizeof deck[0]);

int sum = 0;
for (int i = 0; i < 10; ++i) sum += deck[i];
EXAMPLE_REQUIRE(sum == 45, "a shuffle is a permutation: every card is still there, once");

/* 암호용 생성기도 같은 특성을 만족하므로, 고르기만 하면 되는 것이 아니라 알아맞힐 수
 * 없어야 할 때 같은 도우미를 그대로 쓸 수 있다. */
proven_rng_t secure = proven_chacha_rng(&crypto);
proven_u64 unguessable_index = proven_rng_below(secure, 1000);
EXAMPLE_REQUIRE(unguessable_index < 1000, "the helpers do not care which source they draw from");

(void)token;
(void)key;
return EXAMPLE_OK();
}

```

실전 예제: 정전이 나도 파일이 깨지지 않게 교체하기

파일을 제자리에서 덮어쓰는 방식에는, 시험에서는 결코 보이지 않고 운영에서는 언젠가 반드시 보이는 실패 방식이 있다. 쓰는 도중에 전원이 나가고, 남은 파일은 예전 것도 새 것도 아니게 된다. 해법은 오래된 네 단계 절차이며, 네 단계 모두가 제 몫을 한다.

1. 새 내용을 진짜 파일 옆의 임시 파일에 쓴다.
2. `proven_fs_sync()` — 새 파일의 바이트가 운영체제 캐시가 아니라 저장 장치에 도달한다.
3. `proven_fs_rename()` — 이름이 한 번의 나눌 수 없는 단계로 새 파일로 넘어간다. 읽는 쪽은 옛 파일 전체나 새 파일 전체를 보며, 섞인 것은 결코 보지 않는다.
4. `proven_fs_sync_dir()` — 이름 바꾸기 자체가 저장 장치에 도달한다.

2단계를 건너뛰면 내용이 도착한 적 없는 파일을 이름이 가리킬 수 있다. 4단계를 건너뛰면 내용은 안전한데 그것을 가리키는 이름이 안전하지 않다.

같은 예제가 함께 쓰이는 레코드 단위 호출들도 다룬다.

호출	무엇을 위한 것인가
<code>proven_fs_pread / proven_fs_pwrite</code>	절대 오프셋에서 읽고 쓰되 파일 위치를 옮기지 않는다 . 하나의 핸들을 여러 스레드가 함께 써도 안전한 이유다.
<code>proven_fs_seek / proven_fs_tell</code>	위치를 옮기고, 지금 어디인지 묻는다. 끝에서 음수 오프셋으로 seek하면 길이를 몰라도 마지막 레코드를 찾는다.
<code>proven_fs_truncate</code>	길이를 직접 정한다. 한 번의 호출로 파일 시스템이 숫자 하나를 고치며, 대안은 남길 부분을 복사하는 것이다.
<code>proven_fs_lock</code>	권고(advisory) 잠금 이다. 마찬가지로 잠금을 요청하는 다른 프로세스만 막고, 아예 요청하지 않는 프로그램에는 아무 영향이 없다.
<code>proven_fs_copy</code>	바이트를 복제해 독립된 두 번째 파일을 만든다.
<code>proven_fs_link</code>	같은 파일에 대한 두 번째 이름 (하드 링크). 원본이라는 것이 없고, 마지막 이름이 사라질 때까지 데이터가 산다. 같은 파일 시스템 안에서만 된다.
<code>proven_fs_symlink</code>	경로를 담은 작은 파일(심볼릭 링크). 파일 시스템을 건너뛸 수 있고, 아무것도 가리키지 않을 수도 있다.
<code>proven_fs_is_absolute</code>	이 경로가 루트에서 시작하는가? 규칙이 플랫폼마다 달라서 함수인 것이다.
<code>proven_fs_rmdir</code>	빈 디렉토리를 지운다. 비어 있지 않으면 거절하므로, 재귀 삭제는 명시적인 결정으로 남는다.

/*

* 정전이 파일을 반쯤 쓰인 채로 남겨 두지 못하게 하며 파일을 고치기.

*

* 조리법은 오래되었고, 그 모든 걸음이 하중을 받는다.

*

* 1. 새 내용을 진짜 파일 옆의 *임시* 파일에 쓴다,

* 2. `proven_fs_sync` - 이제 새 파일의 바이트가 장치에 있다,

* 3. `proven_fs_rename` - 이름이 한 걸음에 새 파일로 넘어간다. 읽는 쪽은 옛 파일 전체나

* 새 파일 전체를 보지, 그 사이를 보지 않는다,

* 4. `proven_fs_sync_dir` - 이제 그 이름 바꾸기 자체가 장치에 있다.

*

* 2를 건너뛰면 내용이 도착한 적 없는 파일을 이름이 공표할 수 있다. 4를 건너뛰면 내용은

* 안전하데 그 이름이 안전하지 않을 수 있다. 둘 다 시험에서는 드러나지 않는다. 둘 다

* 운영에서, 한 번, 드러난다.

*

* 이 프로그램은 레코드 단위 호출들 - seek/tell, pread/pwrite, truncate - 과, 이 모든

* 일을 두 별의 프로그램이 동시에 하지 못하게 막는 권고 잠금도 함께 보인다.

```

*/

typedef struct {
    proven_u32 id;
    proven_u32 score;
} record_t;

static proven_err_t write_records(proven_file_t f, const record_t *recs, proven_size_t n) {
    proven_mem_view_t view = { .ptr = (const proven_byte_t *)recs, .size = n * sizeof recs[0] };
    return proven_fs_write_all(f, view);
}

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    proven_u8str_view_t live = PROVEN_LIT("proven_example_durable.dat");
    proven_u8str_view_t temp = PROVEN_LIT("proven_example_durable.dat.new");
    proven_u8str_view_t here = PROVEN_LIT(".");

    /* is_absolute 는 경로를 잊거나 기준 디렉터리에 대고 풀기 전에 물어 둘 값어치가 있는
     * 물음에 답한다 - 이 경로가 이미 뿌리에서 시작하는가? 규칙은 플랫폼마다 다르다 -
     * 여기서는 앞의 '/', 윈도우에서는 드라이브 문자나 UNC 접두사 - 그리고 바로 그 때문에
     * 이것은 '/' 와의 비교가 아니라 호출이다. */
    EXAMPLE_REQUIRE(!proven_fs_is_absolute(live), "the working paths in this example are relative");
    EXAMPLE_REQUIRE(proven_fs_is_absolute(PROVEN_LIT("/etc/hosts")), "a leading slash is absolute on POSIX");

    static const record_t initial[] = {
        { .id = 1, .score = 10 }, { .id = 2, .score = 20 },
        { .id = 3, .score = 30 }, { .id = 4, .score = 40 },
    };

    /* --- 평범한 첫 쓰기 ----- */

    proven_result_file_t f = proven_fs_open(alloc, live, PROVEN_FS_WRITE | PROVEN_FS_CREATE |
    PROVEN_FS_TRUNC);
    EXAMPLE_REQUIRE(proven_is_ok(f.err), "creating the data file must succeed");
    if (!proven_is_ok(f.err)) return 1;

    proven_err_t err = write_records(f.value, initial, 4);
    EXAMPLE_REQUIRE(proven_is_ok(err), "writing the initial records must succeed");
    err = proven_fs_close(f.value);
    EXAMPLE_REQUIRE(proven_is_ok(err), "closing after a write must succeed");

    /* --- 두 벌이 서로 끼어들지 못하게 하는 권고 잠금 ----- */

    proven_result_file_t rw = proven_fs_open(alloc, live, PROVEN_FS_READ | PROVEN_FS_WRITE);
    EXAMPLE_REQUIRE(proven_is_ok(rw.err), "reopening for update must succeed");
    if (!proven_is_ok(rw.err)) return 1;

    /* *배타* 잠금은 같은 것을 청하는 다른 모든 프로세스를 밖에 세운다. "권고" 는 말
     * 그대로다. 협조하는 프로그램을 막을 뿐, 잠금을 아예 청하지 않는 프로그램에는 아무
     * 영향이 없다. `wait = false` 는 막지 않고 곧바로 돌아온다 - 달리 할 일이 있을 때
     * 옳은 선택이고, 잠금을 쥔 쪽이 여러분을 기다리고 있을 수 있을 때는 유일하게 안전한
     * 선택이다. */
    err = proven_fs_lock(rw.value, PROVEN_FS_LOCK_EXCLUSIVE, false);
    EXAMPLE_REQUIRE(proven_is_ok(err), "taking the exclusive lock must succeed when nobody holds it");

    /* --- 레코드 하나를, 위치를 지정해 읽고 쓰기 ----- */

    /* pread 는 절대 위치에서 읽고 파일 위치를 옮기지 *않는다*. 그래서 손잡이 하나를

```

```

* 나눠 쓰는 두 스레드에서도 안전하다 - 그들이 다룰 공유 커서가 없다. */
record_t third = {0};
proven_mem_mut_t into = { .ptr = (proven_byte_t *)&third, .size = sizeof third };
proven_result_size_t got = proven_fs_pread(rw.value, into, 2 * sizeof(record_t));
EXAMPLE_REQUIRE(proven_is_ok(got.err) && got.value == sizeof third, "reading record 2 must succeed");
EXAMPLE_REQUIRE(third.id == 3 && third.score == 30, "and yield the record that was written there");

/* tell 은 위치를 알려 준다. pread 뒤에도 그것은 움직이지 않았다. */
proven_result_u64_t pos = proven_fs_tell(rw.value);
EXAMPLE_REQUIRE(proven_is_ok(pos.err) && pos.val == 0, "pread must not move the file position");

/* pwrite 는 그 레코드를 제자리에서 고친다. 이번에도 커서는 건드리지 않는다. */
third.score = 99;
proven_mem_view_t out_view = { .ptr = (const proven_byte_t *)&third, .size = sizeof third };
proven_result_size_t put = proven_fs_pwrite(rw.value, out_view, 2 * sizeof(record_t));
EXAMPLE_REQUIRE(proven_is_ok(put.err) && put.value == sizeof third, "writing record 2 back must succeed");

/* seek 은 커서를 옮기는 쪽이고, 도착한 위치를 돌려준다. 음수 오프셋으로 *끝*에서
* seek 하는 것이 파일 길이를 먼저 알지 않고 마지막 레코드를 찾는 방법이다. */
proven_result_u64_t last = proven_fs_seek(rw.value, -(proven_i64)sizeof(record_t), PROVEN_FS_SEEK_END);
EXAMPLE_REQUIRE(proven_is_ok(last.err), "seeking to the last record must succeed");
EXAMPLE_REQUIRE(last.val == 3 * sizeof(record_t), "which is three records in");

pos = proven_fs_tell(rw.value);
EXAMPLE_REQUIRE(proven_is_ok(pos.err) && pos.val == last.val, "tell agrees with the seek result");

/* truncate 는 길이를 곧장 정한다. 마지막 레코드를 버리는 것이 호출 하나에 O(1) 이다.
* 옛날 방식 - 전부 읽고 남길 부분을 다시 쓰기 - 은 파일 시스템이 수 하나를 고쳐서
* 하는 일에 O(n) 복사를 치르는 것이었다. */
err = proven_fs_truncate(rw.value, 3 * sizeof(record_t));
EXAMPLE_REQUIRE(proven_is_ok(err), "truncating to three records must succeed");
proven_result_size_t size = proven_fs_size(rw.value);
EXAMPLE_REQUIRE(proven_is_ok(size.err) && size.value == 3 * sizeof(record_t),
    "the file is now exactly three records long");

/* 잠금을 명시적으로 푼다. 손잡이를 달아도 풀리지만, 그렇다고 적어 두면 임계 구역이
* 코드에 눈에 보인 채로 남는다. */
err = proven_fs_lock(rw.value, PROVEN_FS_LOCK_UNLOCK, false);
EXAMPLE_REQUIRE(proven_is_ok(err), "releasing the lock must succeed");
err = proven_fs_close(rw.value);
EXAMPLE_REQUIRE(proven_is_ok(err), "closing the update handle must succeed");

/* --- 건디는 바꿔치기 ----- */

static const record_t replacement[] = {
    { .id = 1, .score = 11 }, { .id = 2, .score = 22 },
};

/* 1. 새 내용을 옛 파일 옆에 쓴다. CREATE_NEW 는 임시 이름이 이미 있으면 거부하고,
* 그것이 죽어 버린 실행이 남긴 찌꺼기를 조용히 재사용하지 않고 알아채는 방법이다. */
proven_result_file_t tmp = proven_fs_open(alloc, tmp,
    PROVEN_FS_WRITE | PROVEN_FS_CREATE | PROVEN_FS_TRUNC);
EXAMPLE_REQUIRE(proven_is_ok(tmp.err), "creating the temporary file must succeed");
if (!proven_is_ok(tmp.err)) return 1;

err = write_records(tmp.value, replacement, 2);
EXAMPLE_REQUIRE(proven_is_ok(err), "writing the new contents must succeed");

/* 2. 그 바이트를 저장 장치까지 끝까지 밀어 준다. 값이 비싸고, 비싸야 마땅하다.
* 정전 뒤에도 자료가 있다는 보장을 사는 것이고, 그 값은 장치까지 실제로 다녀오는

```

```

*   일이다. */
err = proven_fs_sync(tmp.value);
EXAMPLE_REQUIRE(proven_is_ok(err), "syncing the new file's data must succeed");

err = proven_fs_close(tmp.value);
EXAMPLE_REQUIRE(proven_is_ok(err), "closing the temporary file must succeed");

/* 3. 이름을 넘긴다. 한 디렉터리 안의 rename 은 원자적이다. 읽는 쪽은 옛 파일이나 새
*   파일을 보지, 반쯤 쓰인 것을 보지 않는다. */
err = proven_fs_rename(alloc, tmp, live);
EXAMPLE_REQUIRE(proven_is_ok(err), "renaming the temporary file over the live one must succeed");

/* 4. 그 이름 바꾸기 자체를 견디게 만든다. 디렉터리가 장치에 닿기 전까지는, 새 내용이
*   안전하지 않을 수도 있는 이름 아래에 안전하게 있는 것이다. */
err = proven_fs_sync_dir(alloc, here);
EXAMPLE_REQUIRE(proven_is_ok(err), "syncing the directory must succeed");

proven_result_file_t check = proven_fs_open(alloc, live, PROVEN_FS_READ);
EXAMPLE_REQUIRE(proven_is_ok(check.err), "the live path must now open");
size = proven_fs_size(check.value);
EXAMPLE_REQUIRE(proven_is_ok(size.err) && size.value == 2 * sizeof(record_t),
                "and hold exactly the replacement records");
err = proven_fs_close(check.value);
EXAMPLE_REQUIRE(proven_is_ok(err), "closing the verification handle must succeed");

/* --- 복사, 그리고 두 가지 링크 ----- */

/* copy 는 바이트를 복제한다. 이제부터는 서로 독립인 파일 둘이다. 할당자는 복사가
*   자료를 옮겨 가는 임시 버퍼를 위한 것이다. */
proven_u8str_view_t backup = PROVEN_LIT("proven_example_durable.bak");
err = proven_fs_copy(alloc, live, backup);
EXAMPLE_REQUIRE(proven_is_ok(err), "copying the file must succeed");

/* *하드* 링크는 같은 파일의 두 번째 이름이다. 원본이라는 것이 없다. 자료는 마지막
*   이름이 지워질 때까지 산다. 두 이름은 같은 파일 시스템에 있어야 한다. 이름과 그
*   자료가 둘에 걸칠 수는 없기 때문이다. */
proven_u8str_view_t hard = PROVEN_LIT("proven_example_durable.hard");
err = proven_fs_link(alloc, live, hard);
EXAMPLE_REQUIRE(proven_is_ok(err), "creating a hard link must succeed");

proven_fs_stat_t st_live = {0}, st_hard = {0};
EXAMPLE_REQUIRE(proven_is_ok(proven_fs_stat(alloc, live, &st_live)), "stat of the live name");
EXAMPLE_REQUIRE(proven_is_ok(proven_fs_stat(alloc, hard, &st_hard)), "stat of the hard link");
EXAMPLE_REQUIRE(st_live.ino == st_hard.ino && st_live.dev == st_hard.dev,
                "both names refer to the same file, which is what a hard link means");

/* *심볼릭* 링크는 경로를 담은 작은 파일이다. 다른 파일 시스템의 무언가를 가리킬 수도
*   있고, 아무것도 아닌 것을 가리킬 수도 있다 - 그때 따라가기는 실패하는데, 하드 링크는
*   결코 그럴 수 없다. */
proven_u8str_view_t soft = PROVEN_LIT("proven_example_durable.link");
err = proven_fs_symlink(alloc, live, soft);
EXAMPLE_REQUIRE(proven_is_ok(err), "creating a symbolic link must succeed");

proven_fs_stat_t st_soft = {0};
EXAMPLE_REQUIRE(proven_is_ok(proven_fs_stat(alloc, soft, &st_soft)),
                "stat follows the symbolic link to its target");
EXAMPLE_REQUIRE(st_soft.size == st_live.size, "so it reports the target's size");

/* --- 디렉터리, 그리고 뒷정리 ----- */

```

```

proven_u8str_view_t dir = PROVEN_LIT("proven_example_durable_dir");
err = proven_fs_mkdir(alloc, dir);
EXAMPLE_REQUIRE(proven_is_ok(err), "creating a directory must succeed");

/* rmdir 은 *빈* 디렉터리만 지운다. 그 거부가 기능이다. 재귀 삭제는 부르는 쪽이
 * 명시적으로 내려야 하는 결정이지, 잘못 들어온 경로 인자 하나가 일으킬 수 있는 일이
 * 아니다. */
proven_u8str_view_t inside = PROVEN_LIT("proven_example_durable_dir/file.txt");
proven_result_file_t child = proven_fs_open(alloc, inside, PROVEN_FS_WRITE | PROVEN_FS_CREATE);
EXAMPLE_REQUIRE(proven_is_ok(child.err), "creating a file inside it must succeed");
EXAMPLE_REQUIRE(proven_is_ok(proven_fs_close(child.value)), "closing it must succeed");

err = proven_fs_rmdir(alloc, dir);
EXAMPLE_REQUIRE(err != PROVEN_OK, "removing a non-empty directory must be refused");

EXAMPLE_REQUIRE(proven_is_ok(proven_fs_remove(alloc, inside)), "removing the file must succeed");
err = proven_fs_rmdir(alloc, dir);
EXAMPLE_REQUIRE(proven_is_ok(err), "and then the empty directory can be removed");

printf("durable replace complete: %zu byte(s) live\n", (size_t)size.value);

(void)proven_fs_remove(alloc, soft);
(void)proven_fs_remove(alloc, hard);
(void)proven_fs_remove(alloc, backup);
(void)proven_fs_remove(alloc, live);
return EXAMPLE_OK();
}

```

반례 — 이 절차가 대신하려는 제자리 덮어쓰기:

```

proven_result_file_t f = proven_fs_open(alloc, live, PROVEN_FS_WRITE | PROVEN_FS_TRUNC);
proven_err_t e = proven_fs_write_all(f.value, new_contents); /* wrong */

```

자르기와 마지막 바이트 쓰기 사이에서 디스크 위의 파일은 불완전하다. 그 사이의 크래시는 갱신을 잃는 것이 아니라 이미 있던 데이터를 잃는다.

반례 — sync 없이 이름부터 바꾸는 경우:

```

proven_err_t e = proven_fs_close(tmp.value); /* no proven_fs_sync */
e = proven_fs_rename(alloc, temp, live); /* wrong */

```

파일을 닫는 것은 그 바이트를 장치에 올려놓는 일이 아니다. 이름 바꾸기는 지속되는데 그것이 가리키는 내용은 그렇지 않을 수 있다.

반례 — 잠금이 모두를 막는다고 가정하는 경우:

```

proven_err_t e = proven_fs_lock(f.value, PROVEN_FS_LOCK_EXCLUSIVE, true);
/* another program writes the file without ever calling proven_fs_lock */

```

권고 잠금은 그것을 모두가 쓰기로 한 프로그램들 사이의 약속이다. 접근 권한 통제가 아니다.

실전 예제: reader와 writer, 그리고 표준 스트림

writer는 "바이트가 가는 곳", **reader**는 "바이트가 오는 곳"이다. 각각은 포인터 둘이다. 작은 함수 표 하나와, 그 함수들이 다루는 상태 하나. 이 구조의 값어치는 전부 여기서 나온다. writer를 상대로 쓴 코드는 그 바이트가 파일로 가는지, 문자열로 가는지, 터미널로 가는지, 시험용 버퍼로 가는지 알지 못하고 — reader를 상대로 쓴 코드는 디스크의 파일 대신 메모리의 문자열을 상대로 시험할 수 있다.

예제는 그 조각들을 한데 놓는다.

- proven_reader_from_view()는 이미 손에 있는 바이트 위에 reader를 만든다. 파서를 파일 없이 시험하는 방법이다. proven_reader_read()는 버퍼 크기까지 채우고 실제로 얻은 양을 알려 준다. 짧은 읽기는 정상이며, 입력의 끝은 0바이트 성공이 아니라 PROVEN_ERR_EOF다.

- proven_sysio_stdout_writer()와 proven_sysio_stderr_writer()는 버퍼링 없는 표준 스트림이고, proven_sysio_stdin_reader()는 표준 입력이다. proven_writer_write()는 "전부, 아니면 에러"를 뜻하고, proven_writer_write_partial()은 옮길 수 있는 만큼 옮기고 그 개수를 알려 준다 — 재시도나 배압(back-pressure)을 직접 다루는 호출자를 위한 것이다.
- proven_reader_is_valid()와 proven_writer_is_valid()는 만들어진 적 없는 핸들을 첫 읽기·쓰기가 아니라 경계에서 잡는다.
- proven_sysio_file_buffered()는 열린 파일을, **여러분이** 준 버퍼 위의 버퍼드 writer로 감싼다. 버퍼링의 메모리 비용이 여러분이 고른 숫자로 남는다는 뜻이다. 반드시 flush해야 한다. 여기서는 아무것도 종료 시에 대신 flush해 주지 않는다.
- proven_sysio_lines_open()은 같은 방식의 호출자 제공 버퍼로 그 파일을 한 줄씩 읽는다. 예상되는 가장 긴 줄에 맞춰 크기를 잡는다. 더 긴 줄은 조용히 잘리는 대신 PROVEN_ERR_OUT_OF_BOUNDS가 된다.
- proven_scan_fmt_from_file()(내부적으로 proven_sysio_scan_chunk_impl()을 부른다)은 입력이 자유 서식이 아니라 정해진 모양일 때 파일 핸들에서 타입 있는 값을 바로 뽑는다.

```
/*
 * 쓰는 쪽은 "바이트가 가는 어딘가" 이고 읽는 쪽은 "바이트가 오는 어딘가" 이며, 각각
 * 포인터 둘이다. 작은 함수 표 하나와 그 함수들이 다룰 상태 하나. 그것이 전부다. 이
 * 열개의 값어치는, 쓰는 쪽에 기대어 쓴 코드가 그 바이트가 파일로 가는지 문자열로 가는지
 * 터미널로 가는지 시험용 버퍼로 가는지 알지도 못하고 신경 쓰지도 않는다는 것이다 -
 * 그리고 읽는 쪽에 기대어 쓴 코드는 디스크의 파일 대신 기억 속 문자열에 대고 시험할 수
 * 있다.
 *
 * 이 프로그램은 둘 다, 그리고 거기 매달린 표준 스트림과 파일 배관을 보인다.
 *
 * - 뷰(기억 속 문자열) 위의 읽는 쪽. 파일 시스템을 건드리지 않고 파서를 시험하는
 *   방법이다,
 * - 버퍼 없는 stdout 과 stderr 쓰는 쪽, 그리고 "이걸 다 써라" 와 "쓸 수 있는 만큼
 *   써라" 의 차이,
 * - 파일 위의 버퍼 둔 쓰는 쪽. 작은 쓰기 여럿을 큰 쓰기 몇으로 바꿔 준다,
 * - 그 같은 파일 위의 줄 단위 읽는 쪽,
 * - 파일 손잡이에서 곧장 형식화된 값 읽기.
 *
 * 사람들이 반나절을 잃게 만드는 주의 하나: 아래의 상태 구조체들은 선언된 자리에 그대로
 * 있어야 한다. 쓰는 쪽은 자기 상태 구조체 *안*을 가리키는 포인터를 줘므로, 구조체를
 * 복사하면 사본은 죽은 것이 되고 주소는 원본을 가리킨 채로 남는다.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* --- 1. 이미 가진 바이트 위의 읽는 쪽 ----- */

    /* 아래 파서는 이것이 문자열이라는 것을 모른다. 파일이든 파이프든 소켓이든 똑같이
     * 둘 것이다 - 그래서 시험이 값싼 쪽을 쓸 수 있다. */
    proven_reader_view_t src_state;
    proven_reader_t src = proven_reader_from_view(&src_state, PROVEN_LIT("id=41\nid=42\n"));

    /* is_valid 는 그 읽는 쪽이 실제로 만들어졌는지 묻는다 - 0 으로 초기화된 손잡이는
     * 읽는 쪽이 아니고, 그것을 첫 읽기가 아니라 경계에서 말해 주는 검사가 이것이다. */
    EXAMPLE_REQUIRE(proven_reader_is_valid(src), "a reader made from a view must be usable");
    proven_reader_t never_made = {0};
    EXAMPLE_REQUIRE(!proven_reader_is_valid(never_made), "a zero-initialised reader handle is not");

    /* read 는 dest.size 바이트까지 채우고 실제로 얼마나 받았는지 알려 준다. 짧은 읽기는
     * 정상이다 - 파일의 끝이 아니고, 그것을 끝으로 여기는 것이 입력의 꼬리를 읽는
     * 고전적인 방법이다. 파일의 끝에는 제 코드가 따로 있다, PROVEN_ERR_EOF. */
    proven_byte_t chunk[5];
    proven_mem_mut_t into = { .ptr = chunk, .size = sizeof chunk };
}
```

```

proven_result_size_t got = proven_reader_read(src, into);
EXAMPLE_REQUIRE(proven_is_ok(got.err), "the first read must succeed");
EXAMPLE_REQUIRE(got.value == 5, "and fill the buffer from the view");

proven_size_t total = got.value;
for (;;) {
    got = proven_reader_read(src, into);
    if (got.err == PROVEN_ERR_EOF) {
        break;
    }
    EXAMPLE_REQUIRE(proven_is_ok(got.err), "reads before end of file must succeed");
    total += got.value;
}
EXAMPLE_REQUIRE(total == 12, "the loop must consume the whole view, however it was chunked");

/* --- 2. 표준 스트림 ----- */

/* 상태 구조체가 쓰는 쪽이 가리키는 손잡이를 쥐고 있으므로, 쓰는 쪽보다 오래 살아야
 * 한다. 둘을 나란히 선언하는 습관이 그것을 지켜 준다. */
proven_sysio_std_t out_state;
proven_writer_t out = proven_sysio_stdout_writer(&out_state);
EXAMPLE_REQUIRE(proven_writer_is_valid(out), "the stdout writer must be usable");

/* write 는 "전부, 아니면 오류" 라는 뜻이다. 안에서 반복한다. 시스템 수준의 쓰기 한
 * 번은 청한 것보다 적은 바이트를 옮길 수 있기 때문이다. */
proven_err_t err = proven_writer_write(out, proven_mem_view_from_u8(PROVEN_LIT("stream example:
start\n")));
EXAMPLE_REQUIRE(proven_is_ok(err), "writing a whole line to stdout must succeed");

/* write_partial 은 정직한 저수준 쌍둥이다. 옮길 수 있는 만큼 옮기고 그 개수를 알려
 * 준다. 재시도나 역압을 여러분이 다룰 때 쓰고, 그냥 바이트를 내보내고 싶으면 write 를
 * 쓸 것. */
proven_result_size_t part = proven_writer_write_partial(out, proven_mem_view_from_u8(PROVEN_LIT("partial
write\n")));
EXAMPLE_REQUIRE(proven_is_ok(part.err), "a partial write to a terminal or pipe must succeed");
EXAMPLE_REQUIRE(part.value > 0, "and report how many bytes it moved");

/* stderr 에 버퍼를 두지 않은 것은 일부러다. 진단은 다음 줄의 코드가 돌기 전에 나가야
 * 하고, 그 다음 줄이 바로 죽는 줄일 때 그것이 필요하다. */
proven_sysio_std_t err_state;
proven_writer_t diag = proven_sysio_stderr_writer(&err_state);
EXAMPLE_REQUIRE(proven_writer_is_valid(diag), "the stderr writer must be usable");
err = proven_writer_write(diag, proven_mem_view_from_u8(PROVEN_LIT("stream example: diagnostics go
here\n")));
EXAMPLE_REQUIRE(proven_is_ok(err), "writing to stderr must succeed");

/* stdin 위의 읽는 쪽도 같은 방식으로 만든다. 이 예제는 거기서 읽지 않는다 - 시험
 * 실행에는 타이핑하는 사람이 없다 - 다만 만드는 모양을 보이고, 필터 프로그램이 돌
 * 손잡이가 이것이다. */
proven_sysio_std_t in_state;
proven_reader_t stdin_reader = proven_sysio_stdin_reader(&in_state);
EXAMPLE_REQUIRE(proven_reader_is_valid(stdin_reader), "the stdin reader must be usable");

/* --- 3. 파일로 나가는 버퍼 둔 출력 ----- */

proven_u8str_view_t path = PROVEN_LIT("proven_example_streams.txt");
proven_result_file_t f = proven_fs_open(alloc, path, PROVEN_FS_WRITE | PROVEN_FS_CREATE |
PROVEN_FS_TRUNC);
EXAMPLE_REQUIRE(proven_is_ok(f.err), "creating the output file must succeed");
if (!proven_is_ok(f.err)) return 1;

```

```

/* 버퍼는 여러분의 것이다. 라이브러리가 등 뒤에서 하나 할당하지 않으므로, 버퍼링의
 * 기억 비용은 여러분이 고른, 눈에 보이는 수다. 256바이트 버퍼를 지나는 예순 줄은
 * 예순 번이 아니라 몇 번의 쓰기다. */
proven_byte_t outbuf[256];
proven_sysio_out_t file_out;
proven_writer_t w = proven_sysio_file_buffered(&file_out, f.value,
                                              (proven_mem_mut_t){ .ptr = outbuf, .size = sizeof
outbuf });
EXAMPLE_REQUIRE(proven_writer_is_valid(w), "the buffered file writer must be usable");

for (int i = 0; i < 20; ++i) {
    proven_fmt_result_t line_out = proven_fprintln(w, "reading {} = {}",
                                                  proven_arg_i32(i), proven_arg_i32(i * i));
    EXAMPLE_REQUIRE(proven_is_ok(line_out.err), "writing a formatted line must succeed");
}

/* 흘러 보내지 않은 버퍼 출력은 일어나지 않은 출력이다. 여기 어느 것도 여러분 대신
 * 종료 시점에 흘러 주지 않는다. */
err = proven_writer_flush(w);
EXAMPLE_REQUIRE(proven_is_ok(err), "the flush is what actually writes the file");
err = proven_fs_close(f.value);
EXAMPLE_REQUIRE(proven_is_ok(err), "closing the file must succeed");

/* --- 4. 한 줄씩 되읽기 ----- */

proven_result_file_t rf = proven_fs_open(alloc, path, PROVEN_FS_READ);
EXAMPLE_REQUIRE(proven_is_ok(rf.err), "reopening for reading must succeed");
if (!proven_is_ok(rf.err)) return 1;

/* 버퍼는 예상되는 가장 긴 *줄*에 맞춰 잡을 것. 더 긴 줄은 OUT_OF_BOUNDS 로 보고된다 -
 * 조용히 반으로 잘리는 일은 없다. 그 자르기가 나쁜 레코드 하나를 그럴듯한 레코드
 * 둘로 만드는 실패다. */
proven_byte_t linebuf[128];
proven_sysio_lines_t lines;
err = proven_sysio_lines_open(&lines, rf.value, (proven_mem_mut_t){ .ptr = linebuf, .size = sizeof
linebuf });
EXAMPLE_REQUIRE(proven_is_ok(err), "opening a line reader over the file must succeed");

proven_size_t count = 0;
for (;;) {
    proven_result_u8str_view_t line = proven_sysio_read_line(&lines);
    if (line.err == PROVEN_ERR_EOF) {
        break;
    }
    EXAMPLE_REQUIRE(proven_is_ok(line.err), "reading a line must succeed until end of file");
    /* 그 뷰는 linebuf 안을 가리키고 다음 호출 전까지만 쓸 수 있다. 그래서 백만 줄이
     * 백만 번의 할당이 아니라 버퍼 하나만큼의 값이 든다 - 그리고 남겨 둘 줄은 복사해야
     * 하는 이유이기도 하다. */
    if (count == 0) {
        EXAMPLE_REQUIRE(proven_u8str_view_eq(line.val, PROVEN_LIT("reading 0 = 0")),
                        "the first line reads back exactly as it was written");
    }
    ++count;
}
EXAMPLE_REQUIRE(count == 20, "every line written must be read back");
EXAMPLE_REQUIRE(proven_is_ok(proven_fs_close(rf.value)), "closing the read handle must succeed");

/* --- 5. 파일에서 줄이 아니라 값을 읽기 ----- */

```

```

/* 입력이 자유로운 글이 아니라 정해진 모양일 때는, 곧장 파싱하는 편이 자르고 변환하는
 * 반복문을 손으로 쓰는 수고를 덜다. 이것은 파일 손잡이에서 한 토막을 읽어 그 안에서
 * 수 둘을 뽑아낸다. */
proven_result_file_t sf = proven_fs_open(alloc, path, PROVEN_FS_READ);
EXAMPLE_REQUIRE(proven_is_ok(sf.err), "opening the file for scanning must succeed");

proven_i32 index = -1, square = -1;
err = proven_scan_fmt_from_file(sf.value, "reading {} = {}",
                                proven_scan_arg_i32(&index), proven_scan_arg_i32(&square));
EXAMPLE_REQUIRE(proven_is_ok(err), "scanning the first record must succeed");
EXAMPLE_REQUIRE(index == 0 && square == 0, "and produce the values that were written");
EXAMPLE_REQUIRE(proven_is_ok(proven_fs_close(sf.value)), "closing the scan handle must succeed");

printf("streams: %zu byte(s) read from the view, %zu line(s) round-tripped\n",
       (size_t)total, (size_t)count);

(void)proven_fs_remove(alloc, path);
return EXAMPLE_OK();
}

```

반례 — writer를 만든 뒤 상태 구조체를 복사하는 경우:

```

proven_sysio_out_t state;
proven_writer_t w = proven_sysio_file_buffered(&state, file, buf);
proven_sysio_out_t moved = state; /* wrong: w still points into `state` */

```

writer는 그 구조체 **안쪽**을 가리키는 포인터를 들고 있다. writer가 사는 동안 상태는 선언한 자리에 둔다.

반례 — flush를 잊는 경우:

```

proven_fmt_result_t r = proven_fprintln(w, "done");
return 0; /* wrong: the buffer never reached the file */

```

실전 예제: 파일을 메모리에 매핑하고, 그 변경을 지속시키기

메모리 매핑된 파일은 프로세서가 메모리처럼 읽는 파일이다. 운영체제가 그 페이지들을 어떤 주소에 나타나게 해 주고, 읽기는 read 호출 없이 일어난다. 큰 파일에 무작위로 접근할 때, 특히 여러 프로세스가 함께 볼 때 어울린다. 스트리밍에는 어울리지 않는다. 거기서는 버퍼드 reader가 더 단순하고, 파일 크기를 주소 공간에 묶지도 않는다.

쓰기가 살아남는지를 정하는 구분은 이것이다.

매핑	쓰기가 가는 곳	proven_mmap_sync()
PROVEN_MMAP_SHARED	파일로 간다	저장 장치까지 밀어 넣는다
PROVEN_MMAP_PRIVATE	사적인 사본으로 간다(기록 시 복사, copy-on-write) — 이 프로세스에만	되돌려 쓸 것이 없으므로 PROVEN_ERR_UNSUPPORTED를 돌려준다

그 거절은 일부러 그런 것이다. 공유 매핑이라고 믿었던 호출자가, 데이터가 없어진 뒤가 아니라 sync 자리에서 그 사실을 알게 된다.

예제는 달력 형식화로 끝난다. 기록하는 레코드에 날짜가 붙기 때문이다. UTF-8 형태는 proven_time_u8_fmt(), UTF-16 형태는 proven_time_u16_fmt()이며, 후자가 옳은 상황은 딱 하나 — 와이드 문자열을 받는 시스템 호출에 그 텍스트를 바로 넘길 때다.

```
#include <string.h>
```

```
/*
```

```

 * 기억에 사상된 파일은 프로세서가 기억처럼 읽는 파일이다. 운영체제가 그 쪽들을 어떤
 * 주소에 나타나게 해 주고, 읽기는 read 호출 없이 일어난다. 한 모양의 문제에는 옳은
 * 도구다 - 여러 프로세스가 함께 보는 큰 파일에 무작위로 접근하는 일 - 그리고 흘러 읽기에는
 * 틀린 도구다. 거기서는 버퍼를 둔 읽기 쪽이 더 단순하고, 파일 크기를 여러분의 주소 공간에
 * 묶지도 않는다.

```

```

*
* 틀리기 쉬운 자리는 지속성이고, 이 예제가 있는 이유가 그것이다.
*
* PROVEN_MMAP_SHARED - 쓰기가 파일로 간다. proven_mmap_sync 가 그것을 저장 장치까지
*                       밀어 준다.
* PROVEN_MMAP_PRIVATE - 쓰기가 복사-후-쓰기다. 이 프로세스 안에만 있고 다른 어디에도
*                       없다. 되 쓸 것이 없으므로, sync 를 청하면 조용히 아무 일도 하지
*                       않는 대신 그렇다고 말한다.
*
* 이 예제는 달력 형식화기도 나란히 보인다. 여기서 쓰는 레코드가 시각을 심기 때문이고,
* 윈도우 API 를 위해 쓰는 시각이야말로 UTF-16 형식화기가 옳은 호출인 유일한 자리다.
*/

```

```

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();
    proven_u8str_view_t path = PROVEN_LIT("proven_example_mmap.dat");

    /* 사상은 파일을 늘릴 수 없다. 그러니 파일은 사상하기 전에 사상하려는 크기여야 한다. */
    static const char initial[] = "record 0: pending  \n";
    proven_result_file_t create = proven_fs_open(alloc, path,
                                                PROVEN_FS_WRITE | PROVEN_FS_CREATE | PROVEN_FS_TRUNC);
    EXAMPLE_REQUIRE(proven_is_ok(create.err), "creating the backing file must succeed");
    if (!proven_is_ok(create.err)) return 1;

    proven_mem_view_t seed = { .ptr = (const proven_byte_t *)initial, .size = sizeof initial - 1 };
    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_write_all(create.value, seed)), "writing the record must succeed");
    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_close(create.value)), "closing it must succeed");

    /* --- 공유 사상: 쓰기가 파일에 닿는다 ----- */

    proven_result_file_t f = proven_fs_open(alloc, path, PROVEN_FS_READ | PROVEN_FS_WRITE);
    EXAMPLE_REQUIRE(proven_is_ok(f.err), "opening the file for mapping must succeed");
    if (!proven_is_ok(f.err)) return 1;

    /* size 0 은 "파일 끝까지" 라는 뜻이다. */
    proven_result_mmap_t m = proven_mmap_create(f.value, 0, 0,
                                                PROVEN_MMAP_READ | PROVEN_MMAP_WRITE,
                                                PROVEN_MMAP_SHARED);
    EXAMPLE_REQUIRE(proven_is_ok(m.err), "mapping the file must succeed");
    if (!proven_is_ok(m.err)) {
        (void)proven_fs_close(f.value);
        return 1;
    }
    proven_mmap_t map = m.value;

    /* 읽기는 그냥 기억을 읽는 것이다 - 호출도 복사도 없다. as_view 는 사상 전체를 바이트
     * 뷰로 돌려주므로, 보통의 뷰 도우미들이 그대로 먹는다. */
    proven_u8str_view_t contents = proven_mmap_as_view(map);
    EXAMPLE_REQUIRE(contents.size == sizeof initial - 1, "the mapping covers the whole file");
    EXAMPLE_REQUIRE(proven_u8str_view_starts_with(contents, PROVEN_LIT("record 0:")),
                    "and shows the bytes that were written");

    /* 쓰기는 기억에 쓰는 것이다. 상태 필드를 일부러 꼭 고정으로 두었다. 사상은 파일을
     * 길게 만들 수 없으므로, 제자리 편집은 이미 있는 자리에 들어가야 한다. */
    proven_size_t at = proven_u8str_view_find(contents, 0, PROVEN_LIT("pending"));
    EXAMPLE_REQUIRE(at != PROVEN_SIZE_MAX, "the status field must be found");
    memcpy((proven_byte_t *)map.ptr + at, "done  ", 7);

    /* sync 가 지속성의 걸음이다. 그것 없이는 바뀐 내용이 페이지 캐시에 있고, 거기서는 이
     * 프로그램이 끝나는 것은 견디지만 기계가 전원을 잃는 것은 견디지 못한다. */

```

```

EXAMPLE_REQUIRE(proven_is_ok(proven_mmap_sync(&map)), "syncing a shared mapping must succeed");
EXAMPLE_REQUIRE(proven_is_ok(proven_mmap_destroy(&map)), "unmapping must succeed");
EXAMPLE_REQUIRE(proven_is_ok(proven_fs_close(f.value)), "closing the mapped file must succeed");

/* 편집이 이 프로세스의 기억에만이 아니라 파일에 들어가 있다. */
proven_result_u8str_t back = proven_fs_read_all_u8str(alloc, path);
EXAMPLE_REQUIRE(proven_is_ok(back.err), "reading the file back must succeed");
EXAMPLE_REQUIRE(proven_u8str_view_find(proven_u8str_as_view(&back.value), 0, PROVEN_LIT("done")) !=
PROVEN_SIZE_MAX,
    "the mapped write reached the file");
proven_u8str_destroy(alloc, &back.value);

/* --- 사적 사상: 쓰기가 아무 데도 가지 않는다 ----- */

proven_result_file_t pf = proven_fs_open(alloc, path, PROVEN_FS_READ);
EXAMPLE_REQUIRE(proven_is_ok(pf.err), "reopening for a private mapping must succeed");

proven_result_mmap_t pm = proven_mmap_create(pf.value, 0, 0, PROVEN_MMAP_READ, PROVEN_MMAP_PRIVATE);
EXAMPLE_REQUIRE(proven_is_ok(pm.err), "a private read mapping must succeed");
proven_mmap_t priv = pm.value;

/* 사적 사상에 sync 를 청하면 받아 놓고 무시하는 대신 거부된다. 그 거부가 쓸모 있는
 * 동작이다. 사상이 공유라고 믿고 있던 쪽이 자료가 사라진 뒤가 아니라 여기서 알게 된다. */
EXAMPLE_REQUIRE(proven_mmap_sync(&priv) == PROVEN_ERR_UNSUPPORTED,
    "a private mapping has nothing to write back, and says so");

EXAMPLE_REQUIRE(proven_is_ok(proven_mmap_destroy(&priv)), "unmapping the private mapping must succeed");
EXAMPLE_REQUIRE(proven_is_ok(proven_fs_close(pf.value)), "closing it must succeed");

/* --- 시각을, 두 문자열 타입 모두로 ----- */

proven_datetime_t now = proven_time_now_datetime();

proven_result_u8str_t stamp = proven_u8str_create(alloc, 64);
EXAMPLE_REQUIRE(proven_is_ok(stamp.err), "creating the timestamp string must succeed");
EXAMPLE_REQUIRE(proven_is_ok(proven_time_u8_fmt(alloc, &stamp.value, now, &proven_time_locale_en,
    "{year}-{month:0>2}-{day:0>2}")),
    "formatting the date as UTF-8 must succeed");
EXAMPLE_REQUIRE(proven_u8str_as_view(&stamp.value).size == 10, "a YYYY-MM-DD date is ten characters");

/* 같은 호출의 UTF-16 꼴이다. 그것이 옳은 유일한 자리를 위한 것 - 와이드 문자열을 받는
 * 시스템 호출에 글을 곧장 건네는 자리다. */
proven_result_u16str_t wide = proven_u16str_create(alloc, 64);
EXAMPLE_REQUIRE(proven_is_ok(wide.err), "creating the wide timestamp string must succeed");
EXAMPLE_REQUIRE(proven_is_ok(proven_time_u16_fmt(alloc, &wide.value, now, &proven_time_locale_en,
    "{year}-{month:0>2}-{day:0>2}")),
    "formatting the same date as UTF-16 must succeed");
EXAMPLE_REQUIRE(proven_u16str_len(&wide.value) == 10, "ten code units, one per character here");
EXAMPLE_REQUIRE(proven_u16str_as_ptr(&wide.value)[4] == (proven_u16)'-',
    "and the same layout as the UTF-8 form");

printf("mapped record updated; stamped %s\n", proven_u8str_as_cstr(&stamp.value));

proven_u16str_destroy(alloc, &wide.value);
proven_u8str_destroy(alloc, &stamp.value);
(void)proven_fs_remove(alloc, path);
return EXAMPLE_OK();
}

```

반례 — 매핑이 파일을 늘려 줄 것이라고 기대하는 경우:

```
proven_result_mmap_t m = proven_mmap_create(f.value, 0, 1 << 20, ...); /* file is 22 bytes */
memcpy((char *)m.value.ptr + 4096, data, n); /* wrong */
```

파일 길이를 먼저 정하고 — proven_fs_truncate()가 한 번에 해 준다 — 존재하는 만큼을 매핑한다.

실전 예제: 난수는 어디에서 오는가

위의 예제는 어떤 생성기를 고를지를 다룬다. 이것은 그 아래 계층, 곧 바이트의 출처와, 어느 출처를 받았는지 신경 쓰지 않는 코드를 쓰는 법이다.

- proven_rng_t가 그 인터페이스다. 이것을 받는 함수는 운영체제 생성기로도, ChaCha20으로도, xoshiro로도, 시험용으로 여러분이 만든 가짜로도 고치지 않고 동작한다. proven_rng_is_valid()는 출처가 실제로 만들어졌는지 말해 준다. 만들어지지 않은 것에서 뽑으면 숫자를 지어내는 대신 0을 돌려주므로, 뽑을 때마다 믿는 대신 경계에서 한 번 확인한다. proven_rng_u64()는 64비트 낱말 하나를, proven_rng_fill()은 버퍼 전체를 한 번에 채운다.
- 고정된 씨앗(seed)은 시험을 재현 가능하게 만든다. proven_chacha_rng_seed()는 씨앗 바이트를 직접 받으므로 proven_chacha_rng_next()는 알려진 수열을 걷는다. "일주일에 한 번 실패한다"를 다시 돌려볼 수 있는 실패로 바꾸는 것이 바로 이것이다. 운영에서는 씨앗이 진짜 엔트로피에서 와야 한다. 그것을 아는 사람은 이후의 모든 바이트를 알기 때문이다.
- proven_random_u64()는 엔트로피 출처에서 강한 낱말 하나를 바로 뽑는다. 시동 시의 해시 키나 식별자 같은 일회성에는 알맞고, 반복문 안에서는 그르다. 호출마다 운영체제까지 다녀오기 때문이다.
- proven_random_set_source()는 엔트로피 출처 자체를 설치한다. 호스티드(hosted) 프로그램에는 운영체제의 것이 이미 있으니 그대로 두면 되고, 베어메탈 프로그램에는 없으니 그 보드의 하드웨어 출처가 여기로 들어간다.

```
#include <string.h>
```

```
/*
 * 다른 예제는 어느 생성기를 고를지를 보인다. 이것은 그 밑의 층에 대한 것이다. 난수가
 * *어디서* 오는가, 그리고 그것을 신경 쓰지 않는 코드를 어떻게 쓰는가.
 *
 * proven_rng_t 는 난수 바이트의 원천을 포인터 한 쌍으로 나타낸 것이다 - 작은 함수
 * 표와 그 함수들이 다룰 생성기 상태. proven_rng_t 를 받는 코드는 OS 생성기에서도,
 * ChaCha20 에서도, xoshiro 에서도, 시험용으로 여러분이 지어낸 가짜에서도 한 줄도
 * 고치지 않고 돈다.
 *
 * proven_random_set_source 는 *그보다* 아래 층이다. 생성기가 씨를 받는 날 엔트로피가
 * 어디서 오는가. 호스트가 있는 프로그램에는 이미 하나가 있고 - 운영체제의 것 - 그대로
 * 두어야 한다. 베어메탈 프로그램에는 없고, 그 기계의 하드웨어 원천을 다는 자리가
 * 이 고리다.
 *
 * 고정된 씨앗 부분은 보기보다 중요하다. *알려진* 씨앗으로 씨를 뿌린 암호용 생성기는
 * 알려진 수열을 내놓고, 그것이 난수가 끼는 시험을 "일주일에 한 번 실패" 가 아니라
 * 재현 가능한 것으로 만든다.
 */
```

```
/* 전혀 무작위가 아닌 "엔트로피" 원천이다. 그냥 센다. 이런 것이 실제 프로그램에 들어갈
 * 자리는 없고 - 장의 반례를 볼 것 - 다만 이 고리가 어떻게 도는지 보이기예, 그리고 돌
 * 때마다 같은 바이트를 내야 하는 시험에 딱 맞는 모양이다. */
```

```
static bool counting_entropy(void *ctx, void *buf, proven_size_t len) {
    proven_u8 *next = (proven_u8 *)ctx;
    proven_u8 *out = (proven_u8 *)buf;
    for (proven_size_t i = 0; i < len; ++i) {
        out[i] = (*next)++;
    }
    return true;
}
```

```
/* 특성에 기대어 쓴 함수. 자기가 어느 생성기를 받았는지 끝내 알지 못한다. */
```

```

static proven_u64 roll_total(proven_rng_t rng, int rolls) {
    proven_u64 sum = 0;
    for (int i = 0; i < rolls; ++i) {
        sum += proven_rng_below(rng, 6) + 1;
    }
    return sum;
}

int main(void) {
    /* --- 1. 쓰기 전에 확인할 수 있는 원천 ----- */

    proven_rng_t nothing = {0};
    EXAMPLE_REQUIRE(!proven_rng_is_valid(nothing), "a zero-initialised source is not a generator");

    /* 잘못된 원천에서 뽑아도 죽지 않고 수를 지어내지도 않는다. 0 을 돌려준다. 정의된,
     * 심심한 답이다 - 그런데 0 의 연속은 난수가 아니므로, 뽑을 때마다 믿는 대신 받을 때
     * 한 번 원천을 확인할 것. */
    EXAMPLE_REQUIRE(proven_rng_u64(nothing) == 0, "an invalid source yields 0, not a fabricated value");

    /* --- 2. *알려진* 씨앗에서 나온 암호용 생성기 ----- */

    /* proven_chacha_rng_seed 는 씨앗 바이트를 곧장 받으므로 수열이 재현된다. 시험에서는
     * 그것이 원하는 바이고 운영에서는 결코 아니다. 씨앗을 알아낸 사람은 그 생성기가
     * 내놓을 모든 바이트를 안다. */
    proven_byte_t seed[PROVEN_CHACHA_SEED_SIZE];
    memset(seed, 0xA5, sizeof seed);

    proven_chacha_rng_t a, b;
    proven_chacha_rng_seed(&a, seed);
    proven_chacha_rng_seed(&b, seed);

    /* next 는 한 번에 64비트 낱말 하나를 돌려준다. 같은 씨앗을 받은 생성기 둘은 같은
     * 수열을 걷는다 - 시험이 기대는 성질이 그것이다. */
    proven_u64 first = proven_chacha_rng_next(&a);
    EXAMPLE_REQUIRE(first == proven_chacha_rng_next(&b), "the same seed replays the same sequence");
    EXAMPLE_REQUIRE(proven_chacha_rng_next(&a) == proven_chacha_rng_next(&b), "and keeps replaying it");

    /* --- 3. 특성을 통해 쓰기 ----- */

    proven_rng_t rng = proven_chacha_rng(&a);
    EXAMPLE_REQUIRE(proven_rng_is_valid(rng), "a seeded generator makes a valid source");

    proven_u64 word = proven_rng_u64(rng);
    (void)word; /* 어떤 64비트 값이든 옳은 답이다. 단언할 것이 없다 */

    /* fill 은 무더기로 하는 꼴이다. 나머지를 스스로 처리해야 하는 64비트 낱말 반복문
     * 대신, 버퍼 하나를 한 번의 호출로 채운다. */
    proven_byte_t nonce[12] = {0};
    proven_rng_fill(rng, nonce, sizeof nonce);

    bool all_zero = true;
    for (proven_size_t i = 0; i < sizeof nonce; ++i) {
        if (nonce[i] != 0) all_zero = false;
    }
    EXAMPLE_REQUIRE(!all_zero, "filling from a seeded generator must produce something");

    /* 같은 함수를, 서로 다른 생성기 둘이 굴린다. 이 특성이 존재하는 이유는 이것 하나다. */
    proven_chacha_rng_t c;
    proven_chacha_rng_seed(&c, seed);
    proven_u64 crypto_total = roll_total(proven_chacha_rng(&c), 50);

```

```

proven_xoshiro256ss_t fast;
proven_xoshiro256ss_seed(&fast, 7);
proven_u64 fast_total = roll_total(proven_xoshiro256ss_rng(&fast), 50);

EXAMPLE_REQUIRE(crypto_total >= 50 && crypto_total <= 300, "50 dice must total between 50 and 300");
EXAMPLE_REQUIRE(fast_total >= 50 && fast_total <= 300, "whichever generator produced them");

/* --- 4. 생성기를 쥐지 않고 강한 낱말 하나 ----- */

/* proven_random_u64 는 엔트로피 원천에서 곧장 뽑는다. 한 번뿐인 일에는 편하고 -
 * 시작할 때 잡는 표의 해시 키, 요청 번호 - 반복문에는 틀린 도구다. 호출마다 운영체제로
 * 다녀오는 값이 틀기 때문이다. 무더기로 뽑을 때는 생성기에 한 번 씨를 뿌리고 거기서
 * 뽑을 것. */
proven_u64 one_off = proven_random_u64();
proven_u64 another = proven_random_u64();
EXAMPLE_REQUIRE(one_off != another || one_off != 0,
    "two draws from the OS source are essentially never the same value");

/* --- 5. 엔트로피 원천 달기 ----- */

/* 호스트가 있는 대상에서는 운영체제의 원천이 이미 달려 있고 그대로 두어야 한다. 이
 * 고리는 베어메탈을 위한 것이다. 거기서는 그 보드의 엔트로피가 어느 하드웨어
 * 레지스터에 사는지 라이브러리가 알 길이 없다. 여기서는 일부러 가짜 원천을 달았다.
 * 오직 그 장치를 보이고 알아 끼우기가 먹혔음을 증명하기 위해서다 - 진짜는 진짜
 * 하드웨어 엔트로피여야 한다. */
proven_u8 counter = 0;
proven_random_set_source(counting_entropy, &counter);

proven_byte_t drawn[4] = {0};
EXAMPLE_REQUIRE(proven_random_bytes(drawn, sizeof drawn), "the installed source must answer");
EXAMPLE_REQUIRE(drawn[0] == 0 && drawn[1] == 1 && drawn[2] == 2 && drawn[3] == 3,
    "and it is the source we installed that answered");

/* 플랫폼 기본값을 되돌려 놓는다. 시험용 원천을 그대로 둔 채로 두는 것이, 프로그램이
 * 운영에서 예측 가능한 키를 만들어 내게 되는 방법이다. */
proven_random_set_source(NULL, NULL);

proven_byte_t real[8] = {0};
EXAMPLE_REQUIRE(proven_random_bytes(real, sizeof real), "the OS source is back and working");

printf("random: first word %llu, dice totals %llu and %llu\n",
    (unsigned long long)first, (unsigned long long)crypto_total, (unsigned long long)fast_total);

return EXAMPLE_OK();
}

```

반례 — 무작위처럼 보이기만 하는 것을 설치하는 경우:

```

static bool clock_entropy(void *ctx, void *buf, proven_size_t len) {
    proven_time_t t = proven_time_now(); /* wrong: predictable */
    memcpy(buf, &t, len < sizeof t ? len : sizeof t);
    return true;
}
proven_random_set_source(clock_entropy, NULL);

```

시계, 일련번호, 초기화되지 않은 버퍼, 의사 난수 생성기는 모두 얼핏 보아 넘어가지만 추측할 수 있는 것을 만든다. 보드에 진짜 엔트로피가 없다면 아무것도 설치하지 않는 것으로 그 사실을 말한다. 거절은 호출자가 대응할 수 있는 사실이고, 조용한 예측 가능성은 그렇지 않다.

반례 — 시험용 출처를 설치한 채 두는 경우:

```
proven_random_set_source(counting_entropy, &counter);  
/* ... the rest of the program, now generating predictable keys ... */
```

시험이 끝나는 즉시 proven_random_set_source(NULL, NULL)로 플랫폼 기본값을 되돌린다.

6장: 실행, 별칭, PAL, Freestanding, 크로스 빌드

6부 — 더 나아가기. 선행 조건: 2-5부. 이 장을 마치면 메모리 모델에 놀라는 일 없이 둘 이상의 스레드에서 작업을 실행할 수 있고, 루프처럼 읽히는 상태 기계를 작성할 수 있으며, 운영체제가 없는 대상으로 빌드할 수 있다.

이 장은 `coro.h`, `job.h`, `alias_xcv.h`, 스레드 안전성과 포인터 프로버넌스, PAL 계약, 프리스탠딩 (freestanding) 모드, 플랫폼 빌드 관련 사항을 다룬다. 이것은 이 매뉴얼에서 가장 어려운 내용이며, 의도적으로 맨 마지막에 놓았다 — 1-5부의 어떤 것도 여기에 의존하지 않는다.

목차

1. [스택리스 코루틴](#)
2. [Job system](#)
3. [스레드, allocator, 포인터 프로버넌스](#)
4. [별칭 계층](#)
5. [PAL 계약](#)
6. [Freestanding](#) 부분집합
7. [크로스 컴파일](#)
8. [예제와 오용 사례](#)

1. 스택리스 코루틴

코루틴 매크로는 `switch` 안에서 `__LINE__` 라벨을 사용하여 스택리스 상태 기계를 구현한다. 코루틴 함수는 `proven_i32`를 반환해야 한다: 0은 `yield` 또는 대기 중을 의미하고, 1은 완료를 의미한다.

구조체

```
typedef struct {
    proven_i32 state;
} proven_coro_t;
```

매크로

매크로	의도
<code>PROVEN_CORO_INIT(c)</code>	첫 사용 전에 <code>state</code> 를 0으로 설정한다.
<code>PROVEN_CORO_BEGIN(c)</code>	코루틴 <code>switch</code> 블록을 시작한다.
<code>PROVEN_CORO_YIELD(c)</code>	재개 지점을 저장하고 0을 반환한다.
<code>PROVEN_CORO_AWAIT(c, cond)</code>	<code>cond</code> 가 참이 될 때까지 0을 반환하고, 참이 되면 계속 진행한다.
<code>PROVEN_CORO_END(c)</code>	완료로 표시하고 1을 반환한다.
<code>PROVEN_CORO_IS_DONE(c)</code>	<code>state</code> 가 -1인지 확인한다.

어떻게 전개되는가 (그리고 유일하게 중요한 규칙 하나)

이 매크로들은 고전적인 Protothreads/Duff's-device 기법이다. `BEGIN`은 `switch (state)`를 열고, 각 `YIELD/AWAIT`는 `__LINE__`을 재개 라벨로 기록한 뒤 `return`하며, `END`는 `switch`를 닫는다. 개념적으로는 다음과 같다:

```
proven_i32 f(Ctx *c) {
    switch (c->coro.state) {          /* PROVEN_CORO_BEGIN */
    case 0:
        /* ... code before the first yield ... */
        c->coro.state = 42; return 0; case 42:; /* PROVEN_CORO_YIELD on line 42 */
        /* ... code after the yield ... */
    }
    c->coro.state = -1; return 1; /* PROVEN_CORO_END */
}
```

함수는 각 `yield` 지점에서 반환되고, 맨 위에서 다시 진입하여 재개 `case`로 곧장 점프한다. 그 결과로 기억해야 할 단 하나의 규칙:

지역(스택) 변수는 **yield**를 넘어 살아남지 못한다. **YIELD/AWAIT**를 넘어 값이 유지되어야 하는 것은 코루틴 자신의 구조체 안에(또는 **static**으로) 있어야 한다. 지역 변수는 호출할 때마다 다시 생성되기 때문이다.

전개에서 따라 나오는 다른 제약들: 두 코루틴 매크로는 같은 소스 라인을 공유해서는 안 되고(**__LINE__**이 충돌한다), **YIELD/AWAIT**는 여러분 자신의 **switch** 안에 있으면 안 된다(잘못된 **case**에 착지하게 된다). 코루틴은 스택이 없으므로 자신이 호출하는 헬퍼 함수에서는 **yield**할 수 없다 — 오직 자기 본문에서만 가능하다.

반례 — **yield**를 넘는 지역 변수

```
static proven_i32 bad(Ctx *c) {
    PROVEN_CORO_BEGIN(&c->coro);
    int i = 0;                      /* WRONG: a local */
    while (i < 3) {
        i += 1;
        PROVEN_CORO_YIELD(&c->coro); /* on re-entry `i` is re-initialized to 0 */
    }
    PROVEN_CORO_END(&c->coro);      /* loop never ends: infinite yields */
}
/* RIGHT: put `i` in Ctx (like `value` in the Counter example below). */
```

예제(스케치: 코루틴은 함수 전체이므로 문장 조각으로는 보여줄 수 없다 — 이 형태를 컴파일한 버전은 [8절](#)의 다루어진 예제, `manual/examples/ex_06_coro.c`이다):

```
typedef struct Counter {
    proven_coro_t coro;
    int value;
} Counter;

static proven_i32 counter_next(Counter *c) {
    PROVEN_CORO_BEGIN(&c->coro);
    c->value = 0;
    while (c->value < 3) {
        c->value += 1;
        PROVEN_CORO_YIELD(&c->coro);
    }
    PROVEN_CORO_END(&c->coro);
}

Counter c = {0};
PROVEN_CORO_INIT(&c.coro);
while (!counter_next(&c)) {
    use_value(c.value);
}
```

2. Job 시스템

작업(job) 시스템은 워커 스레드와 경계가 있는 MPMC 방식 큐를 소유한다. 생산자는 `routine(arg)` 작업 항목을 제출한다. 워커는 시스템이 닫히고 비워질 때까지 큐에 들어온 작업을 실행한다.

구조체

```
typedef struct {
    void (*routine)(void *arg);
    void *arg;
} proven_job_t;
```

```
typedef struct proven_job_sys proven_job_sys_t;
```

`proven_job_sys_t`는 불투명(`opaque`) 타입이다.

함수

API	의도	반환
-----	----	----

proven_job_system_init(alloc, num_workers, max_queue_capacity, out_sys)	job system을 할당하고 시작한다. 큐 용량은 2의 거듭제곱이어야 한다.	proven_err_t.
proven_job_system_close(sys)	새 작업 수락을 중단한다.	void.
proven_job_system_destroy(sys)	필요하면 닫고, 큐를 비우고, 워커를 join하고, 자원을 해제한다.	void.
proven_job_submit(sys, routine, arg)	작업 하나를 제출한다. 다른 제출자와 스레드 안전하다.	큐에 들어가면 true, 가득 차거나 닫혀 있으면 false.
proven_job_execute_one(sys)	호출 스레드가 사용 가능한 작업 하나를 실행하게 한다.	작업이 실행되었으면 true.

중요한 제약:

- proven_job_system_close()는 제출자와 경쟁할 수 있다. close 전에 진입을 허가받은 제출은 close가 반환되기 전에 작업을 commit하거나 false를 반환하며, 그 이후의 제출은 거부된다.
- proven_job_system_destroy()는 여전히 proven_job_submit()을 호출하는 생산자 스레드와 경쟁(race)해서는 안 된다.
- close를 정지 신호로 사용하려면 close를 호출하고, 모든 생산자를 join한 뒤에만 destroy해야 한다.
- 큐 시퀀스 카운터는 하나의 job-system 수명 동안 부호 있는 포인터 차이(signed pointer-difference) 범위를 넘어 랩(wrap)하지 않는다고 가정한다.

동시성 모델

큐는 max_queue_capacity개의 슬롯을 가진 고정 크기 링 버퍼이다(2의 거듭제곱이어야 하며, 그래서 슬롯 인덱스는 $\text{seq} \ \& \ (\text{capacity} - 1)$ 이다). 생산자와 소비자는 큐 전체를 잠그지 않고 **원자적 시퀀스 카운터**로 협조한다:

- 생산자(proven_job_submit)는 다음 enqueue 위치를 원자적으로 확보한다. 확보가 아직 소비되지 않은 가장 느린 슬롯을 덮어쓰게 되는 경우, 큐가 가득 찬 것이며 submit은 용량이 생기기까지 기다리거나 대기 중인 작업을 덮어쓰는 대신 false를 반환한다. 워커 wake를 post하는 과정에서는 플랫폼 동기화에 잠시 진입할 수 있다.
- 소비자(워커 스레드, 또는 proven_job_execute_one을 통한 여러분 자신의 스레드)는 다음 dequeue 위치를 원자적으로 확보하고, routine/arg를 읽고, 슬롯을 재사용 가능으로 표시한 뒤, 확보 구간 바깥에서 routine을 실행한다.

유향 워커는 계속 폴링하고 yield하는 대신 플랫폼 카운팅 세마포어에서 블록한다. 성공한 submit은 슬롯을 먼저 공개한 뒤 permit 하나를 post한다. 대기 중인 워커가 없어도 permit은 남으므로 빈 큐를 확인한 시점과 대기에 들어가는 시점 사이에서 wake가 유실되지 않는다. close는 먼저 새 진입을 막고 이미 허가된 submit이 끝나기를 기다린 다음, 워커마다 최종 permit 하나를 post한다. 외부 proven_job_execute_one이 워커보다 먼저 작업을 가져가면 남은 permit은 단지 stale 상태가 된다. 워커는 빈 큐를 다시 확인한 뒤 다시 대기하거나, close 뒤라면 종료한다.

submit과 execute가 MPMC-안전하므로, 여러 스레드가 동시에 제출할 수 있고 여러 스레드가 동시에 소비할 수 있다. 라이브러리가 여러분을 위해 **하지 않는 것**: 그것은 *작업이 건드리는 데이터*를 동기화하지 않는다. 같은 변수를 쓰는 두 작업은 여전히 자체 잠금/원자 연산이 필요하다 — 큐는 인계(handoff)만 순서 짓고 작업 자체는 아니다. 그 작업이 사용하는 할당자(allocator)의 스레드 안전성, 할당을 공유할 때의 포인터 프로버넌스 위험, 그리고 락프리 도구 상자(CAS, ABA, 태그 포인터, hazard pointer, 에폭 기반 회수)에 대해서는 2장 § 7 "Allocator 스레드 안전성과 프로버넌스"를 참조하라.

수명 주기 상태 기계(이 순서로 구동하라):

```
init ---> running --(close)--> closed --(destroy: drain + join)--> freed
```

- init은 num_workers개의 OS 스레드와 슬롯 버퍼를 미리 예약한다.
- close는 플래그를 뒤집어 이후의 모든 submit이 false를 반환하게 한다. 이미 허가된 submit은 close가 반환되기 전에 정리되며, 수락된 모든 작업은 여전히 실행된다.

- destroy는 필요하면 다른 다음, 큐가 비고 모든 워커가 join될 때까지 호출 스레드를 블록한 뒤, 모든 것을 해제한다.

메모리 가시성. destroy 내부에서 워커가 join되는 것은 동기화 지점이다: 모든 작업의 모든 메모리 효과는 destroy가 반환된 후 destroy를 호출한 스레드에 가시적이다. 따라서 결과를 수집하는 안전한 패턴은 "모두 submit → close → destroy → 결과 읽기"이다. 그 join 이전에 다른 스레드에서 작업의 출력을 읽으려면 여러분 자신의 동기화가 필요하다.

반례

```
/* WRONG: capacity must be a power of two. */
proven_job_system_init(alloc, 4, 100, &sys); /* 100 is not 2^n */

/* WRONG: reading a result before the join makes it visible. */
int total = 0;
(void)proven_job_submit(sys, accumulate, &total);
proven_println("total = {}", PROVEN_ARG(total)); /* data race: job may still be running */
/* RIGHT: */
proven_job_system_close(sys);
proven_job_system_destroy(sys); /* joins workers -> writes now visible */
proven_println("total = {}", PROVEN_ARG(total));
```

```
/* WRONG: ignoring the submit result drops work silently when the ring is full. */
proven_job_submit(sys, work, arg); /* [[nodiscard]]: a false return means NOT queued */
```

예제(스케치인 이유는 job routine이 자체 함수여야 하기 때문이다. 공유 카운터가 실제로 필요로 하는 원자 연산을 포함한 컴파일된 버전은 8절의 다루어진 예제, manual/examples/ex_06_job.c이다):

```
static void increment(void *arg) {
    int *p = arg;
    *p += 1;
}

proven_job_sys_t *sys = NULL;
proven_err_t e = proven_job_system_init(alloc, 2, 64, &sys);
if (!proven_is_ok(e)) return e;
```

```
int value = 0;
if (!proven_job_submit(sys, increment, &value)) {
    proven_job_system_close(sys);
    proven_job_system_destroy(sys);
    return PROVEN_ERR_BUSY;
}
```

```
proven_job_system_close(sys);
proven_job_system_destroy(sys);
```

routine이 전혀 보이지 않는, 수명 주기 그 자체는 평범한 문장 코드이다:

```
proven_job_sys_t *sys = NULL;
proven_err_t e = proven_job_system_init(alloc, 2, 64, &sys); /* capacity: power of two */
if (proven_is_ok(e)) {
    /* ... submit work here, from this thread or from producers you own ... */
    proven_job_system_close(sys); /* every later submit now returns false */
    proven_job_system_destroy(sys); /* drains the queue, joins the workers */
}
```

3. 스레드, allocator, 포인터 프로버넌스

이 절은 2장에서 여기로 옮겨 왔다. 매뉴얼에서 가장 어려운 내용이 독자가 가장 먼저 여는 장 중 하나에 놓여 있었고, 1-5장의 어떤 것도 여기에 의존하지 않는다. proven 객체를 스레드 간에 공유하고 있거나 그 위에 락

프리 구조를 만들고 있다면 이 절이 필요한 절이다. 그렇지 않다면 통째로 건너뛰어도 되고, 이후에 아무 지장도 없다.

proven_allocator_t 트레이트는 ctx 하나에 함수 포인터 셋일 뿐이며, **자체적인 동기화를 전혀 담고 있지 않다**. 동시 사용이 안전한지는 전적으로 트레이트 뒤에 있는 구체 allocator에, 그리고 allocator가 만들어 낸 *객체*를 어떻게 공유하는지에 달려 있다. 이 절은 먼저 보장 사항을 밝히고, 그다음 더 깊은 포인터 프로버넌스 위험과 그 위에 동시성 구조를 만들 때 필요한 락프리 개념들을 설명한다.

3.1 무엇이 스레드 안전하고 무엇이 아닌가

Allocator	한 인스턴스의 동시 사용	이유
proven_heap_allocator()	동시 alloc/realloc/free에 안전	트레이트는 상태가 없다(ctx == NULL). 플랫폼의 aligned_alloc/posix_memalign/_aligned_malloc+free로 넘기며, 이들은 C11 이래 스레드 안전하다.
proven_arena_t	안전하지 않음	proven_arena_alloc_*는 arena->offset을 비원자적으로 읽고-수정하고-쓴다.
proven_pool_t	안전하지 않음	proven_pool_*는 프리리스트(bin, bin_len)를 비원자적으로 pop/push한다.

여기서 두 가지 규칙이 따라 나온다:

1. **"allocator 안전"은 "객체 안전"이 아니다.** 힙 allocator를 쓰더라도 그 위에 세워진 컨테이너들(proven_u8str_t, proven_array_t, proven_map_t, ...)은 내부 잠금을 전혀 추가하지 않는다. 한 스레드가 append하는 동안 다른 스레드가 읽는 proven_u8str_t는 allocator가 얼마나 스레드 안전하든 데이터 경쟁이다. 공유 가변 객체에는 언제나 여러분의 동기화가 필요하다.
2. **Arena와 풀(pool)은 공유해서는 안 된다.** 동시 proven_arena_alloc은 offset을 찢어 놓아 *같은 바이트*를 두 스레드에 넘기거나 할당 하나를 통째로 잃어버릴 수 있고, 동시 pool pop/push는 같은 슬롯을 두 번 내주거나 bin_len을 언더플로시킬 수 있다. 데이터 경쟁은 그 자체로 미정의 동작이다 — 그 결과가 왜 "숫자가 하나 틀린 것"보다 나쁜지는 § 7.3을 보라.

3.2 한 문단으로 보는 포인터 프로버넌스

C의 추상 기계에서 모든 포인터는 **프로버넌스**를 지닌다: 그 포인터가 유래한 저장소 인스턴스의 정체성이다. 여기서 중요한 규칙은 둘이다: (a) 포인터를 그 프로버넌스 객체의 수명이나 경계 바깥에서 사용하는 것은 미정의이며, (b) 최적화기는 **프로버넌스가 다른 포인터끼리는 엘리머싱하지 않는다**고 가정하고 그 근거로 메모리 접근을 재정렬하거나 제거해도 된다. 할당, 재할당, 해제는 모두 프로버넌스를 만들고 없앤다 — 바로 그래서 동기화되지 않은 공유와 나쁘게 상호작용한다.

3.3 스레드에서 할당 + 프로버넌스가 물어뜯는 지점

- **여기서 realloc은 항상 재배치한다.** 플랫폼의 realloc은 *새로 할당 + 복사 + 옛것 해제*이므로(정렬된 블록은 이식성 있게 제자리에서 크기를 바꿀 수 없다), 성공한 성장은 **항상** 새 프로버넌스를 가진 새 객체를 반환하고 옛 객체를 끝낸다. 옛 포인터를 남겨 둔 복사본은 무엇이든 — 캐시해 둔 원소 포인터, 빌려온 proven_mem_view_t/proven_u8str_view_t — 이제 땀글링이다. 단일 스레드에서는 성장을 넘어 뷰를 들고 있지 않음으로써 이를 피하지만, 스레드가 여럿이면 다른 스레드가 아무 순간에나 공유 컨테이너를 성장시킬 수 있고, 여러분의 뷰는 해제된 저장소를 가리키게 된다(규칙 (a): UB, 게다가 데이터 경쟁).
- **주소 재사용 / ABA.** free 후 alloc(특히 pool의 프리리스트 재활용)은 *같은 주소*를 다른 객체에 새 프로버넌스로 돌려줄 수 있다. 옛 포인터를 여전히 들고 있는 스레드는 옛 프로버넌스를 가정한다. 규칙 (b)에 따라 컴파일러는 바이트가 일치하더라도 둘을 엘리머싱하지 않는 것으로 취급할 수 있고, happens-before 간선이 없으면 새 객체에 대한 쓰기가 가시적일 필요도 없다. 같은 주소, 다른 프로버넌스 — 여전히 UB다.
- **uintptr_t 왕복 + 찢어진 읽기.** 아레나(arena)는 정렬 계산을 위해 backing.ptr을 정수로 변환한다. arena는 정수로부터 포인터를 날조하는 대신 원래 backing.ptr을 오프셋하여 결과 포인터를 유도하도록(그 프로버넌스를 보존하도록) 조심한다 — 프로버넌스 관점에서 올바른 기법이다. 하지만 offset이 경쟁 상황에서 찢어진 채로 읽히면, 계산된 포인터가 backing 객체 *바깥*에 착지할 수 있다. 즉 그 위치에 대한 유효한 프로버

년스가 없는 접근이 된다. 경쟁은 단지 틀린 오프셋이 아니라 그곳을 가리킬 권리가 없는 포인터를 만들어 낸다.

- **데이터 경쟁 × 프로버넌스 추론 = 잘못된 값이 아니라 미스컴파일.** 컴파일러는 각 스레드 안에서 단일 스레드적, 프로버넌스 기반의 비앨리어싱 추론을 적용하므로, 경쟁하는 allocator는 최적화기가 런타임에는 성립하지 않는 비앨리어싱을 "증명"하게 만들 수 있다 — 찢어진 포인터, 컴파일러가 겹칠 수 없다고 믿는 이중 할당, 사라진 저장에 그 결과다. 실패 양상은 어쩌다 나오는 값이 아니라 구조적이다.

3.4 락프리 도구 상자 (그 위에 동시성을 만든다면, 개념 정리)

proven은 락프리 allocator나 안전 메모리 회수(safe-memory-reclamation) 기법을 전혀 구현하지 않는다. 이 allocator들 위에 동시 자료구조를 만든다면 다음은 여러분이 직접 공급한다(<stdatomic.h>는 사용 가능하다 — 6장의 job system이 큐 인덱스에 이를 쓴다). 아래는 표준적인 조각들과 그것들이 위의 프로버넌스 위험과 어떻게 관계되는지다.

- **CAS (compare-and-swap).** 거의 모든 락프리 코드의 바탕이 되는 원자 기본 연산이다:

atomic_compare_exchange_strong(&p, &expected, desired)는 $p == \text{expected}$ 일 때만 원자적으로 $p = \text{desired}$ 로 설정하고, 아니면 현재 값을 보고한다. 다른 스레드가 그 워드를 먼저 바꾸지 않았을 때만 한 스레드가 변경을 공표하게 해 준다.

```
text /* Treiber-stack style push (sketch): node_t and n are illustrative, so this is a listing rather than a
compiled block. */ _Atomic(node_t *) head; node_t *old = atomic_load(&head); do { n->next = old; } while (!
atomic_compare_exchange_weak(&head, &old, n));
```

- **ABA 문제.** CAS는 *값*을 비교하지 이력을 비교하지 않는다. 락프리 pop이 head == A를 읽고 A->next를 설치할 계획을 세운 뒤 CAS를 한다. 그 사이에 다른 스레드들이 A를 pop하고, 그 후속을 pop하고, 그것들을 해제한 뒤, A를 다시 push하면(주소가 재사용되어), CAS는 여전히 head == A를 보고 성공한다 — 해제된 메모리를 가리키는 포인터를 설치하면서. 값은 일치했지만(A→B→A) 세상은 바뀌었다. pool의 프리리스트는 순진하게 락프리로 만들면 교과서적인 ABA 후보다.
- **태그 포인터 / 버전 카운터.** 단조 증가하는 태그를 포인터 옆에 담아 함께 CAS한다(더블 폭 CAS를 쓰거나, 낮은 정렬 비트나 상위 비트를 훔쳐서). 성공한 갱신마다 태그가 올라가므로 A→B→A 시퀀스는 다른 태그로 돌아오고 CAS는 실패한다 — ABA가 탐지된다. 비용/한계: 비트 훔치기는 정렬이 보장되어야 하고 쓸 수 있는 주소 범위를 줄인다. 전체 폭 태그는 하드웨어 더블 워드 CAS(예: cmpxchg16b)를 필요로 한다. 태그는 원리상 랩할 수 있다. **결정적으로, 태그는 공유 워드에서의 ABA *탐지*를 해결할 뿐 프로버넌스를 복원하지는 않는다. ** 재사용된 주소는 여전히 새 객체이며, 태그는 낡은 관점에 따라 행동하는 것을 막아 줄 뿐이다.
- **Hazard pointer (안전 메모리 회수).** 각 스레드가 단일 기록자/다중 독자 "hazard" 슬롯을 몇 개 소유한다. 공유 포인터를 역참조하기 전에 그 포인터를 슬롯에 공표하고 다시 검증한다. 객체를 해제하려는 스레드는 먼저 모든 hazard 슬롯을 훑고, 누군가 보호 중이면 해제를 **미룬다**(retire 목록). 이는 메모리를 유계로 유지하고 use-after-free와 회수-ABA를 막아 주며, 대가는 보호되는 접근마다 저장 한 번 + 펜스 하나, 그리고 스레드당 동시에 보호할 수 있는 포인터 개수가 고정된다는 점이다.
- **에폭 기반 회수(EBR).** 전역 에폭 카운터를 둔다. 스레드는 공유 구조를 건드리는 동안 현재 에폭을 "핀"하고, 은퇴시킨 메모리에는 은퇴한 에폭을 태그한다. 에폭 *e*에서 은퇴한 메모리는 모든 스레드가 *e*를 지난 것이 관찰된 뒤에야 해제된다(에폭 두어 개 분량의 유예 기간). 연산당 비용은 hazard pointer보다 싸지만(핀 플래그 하나뿐), 핀한 채로 멈춰 버린 스레드는 모든 회수를 막는다 — 무한한 메모리 증가. 변종: 정지 상태 기반(QSBR)과 구간 기반 회수.
- **이것들이 프로버넌스와 어떻게 이어지는가.** 회수 기법(hazard pointer, EBR)은 정확히 해제된 객체의 저장소를 살려 두기 위해 — 그 프로버넌스를 유효하게 유지하기 위해 — 존재한다. 어떤 스레드도 더는 참조할 수 없을 때까지 말이다. CAS + 태그는 공유 *워드*를 일관되게 유지하지만 그것이 가리키는 대상의 수명에 대해서는 아무 말도 하지 않는다. 그래서 올바른 락프리 스택은 보통 **둘 다** 필요하다: 태그(head 워드의 ABA용) 그리고 SMR 기법(pop된 노드의 안전한 회수용). proven의 pool이나 arena는 둘 중 아무것도 주지 않으므로, 동시성은 그것들 위에 층으로 쌓아야지 가정해서는 안 된다.

3.5 proven에서의 안전한 패턴

- **스레드별 arena/pool.** 각 스레드에 자신의 proven_arena_t / proven_pool_t를 준다. 공유가 없으면 경쟁도 없고 스레드 간 프로버넌스도 없다 — 가장 단순하게 올바른 설계이며, 보통 가장 빠르기도 하다.
- **스레드 간 alloc/free에는 힙을 쓰되, 객체는 동기화하라.** 스레드 A가 할당하고 스레드 B가 proven_heap_allocator()로 해제하는 것은 괜찮다. 하지만 둘이 잠금 없이 같은 proven_array_t/proven_u8str_t를 변경하는 것은 괜찮지 않다.
- **happens-before 간선과 함께 소유권을 넘겨라.** 포인터를 다른 스레드로 넘길 때는 뮤텍스, release/acquire 순서를 가진 원자 연산, 또는 (job system 같은) 큐를 통해서 넘겨, 한 번에 한 스레드만 그것을 관찰하게 하라. 이렇게 하면 각 할당의 프로버넌스가 한 스레드 안에 갇히고, 생산자의 쓰기가 소비자에게 가시화된다.
- **빌려온 뷰를 스레드 간에 넘기지 말라.** 빌림이 지속되는 내내 소유자가 성장/이동/해제 하지 않는다고 보장되지 않는 한 그렇다 — 그리고 여기서 성장은 항상 재배치임을 기억하라.
- **꼭 arena/pool을 공유해야 한다면 감싸라.** 여러분 자신의 뮤텍스로 감싸거나(또는 § 7.4의 도구로 진짜 락프리 allocator를 만들거나) 하라. 내장된 것들은 한 번에 한 소유자를 가정한다.

4. 별칭 계층

왜 두 번째 이름 집합이 존재하는가

proven_u8str_view_slice는 26자다. 그런 함수를 스무 번 호출하는 파일에서 접두사는 한 줄의 3분의 1을 차지하고, 정작 달라지는 부분 — 여러분이 실제로 읽는 부분 — 은 남은 자리에 눌러 들어간다.

C에서 접두사는 장식이 아니다. 이 언어에는 함수를 위한 전역 이름 공간이 하나뿐이므로, 슬라이스(slice)를 뜻하는 slice나 create를 내보내는 라이브러리는 언젠가 프로그램이 링크하는 다른 무언가와 충돌하게 된다. 접두사는 C 라이브러리를 다른 것들과 안전하게 조합할 수 있게 해 주는 장치이며, 진지한 C 라이브러리마다 접두사가 있는 이유다.

alias_xcv.h는 그 타협안이다: 모든 공개 심벌에 대한 더 짧은 xcv_ 표기를, 여러분이 선택해서 포함하는 헤더에 담았다. 포함하면 xcv_u8str_view_slice가 동작하고, 포함하지 않으면 짧은 이름은 아예 존재하지 않으므로 무엇과도 충돌하지 않는다.

정규(canonical) 이름은 여전히 진실의 원천으로 남는다 — ABI에 대해서도, 이 매뉴얼에 대해서도, 테스트에 대해서도 그렇다. 별칭 계층은 표기이지 API가 아니다: 긴 이름으로 내보내지 않은 것은 짧은 이름으로도 내보내지 않으며, 완전성 게이트(tests/test_docs_alias_completeness.c)는 공개 함수가 별칭 없이 추가되면 빌드를 실패시킨다. 구멍 난 별칭 계층은 아예 없느니만 못하기 때문이다 — 그것을 채택한 호출자는 컴파일 오류를 하나씩 만나며 그 구멍들을 발견하게 된다.

잘못됨 — 헤더를 포함하지 않고 별칭을 기대하기:

```
#include "proven.h"
xcv_u8str_view_t v; /* wrong: proven.h does not pull in the alias layer */
```

잘못됨 — 한 파일에서 같은 개념에 두 표기를 섞기:

```
proven_result_u8str_t s = xcv_u8str_create(alloc, 32); /* wrong: pick one and stay with it */
```

둘 다 컴파일된다. 둘 다 좋은 생각은 아니다: 이제 독자는 함수 하나를 따라가기 위해 두 어휘를 모두 알아야 한다.

정규 헤더 다음에 포함하라:

```
#include "proven.h"
#include "proven/alias_xcv.h"
```

```
xcv_allocator_t heap = xcv_heap_allocator();
(void)heap;
xcv_println("{} ", XCV_ARG(123));
```

별칭 설계 규칙:

- 별칭은 전처리기 편의 기능이지 별도의 ABI가 아니다.

- 정규 API가 진실의 원천으로 남는다.
- 프로젝트가 자체적인 더 짧은 로컬 접두사를 원한다면 `alias_xcv.h`를 템플릿으로 취급하라.
- 공개 API 이름이 추가되거나 제거될 때 별칭 테스트를 동기화 상태로 유지하라.

별칭 매크로 계열:

- 포매팅: `XCV_ARG`, `XCV_ARG_CSTR`, `XCV_ARG_CSTR_N`, `XCV_FMT_IS_OK`.
- 컨테이너: `XCV_ARRAY_*`, `XCV_RING_*`, `XCV_MAP_*`, `XCV_LIST_*`.
- 오류 및 산술: `XCV_ERR_*`, `XCV_IS_OK`, `XCV_CKD_*`.
- 코루틴: `XCV_CORO_*`.
- 파일시스템 및 sysio: `xcv_fs_*`, `xcv_print*`, `xcv_scan*` 별칭.
- 타입 및 allocator: `xcv_*_t` 타입 별칭과 `allocator` 별칭.

소스에 근거한 완전한 별칭 테이블은 [7장: 별칭 인덱스](#)를 참조하라.

5. PAL 계약

왜 syscall을 격리했는가

운영체제를 건드리는 모든 라이브러리는 OS별 코드를 어디에 둘지 결정해야 한다. 흔한 답은 "필요한 곳마다"이고, 그러면 소스 곳곳에 `#ifdef _WIN32`가 흩뿌려진다. 그것도 동작은 하지만, 누적되는 두 가지 비용이 있다: 이 라이브러리가 OS에 실제로 무엇을 요구하는지 아무도 알 수 없고, 이식은 모든 파일을 감사한다는 뜻이 된다.

이 라이브러리는 그 전부를 한 디렉터리에 넣는다. `platform/`은 **syscall을 하는 유일한 장소다**. `src/proven/` 안의 모든 것은 PAL을 통해 호출하는 이식 가능한 C이며, 그 결과:

- **요구 사항 목록이 곧 파일 목록이다.** 이 라이브러리가 운영체제에 요구하는 것은 정확히 `proven_sys_*` 함수들의 집합이다 — 숨은 것도 없고, 뒤늦게 발견될 것도 없다.
- **이식의 범위가 유계다.** 새 대상은 `platform/`을 다시 구현한다. `src/proven/` 안의 것은 아무것도 바뀌지 않고, 이식 가능한 절반을 검사하는 테스트들은 계속 통과한다.
- **Freestanding은 특수 사례가 아니라 같은 메커니즘이다.** 호스티드 PAL 파일 없이 빌드하면 남는 것이 이식 가능한 코어다. [freestanding 가이드](#)가 별도의 이식 작업이 아니라 어떤 파일을 빼는지의 목록인 이유가 그것이다.

PAL은 **내부다**. `proven_sys_*` 함수는 공개 API가 아니며 시그니처가 바뀔 수 있다. 여러분이 호출할 것은 `fs.h`, `sysio.h`, `time.h`, `random.h`의 공개 래퍼다. 심벌 게이트도 이를 알고 있어서 PAL을 "문서화되어야 한다"는 요구에서 면제한다. 바로 그것이 여러분이 대상으로 프로그래밍할 표면이 아니기 때문이다.

잘못됨 — 애플리케이션 코드에서 PAL을 직접 호출하기:

```
proven_sys_fs_open(path, flags); /* wrong: internal. Use proven_fs_open. */
```

오류 변환, 인자 검증, 그리고 다음 릴리스에서도 그 호출이 계속 동작한다는 보장을 모두 잃는다.

PAL 영역:

- `proven_sys_mem`: 힙 할당 백엔드.
- `proven_sys_fs`: 파일, 디렉터리, 경로, 링크, 권한, 잠금.
- `proven_sys_time`: 클럭, 슬립, 시간 분해.
- `proven_sys_env`: 환경 변수.
- `proven_sys_thread`: job system을 위한 스레드와 동기화.
- `proven_sys_io`: 표준 스트림 I/O.
- `proven_sys_math`: 필요한 경우의 수학 헬퍼.

공개 애플리케이션 코드는 고수준 API를 선호해야 한다:

- 파일시스템 연산에는 `proven_fs_*`.
- 콘솔 I/O에는 `proven_sysio_*`와 `print/scan` 매크로.
- 시간에는 `proven_time_*`.

- 스레드 작업에는 proven_job_*.

PAL 내부 예외: 스레드 수명 주기 코드는 불투명한 OS 메타데이터를 진입점을 넘어 살려 두기 위해 내부 플랫폼 할당이 필요할 수 있다. 이 예외는 PAL 코드 안에 머물러야 하며 핵심 컨테이너 API로 새어 나가서는 안 된다.

6. Freestanding 부분집합

PROVEN_FREESTANDING는 OS가 없거나 libc가 최소인 대상을 위한 축소된 부분집합을 빌드한다. 현재 freestanding 빌드는 또한 다음을 정의한다:

`-DPROVEN_FREESTANDING -DPROVEN_FMT_NO_FLOAT -DPROVEN_NO_U16STR -ffreestanding`

현재 freestanding 구성에서 사용 가능한 모듈:

- types
- error
- memory
- align
- allocator
- arena
- pool
- buffer
- array
- list
- ring
- map
- algorithm
- u8str
- coro
- scan
- 부동소수점 포매팅이 없는 fmt
- panic — 패닉(panic) 처리

제외되거나 스텝 처리된 모듈:

- heap — 힙(heap): 컴파일되지만, proven_heap_allocator()는 유효하지 않은 0 allocator를 반환한다.
- u16str: 현재 freestanding 빌드에서 PROVEN_NO_U16STR에 의해 제외됨.
- time: 부분적. src/proven/time.c는 컴파일되므로 proven_time_breakdown()과 datetime 포매터는 사용 가능하지만, 클럭 백엔드 (platform/proven_sys_time.c)는 포함되지 않는다 — 따라서 proven_time_now(), proven_time_now_datetime(), proven_time_sleep()은 링크할 구현이 없다.
- fs, sysio, mmap, job: 호스티드/PAL 서비스로, 현재 freestanding 부분집합에서 제외됨.

정확한 소스 목록과 명령 예제는 manual-freestanding-ko.md를 참조하라.

7. 크로스 컴파일

왜 실행할 수도 없는 대상으로 빌드하는가

이식 가능하다고 주장하는 라이브러리는 조용히 썩어 가는 주장을 하고 있는 것이다. 포인터가 64비트라고, char가 부호 있다고, 정렬되지 않은 로드가 괜찮다고 가정한 코드는 그것이 작성된 기계에서는 완벽하게 컴파일되고 여섯 달 뒤 ARM 보드에서 깨진다 — 그리고 그것을 깨뜨린 커밋은 무해해 보였다.

./nob cross는 아무것도 실행하지 않고 매트릭스의 모든 대상에 대해 라이브러리를 컴파일한다. 설정된 MinGW 경로는 smoke 실행 파일을 추가로 링크한다. 약해 보이지만 그렇지 않다: **대부분의 이식성 실패는 컴파일 타임 실패다.** 가정한 폭이 아닌 타입, 없는 인트린식, 대상이 실제로 강제하는 정렬 요구, 거기에는 존재하지 않는 헤더 — 전부 컴파일러에서 실패한다. 이 기계에서, 몇 초 만에, 그것을 도입한 커밋에서.

컴파일 전용 검사가 잡을 수 없는 것은 동작이다: 엔디언 버그, 런타임에만 일어나는 정렬 폴트, 그리고 타이밍에 의존하는 모든 것. 그것들에는 실제 하드웨어나 에뮬레이터가 필요하고, 이 매트릭스는 그런 척하지 않는다. 이것은 이식성 테스트의 값싼 절반이며, 비싼 절반을 결코 하지 않는 대신 모든 빌드에서 돌아간다.

크로스 빌드는 또한 freestanding 프로파일이 실제 대상 — 그중에도 Cortex-M과 RISC-V — 에 대해 실전으로 검증되는 곳이다. 그래서 "운영체제 없음"이라는 주장은 가이드의 한 문단이 아니라 컴파일러가 검사한다.

잘못됨 — 크로스 매트릭스가 초록색이라고 해서 라이브러리가 그 대상에서 동작한다는 증거로 취급하기:

```
/* wrong: ./nob cross compiles and links objects. It does not execute anything.
   Endianness, alignment faults and real timing still need the hardware. */
```

빌드 드라이버 명령은 다음과 같다:

```
./nob cross -build-root build-out/proven_c_lib
```

현재 대상 범주:

- 네이티브 GCC 호스티드.
- 사용 가능한 경우 네이티브 Clang 호스티드.
- Linux AArch64 호스티드 컴파일 전용.
- Linux ARM 하드플로트 호스티드 컴파일 전용.
- i686-linux-gnu-gcc 또는 gcc -m32를 통한 Linux i686 호스티드 컴파일 전용.
- MinGW를 통한 Windows x86_64 및 i686 WinAPI 컴파일·링크 smoke 검사.
- ARM Cortex-M freestanding 컴파일 전용.
- RISC-V ELF freestanding 컴파일 전용.

규칙:

- 없는 선택적 컴파일러는 건너뛴다.
- 실제 컴파일 오류는 명령을 실패시킨다.
- 크로스 빌드는 설정된 MinGW 컴파일·링크 smoke를 제외하면 컴파일 전용이며, 어느 경로도 런타임 검증은 아니다.
- 런타임 동작은 여전히 대상 러너, 에뮬레이터, 장치, 또는 OS 환경이 필요하다.

8. 예제와 오용 사례

코루틴 매크로는 하나의 소스 라인을 공유해서는 안 된다

잘못됨:

```
PROVEN_CORO_YIELD(&c->coro); PROVEN_CORO_YIELD(&c->coro);
/* wrong: both use the same __LINE__ value */
```

올바름:

```
PROVEN_CORO_YIELD(&c->coro);
PROVEN_CORO_YIELD(&c->coro);
```

코루틴 지역 변수는 평범한 지역 변수이다

스택리스 코루틴은 호출자에게 반환된다. yield를 넘어 값이 유지되어야 하는 지역 변수는 코루틴 함수 안의 자동 지역 변수가 아니라 코루틴 상태 객체 안에 있어야 한다.

잘못됨:

```
static proven_i32 next(Task *t) {
    int temporary = 0;
    PROVEN_CORO_BEGIN(&t->coro);
    temporary = 42;
    PROVEN_CORO_YIELD(&t->coro);
    use_int(temporary); /* wrong assumption: ordinary local lifetime/state is not persistent */
}
```

```
PROVEN_CORO_END(&t->coro);
}
```

올바름:

```
typedef struct Task {
    proven_coro_t coro;
    int temporary;
} Task;
```

Job destroy는 제출자와 경쟁해서는 안 된다

잘못됨:

```
/* thread A */
proven_job_system_destroy(sys);

/* thread B at the same time */
proven_job_submit(sys, work, arg); /* wrong: external synchronization missing */
```

올바름:

```
proven_job_system_close(sys); /* submit과 경쟁할 수 있으며 submit을 false로 만든다 */
join_producer_threads();      /* 이 지점 뒤에는 어떤 생산자도 sys에 접근하지 않는다 */
proven_job_system_destroy(sys);
```

Job 인자 수명

잘못됨:

```
void submit_bad(proven_job_sys_t *sys) {
    int value = 10;
    proven_job_submit(sys, work, &value);
} /* wrong: value may be dead before work runs */
```

올바름:

```
JobData *data = allocate_job_data();
proven_job_submit(sys, work_and_free_data, data);
```

별칭 계층은 정규 문서를 가려서는 안 된다

잘못됨:

```
/* Document only xcv_map_t and forget proven_map_t. */
```

올바름: 정규 proven_ API를 먼저 문서화한 다음, 별칭을 선택적 로컬 표기로 언급하라.

다루어진 예제: 경계가 있는 job system

테스트 스위트가 컴파일하고 실행한다. 계약이 요구하는 순서에 주목하라: submit, 그다음 close, 그다음 destroy. proven_job_system_destroy는 proven_job_submit과 경쟁해서는 안 된다.

```
#include <stdatomic.h>
```

```
/*
 * 작업 시스템: 일꾼 스레드와 경계가 있는 원자적 MPMC 큐. 놀고 있는 일꾼은 플랫폼
 * 동기화에 세워 둔다. 이것이 순서를 정해 주는 것은 일의 *넘김*이지, 그 일이 건드리는
 * 자료가 아니다. 아래 계수기가 그냥 int 가 아니라 atomic 인 이유가 그것이다 - 두 작업이
 * 같은 변수를 올리는 것은 부르는 쪽이 달리 말하지 않는 한 자료 경합이다.
 *
 * 수명은 곧은 한 줄이고, 선택 사항이 아니다.
 *
 *      init -> submit... -> close -> destroy
 *
 * destroy 는 submit 과 경합해서는 안 된다. 라이브러리 안의 어느 것도 그것을 강제하지
 * 않는다. 부르는 쪽이 먼저 생산자를 멈춰야 한다. 여기서는 생산자가 이 스레드 하나뿐이라,
 * "생산자를 멈춘다" 는 것은 "닫기 전에 submit 반복문을 끝낸다" 는 뜻이다.
 */
```

```

#define JOB_COUNT 64

static void increment(void *arg) {
    atomic_int *counter = arg;
    /* relaxed 로 충분하다. 우리는 합계가 맞기만 하면 되지, 그것에 맞춰 무엇이 순서를
     * 정할 필요가 없다. 결과를 우리에게 공표하는 것은 destroy 안의 join 이다. */
    atomic_fetch_add_explicit(counter, 1, memory_order_relaxed);
}

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    proven_job_sys_t *sys = NULL;
    /* 큐 용량은 2의 거듭제곱이어야 한다 - 고리가 마스크로 순번을 칸에 대응시킨다.
     * JOB_COUNT 보다 크게 잡아, 일꾼이 모두 아직 뜨는 중이어도 submit 이 고리가 가득 찬
     * 것을 만나지 않게 했다. */
    proven_err_t err = proven_job_system_init(alloc, 4, 128, &sys);
    EXAMPLE_REQUIRE(proven_is_ok(err), "starting a job system with 4 workers should succeed");
    if (!proven_is_ok(err)) return 1;

    /* destroy 뒤까지 산다. 작업의 인자는 작업보다 오래 살아야 하고, 작업은 destroy 가
     * 큐를 비울 때까지 돈다. */
    atomic_int counter = 0;

    proven_size_t submitted = 0;
    for (proven_size_t i = 0; i < JOB_COUNT; ++i) {
        /* submit 은 고리가 가득 찼거나 시스템이 닫혔을 때 false 를 돌려준다. 큐에 자리가
         * 나기를 기다리는 일도, 일을 조용히 버리는 일도 없다. 깨우는 경로에서 잠깐 플랫폼
         * 동기화에 들어갈 수는 있다. 그 답을 무시하는 것이 작업을 잃는 방법이고, 그래서
         * [[nodiscard]] 가 붙어 있다. */
        if (!proven_job_submit(sys, increment, &counter)) {
            /* 실제 부르는 쪽이라면 물러났다 다시 시도하거나 proven_job_execute_one 으로
             * 그 자리에서 돌릴 것이다. 여기서 고리가 가득 찼다는 것은 위의 크기 잡기가
             * 틀렸다는 뜻이므로, 덮어 두지 말고 그렇다고 말한다. */
            EXAMPLE_REQUIRE(false, "the queue was sized to hold every job");
            break;
        }
        ++submitted;
    }

    /* 이 스레드가 유일한 생산자이고 submit 을 마쳤다 - 그러니 닫아도 안전하다. close 는
     * 그 뒤의 submit 을 모두 실패시킨다. 이미 줄에 선 작업은 그대로 돈다. */
    proven_job_system_close(sys);

    /* destroy 는 큐가 비고 모든 일꾼이 join 될 때까지 막는다. 그 join 이 동기화 지점이다.
     * destroy 가 돌아온 뒤에는 모든 작업의 모든 기억 효과가 이 스레드에 보인다. 이 줄
     * 앞에서 `counter` 를 읽는 것은 일꾼들이 아직 쓰고 있는 값을 읽는 것이다. */
    proven_job_system_destroy(sys);

    int ran = atomic_load(&counter);
    EXAMPLE_REQUIRE(submitted == JOB_COUNT, "every job should have been accepted");
    EXAMPLE_REQUIRE(ran == (int)submitted, "every submitted job should have run exactly once");

    printf("submitted %zu jobs, %d ran\n", (size_t)submitted, ran);

    return EXAMPLE_OK();
}

```

다루어진 예제: 스택리스 코루틴

테스트 스위트가 컴파일하고 실행한다. 모두를 잡는 규칙: 지역 변수는 yield를 넘어 살아남지 못하므로, 코루틴이 서스펜션을 넘어 필요로 하는 모든 상태는 자신의 구조체 안에 있어야 한다.

```
/*
 * 스택 없는 코루틴은 사실 switch 문이다. BEGIN 이 저장된 상태를 두고 switch 를 열고,
 * YIELD 마다 __LINE__ 을 재개 표지로 적고 *돌아간다*. 다음 호출은 함수를 위에서부터 다시
 * 들어와 그 표지로 곧장 뛴다.
 *
 * 뒤따르는 모든 것이 그 사실 하나에서 나온다.
 *
 * - 지역 변수는 yield 를 넘겨 살아남지 *못한다*. 함수가 돌아갔고 그 스택 프레임은
 *   사라졌다. 남아 있어야 할 것은 코루틴 자신의 구조체에 산다 - 아래의 `value` 와
 *   `remaining` 이 하는 일이 그것이다.
 * - 코루틴 매크로 둘이 같은 소스 줄에 있으면 안 된다(__LINE__ 이 부딪친다).
 * - 자기가 부른 도우미 안에서 yield 할 수 없다. 멈춰 둘 스택이 없기 때문이다.
 *
 * 그 대가로, 멈춰 있는 코루틴은 정확히 자기 구조체만큼의 값이 든다 - 상태 4바이트에
 * 여러분이 곁에 둔 것 - 그리고 스레드도, 스택도, 문맥 전환도 없다.
 */

typedef struct {
    proven_coro_t coro;
    /* 생성기의 상태. 보통의 반복문이라면 `int i` 같은 지역 변수였을 것들이다. 여기서는
     * 필드여야 한다. 아니면 재개할 때마다 처음 값으로 되돌아가 반복문이 끝나지 않는다. */
    int value;
    int remaining;
} squares_t;

/* 코루틴은 proven_i32 를 돌려준다. 0 = 멈췄음(다시 불러 달라), 1 = 끝. */
static proven_i32 squares_next(squares_t *g) {
    PROVEN_CORO_BEGIN(&g->coro);

    g->remaining = 5;
    g->value = 1;

    while (g->remaining > 0) {
        g->value = g->value * g->value;
        PROVEN_CORO_YIELD(&g->coro);    /* 부르는 쪽이 여기서 g->value 를 읽는다 */
        g->value = g->value + 1;          /* 정확히 이 줄에서 재개한다 */
        g->remaining -- 1;
    }

    PROVEN_CORO_END(&g->coro);
}

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* 코루틴은 기억을 소유하지 않으므로 지울 것이 없다 - 다만 그것이 내놓는 값들은
     * 어딘가로 가야 하고, 그 문자열에는 소유자가 있다. */
    proven_result_u8str_t out = proven_u8str_create(alloc, 32);
    EXAMPLE_REQUIRE(proven_is_ok(out.err), "creating the output string should succeed");
    if (!proven_is_ok(out.err)) return 1;

    squares_t gen = {0};
    PROVEN_CORO_INIT(&gen.coro);    /* 조건 없이, 첫 호출 전에, 정확히 한 번 */

    int produced = 0;
    int last = 0;
```

```

/* 끝까지 굴린다. squares_next 는 본문의 끝을 지나 달리는 호출에서 1 을 돌려준다 -
 * 그 호출은 값을 내놓지 않으므로, 반복문 본문은 0 을 돌려준 동안에만 돈다. */
while (!squares_next(&gen)) {
    proven_fmt_result_t r = proven_u8str_append_fmt_grow(alloc, &out.value, "{} ",
                                                         PROVEN_ARG(gen.value));

    EXAMPLE_REQUIRE(PROVEN_FMT_IS_OK(r), "appending a generated value should succeed");
    last = gen.value;
    ++produced;
}

/* 끝난 상태는 돌려볼는다. 상태는 -1 이고 그대로 있다. 다시 불러도 본문을 다시 돌리지
 * 않고 1 만 돌려준다. */
EXAMPLE_REQUIRE(PROVEN_CORO_IS_DONE(&gen.coro), "the generator should have finished");
EXAMPLE_REQUIRE(squares_next(&gen) == 1, "a finished coroutine stays finished");

/* 1, 그다음 (1+1)^2 = 4, 그다음 (4+1)^2 = 25, 그다음 676, 그다음 458329. */
EXAMPLE_REQUIRE(produced == 5, "the generator yields once per iteration");
EXAMPLE_REQUIRE(last == 458329, "the state carried across every yield");
EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&out.value),
                                     PROVEN_LIT("1 4 25 676 458329 ")),
                 "the generated sequence should be exactly this");

printf("squares: %s\n", proven_u8str_as_cstr(&out.value));

proven_u8str_destroy(alloc, &out.value);
return EXAMPLE_OK();
}

```

Proven Freestanding 모드 (v0.1.1)

6부 — 더 나아가기. 선행 조건: 2-5부, 그리고 6장. 이 가이드를 마치면 운영체제도 libc도 없는 대상으로 이 라이브러리를 빌드할 수 있고, 그 상황에서 어떤 모듈이 살아남고 어떤 모듈이 살아남지 못하는지 정확히 알게 된다.

이 가이드는 nob.c와 공개 헤더가 구현한 현재의 PROVEN_FREESTANDING 구성을 설명한다.

이 가이드의 생김새에 대한 한마디. 다른 장들과 달리 이 가이드의 대략 절반은 절차다 — 플래그 목록, 파일 목록, 그리고 컴파일 명령 두 개. 그 절들은 의도적으로 짧다: 길게 설명된 컴파일 명령은 아무도 읽지 않는 컴파일 명령이다. *결정*을 담은 절들 — 이 모드가 존재하는 이유 (§ 0), "제외됨"이 정말 무슨 뜻인지 (§ 3), 여러분의 패닉(panic) 핸들러가 무엇을 해야 하는지 (§ 5), 부동소수점을 뺀으로써 무엇을 잃는지 (§ 8), 호스티드 호출이 왜 링크 타임에 실패하는지 (§ 9), 수명 규칙이 왜 여기서 더 중요한지 (§ 10), 그리고 이 가이드의 주장들이 어떻게 검증되는지 (§ 11) — 은 온전히 풀어 썼다.

0. 이 모드가 존재하는 이유, 그리고 라이브러리 전체가 이 제약에 의해 빚어진 이유

Freestanding 모드는 펌웨어, 커널, 부트로더, 하이퍼바이저, 그리고 그 밑에 운영체제가 없는 그 밖의 모든 곳을 위한 것이다 — malloc도, open도, printf도 없고, 흔히 libc 자체가 없다.

C 표준에는 이를 가리키는 이름이 있다. **호스티드(hosted)** 구현은 표준 라이브러리 전체를 주고 프로그램을 main에서 시작한다. **Freestanding** 구현은 소수의 헤더만을 보장한다 — <stddef.h>, <stdint.h>, <limits.h>와 몇 가지 더 — 그리고 이것이 베어메탈을 대상으로 하는 컴파일러가 주는 것이다. -ffreestanding은 컴파일러에게 정확히 그렇게 가정하라고 알린다.

대부분의 C 라이브러리는 여기서 아예 쓸 수 없으며, 그 이유가 거창한 경우는 드물다. 세 단계 아래 호출에 묻혀 있는 malloc 하나, 오류 경로의 printf 하나, 또는 결코 실행되지 않는 무언가에 의해 초기화되는 전역 하나다. 호스트에 대한 숨은 의존성 하나면 마이크로컨트롤러에서 라이브러리를 쓸 수 없게 만들기에 충분하다.

이것이 이 라이브러리의 설계 결정 대부분을 낳은 제약이며, 그 연결을 보아 둘 가치가 있다. 그렇지 않으면 취향처럼 보이는 선택들을 설명해 주기 때문이다:

설계 결정	앞선 장에서는 이렇게 읽힌다	그리고 진짜 이유는
Allocator는 매개변수다 (2장)	"전략을 갈아 끼울 수 있도록"	여기에는 malloc이 없다. static 배열 위의 arena가 유일한 allocator이며, allocator를 매개변수로 받던 코드는 이미 그대로 동작한다.
오류는 반환되는 값이다 (1장)	"무시할 수 없도록"	여기에는 errno가 없고 풀어낼 스택도 없다. 반환값이 존재하는 유일한 메커니즘이다.
Panic은 혹은 거친다 (1장 § 6)	"오버라이드할 수 있도록"	여기에는 abort()도 stderr도 없다. 기본 동작이 trap인 것은 이식성 있게 할 수 있는 전부가 그것이기 때문이다.
Syscall은 platform/에만 산다	"관심사의 분리"	새 대상을 위해 교체해야 하는 유일한 디렉터리다. src/proven/ 안의 모든 것은 구성상 이식 가능하다.
C 문자열 대신 뷰 (3장)	"길이를 잃어버리지 않도록"	strlen은 libc다. 뷰는 자기 길이를 함께 가지고 다니며 아무것도 필요로 하지 않는다.

달리 말하면: 프리스탠딩(freestanding) 지원은 옆에 덧붙인 기능이 아니다. 이 매뉴얼의 나머지는 여기서 살아남도록 지어진 라이브러리를 계속 설명해 온 것이고, 이 가이드는 무엇이 살아남고 무엇이 살아남지 못하는지 알아내는 곳이다.

무엇을 포기하는가

운영체제를 필요로 하는 모든 것, 그리고 그 밖에는 아무것도 아니다:

- **힙 없음.** proven_heap_allocator는 존재하지 않는다. 여러분이 메모리를 공급하고 — 보통은 static 배열이다 — 그 위에 아레나(arena)나 풀(pool)을 얹는다.
- **파일시스템 없음, 표준 스트림 없음, 클럭 없음, OS 난수 없음.** fs.h, sysio.h, mmap.h, time.h, 그리고 OS 엔트로피 소스는 모두 호스티드 서비스다. proven_chacha_rng_t는 여러분이 직접 시드를 주면 여전히 동작하며, proven_random_set_source가 그것을 위한 것이다.
- **기본적으로 부동소수점 포매팅 없음.** PROVEN_FMT_NO_FLOAT가 그것을 뺀다. 부동소수점 경로는 포맷터에서 가장 큰 코드 덩어리이고 대부분의 펌웨어는 double을 출력할 일이 없기 때문이다. 다시 켜는 방법은 8절이 설명한다.
- **UTF-16 문자열 없음.** PROVEN_NO_U16STR가 켜져 있다. 여기에는 대화할 Windows API가 없다.

여러분이 유지하는 것은 이식 가능한 코어 전체다: 오류, 뷰, 검사된 산술, 정렬, arena, pool, 버퍼, 문자열, 배열, 맵, 리스트, 링, 정렬, 검색, 해싱, 인코딩, 코루틴, 그리고 정수 포매팅과 스캐닝.

여전히 물어뜯는 것

대상이 작아졌다고 해서 수명 규칙이 느슨해지지 않는다는 오�히려 더 중요해진다. 실수를 알아챌 운영체제도 없고, 여러분이 아직 들고 있는 메모리를 재사용하지 않으려 해 줄 할당자(allocator)도 없기 때문이다. 10절은 짧고, 사람들이 건너뛰는 절이다.

1. 빌드 프로파일

빌드 드라이버는 다음 freestanding 플래그를 사용한다:

```
-std=c23
-ffreestanding
-DPROVEN_FREESTANDING
-DPROVEN_FMT_NO_FLOAT
-DPROVEN_NO_U16STR
```

드라이버는 여전히 -std=c23을 먼저 탐지하고, 컴파일러가 과도기 표기를 사용하는 경우 -std=c2x로 폴백한다. 빌드 래퍼가 두 플래그 표기 중 어느 쪽도 받아들이더라도 freestanding 소스 계약은 C23 우선으로 유지된다.

호스티드 ./nob freestanding 명령은 또한 로컬 freestanding 테스트 실행 파일을 빌드 호스트에서 정적으로 링크한다. ./nob cross의 크로스 freestanding 대상은 컴파일 전용 오브젝트 검사를 사용한다.

프로젝트 freestanding 검사를 실행하라:

```
cc nob.c -o nob
./nob freestanding -build-root build-out/proven_c_lib
```

크로스 빌드 매트릭스를 실행하라:

```
./nob cross -build-root build-out/proven_c_lib
```

2. 사용 가능한 소스 파일

현재 freestanding 빌드에서 nob.c는 호스티드 전용 모듈을 제외한 이식 가능한 소스 파일을 컴파일한다.

포함된 핵심 모듈:

```
src/proven/memory.c
src/proven/arena.c
src/proven/pool.c
src/proven/buffer.c
src/proven/heap.c
src/proven/u8str.c
src/proven/array.c
src/proven/ring.c
src/proven/map.c
```

```
src/proven/algorithm.c
src/proven/hash.c
src/proven/encode.c
src/proven/random.c
src/proven/float_decimal.c
src/proven/float_parse.c
src/proven/float_format.c
src/proven/time.c
src/proven/fmt.c
src/proven/scan.c
src/proven/panic.c
platform/proven_sys_math.c
```

제외된 호스티드 모듈:

```
src/proven/u16str.c
src/proven/fs.c
src/proven/stream.c
src/proven/sysio.c
src/proven/mmap.c
src/proven/job.c
platform/proven_sys_fs.c
platform/proven_sys_thread.c
platform/proven_sys_io.c
platform/proven_sys_env.c
platform/proven_sys_time.c
platform/proven_sys_random.c
platform/proven_sys_mem.c
```

대상에 대한 실제 플랫폼 백엔드를 함께 제공하지 않는 한, 제외된 호스티드 모듈을 베어메탈 빌드에 추가하지 말라.

3. 모듈 사용 가능성

이 표를 읽는 법, 그리고 "제외됨"의 의미

여기서 모듈이 제외되는 이유는 정확히 하나다: 운영체제만이 제공할 수 있는 무언가를 필요로 하기 때문이다. 코드가 작은 대상에서 검증되지 않았거나 누군가 아직 손을 못 댔다는 뜻이 아니다 — `fs.h`는 파일시스템이, `sysio.h`는 표준 스트림이, `mmap.h`는 가상 메모리가, `job.h`는 스레드가 필요하다. 이식할 것이 없다.

흥미로운 행은 사용 가능하지도 제외되지도 않은 두 개다:

- **heap.h는 스택이다.** `proven_heap_allocator()`는 여전히 존재하고 여전히 컴파일되며, 함수 포인터가 널인 `allocator`를 반환한다 — `proven_alloc_is_valid`가 유효하지 않다고 보고하는 그것이다. 조용히 다른 어딘가에서 할당하지도 않고 링크에 실패하지도 않는다. 이 선택은 중요하다: `allocator` 트레이트를 대상으로 작성된 코드는 베어메탈 대상에서도 계속 컴파일되고, 깨지는 지점은 힙을 요구한 바로 그 한 줄이며, 런타임에, 여러분이 직접 쓸 수 있는 검사와 함께 드러난다. 대신 무엇을 넘길지는 4절이 보여 준다.
- **time.h는 제한적이다.** `datetime` 포매팅은 여러분이 공급한 숫자에 대한 순수 산술이므로 컴파일된다. 없는 것은 그 숫자다: 클럭을 읽는 것은 `syscall`이므로 여기서 `proven_time_now`에는 백엔드가 없다. 여러분 자신의 하드웨어 타이머에서 얻은 타임스탬프를 포매팅하라.

"사용 가능"으로 표시된 모든 것은 이식 가능한 코어이며, "사용 가능"이란 호스티드 빌드에서 도는 바로 그 테스트들이 여기에 대해서도 돈다는 뜻이다 — `freestanding` 프로파일은 이 표에서 주장되는 것이 아니라 릴리스마다 `./nob freestanding`이 빌드하고 검사한다.

모듈	현재 <code>freestanding</code> 프로파일에 서의 상태	비고
<code>types.h</code> , <code>error.h</code> , <code>align.h</code> , <code>memory.h</code>	사용 가능	고정 폭 정수와 <code>uintptr_t</code> 지원이 필요하다.

allocator.h	사용 가능	트레이트만 제공. 호출자가 뒷받침 allocator를 제공한다.
arena.h	사용 가능	정적 메모리 영역을 위한 주 allocator.
pool.h	사용 가능	호출자가 제공하는 기반 allocator(흔히 arena)를 사용한다.
buffer.h	사용 가능	고정 용량 바이트 버퍼.
u8str.h	사용 가능	U8 문자열과 뷰.
u16str.h	제외됨	현재 프로파일은 PROVEN_NO_U16STR를 정의한다.
array.h, list.h, ring.h, map.h	사용 가능	숨겨진 OS 의존성 없음.
algorithm.h	사용 가능	배열용 정렬/검색 헬퍼.
hash.h	사용 가능	FNV-1a, SipHash-2-4, CRC-32, SHA-256 — 바이트 단위로 정확, OS 의존성 없음.
encode.h	사용 가능	Hex와 Base64 — 순수 계산, OS 없음.
fmt.h	부동소수점 없이 사용 가능	현재 프로파일은 PROVEN_FMT_NO_FLOAT를 정의한다.
scan.h	사용 가능	메모리 뷰용 스캐너.
float_parse.h	사용 가능	proven_strtod, proven_parse_double_ascii, proven_parse_f64_ascii가 모두 여기서 컴파일된다. 십진수 → binary64 엔진은 정수 연산만 쓰므로 libc가 필요 없다. 유일한 차이는 freestanding 빌드가 오버플로/언더플로에서 errno를 설정하지 않는다는 점이며, 대신 반환된 proven_err_t가 그 정보를 담는다 — errno 자체가 없는 타겟에서 확인해야 할 값이다. 이는 이 프로파일이 실제로 컴파일에서 제외하는 fmt.h의 부동소수점 출력과는 별개다.
float_format.h	fmt.h 연동 없이 사용 가능	binary64 포매터는 정수 연산만 쓰므로 컴파일되지만, 현재 프로파일이 PROVEN_FMT_NO_FLOAT를 정의하므로 {}는 부동소수점을 렌더링하지 않는다. 코드 크기를 감수할 가치가 있다고 판단한 타겟에서 자릿수 출력이 필요하다면 float_format.h의 진입점을 직접 호출할 것.

time.h	제한적	핵심 datetime 포매팅은 컴파일 가능하지만, 실제 PAL 시간은 제외된다.
heap.h	스텝	proven_heap_allocator()는 유효하지 않은 allocator를 반환한다.
fs.h, stream.h, mmap.h, sysio.h, job.h	제외됨	호스티드 PAL 서비스가 필요하다.
random.h	사용 가능	생성기와 헬퍼는 순수 산술이다. proven_random_bytes도 여기서 동작하지만, proven_random_set_source로 엔트로피 소스 — 보드의 TRNG, 링 오실레이터, 또는 ADC 잡음 바닥 — 를 설치한 후에만 가능하다. 아무것도 설치되지 않았을 때는 클럭으로 시드된 PRNG로 폴백하지 않고 false 를 반환한다. 폴백은 성공처럼 보이면서 아무것도 보고하지 않는 보안 구멍이 될 것이기 때문이다. 그런 다음 proven_chacha_rng_seed_from_entropy가 보드의 엔트로피를 끝없는 암호학적 스트림으로 바꾼다.
coro.h	사용 가능	매크로 전용 스택리스 코루틴 지원.
panic.h	사용 가능	대상별 trap/reset 동작을 위한 오버라이드.

4. 최소 정적 arena 설정

힙이 없으므로 메모리는 정적 블록에서 나오고 그 위에 arena를 얹는다. arena는 그 블록을 소유하지 않는다: alloc은 오프셋을 증가시키고, free는 no-op이며, reset은 전체를 한 번에 되감는다.

이것이 라이브러리 전체가 가능하게 하려고 빚어진 패턴이며, 왜 그것이 통하는지 볼 가치가 있다. 할당할 수 있는 모든 함수는 proven_allocator_t를 매개변수로 받으므로, 호스티드 빌드에서 proven_heap_allocator()를 대상으로 작성된 코드는 arena를 대신 건네주기만 하면 여기서 그대로 돈다. u8str.h, array.h, map.h의 어떤 것도 메모리가 malloc이 아니라 .bss의 static 배열에서 왔다는 것을 알지도, 상관하지도 않는다.

그 블록의 크기를 정할 때 내려야 할 세 가지 결정이 있고, 어느 것도 라이브러리가 대신 내려 줄 수 없다:

- **얼마나 크게.** arena는 성장할 수 없으므로 그 크기는 여러분이 컴파일 타임에 고르는 단단한 한계다. 너무 작으면 할당이 PROVEN_ERR_NOMEM을 반환하기 시작하고, 너무 크면 펌웨어의 나머지가 필요로 하던 RAM을 써 버린 것이다. 이것은 몇 달 뒤 힙 단편화로 발견하는 대신 명시적으로 하라고 요구되는 거래다.
- **언제 reset할지.** arena의 이점 전부는 한 번에 모든 것을 해제하는 데 있다. 자연스러운 지점은 루프 한 번, 수신한 패킷 하나, 명령 하나 — 작업 묶음에 분명한 끝이 있는 곳이면 어디든이다. 그 묶음 동안 할당된 모든 것은 reset에서 죽으므로, **그것보다 오래 살아남아도 되는 것은 없다.**
- **뒷받침 배열의 정렬.** 아래처럼 선언에 alignas(max_align_t)를 붙인다. 그것이 없으면 배열이 double을 담을 수 없는 주소에서 시작할 수 있고, arena에는 그것을 고칠 수단이 없다.

잘못됨 — reset보다 오래 사는, arena 안에 만들어진 뷰:

```

proven_u8str_view_t label = build_label(arena_alloc); /* lives in the arena */
proven_arena_reset(&arena);
send(label); /* wrong: those bytes are free space now, and the next allocation takes them */

```

호스티드 빌드에서는 이것이 테스트에서 꽤찮아 보일 만큼은 자주 살아남는다. 여기서는 다음 할당이 곧바로 같은 바이트를 가져가므로 그렇지 않다.

```

#include "proven/types.h"
#include "proven/memory.h"
#include "proven/arena.h"
#include "proven/array.h"
#include "proven/panic.h"

/* static storage: no OS, no heap, known at link time */
static alignas(PROVEN_MAX_ALIGN) proven_byte_t storage[4096];

proven_arena_t arena = proven_arena_create((proven_mem_mut_t){
    .ptr = storage,
    .size = sizeof storage,
});
proven_allocator_t arena_alloc = proven_arena_as_allocator(&arena);

proven_result_array_t r = PROVEN_ARRAY_INIT(arena_alloc, proven_i32, 16);
if (proven_is_ok(r.err)) {
    proven_array_t values = r.value;
    proven_err_t e = PROVEN_ARRAY_PUSH(&values, proven_i32, 42);
    (void)e;
    PROVEN_ARRAY_DESTROY(&values); /* arena free is a no-op */
}

proven_arena_destroy(&arena); /* also a no-op: the caller owns `storage` */

```

흔한 실수:

```

proven_allocator_t heap = proven_heap_allocator();
heap.alloc_fn(heap.ctx, 64, 8); /* wrong: heap is invalid in PROVEN_FREESTANDING */

```

올바름:

```

proven_allocator_t heap = proven_heap_allocator();
if (!proven_alloc_is_valid(heap)) {
    /* use an arena, pool, or target-provided allocator instead */
    proven_panic("no heap on this target");
}

```

5. Panic 핸들러 오버라이드

여기서는 왜 핸들러 설치가 선택 사항이 아닌가

호스티드 시스템에서 trap하는 panic은 건달 만하다: 프로세스가 죽고, 운영체제가 뒷정리를 하고, 무언가가 그것을 다시 띄운다. 베어메탈에는 그 밑에 아무것도 없다. trap은 코어를 멈추고, 장치가 하던 일이 무엇이든 — 모터를 어떤 속도로 유지하는 것, 무선 링크를 유지하는 것, 히터를 구동하는 것 — CPU가 멈춘 뒤에도 여전히 그러고 있다.

그래서 기본 핸들러는 여러분이 반드시 작성해야 할 핸들러의 자리 표시자이며, 여러분의 핸들러가 무엇을 하는지는 프로그래밍 결정이라기보다 제품 결정이다. 흔한 형태들:

- **하드웨어를 안전 상태로 만든 다음 멈춘다.** 모터 끄기, 출력을 알려진 수준으로, 그다음 회전 대기하거나 디버거를 기다린다. 물리적 액추에이터가 있는 것이라면 무엇에든 올바르다.
- **기록하고 리셋한다.** 리셋을 넘어 살아남는 레지스터나 예약된 RAM 영역에 사유 코드를 쓴 다음, 위치독을 발동시킨다. 다음 부팅이 지난 부팅이 왜 끝났는지 보고한다.

- **요란하게 멈춘다.** 알아볼 수 있는 패턴으로 LED를 깜빡인다. 콘솔이 없는 보드에서는 이것이 진단 채널의 전부이며, 들리는 것보다 값어치가 크다.

1장 §6의 규칙이 여기서는 더 강하게 적용된다: **핸들러는 반환해서는 안 된다.** 반환하면

proven_arena_alloc_or_panic은 애초에 할당되지 않은 블록을 가지고 진행하고, 실패는 "장치가 멈췄다"에서 "장치가 널 포인터를 통해 쓰고 있다"로 옮겨 간다.

잘못됨 — 로그를 남기고 반환하는 핸들러:

```
static void my_panic(const char *msg) {
    uart_write(msg); /* wrong: this returns, and the caller uses a block it never got */
}
```

proven_arena_alloc_or_panic()과 관련 panic 경로는 proven_panic()을 호출하며, 이는 proven_set_panic_handler()로 설치된 핸들러로 디스패치한다. 기본 핸들러는 trap한다.

핸들러는 함수 전체이므로, 이것은 조각이 아니라 리스팅이다:

```
#include "proven/panic.h"

static void my_panic(const char *msg) {
    (void)msg;
    /* optional: write msg to UART or crash log */
    for (;;) {
        /* or reset the MCU */
    }
}

/* install once during startup; pass NULL to restore the default */
proven_set_panic_handler(my_panic);
```

프로덕션 panic 핸들러는 반환해서는 안 된다. 반환하면 panic을 유발한 할당 결과가 유효하다고 보장되지 않는다.

6. Cortex-M 컴파일 명령 예제

```
arm-none-eabi-gcc \
-std=c23 \
-mcpu=cortex-m4 -mthumb \
-ffreestanding -nostdlib \
-DPROVEN_FREESTANDING \
-DPROVEN_FMT_NO_FLOAT \
-DPROVEN_NO_U16STR \
-Iinclude -Iplatform \
-c src/proven/memory.c -o memory.o
```

실제 펌웨어 빌드는 위에 나열된 포함 freestanding 모듈을 컴파일하고, 대상 startup 코드를 컴파일하고, 여러분의 linker script로 링크해야 한다.

7. RISC-V ELF 컴파일 명령 예제

```
riscv64-elf-gcc \
-std=c23 \
-ffreestanding -nostdlib \
-DPROVEN_FREESTANDING \
-DPROVEN_FMT_NO_FLOAT \
-DPROVEN_NO_U16STR \
-Iinclude -Iplatform \
-c src/proven/arena.c -o arena.o
```

여러분의 툴체인이 riscv64-unknown-elf-gcc 이름을 사용하면 그 컴파일러를 대신 사용하라. 프로젝트 크로스 매트릭스는 사용 가능한 경우 두 이름을 모두 검사한다.

8. Freestanding 모드에서의 포매팅

무엇이 남는가, 그리고 왜 대개 그것으로 충분한가

포매터는 이식 가능한 계산이다 — 여러분이 공급한 목적지에 바이트를 만들어 넣는다 — 그래서 거의 전부가 여기서 살아남는다. 사라지는 것은 자리 표시자 타입 하나: `double`이다.

큰 손실처럼 들리지만 대개 그렇지 않다. 올바른 부동소수점 포매팅이란 서브노멀을 포함한 모든 입력에 대해 같은 값으로 되입히는 가장 짧은 십진수를 내보내는 것을 뜻하고, 그러려면 큰 정수 산술과 록업 테이블이 필요하다. 그것이 포매터에서 단일로 가장 큰 코드 덩어리다. 대부분의 펌웨어는 정수, 문자열 뷰, 문자, 포인터를 포매팅한다 — 센서 읽은 값은 스케일된 정수이고, 상태 줄은 텍스트다 — 그래서 이 프로파일은 기본적으로 부동소수점 경로를 빼고, 그 덕분에 바이너리가 더 작다.

남는 것: 모든 폭의 정수, 뷰, 문자, 불리언, 포인터에 대한 `{}`. 스펙 문법 전체(폭, 채움, 정렬, 진법, 부호). 여러분 자신의 타입을 위한 `PROVEN_ARG_OF`. 그리고 스캐너 전체 — 같은 무게를 지지 않는 부동소수점 *파싱*까지 포함해서.

작은 대상에서 정말로 부동소수점이 필요하다면 스위치는 `PROVEN_FMT_NO_FLOAT`이고, 그와 함께 가는 용량 조절 손잡이는 § 8a가 다룬다. 결정하기 전에 크기 변화를 측정하라 — 플래시가 32 KB인 부품에서 그것은 반올림 오차가 아니다.

다른 귀결에도 주의하라: **여기에는 `proven_printf`가 없다.** 포매팅은 `proven_u8str_t`나 버퍼에 덧붙이며, 그 바이트를 UART나 RTT 채널로 보내는 것은 여러분의 플랫폼 코드다. freestanding 빌드에는 가정할 수 있는 표준 출력이 없기 때문이다.

부동소수점 포매팅은 `PROVEN_FMT_NO_FLOAT`에 의해 비활성화된다. 정수와 문자열 뷰 포매팅은 계속 사용 가능하다.

올바름(여기서 `alloc`은 4절에서처럼 arena allocator이다 — 힙이 없다):

```
proven_result_u8str_t r = proven_u8str_create(alloc, 32);
if (proven_is_ok(r.err)) {
    proven_fmt_result_t f = proven_u8str_append_fmt_grow(alloc, &r.value,
                                                         "value={}", PROVEN_ARG(123));
    (void)f;
    proven_u8str_destroy(alloc, &r.value);
}
```

잘못됨:

```
proven_u8str_append_fmt_grow(alloc, &s, "{}", PROVEN_ARG(3.14));
/* wrong in the current freestanding profile: float args are excluded, so
   PROVEN_ARG has no _Generic association for double and this fails to compile */
```

8a. Float 큰 정수 용량 조정

정확한 십진수-to-binary64 폴백과 큰 정수 나눗셈 헬퍼는 용량이 `PROVEN_FLOAT_BIGINT_LIMBS`(기본값 160, `proven/float_config.h`에 있음)로 설정되는 고정 크기 큰 정수를 사용한다. 이것이 그들의 스택 사용량을 지배하는 요인이다: 각 큰 정수는 $8 * \text{PROVEN_FLOAT_BIGINT_LIMBS}$ 바이트이며, 나눗셈은 여러 개에 더해 32비트 스크래치를 사용한다.

스택이 빠듯한 대상은 이를 낮출 수 있다. 예를 들면:

```
-DPROVEN_FLOAT_BIGINT_LIMBS=48
```

유지되는 가수 상한(`PROVEN_FLOAT_MAX_SIGNIFICAND_DIGITS`)은 이 용량에서 파생되므로, 더 작은 값이 절대 오버플로하지 않는다: 파서는 파생된 유효 자릿수까지의 입력에 대해 여전히 올바르게 반올림하고, 더 긴 입력에 대해서는 1 ULP 이내에 머문다. Clinger와 Eisel-Lemire 빠른 경로는 큰 정수를 사용하지 않으므로, 일반적인 짧은 입력은 영향을 받지 않는다. binary64 반올림 경계는 최대 767개의 유효 자릿수를 필요로 할 수 있으므로, 병적으로 긴 십진수의 정확한 반올림이 중요할 때는 기본값(160)을 유지하라.

9. 호스티드 API 피하기

왜 이것이 런타임의 뜻밖이 아니라 링크 오류인가

호스티드 모듈은 freestanding 빌드에 아예 컴파일되어 들어가지 않는다. 따라서 그중 하나를 호출하면 링크 오류가 된다 — 빌드 타임에, 쓰지 말았어야 할 함수의 이름을 대는 미정의 심벌로.

그것이 설계가 작동하는 모습이다. 많은 임베디드 라이브러리가 택하는 대안은 런타임에 오류를 반환하는 스텝을 제공하는 것이다. 그러면 마이크로컨트롤러에서 proven_fs_open 호출이 컴파일되고, 링크되고, 출하되고, 아무도 봐야 보지 않은 경로에서 현장에서 실패한다. 여기서는 빌드 밖으로 나갈 수가 없다.

여러분 자신의 코드에 대한 실질적인 귀결: 어떤 모듈이 호스티드 도구와 펌웨어 양쪽에 공유된다면, 호스티드 전용 호출은 `#ifndef PROVEN_FREESTANDING` 뒤에 두어야 하고, 링크 오류가 어떤 것을 놓쳤는지 정확히 알려준다.

현재 freestanding 프로파일에서 다음을 호출하지 말라:

```
proven_fs_open(...);
proven_println(...);
proven_env_get(...);
proven_mmap_create(...);
proven_job_system_init(...);
```

이들은 의도적으로 제외된 호스티드 PAL 파일을 필요로 한다.

10. 수명 규칙은 여전히 적용된다

사람들이 건너뛰는 절, 그리고 여기서 그 대가가 더 큰 이유

Freestanding는 0장 §5의 소유권 규칙을 하나도 완화하지 않는다. 오히려 날을 세운다. 모두 같은 방향을 가리키는 세 가지 이유가 있다:

- **여러분과 메모리 사이에 allocator가 없다.** 힙 블록을 가리키는 덩글링 포인터는 allocator가 그 블록을 아직 다시 내주지 않았다는 이유로 한동안 살아남는 일이 흔하다. arena는 reset 직후 바로 다음 할당에서 같은 바이트를 내주므로, 낡은 뷰는 즉시, 그리고 결정론적으로 덮어써진다.
- **여러분을 붙잡아 줄 것이 없다.** 작은 대상에는 MMU 트랩도 없고, 프로세스를 죽여 줄 운영체제도 없고, 현상에는 새니타이저도 없다. use-after-reset은 크래시하지 않는다 — 남의 데이터를 읽고 그냥 계속 가며, 증상은 전혀 관계없는 곳에서 나타난다.
- **결과가 물리적이다.** 여기서 잘못된 바이트 하나는 모터 명령이거나 무선 패킷일 수 있다.

규칙은 여러분이 이미 아는 그것들이다: 뷰는 소유자가 살아 있는 동안에만 유효하고, 소유된 객체는 그것을 만든 allocator로 정확히 한 번 파괴되며, 호출자 소유 상태 구조체는 복사해서는 안 된다. 달라지는 것은 오차 허용 범위이고, 그것은 0이다.

Freestanding는 컨테이너 규칙을 완화하지 않는다.

잘못됨:

```
int *p = PROVEN_ARRAY_GET_MUT(&arr, int, 0);
PROVEN_ARRAY_PUSH(&arr, int, 9);
*p = 1; /* wrong: push may have moved the array */
```

잘못됨:

```
proven_u8str_view_t key = make_stack_view();
PROVEN_MAP_SET_U8_BORROWED(&map, key, int, 1);
return; /* wrong if map survives after key bytes go out of scope */
```

11. 검증

왜 이 가이드는 믿는 것이 아니라 검사되는가

위의 모든 것은 운영체제 없이 무엇이 컴파일되고 링크되는지에 대한 주장이며, 그런 주장은 조용히 씩는다. 관계없는 커밋에서 이식 가능한 소스 파일에 `#include <stdio.h>` 하나가 추가되면 freestanding 프로파일은 깨진다 — 호스트 빌드에서는 아무도 알아채지 못한다. `stdio.h`는 바로 거기에 있으니까.

그래서 이 프로파일은 서술되는 대신 릴리스마다 빌드된다. `./nob freestanding`은 이식 가능한 코어를 `ffreestanding`과 이 프로파일의 정의들로 컴파일하고, 아래의 검사들을 빌드 호스트에서 **정적으로** 링크한 뒤 실행한다. 그런 다음 `./nob cross`가 같은 프로파일을 실제 임베디드 대상 — 그중에도 Cortex-M과 RISC-V — 에 대해 컴파일 전용 매트릭스로 컴파일한다.

두 검사는 무엇을 잡아낼지를 기준으로 골랐다:

- **힙(heap) 스텝** 검사는 `proven_heap_allocator()`가 여전히 존재하고 `proven_alloc_is_valid`가 거부하는 무언가를 반환함을 증명한다. 누군가 그것을 대신 링크 실패로 만들었다면 트레이트 기반 코드가 베어메탈용으로 컴파일되지 않게 된다 — §3이 설명하는, 이 설계가 피하는 그 실패다.
- **컴파일 검사**는 이 프로파일에 대해 대표적인 프로그램을 빌드하며, 이것이 호스티드 헤더가 이식 가능한 파일에 몰래 들어오는 것을 잡아낸다.

어느 쪽도 실제 하드웨어에서 돌지 않으며, 이 가이드는 그렇다고 주장하지 않는다: 정렬 폴트, 엔디안, 타이밍에는 보드가 필요하다. 검증되는 것은 검증될 수 있는 부분이고, 그것이 모든 빌드에서 돈다. 전부를 아무 빌드에서도 하지 않는 대신에.

프로젝트 freestanding 명령은 빌드 호스트에서 다음 로컬 검사를 빌드하고 실행한다:

```
tests/test_portability_freestanding_heap_stub
tests/test_portability_compile_freestanding
tests/test_portability_compile_nofloat
tests/test_portability_compile_nou16str
tests/test_portability_freestanding
```

크로스 명령은 사용 가능한 임베디드 컴파일러에 대해 컴파일 전용 검사를 수행한다:

```
freestanding-arm-cortex-m4      arm-none-eabi-gcc -mcpu=cortex-m4 -mthumb
freestanding-riscv64-elf        riscv64-elf-gcc
freestanding-riscv64-unknown-elf riscv64-unknown-elf-gcc
```

없는 툴체인은 건너뛰다. 실제 컴파일 실패는 명령을 실패시킨다. 런타임 동작은 여전히 대상 또는 에뮬레이터에서의 검증이 필요하다.

부록 A: alias_xcv.h Alias 인덱스 (7장)

영문 [manual/manual-07-alias-xcv-index.md](#)의 한국어 번역본. 아래 매핑 표는 심볼 대응이라 원문 그대로 둔다.

이것은 조회표이지 읽을거리가 아니다. 매뉴얼에서 훑어 나갈 산문이 없는 유일한 파일이다: 모든 xcv_ 짧은 이름을 정본 proven_ 이름으로 대응시키는 500여 행. 처음 읽을 때는 건너뛰고, 알아보지 못하는 xcv_ 철자를 만났을 때, 또는 자기 코드에 짧은 이름을 도입할지 정할 때 돌아오라.

이 파일이 manual-07 번호를 유지하는 것은 그 번호가 링크와 빌드가 쓰는 안정적인 식별자이기 때문이다. 읽기 순서에서의 위치는 "부록"이며, 이 제목이 말하는 바가 그것이다.

이 부록은 include/proven/alias_xcv.h에서 생성된다. alias 계층은 선택 사항이다. 정본 proven_·PROVEN_ 이름이 ABI, 문서, 테스트의 기준으로 유지된다.

사용 규칙

alias는 일반 헤더 뒤에 include하라:

```
#include "proven.h"
#include "proven/alias_xcv.h"
```

alias 문서와 정본 API 문서를 섞지 마라. 이 인덱스는 철자 맵으로만 사용하라.

Alias 표

alias 501개: 소문자 xcv_ 함수 이름 375개와 대문자 XCV_ 매크로 이름 126개. 표는 include/proven/alias_xcv.h에서 생성된다. 공개 함수에 alias가 없으면 tests/test_docs_alias_completeness가 빌드를 실패시킨다. 다만 그 게이트는 헤더끼리 비교할 뿐 이 부록과는 비교하지 않는다 — 재생성 전에 표는 헤더보다 61행 뒤쳐져 있었으므로, 행을 손으로 고치지 말고 헤더에서 재생성할 것.

행 번호 열은 의도적으로 두지 않았다. 위에 alias가 삽입될 때마다 틀려졌고, 그건 열이 아예 없느니만 못했다.

Alias macro	Expands to
XCV_ARG	PROVEN_ARG
XCV_ARG_CSTR	proven_arg_cstr
XCV_ARG_DATETIME	proven_arg_datetime
XCV_ARG_F64	proven_arg_f64
XCV_ARG_FN	PROVEN_ARG_FN(f)
XCV_ARG_I32	proven_arg_i32
XCV_ARG_I64	proven_arg_i64
XCV_ARG_NONE	proven_arg_none
XCV_ARG_PTR	proven_arg_ptr
XCV_ARG_STR_VIEW	proven_arg_str_view
XCV_ARG_U32	proven_arg_u32
XCV_ARG_U64	proven_arg_u64
XCV_ARRAY_DESTROY	PROVEN_ARRAY_DESTROY
XCV_ARRAY_GET	PROVEN_ARRAY_GET
XCV_ARRAY_GET_MUT	PROVEN_ARRAY_GET_MUT
XCV_ARRAY_INIT	PROVEN_ARRAY_INIT
XCV_ARRAY_IS_VALID	proven_array_is_valid
XCV_ARRAY_POP	PROVEN_ARRAY_POP

XCV_ARRAY_PUSH	PROVEN_ARRAY_PUSH
XCV_CKD_ADD	PROVEN_CKD_ADD
XCV_CKD_MUL	PROVEN_CKD_MUL
XCV_CKD_SUB	PROVEN_CKD_SUB
XCV_CONTAINER_OF	PROVEN_CONTAINER_OF
XCV_CORO_AWAIT	PROVEN_CORO_AWAIT
XCV_CORO_BEGIN	PROVEN_CORO_BEGIN
XCV_CORO_END	PROVEN_CORO_END
XCV_CORO_INIT	PROVEN_CORO_INIT
XCV_CORO_IS_DONE	PROVEN_CORO_IS_DONE
XCV_CORO_YIELD	PROVEN_CORO_YIELD
XCV_DEFAULT_ALIGNMENT	PROVEN_DEFAULT_ALIGNMENT
XCV_ERR_AGAIN	PROVEN_ERR_AGAIN
XCV_ERR_BUSY	PROVEN_ERR_BUSY
XCV_ERR_EOF	PROVEN_ERR_EOF
XCV_ERR_INVALID_ARG	PROVEN_ERR_INVALID_ARG
XCV_ERR_INVALID_ENCODING	PROVEN_ERR_INVALID_ENCODING
XCV_ERR_INVALID_STATE	PROVEN_ERR_INVALID_STATE
XCV_ERR_IO	PROVEN_ERR_IO
XCV_ERR_NOMEM	PROVEN_ERR_NOMEM
XCV_ERR_NOT_FOUND	PROVEN_ERR_NOT_FOUND
XCV_ERR_NEED_MORE	PROVEN_ERR_NEED_MORE
XCV_ERR_OUT_OF_BOUNDS	PROVEN_ERR_OUT_OF_BOUNDS
XCV_ERR_OVERFLOW	PROVEN_ERR_OVERFLOW
XCV_ERR_PERMISSION	PROVEN_ERR_PERMISSION
XCV_ERR_UNSUPPORTED	PROVEN_ERR_UNSUPPORTED
XCV_FS_APPEND	PROVEN_FS_APPEND
XCV_FS_CREATE	PROVEN_FS_CREATE
XCV_FS_LOCK_EXCLUSIVE	PROVEN_FS_LOCK_EXCLUSIVE
XCV_FS_LOCK_SHARED	PROVEN_FS_LOCK_SHARED
XCV_FS_LOCK_UNLOCK	PROVEN_FS_LOCK_UNLOCK
XCV_FS_PATH_SEP	PROVEN_FS_PATH_SEP
XCV_FS_PERM_DEFAULT	PROVEN_FS_PERM_DEFAULT
XCV_FS_PERM_GROUP_R	PROVEN_FS_PERM_GROUP_R
XCV_FS_PERM_GROUP_W	PROVEN_FS_PERM_GROUP_W
XCV_FS_PERM_GROUP_X	PROVEN_FS_PERM_GROUP_X
XCV_FS_PERM_OTHER_R	PROVEN_FS_PERM_OTHER_R
XCV_FS_PERM_OTHER_W	PROVEN_FS_PERM_OTHER_W
XCV_FS_PERM_OTHER_X	PROVEN_FS_PERM_OTHER_X
XCV_FS_PERM_OWNER_R	PROVEN_FS_PERM_OWNER_R
XCV_FS_PERM_OWNER_W	PROVEN_FS_PERM_OWNER_W
XCV_FS_PERM_OWNER_X	PROVEN_FS_PERM_OWNER_X
XCV_FS_READ	PROVEN_FS_READ
XCV_FS_TRUNC	PROVEN_FS_TRUNC
XCV_FS_TYPE_DIR	PROVEN_FS_TYPE_DIR

XCV_FS_TYPE_FILE	PROVEN_FS_TYPE_FILE
XCV_FS_TYPE_OTHER	PROVEN_FS_TYPE_OTHER
XCV_FS_WRITE	PROVEN_FS_WRITE
XCV_INDEX_NOT_FOUND	PROVEN_INDEX_NOT_FOUND
XCV_IS_OK	PROVEN_IS_OK
XCV_KEY_TYPE_INT	PROVEN_KEY_TYPE_INT
XCV_KEY_TYPE_U8_BORROWED	PROVEN_KEY_TYPE_U8_BORROWED
XCV_KEY_TYPE_U8_OWNED	PROVEN_KEY_TYPE_U8_OWNED
XCV_LIST_ENTRY	PROVEN_LIST_ENTRY
XCV_LIST_FOR_EACH	PROVEN_LIST_FOR_EACH
XCV_LIST_FOR_EACH_SAFE	PROVEN_LIST_FOR_EACH_SAFE
XCV_LIT	PROVEN_LIT
XCV_MAP_DESTROY	PROVEN_MAP_DESTROY
XCV_MAP_GET_INT	PROVEN_MAP_GET_INT
XCV_MAP_GET_MUT_INT	PROVEN_MAP_GET_MUT_INT
XCV_MAP_GET_MUT_U8_BORROWED	PROVEN_MAP_GET_MUT_U8_BORROWED
XCV_MAP_GET_U8_BORROWED	PROVEN_MAP_GET_U8_BORROWED
XCV_MAP_GET_U8_OWNED	PROVEN_MAP_GET_U8_OWNED
XCV_MAP_INIT_INT	PROVEN_MAP_INIT_INT
XCV_MAP_INIT_U8_BORROWED	PROVEN_MAP_INIT_U8_BORROWED
XCV_MAP_INIT_U8_OWNED	PROVEN_MAP_INIT_U8_OWNED
XCV_MAP_IS_VALID	proven_map_is_valid
XCV_MAP_REMOVE_INT	PROVEN_MAP_REMOVE_INT
XCV_MAP_REMOVE_U8_BORROWED	PROVEN_MAP_REMOVE_U8_BORROWED
XCV_MAP_REMOVE_U8_OWNED	PROVEN_MAP_REMOVE_U8_OWNED
XCV_MAP_SET_INT	PROVEN_MAP_SET_INT
XCV_MAP_SET_WITH_SCRATCH_INT	PROVEN_MAP_SET_WITH_SCRATCH_INT
XCV_MAP_SET_U8_BORROWED	PROVEN_MAP_SET_U8_BORROWED
XCV_MAP_SET_U8_OWNED	PROVEN_MAP_SET_U8_OWNED
XCV_MAP_SET_WITH_SCRATCH_U8_BORROWED	PROVEN_MAP_SET_WITH_SCRATCH_U8_BORROWED
XCV_MAX_ALIGN	PROVEN_MAX_ALIGN
XCV_MMAP_EXEC	PROVEN_MMAP_EXEC
XCV_MMAP_PRIVATE	PROVEN_MMAP_PRIVATE
XCV_MMAP_READ	PROVEN_MMAP_READ
XCV_MMAP_SHARED	PROVEN_MMAP_SHARED
XCV_MMAP_WRITE	PROVEN_MMAP_WRITE
XCV_OK	PROVEN_OK
XCV_RING_DESTROY	PROVEN_RING_DESTROY
XCV_RING_INIT	PROVEN_RING_INIT
XCV_RING_IS_VALID	proven_ring_is_valid
XCV_RING_POP	PROVEN_RING_POP
XCV_RING_PUSH	PROVEN_RING_PUSH
XCV_SCAN_ARG	PROVEN_SCAN_ARG(x)
XCV_SCAN_ARG_LONG	PROVEN_SCAN_ARG_LONG(ptr)
XCV_SCAN_ARG_TYPE_F64	PROVEN_SCAN_ARG_TYPE_F64

XCV_SCAN_ARG_TYPE_I32	PROVEN_SCAN_ARG_TYPE_I32
XCV_SCAN_ARG_TYPE_I64	PROVEN_SCAN_ARG_TYPE_I64
XCV_SCAN_ARG_TYPE_INT	PROVEN_SCAN_ARG_TYPE_INT
XCV_SCAN_ARG_TYPE_LLONG	PROVEN_SCAN_ARG_TYPE_LLONG
XCV_SCAN_ARG_TYPE_LONG	PROVEN_SCAN_ARG_TYPE_LONG
XCV_SCAN_ARG_TYPE_NONE	PROVEN_SCAN_ARG_TYPE_NONE
XCV_SCAN_ARG_TYPE_SHORT	PROVEN_SCAN_ARG_TYPE_SHORT
XCV_SCAN_ARG_TYPE_STR_VIEW	PROVEN_SCAN_ARG_TYPE_STR_VIEW
XCV_SCAN_ARG_TYPE_U32	PROVEN_SCAN_ARG_TYPE_U32
XCV_SCAN_ARG_TYPE_U64	PROVEN_SCAN_ARG_TYPE_U64
XCV_SCAN_ARG_TYPE_UINT	PROVEN_SCAN_ARG_TYPE_UINT
XCV_SCAN_ARG_TYPE_ULLONG	PROVEN_SCAN_ARG_TYPE_ULLONG
XCV_SCAN_ARG_TYPE_ULONG	PROVEN_SCAN_ARG_TYPE_ULONG
XCV_SCAN_ARG_TYPE_USHORT	PROVEN_SCAN_ARG_TYPE_USHORT
XCV_SCAN_ARG_ULONG	PROVEN_SCAN_ARG_ULONG(ptr)
XCV_U16_LIT	PROVEN_U16_LIT
XCV_VERSION_ENCODE	PROVEN_VERSION_ENCODE(major, minor, patch)
XCV_VERSION_MAJOR	PROVEN_VERSION_MAJOR
XCV_VERSION_MINOR	PROVEN_VERSION_MINOR
XCV_VERSION_NUM	PROVEN_VERSION_NUM
XCV_VERSION_PATCH	PROVEN_VERSION_PATCH
XCV_VERSION_STRING	PROVEN_VERSION_STRING
xcv_alloc_fn_t	proven_alloc_fn_t
xcv_alloc_is_valid	proven_alloc_is_valid
xcv_allocator_t	proven_allocator_t
xcv_arena_alloc	proven_arena_alloc
xcv_arena_alloc_aligned	proven_arena_alloc_aligned
xcv_arena_alloc_aligned_or_panic	proven_arena_alloc_aligned_or_panic
xcv_arena_alloc_or_panic	proven_arena_alloc_or_panic
xcv_arena_alloc_trait	proven_arena_alloc_trait
xcv_arena_as_allocator	proven_arena_as_allocator
xcv_arena_create	proven_arena_create
xcv_arena_destroy	proven_arena_destroy
xcv_arena_free_trait	proven_arena_free_trait
xcv_arena_realloc_aligned	proven_arena_realloc_aligned
xcv_arena_realloc_trait	proven_arena_realloc_trait
xcv_arena_reset	proven_arena_reset
xcv_arena_t	proven_arena_t
xcv_arg_bool	proven_arg_bool
xcv_arg_char	proven_arg_char
xcv_arg_cstr	proven_arg_cstr
xcv_arg_cstr_n	proven_arg_cstr_n
xcv_arg_custom	proven_arg_custom
xcv_arg_datetime	proven_arg_datetime
xcv_arg_f64	proven_arg_f64

xcv_arg_fn	proven_arg_fn
xcv_arg_i32	proven_arg_i32
xcv_arg_i64	proven_arg_i64
xcv_arg_identity	proven_arg_identity
xcv_arg_none	proven_arg_none
xcv_arg_ptr	proven_arg_ptr
xcv_arg_str_view	proven_arg_str_view
xcv_arg_t	proven_arg_t
xcv_arg_type_t	proven_arg_type_t
xcv_arg_u32	proven_arg_u32
xcv_arg_u64	proven_arg_u64
xcv_arg_ucstr	proven_arg_ucstr
xcv_array_binary_search	proven_array_binary_search
xcv_array_create	proven_array_create
xcv_array_destroy	proven_array_destroy
xcv_array_get	proven_array_get
xcv_array_get_mut	proven_array_get_mut
xcv_array_is_valid	proven_array_is_valid
xcv_array_linear_search	proven_array_linear_search
xcv_array_pop	proven_array_pop
xcv_array_push	proven_array_push
xcv_array_reserve	proven_array_reserve
xcv_array_sort	proven_array_sort
xcv_array_t	proven_array_t
xcv_buf_append	proven_buf_append
xcv_buf_create	proven_buf_create
xcv_buf_destroy	proven_buf_destroy
xcv_buf_t	proven_buf_t
xcv_byte_t	proven_byte_t
xcv_c_lib	proven_c_lib
xcv_compare_fn_t	proven_compare_fn_t
xcv_coro_t	proven_coro_t
xcv_crc32	proven_crc32
xcv_crc32_update	proven_crc32_update
xcv_cstr_len	proven_cstr_len
xcv_datetime_t	proven_datetime_t
xcv_env_get	proven_env_get
xcv_eprint	proven_eprint
xcv_eprintln	proven_eprintln
xcv_err_t	proven_err_t
xcv_file_t	proven_file_t
xcv_float_format_f32_policy	proven_float_format_f32_policy
xcv_float_format_f64_policy	proven_float_format_f64_policy
xcv_float_format_options_fixed_default	proven_float_format_options_fixed_default
xcv_float_format_options_scientific	proven_float_format_options_scientific

xcv_float_format_options_shortest	proven_float_format_options_shortest
xcv_fmt_put	proven_fmt_put
xcv_fmt_to_writer_impl	proven_fmt_to_writer_impl
xcv_fprint	proven_fprint
xcv_free_fn_t	proven_free_fn_t
xcv_fs_chmod	proven_fs_chmod
xcv_fs_close	proven_fs_close
xcv_fs_copy	proven_fs_copy
xcv_fs_dir_close	proven_fs_dir_close
xcv_fs_dir_next	proven_fs_dir_next
xcv_fs_dir_open	proven_fs_dir_open
xcv_fs_entry_t	proven_fs_entry_t
xcv_fs_handle_t	proven_fs_handle_t
xcv_fs_is_absolute	proven_fs_is_absolute
xcv_fs_link	proven_fs_link
xcv_fs_list	proven_fs_list
xcv_fs_list_destroy	proven_fs_list_destroy
xcv_fs_lock	proven_fs_lock
xcv_fs_lock_type_t	proven_fs_lock_type_t
xcv_fs_mkdir	proven_fs_mkdir
xcv_fs_mode_t	proven_fs_mode_t
xcv_fs_open	proven_fs_open
xcv_fs_perms_t	proven_fs_perms_t
xcv_fs_pread	proven_fs_pread
xcv_fs_pwrite	proven_fs_pwrite
xcv_fs_read	proven_fs_read
xcv_fs_read_all	proven_fs_read_all
xcv_fs_read_all_u8str	proven_fs_read_all_u8str
xcv_fs_remove	proven_fs_remove
xcv_fs_rename	proven_fs_rename
xcv_fs_rmdir	proven_fs_rmdir
xcv_fs_seek	proven_fs_seek
xcv_fs_size	proven_fs_size
xcv_fs_stat	proven_fs_stat
xcv_fs_stat_t	proven_fs_stat_t
xcv_fs_symlink	proven_fs_symlink
xcv_fs_sync	proven_fs_sync
xcv_fs_sync_dir	proven_fs_sync_dir
xcv_fs_tell	proven_fs_tell
xcv_fs_truncate	proven_fs_truncate
xcv_fs_type_t	proven_fs_type_t
xcv_fs_walk_close	proven_fs_walk_close
xcv_fs_walk_next	proven_fs_walk_next
xcv_fs_walk_open	proven_fs_walk_open
xcv_fs_write	proven_fs_write

xcv_fs_write_all	proven_fs_write_all
xcv_fs_write_file	proven_fs_write_file
xcv_fs_write_file_atomic	proven_fs_write_file_atomic
xcv_fs_write_file_durable	proven_fs_write_file_durable
xcv_hash_bytes	proven_hash_bytes
xcv_hash_keyed	proven_hash_keyed
xcv_base64_decode	proven_base64_decode
xcv_base64_decoded_size	proven_base64_decoded_size
xcv_base64_encode	proven_base64_encode
xcv_base64_encoded_size	proven_base64_encoded_size
xcv_base64url_encode	proven_base64url_encode
xcv_hex_decode	proven_hex_decode
xcv_hex_decoded_size	proven_hex_decoded_size
xcv_hex_encode	proven_hex_encode
xcv_hex_encoded_size	proven_hex_encoded_size
xcv_heap_allocator	proven_heap_allocator
xcv_i16	proven_i16
xcv_i32	proven_i32
xcv_i64	proven_i64
xcv_i8	proven_i8
xcv_intptr_t	proven_intptr_t
xcv_is_ok	proven_is_ok
xcv_is_pow2	proven_is_pow2
xcv_job_execute_one	proven_job_execute_one
xcv_job_submit	proven_job_submit
xcv_job_sys	proven_job_sys
xcv_job_sys_t	proven_job_sys_t
xcv_job_system_close	proven_job_system_close
xcv_job_system_destroy	proven_job_system_destroy
xcv_job_system_init	proven_job_system_init
xcv_job_t	proven_job_t
xcv_key_type_t	proven_key_type_t
xcv_list_init	proven_list_init
xcv_list_insert_after	proven_list_insert_after
xcv_list_is_empty	proven_list_is_empty
xcv_list_node_t	proven_list_node_t
xcv_list_push_back	proven_list_push_back
xcv_list_remove	proven_list_remove
xcv_list_t	proven_list_t
xcv_map_create	proven_map_create
xcv_map_create_trusted	proven_map_create_trusted
xcv_map_create_with_capacity	proven_map_create_with_capacity
xcv_map_destroy	proven_map_destroy
xcv_map_get	proven_map_get
xcv_map_get_mut	proven_map_get_mut

xcv_map_hash	proven_map_hash
xcv_map_is_valid	proven_map_is_valid
xcv_map_key_t	proven_map_key_t
xcv_map_remove	proven_map_remove
xcv_map_reserve	proven_map_reserve
xcv_map_set	proven_map_set
xcv_map_set_u8_owned	proven_map_set_u8_owned
xcv_map_set_with_scratch	proven_map_set_with_scratch
xcv_map_t	proven_map_t
xcv_mem_align_up	proven_mem_align_up
xcv_mem_copy	proven_mem_copy
xcv_mem_move	proven_mem_move
xcv_mem_mut_from_owned	proven_mem_mut_from_owned
xcv_mem_mut_slice_checked	proven_mem_mut_slice_checked
xcv_mem_mut_slice_unchecked	proven_mem_mut_slice_unchecked
xcv_mem_mut_t	proven_mem_mut_t
xcv_mem_t	proven_mem_t
xcv_mem_view_from_owned	proven_mem_view_from_owned
xcv_mem_view_from_u8	proven_mem_view_from_u8
xcv_mem_view_slice_checked	proven_mem_view_slice_checked
xcv_mem_view_slice_unchecked	proven_mem_view_slice_unchecked
xcv_mem_view_t	proven_mem_view_t
xcv_memcmp	proven_memcmp
xcv_mmap_as_view	proven_mmap_as_view
xcv_mmap_create	proven_mmap_create
xcv_mmap_destroy	proven_mmap_destroy
xcv_mmap_flags_t	proven_mmap_flags_t
xcv_mmap_prot_t	proven_mmap_prot_t
xcv_mmap_sync	proven_mmap_sync
xcv_mmap_t	proven_mmap_t
xcv_panic	proven_panic
xcv_parse_double_ascii	proven_parse_double_ascii
xcv_parse_f64_ascii	proven_parse_f64_ascii
xcv_pool_as_allocator	proven_pool_as_allocator
xcv_pool_destroy	proven_pool_destroy
xcv_pool_init	proven_pool_init
xcv_pool_t	proven_pool_t
xcv_print	proven_print
xcv_println	proven_println
xcv_ptrdiff_t	proven_ptrdiff_t
xcv_chacha_rng	proven_chacha_rng
xcv_chacha_rng_fill	proven_chacha_rng_fill
xcv_chacha_rng_next	proven_chacha_rng_next
xcv_chacha_rng_seed	proven_chacha_rng_seed
xcv_chacha_rng_seed_from_entropy	proven_chacha_rng_seed_from_entropy

xcv_entropy_fn	proven_entropy_fn
xcv_random_bytes	proven_random_bytes
xcv_random_set_source	proven_random_set_source
xcv_random_u64	proven_random_u64
xcv_rng_below	proven_rng_below
xcv_rng_f64	proven_rng_f64
xcv_rng_fill	proven_rng_fill
xcv_rng_is_valid	proven_rng_is_valid
xcv_rng_range	proven_rng_range
xcv_rng_shuffle	proven_rng_shuffle
xcv_rng_u64	proven_rng_u64
xcv_xoshiro256ss_next	proven_xoshiro256ss_next
xcv_xoshiro256ss_rng	proven_xoshiro256ss_rng
xcv_xoshiro256ss_seed	proven_xoshiro256ss_seed
xcv_range_contains_ptr	proven_range_contains_ptr
xcv_reader_buffered	proven_reader_buffered
xcv_reader_from_file	proven_reader_from_file
xcv_reader_from_view	proven_reader_from_view
xcv_reader_is_valid	proven_reader_is_valid
xcv_reader_read	proven_reader_read
xcv_reader_read_line	proven_reader_read_line
xcv_realloc_fn_t	proven_realloc_fn_t
xcv_result_array_t	proven_result_array_t
xcv_result_buf_t	proven_result_buf_t
xcv_result_cstr_t	proven_result_cstr_t
xcv_result_f64_t	proven_result_f64_t
xcv_result_file_t	proven_result_file_t
xcv_result_i64_t	proven_result_i64_t
xcv_result_map_t	proven_result_map_t
xcv_result_mem_mut_t	proven_result_mem_mut_t
xcv_result_mem_view_t	proven_result_mem_view_t
xcv_result_mmap_t	proven_result_mmap_t
xcv_result_ring_t	proven_result_ring_t
xcv_result_size_t	proven_result_size_t
xcv_result_u16str_t	proven_result_u16str_t
xcv_result_u64_t	proven_result_u64_t
xcv_result_u8str_t	proven_result_u8str_t
xcv_result_u8str_view_t	proven_result_u8str_view_t
xcv_ring_create	proven_ring_create
xcv_ring_destroy	proven_ring_destroy
xcv_ring_is_valid	proven_ring_is_valid
xcv_ring_pop	proven_ring_pop
xcv_ring_push	proven_ring_push
xcv_ring_t	proven_ring_t
xcv_scan_arg_f64	proven_scan_arg_f64

xcv_scan_arg_i32	proven_scan_arg_i32
xcv_scan_arg_i64	proven_scan_arg_i64
xcv_scan_arg_identity	proven_scan_arg_identity
xcv_scan_arg_int	proven_scan_arg_int
xcv_scan_arg_llong	proven_scan_arg_llong
xcv_scan_arg_long	proven_scan_arg_long
xcv_scan_arg_none	proven_scan_arg_none
xcv_scan_arg_short	proven_scan_arg_short
xcv_scan_arg_str_view	proven_scan_arg_str_view
xcv_scan_arg_t	proven_scan_arg_t
xcv_scan_arg_type_t	proven_scan_arg_type_t
xcv_scan_arg_u32	proven_scan_arg_u32
xcv_scan_arg_u64	proven_scan_arg_u64
xcv_scan_arg_uint	proven_scan_arg_uint
xcv_scan_arg_ullong	proven_scan_arg_ullong
xcv_scan_arg_ulong	proven_scan_arg_ulong
xcv_scan_arg_ushort	proven_scan_arg_ushort
xcv_scan_f64	proven_scan_f64
xcv_scan_fmt	proven_scan_fmt
xcv_scan_fmt_cursor	proven_scan_fmt_cursor
xcv_scan_fmt_from_file	proven_scan_fmt_from_file
xcv_scan_fmt_from_stdin	proven_scan_fmt_from_stdin
xcv_scan_fmt_internal	proven_scan_fmt_internal
xcv_scan_fmt_internal_view	proven_scan_fmt_internal_view
xcv_scan_i64	proven_scan_i64
xcv_scan_init	proven_scan_init
xcv_scan_skip_until	proven_scan_skip_until
xcv_scan_skip_until_number	proven_scan_skip_until_number
xcv_scan_skip_whitespace	proven_scan_skip_whitespace
xcv_scan_str	proven_scan_str
xcv_scan_t	proven_scan_t
xcv_scan_u64	proven_scan_u64
xcv_set_panic_handler	proven_set_panic_handler
xcv_sha256	proven_sha256
xcv_sha256_final	proven_sha256_final
xcv_sha256_init	proven_sha256_init
xcv_sha256_to_hex	proven_sha256_to_hex
xcv_sha256_update	proven_sha256_update
xcv_size_t	proven_size_t
xcv_strtod	proven_strtod
xcv_sysio_file_buffered	proven_sysio_file_buffered
xcv_sysio_lines_open	proven_sysio_lines_open
xcv_sysio_lines_t	proven_sysio_lines_t
xcv_sysio_out_t	proven_sysio_out_t
xcv_sysio_read_line	proven_sysio_read_line

xcv_sysio_std_t	proven_sysio_std_t
xcv_sysio_stderr_writer	proven_sysio_stderr_writer
xcv_sysio_stdin_lines	proven_sysio_stdin_lines
xcv_sysio_stdin_reader	proven_sysio_stdin_reader
xcv_sysio_stdout_buffered	proven_sysio_stdout_buffered
xcv_sysio_stdout_writer	proven_sysio_stdout_writer
xcv_sysio_print_impl	proven_sysio_print_impl
xcv_sysio_scan_chunk_impl	proven_sysio_scan_chunk_impl
xcv_sysio_scanner_deinit	proven_sysio_scanner_deinit
xcv_sysio_scanner_init	proven_sysio_scanner_init
xcv_sysio_scanner_scan_impl	proven_sysio_scanner_scan_impl
xcv_sysio_stderr	proven_sysio_stderr
xcv_sysio_stdin	proven_sysio_stdin
xcv_sysio_stdout	proven_sysio_stdout
xcv_time_breakdown	proven_time_breakdown
xcv_time_locale_en	proven_time_locale_en
xcv_time_locale_t	proven_time_locale_t
xcv_time_now	proven_time_now
xcv_time_now_datetime	proven_time_now_datetime
xcv_time_sleep	proven_time_sleep
xcv_time_t	proven_time_t
xcv_time_u16_fmt	proven_time_u16_fmt
xcv_time_u8_fmt	proven_time_u8_fmt
xcv_u16	proven_u16
xcv_u16str_append	proven_u16str_append
xcv_u16str_append_grow	proven_u16str_append_grow
xcv_u16str_append_partial	proven_u16str_append_partial
xcv_u16str_as_ptr	proven_u16str_as_ptr
xcv_u16str_create	proven_u16str_create
xcv_u16str_create_from_view	proven_u16str_create_from_view
xcv_u16str_destroy	proven_u16str_destroy
xcv_u16str_len	proven_u16str_len
xcv_u16str_t	proven_u16str_t
xcv_u16str_view_t	proven_u16str_view_t
xcv_u32	proven_u32
xcv_u64	proven_u64
xcv_u8	proven_u8
xcv_u8str_append	proven_u8str_append
xcv_u8str_append_byte	proven_u8str_append_byte
xcv_u8str_append_fmt	proven_u8str_append_fmt
xcv_u8str_append_fmt_grow	proven_u8str_append_fmt_grow
xcv_u8str_append_fmt_trunc	proven_u8str_append_fmt_trunc
xcv_u8str_append_fmt_with_scratch	proven_u8str_append_fmt_with_scratch
xcv_u8str_append_grow	proven_u8str_append_grow
xcv_u8str_append_partial	proven_u8str_append_partial

xcv_u8str_as_cstr	proven_u8str_as_cstr
xcv_u8str_as_view	proven_u8str_as_view
xcv_u8str_borrow	proven_u8str_borrow
xcv_u8str_create	proven_u8str_create
xcv_u8str_create_from_view	proven_u8str_create_from_view
xcv_u8str_destroy	proven_u8str_destroy
xcv_u8str_fmt_internal	proven_u8str_fmt_internal
xcv_u8str_insert	proven_u8str_insert
xcv_u8str_insert_grow	proven_u8str_insert_grow
xcv_u8str_is_valid	proven_u8str_is_valid
xcv_u8str_mut_t	proven_u8str_mut_t
xcv_u8str_remove	proven_u8str_remove
xcv_u8str_replace_at	proven_u8str_replace_at
xcv_u8str_replace_at_grow	proven_u8str_replace_at_grow
xcv_u8str_replace_first	proven_u8str_replace_first
xcv_u8str_reserve	proven_u8str_reserve
xcv_u8str_reset	proven_u8str_reset
xcv_u8str_t	proven_u8str_t
xcv_u8str_view_ends_with	proven_u8str_view_ends_with
xcv_u8str_view_eq	proven_u8str_view_eq
xcv_u8str_view_find	proven_u8str_view_find
xcv_u8str_view_from_cstr	proven_u8str_view_from_cstr
xcv_u8str_view_slice	proven_u8str_view_slice
xcv_u8str_view_starts_with	proven_u8str_view_starts_with
xcv_u8str_view_t	proven_u8str_view_t
xcv_u8str_view_to_cstr	proven_u8str_view_to_cstr
xcv_uintptr_align_up	proven_uintptr_align_up
xcv_uintptr_t	proven_uintptr_t
xcv_writer_buffered	proven_writer_buffered
xcv_writer_flush	proven_writer_flush
xcv_writer_from_buffer	proven_writer_from_buffer
xcv_writer_from_file	proven_writer_from_file
xcv_writer_from_u8str	proven_writer_from_u8str
xcv_writer_is_valid	proven_writer_is_valid
xcv_writer_write	proven_writer_write
xcv_writer_write_partial	proven_writer_write_partial
xcv_writer_write_str	proven_writer_write_str

